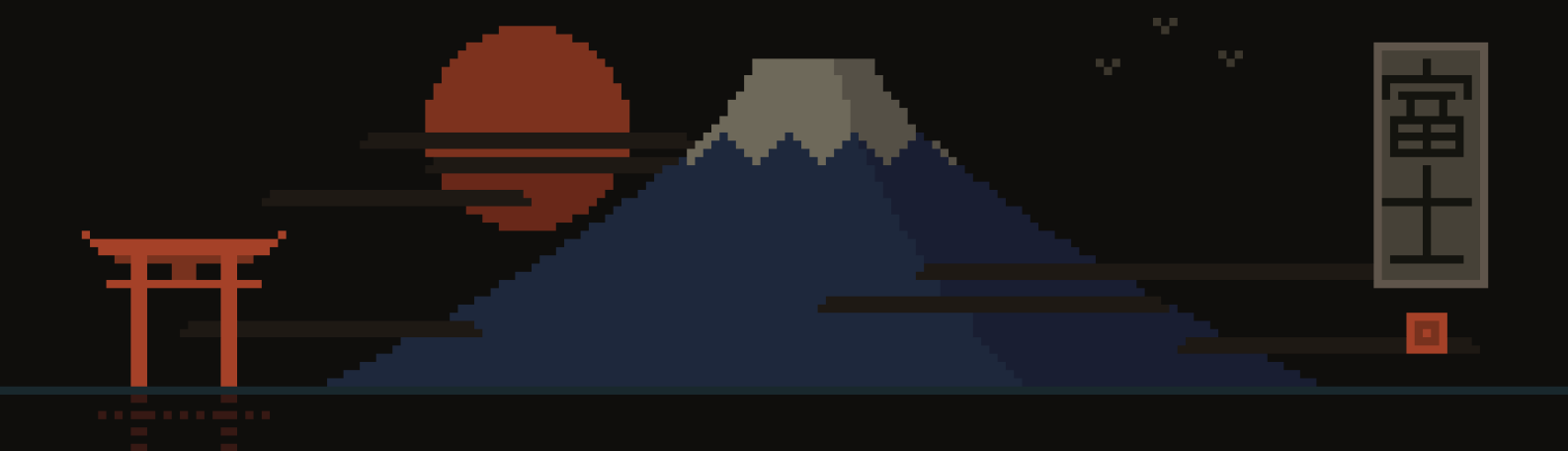
Fedokawaii — the handbook

Learning Linux on a hand-drawn Fedora desktop, from the first USB stick to how it all works

October 2026 · Fedora 44 / 45 · dwl 0.9 · wlroots 0.20



Contents

How to read this book 7
Where to read, by what you want 8

Part I — Before you start 10

1. What this is 11

2. Linux, Fedora and a desktop, in plain words 13
2.1 What an operating system does 13
2.2 Linux is a kernel; Fedora is a distribution 13
2.3 Words you will meet first 15
2.4 Free software, and why there are many Linuxes 15
2.5 The whole machine in one picture 15

3. Preparing: the computer and two USB sticks 17
3.1 What you need 17
3.2 Save what is on the computer 17
3.3 Download Fedora Server, and check it 18
3.4 Write the installer stick 19
3.5 Put the project on a second stick 20
3.6 The computer's start-up settings 20

4. Installing Fedora Server 22
4.1 Starting the installer 22
4.2 The installer, screen by screen 22
4.3 Your first login: the text console 24

5. Running the setup 27
5.1 Getting the project onto the machine 27
5.2 Your settings: fedokawaii.conf 28
5.3 The menu and its five scripts 30
5.4 What setup does, phase by phase 32
5.5 After setup: the toolkit 34
5.6 If setup stops 34

Part II — First days on the desktop 36

6. Your first login, and the screen 37
6.1 Logging in 37
6.2 The screen at a glance 37
6.3 Six keys to start with 38
6.4 Words for what you see 38
6.5 The bar 39
6.6 Locking the screen 40
6.7 Logging out, restarting and shutting down 41

7. Windows, workspaces and keys 42
7.1 Tiling, in pictures 42
7.2 Keybindings 43
7.3 Workspaces 45
7.4 Frames: which window has the focus 45
7.5 Recipe: plug in a second monitor 46

8. Menus, settings and notices 47
8.1 Menus 47
8.2 Getting online: cable and Wi-Fi 49
8.3 Bluetooth: pairing headphones 50
8.4 Notices and do not disturb 51
8.5 The system's own control, in one minute 51

9. Theme and wallpaper 53
9.1 Dark and light 53
9.2 Wallpapers 54
9.3 Recipe: switch the theme and the wallpaper 56

10. Files and folders 58
10.1 Folders, paths and your home 58
10.2 Where your own files go 59
10.3 yazi, the file manager 60
10.4 Open, upload and save: the file dialog 63
10.5 Which program opens a file 65
10.6 Recipe: choose which program opens a type of file 65

11. The terminal: first steps 67
11.1 Terminal, shell and console: five words 67
11.2 The shell's words 68
11.3 The prompt 68
11.4 Your first commands 70
11.5 Copy, move and remove 72
11.6 The anatomy of a command 73
11.7 Getting help: tldr, man and --help 74
11.8 Typing less: Tab, history and Ctrl+R 75
11.9 foot, the terminal 76
11.10 When the terminal seems stuck 77

12. Copy, paste and screenshots 79
12.1 Copy and paste 79
12.2 Screenshots 80
12.3 Marking up a screenshot: satty 81
12.4 Recipe: take, annotate and paste a screenshot 81

13. USB sticks, disks and SD cards 83
13.1 What happens when you plug one in 83
13.2 Ejecting 84
13.3 In yazi and every file dialog 85
13.4 Encrypted drives and disk images 86
13.5 From a terminal 86
13.6 Looking after a drive 87
13.7 Pitfalls, and where it lives 87
13.8 Part II, looking back 88

Part III — Everyday work 89

14. Programs and the web 90
14.1 The launcher: fuzzel 90
14.2 Brave Origin, the browser 91
14.3 Recipe: upload a file from Brave 92
14.4 Screen sharing — picture and sound 93
14.5 Recipe: share your screen, with sound, in a call 93

15. Pictures, video, music and sound 95
15.1 Pictures: imv 95
15.2 Video: mpv and yt-dlp 96
15.3 Recipe: watch or keep a video from the web 98
15.4 Music: spotify_player and cava 98
15.5 Now playing, and the media keys 99
15.6 Sound: the mixer and effects 100

16. Reading, documents and notes 102
16.1 PDFs, EPUBs and comics: zathura 102
16.2 Your library: books (Super+R) 104

16.3 Notes (Super+N) 105
16.4 Documents: Xournal++, AbiWord, Gnumeric and the PDF tools 106
16.5 Slides in the terminal: presenterm 107
16.6 The calculator: qalc 108

17. Writing text: neovim 110
17.1 Modes 111
17.2 Never stuck 113
17.3 The first hour 114
17.4 The grammar: verb, count, motion 116
17.5 The dot 117
17.6 Text objects 117
17.7 Visual mode and block mode 118
17.8 Registers and the clipboard 119
17.9 Macros 120
17.10 Files, buffers and windows 121
17.11 Finding 122
17.12 The file tree 123
17.13 The leader key 123
17.14 Search and replace across a project 125
17.15 Language servers 125
17.16 Completion 127
17.17 Formatting 127
17.18 Extras and Mason: adding a language 127
17.19 How LazyVim is set up here 128
17.20 Keeping it updated and healthy 128
17.21 A daily workflow 128
17.22 A learning path 128

18. Finding anything: the shell's helpers 131
18.1 bash on this desktop, and its helpers 131
18.2 bat, ripgrep, fd, fzf, eza, zoxide and tldr 133
18.3 Practice: find a file, a word, a folder 134

19. Do it yourself: what websites charge for 136
19.1 Before you start 136
19.2 PDFs: join, split, pick and turn pages 137
19.3 PDFs and pictures 138
19.4 PDFs: smaller, locked, unlocked 139
19.5 PDFs: text, scans and OCR 140
19.6 Fill in and sign a PDF 141
19.7 Pictures 141
19.8 Video and sound 143
19.9 Download and record 145
19.10 Files: archives, checksums, trash, renaming, text 146
19.11 Office documents and spreadsheets 147
19.12 Quick conversions: units, money, time 147

20. Torrents: searching and downloading 149
20.1 The first time: Prowlarr 150
20.2 Searching and downloading with torrra 150
20.3 How it behaves 151
20.4 When something is wrong 152

21. Games and hardware 154
21.1 Steam — games 154
21.2 Hardware — OpenRGB, Solaar, liquidctl, LACT 155

Part IV — Understanding Linux 157

22. A map of the system 158
22.1 The layers of this desktop 158
22.2 How Fedora got here: the changes that matter 159
22.3 How to explore any Linux machine 160

23. Files and the file system 162
23.1 Everything is a file — nearly 162
23.2 The hierarchy 162
23.3 Where programs keep their files 163
23.4 Mounts 165
23.5 Btrfs: subvolumes, snapshots and copy-on-write 166
23.6 Inodes and links 166
23.7 Practice: a walk through the tree 167

24. Users, permissions and sudo 170
24.1 Users and groups 170
24.2 Permissions 171
24.3 sudo 173
24.4 polkit 173
24.5 Why the desktop never runs as root 174
24.6 Practice: permissions by hand 174

25. The shell as a language 176
25.1 The shell's ideas, by example 176
25.2 Three streams, redirection and pipes 178
25.3 Environment and PATH 179
25.4 Quoting, globs and regular expressions 179
25.5 Scripts and the shebang 180
25.6 Your first script, step by step 180
25.7 History rules 182
25.8 Secrets: keys and passwords 183
25.9 Why a new terminal opens at once 183

26. Processes and signals 184
26.1 How a program starts: fork and exec 184
26.2 Signals 185
26.3 Jobs: the shell's own processes 186
26.4 Priority: nice 186
26.5 Watching processes: btop 186
26.6 Practice: start, pause and stop a process 188

27. Packages: rpm and dnf 190
27.1 What an RPM holds 190
27.2 dnf5 191
27.3 DNF's settings on this desktop 192
27.4 Weak dependencies, install reasons and groups 192
27.5 Keys and gpgcheck 193
27.6 The repositories this desktop uses 193
27.7 Pinned tools, and why 193
27.8 A worked chain: a video, from mpv down to the GPU 194
27.9 A second chain: the music in the bar 196
27.10 The manifest, setup and verify 196
27.11 Practice: questions to ask rpm and dnf 197

28. Services, timers and logs: systemd 199
28.1 systemd and its units 199
28.2 The system manager and yours 200
28.3 Drop-in files: changing a unit without editing it 200
28.4 Timers, not cron 201
28.5 The journal: reading the logs 201
28.6 cgroups 202
28.7 Practice: look at a service 203

29. How the machine starts 206
29.1 Stages 1–2: UEFI, shim and Secure Boot 207

29.2 Stage 3: GRUB and the boot entries 207
29.3 The boot menu 208
29.4 Stage 4: the kernel and its arguments 209
29.5 Two kernels, and the way back 209
29.6 Stage 5: the initramfs and early microcode 209
29.7 Stage 6: systemd 210
29.8 Stages 7–9: greetd, the login, the session 210
29.9 Stages 10–11: dwl and the autostart 211

30. Hardware: drivers, firmware and devices 214
30.1 Drivers and firmware 214
30.2 Devices: udev and /dev 215
30.3 Bluetooth: BlueZ 215
30.4 The performance profile: tuned 216
30.5 Practice: what is inside this computer? 216

31. Graphics and sound 219
31.1 DRM and KMS 219
31.2 The drivers 220
31.3 Wayland, wlroots and dwl 220
31.4 Layer shell: the bar, notifications and menus 220
31.5 XWayland 221
31.6 Toolkits 221
31.7 Fonts 221
31.8 Sound: ALSA, PipeWire, WirePlumber 221

32. How desktop programs talk: D-Bus and portals 223
32.1 D-Bus 223
32.2 The desktop portals 224
32.3 How the file dialog is wired 224
32.4 Notifications 225
32.5 MPRIS 225
32.6 polkit and udisks 225
32.7 Practice: listen to the bus 225

33. The network 228
33.1 NetworkManager 228
33.2 Names: systemd-resolved 228
33.3 The firewall: firewalld 229
33.4 Practice: is the network up? A checklist 229

34. Security and containers 232
34.1 SELinux 232
34.2 SELinux labels on files 233
34.3 Reading a denial 233
34.4 Rootless containers 233
34.5 podman and the Prowlarr container 234
34.6 A container as a systemd unit: the quadlet 235
34.7 Good habits 235

Part V — Looking after it 237

35. The system's control: Super+U 238
35.1 The menu, entry by entry 238
35.2 What verify proves 240

36. Updates and new Fedora releases 243
36.1 The update season 243
36.2 The season's update 244
36.3 Recipe: update everything 244
36.4 Between seasons 245
36.5 A new Fedora — upgrade 245
36.6 Recipe: move to a new Fedora 246
36.7 Built to be left alone 246

37. Snapshots: going back in time 248
37.1 What a snapshot is, with one example 248
37.2 Snapshots on this desktop 248
37.3 How many are kept 251
37.4 snapper at a glance 252
37.5 Recipe: undo a bad update 253

38. Keeping it clean 254
38.1 Keeping packages lean 254
38.2 Disk space and logs 255
38.3 Recipe: free disk space 257
38.4 A seasonal routine 258

39. When something goes wrong 259
39.1 A calm method 259
39.2 Symptoms, and where to look 260
39.3 The desktop's own self-tests 261
39.4 When the desktop will not start 262

Part VI — Making it yours 265

40. How the look is built 266
40.1 The two palettes 266
40.2 Where every app gets its colours 267
40.3 Fonts 268
40.4 Icons 270
40.5 Pointers 271
40.6 Transitions 272
40.7 The terminal's invisible marks 272
40.8 fetch and the sixteen colours 272
40.9 Why the bar costs almost nothing 273

41. Keeping history: git and lazygit 274
41.1 What git is 274
41.2 Once, before the first commit 275
41.3 Opening it 276
41.4 The screen 276
41.5 The colours here 277
41.6 A first commit, step by step 277
41.7 Practice: a first commit of your own 278
41.8 Staging part of a file 280
41.9 Commit, amend, undo 281
41.10 Stash 281
41.11 Branches 282
41.12 Pull and push 283
41.13 Resolving a conflict 283
41.14 Rewriting history: interactive rebase 284
41.15 History and diffs 285
41.16 Cherry-pick 285
41.17 Recipes 286

42. Changing it yourself 288
42.1 Two kinds of change 288
42.2 Who owns which config 289
42.3 Where each thing lives 290
42.4 The working copy 292
42.5 The one rule for configuration 293
42.6 Check before you apply: the self-tests 293
42.7 Apply: the copy becomes the desktop 294
42.8 Keep it: commit, push, undo 295

42.9 Rules the scripts count on 296
42.10 Recipe: change a setting for good 296
42.11 Adding a package properly 296
42.12 Recipe: install a program and keep verify happy 296
42.13 Taking a package out 297

43. Learning to program here 298
43.1 How to learn here 298
43.2 Languages worth learning now 298
43.3 Read a real script: the weather line 300
43.4 Ideas, defined, with an example each 301
43.5 Where to read 303
43.6 A ladder of first projects 303

44. Learning with an AI assistant 305
44.1 Claude Code, an assistant in the terminal 305
44.2 Learning with it, safely 306

Appendices 308

A. Software, and why each piece 309

B. Where everything lives 310
B.1 Your files 310
B.2 Each program's files 310
B.3 On the system 312

C. Keys at a glance 313

D. Commands by job 316
D.1 Files and folders 316
D.2 Viewing 316
D.3 Searching 317
D.4 Text processing 317
D.5 Archives 317
D.6 Processes 318
D.7 Services and logs 318
D.8 Packages 318
D.9 Users and permissions 319
D.10 Disks and file systems 319
D.11 Network 320
D.12 Hardware 320

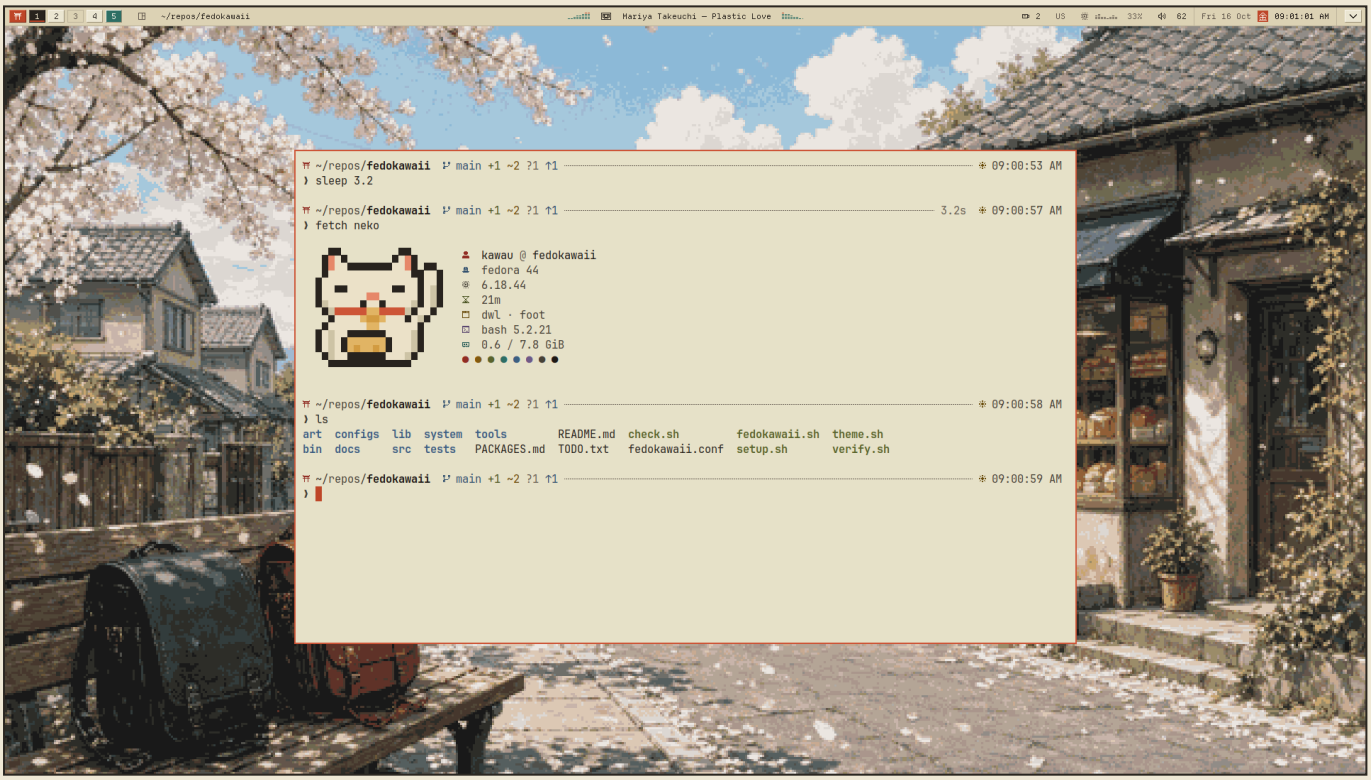
E. Glossary, A–Z 322

F. Where things stand, October 2026 330
F.1 The Linux kernel 330
F.2 Fedora 331
F.3 The Linux desktop 331
F.4 Gaming 332
F.5 The software industry and AI 333
F.6 Security 334
F.7 How to keep up 335

G. Sources 337



The desktop on the dark theme: the bar on top, a terminal with the prompt, fetch and a listing.



The same desktop on the light theme — switched live with Super+Shift+W.

How to read this book

The two pictures above are where this book ends up: a desktop drawn pixel by pixel, on a Linux you install and understand yourself. The book takes you there from nothing — from making the installer stick on Windows, one rung at a time, up to how the kernel starts the machine.

Who it is for. One reader: new to Linux, at home on Windows, at ease with SQL, Power BI and DAX. You learn best from a small example first, then the rule it shows — so every chapter is written that way. Where a Windows habit helps or misleads, a note says so; where a query explains a command better than prose, a note shows the query.

The ladder. Read it in order the first time; each Part stands on the one before.

Part	What it gives you	When to read it
I — Before you start, ch. 1–5	Fedora installed, and the setup building the desktop	on Windows, before you touch the PC; then at the new machine
II — First days on the desktop, ch. 6–13	windows, keys, menus, files, the terminal's first steps, copying, USB drives	the first week
III — Everyday work, ch. 14–21	the browser, pictures and music, reading, neovim, finding anything, converting files	as you need each program
IV — Understanding Linux, ch. 22–34	how the system works underneath, layer by layer, checked on this machine	once the desktop is familiar, a chapter at a time
V — Looking after it, ch. 35–39	updates, new releases, snapshots, cleaning, recovering	ch. 35 within the first season (90 days); the rest when you need it
VI — Making it yours, ch. 40–44	changing anything so updates keep it; git; learning to program	when you want to change something
Appendices A–G	software, paths, keys, commands, words, sources	any time, to look something up

How a chapter is built. Every chapter has the same five parts:

1. **In this chapter** — a box at the top: what you will be able to do at the end, and *You need* — the chapters it builds on.
2. **Word** boxes — each new term, defined once, the first time it is used, with an everyday example. Later chapters use the word plainly.
3. **Try this** — something to do at the machine, always in three parts: the command, what you see, what it means.
4. **What you learnt** — the chapter in a few lines.
5. **Check yourself** — three questions. The answer follows each arrow, in italics: cover it before you read it.

A **Try this** looks like this one, from your first login (§4.3):

```
$ whoami
you
```

You typed	It means
`whoami`	*who am I*: the user you are logged in as

Conventions.

You see	It means
`$ whoami` in a grey block	a command: type what follows the `$`, then **Enter** — not the `$` itself
lines under it without a `$`	what the computer answers

You see	It means
`PS C:\Users\you>`	the same, in PowerShell on Windows (Part I only)
`# a comment` after a command	a note to you; the computer ignores it, and you need not type it
(for example)	your output differs: numbers, sizes, dates and versions vary
`…` and *(shortened)*	lines or characters left out to fit the page
Super+Return	keys pressed together; **Super** is the key with the Windows logo, the one that opens the Start menu on Windows
Settings → Shields	a path through menus
`~`	your home folder, `/home/you`
`you`, `fedora`	this book's user name and machine name; yours appear instead
§… and ch. …	a section or a chapter of this book, by number

The boxes. Five kinds, each with its own label:

Box	What it holds
In this chapter	the chapter's aims, and what it needs
Word — …	one term, defined
On Windows	a bridge from a Windows habit: what is the same, what is not
In SQL terms	a command explained as a query, or a Power BI idea
Careful	something that deletes data or locks you out — read it before the step under it

Looking things up. The glossary (appendix E) lists every Word box's term in one line, with the section that teaches it. *Keys at a glance* (appendix C) puts the keys of the desktop, the terminal and the main programs on two pages; *Commands by job* (appendix D) finds a command by what you want done. On the desktop, **Super+/** lists every key, and **Super+U** → *handbook* opens this book.

Type, do not paste. Type every command yourself, even a long one. Your fingers learn it, and you read it as you go. Pasting from a PDF can also bring in characters you cannot see, or turn straight quotes into curly ones — `fedokawaii.conf` refuses a curly `“` at once (§5.2). And whatever you paste runs as you: read a command before you run it (§34.7).

Where to start. If Fedora is not on the PC yet, start at ch. 1. If the setup has already built the desktop, start at ch. 6.

Where to read, by what you want

You want to	Read
install it	chapters 1–5
use it day to day	chapter 6
use the terminal	chapter 11
use a USB stick, a disk or an SD card	chapter 13
browse, watch, listen, read	chapters 14–16
write and edit text in neovim	chapter 17
convert a PDF, a picture or a video without a website	chapter 19
understand Linux itself: the boot, files, processes, packages, the graphics and sound stacks	chapters 22–34
keep it healthy, update it, move to a new Fedora	chapters 35 and 36

You want to	Read
undo a change	chapter 37
free disk space	§38.3
fix something that broke	chapter 39
change anything about it: keys, colours, the bar, packages	chapter 42 (**Super+Shift+U**)
keep a history of your changes	chapter 41
learn programming on it	chapters 43 and 44
know where Linux, the desktop and the software trade stand now	appendix F
look up a key, a command or a word	appendices C, D and E

Part I — Before you start

You are still on Windows: nothing on the new computer has changed yet. By the end of Part I, Fedora is installed on it and the setup has turned it into the desktop in the two pictures at the front of this book.

Chapter	What you do there
1. What this is	see what you are building, and the four ideas behind it
2. Linux, Fedora and a desktop, in plain words	learn the words: operating system, kernel, distribution, package, desktop
3. Preparing: the computer and two USB sticks	check the PC, save its files, make the installer stick and the project stick — all on Windows
4. Installing Fedora Server	install a minimal Fedora and type your first commands
5. Running the setup	choose your settings and let the setup build the desktop

1. What this is

> **In this chapter**
> - say in two sentences what Fedokawaii is
> - name what you will have at the end of Part I
> - tell what is the same as on Windows, and what is new
> - name the four ideas every file follows
>
> **You need:** nothing yet.

This is the first rung of the ladder: no commands, no settings — only the picture of where you are going.

Fedokawaii takes a **minimal Fedora Server** install and turns it into a small, fast Wayland desktop built around **dwl**, a tiling compositor of about three thousand lines (the words are explained in §2.3). Everything you see on screen was drawn for it: the icons, the Fuji wallpaper scene, the lock screen, the boot menu, the pictures in `fetch` (the terminal's summary of the machine). Nothing comes from a theme pack. The name is Fedora + *kawaii* (かわいい), the Japanese word for cute.

What you will have at the end.

- **A working desktop**: a bar across the top, windows that arrange themselves, a browser, a terminal, a file manager, music and video — the two pictures at the front of the book.
- **A system that looks after itself**: one menu, **Super+U** (**Super** is the Windows key), updates it every 90 days, checks it, repairs it, and goes back in time when a change goes wrong.
- **A book that explains it**: every program, every key and every file, from the first USB stick up to how Linux works underneath.

What is the same, and what is new. Much of it will feel familiar from Windows: a browser, folders full of files, a terminal like PowerShell, a key with the Windows logo that does a lot. Three things are new. **Keys come before the mouse**: almost everything has a key, and the mouse is there when you want it. **One tool per job**: one browser, one editor, one file manager — not three that overlap. **Every setting is a text file** you can read, change and keep a history of, instead of a dialog box with tabs.

Try this (on Windows, now). Press **Windows+Left** in any window, then click a second window in the row that appears.

You did	You see	It means
Windows+Left	the window fills the left half of the screen	Windows *snaps* it
click a second window	it fills the right half	two windows side by side, none hidden

On this desktop that happens to **every** window, always, without a key: open a third and the screen is shared again. That is *tiling* (§2.3).

Four ideas run through every file:

Idea	What it means in practice
One tool per job	one editor (neovim), one browser (Brave Origin), one terminal (foot), one launcher (fuzzel, also every menu). `verify.sh` fails if a second one appears, and update and repair take out what is doubled or fights (§36.2)
Nothing hidden	every package is listed in `lib/manifest.sh`, every release binary in `lib/upstream.sh`, every colour in `tools/build-themes.py`, every icon in `art/icons.txt`
Proof, not hope	`verify.sh` checks the machine against the lists; `theme.sh --check` checks every config against the repository
Pixel-exact	the UI fonts and icons sit on an 11-pixel grid and are drawn at whole multiples of it only (11, 22, 33 — the boot menu 44 on a 5K screen); pixel art is enlarged by whole numbers only, so every pixel stays a sharp square

In plain words: the project keeps a written list of everything it installs, and a checker that compares the machine with that list. If the two differ, you are told — nothing drifts quietly.

What you learnt

- Fedokawaii turns a minimal Fedora Server into a small, hand-drawn tiling desktop.
- At the end of Part I you have the desktop, its upkeep menu (**Super+U**) and this book.
- New here: keys before the mouse, one tool per job, every setting a text file.
- Four ideas: one tool per job, nothing hidden, proof not hope, pixel-exact.

Check yourself

1. What does Fedokawaii start from? → *a minimal Fedora Server install*
2. Which key opens the system's own upkeep menu? → *Super+U*
3. What does "proof, not hope" mean here? → *a checker compares the machine with the project's lists*

2. Linux, Fedora and a desktop, in plain words

> **In this chapter**
> - explain what an operating system does, with one key press as the example
> - tell the kernel, the distribution and the desktop apart
> - recognise Wayland, compositor, tiling, dwl and package
> - draw the machine as four layers
>
> **You need:** ch. 1.

Before you install anything, you need the words. This chapter gives them to you with Windows as the reference; nothing here asks you to type a Linux command.

2.1 What an operating system does

Press the letter **a** in Notepad. Four things happen, faster than you can see:

1. The keyboard sends a small code down the cable: "key 30 went down".
2. The **kernel** — the core of the operating system — receives it through its keyboard driver.
3. The program that owns the screen decides which window has the focus: Notepad.
4. Notepad receives "a" and draws the letter.

On this desktop the path is the same, with other names: the Linux kernel, then dwl (the program that owns the screen here), then the program in focus — a terminal, say.

> **Word — operating system.** The layer of software between programs and the hardware. It shares out the processor, memory and disk, keeps programs apart, and gives them files, the network and a screen. Windows is one; Fedora Linux is another.

An operating system has three jobs. Each has a Windows name you know, and a name here:

Job	On Windows	Here
share the hardware: processor time, memory, disk	the Windows kernel; Task Manager shows the sharing	the Linux kernel; btop shows it (ch. 26)
keep programs apart	each program in its own memory; a prompt before an administrator's change	the same, plus users, permissions (ch. 24) and SELinux
give programs files, the network and a screen	NTFS and drive letters; the network settings; the Windows desktop	Btrfs and one tree of folders; NetworkManager; the compositor, dwl

> **Word — kernel.** The program that starts first and stays in charge: it talks to the hardware and decides which program runs when. On Windows it is a file called `ntoskrnl.exe` you never see; here it is Linux.

The kernel does not know every keyboard, disk and graphics card by itself. For each kind of device it has a small part that knows it:

> **Word — driver.** The part of the system that operates one kind of hardware. The graphics driver drives the graphics card; the Wi-Fi driver drives the Wi-Fi card. Windows downloads them from the maker; on Linux most are part of the kernel.

2.2 Linux is a kernel; Fedora is a distribution

Strictly, **Linux** is only the kernel. What people call "Linux" is a whole system built around it.

> **Word — distribution.** The kernel and thousands of programs around it, tested together and released as one system, with one way to install and update them. Fedora, Ubuntu and Debian are distributions, as Windows 11 Home and Pro are editions of one system.

Layer	What it is	On this machine	See it (you will run these in ch. 11)
kernel	Linux itself, plus its drivers (modules)	the Fedora kernel package	`uname -r` → `7.2.8-200.fc44.x86_64` (for example)
distribution	the kernel and everything around it, as one tested release	Fedora Linux 44 or 45	`cat /etc/fedora-release` → `Fedora release 44 (Forty Four)`
desktop	the programs that draw windows and take input	dwl, waybar, mako, fuzzel, foot	`dwl -v` → `dwl v0.9-dirty` (*dirty*: built with this desktop's patch)

The same kernel runs phones, servers and supercomputers. What makes this machine a desktop is the userspace on top — every program outside the kernel — and Fedokawaii chooses that userspace piece by piece.

Try this (in PowerShell on Windows). Right-click the Start button and choose *Terminal* (on Windows 10, *Windows PowerShell*). Type this line and press **Enter**:

```
PS C:\Users\you> Get-CimInstance Win32_OperatingSystem | Select-Object Caption, Version

Caption                  Version
-------                  -------
Microsoft Windows 11 Pro 10.0.26100
```

(for example)

Column	It means
`Caption`	the name of the whole system — like *Fedora Linux 44*
`Version`	the version of its core: Windows 11 still has kernel version 10.0 — like `7.2.8` for Linux

The name of the whole and the version of its kernel are two different things — on Windows and on Linux alike.

Fedora

Fedora is a community distribution sponsored by Red Hat. It comes in **editions**, each a different starting set of packages for one purpose: Workstation (GNOME), KDE Plasma, Server, Cloud, CoreOS and IoT, with Atomic desktops, Spins and Labs beside them. This desktop starts from **Server** with *Minimal Install* (§4.2): the smallest set that boots, logs in and connects to a network — Fedora's `@core` package group — and adds everything else on purpose.

Fact	Value
a new release	about every six months, aimed at April and October
support for each release	until four weeks after the release two later: about thirteen months
recent releases	Fedora 43 on 28 October 2025, Fedora 44 on 28 April 2026; Fedora 45 is in beta in October 2026
where the date is written	`grep SUPPORT_END /etc/os-release`

> **Word — upstream.** The project that writes a program, as opposed to the distributions that package it. The kernel's upstream is kernel.org; Fedora takes the kernel from there and passes its fixes back.

Upstream first. Fedora tries not to differ from the projects it packages: a fix goes to the project that wrote the program, not into a Fedora-only patch, so everyone gets it and Fedora carries less of its own. The effect for you:

Fedora is usually among the first to ship new versions of the kernel, Mesa, GNOME, systemd and PipeWire, close to how their authors made them, and the upstream documentation fits what you have.

2.3 Words you will meet first

Five words come up from the first page of the setup onwards. Each has an everyday example:

Term	Plain meaning	For example
Wayland	the way programs and the screen talk on modern Linux, the successor of X11: each program draws its own window, and one central program shows them all	like a meeting where each person writes on their own card and one chair pins the cards on the board
compositor	that central program: it places the windows, puts them on the screen and hands them the keyboard and mouse. Under Wayland it is also the display server	the chair of that meeting; on Windows, the Desktop Window Manager does the placing
tiling	the compositor arranges the windows side by side for you, never overlapping, instead of you dragging them about	Windows Snap with two windows side by side — but always, for every window
dwl	the compositor here: a small one written in C on the *wlroots* library, configured by editing its source (`src/dwl/config.h`) and building it again	a program whose settings are a recipe you change and cook again
package	one program or library in Fedora's format (`.rpm`), installed, updated and removed by `dnf`	like an app from the Microsoft Store or `winget`, with a list of what it needs

Read more: wayland.freedesktop.org · dwl's README, codeberg.org/dwl/dwl.

2.4 Free software, and why there are many Linuxes

> **Word — free software.** Software whose source code anyone may read, change and pass on, changed or not. *Free* as in freedom, not as in price. Linux, Fedora and every program on this desktop are free software.

Windows is made by one company, which alone may change it. Linux and the programs around it are made in the open: anyone may take them, build on them and share the result. That is why there are many "Linuxes". Each distribution assembles the same parts — the Linux kernel, systemd, the GNU tools, Mesa for graphics — in its own way, with its own choices of version, defaults and release rhythm. Think of the same ingredients cooked by different kitchens.

Why Fedora for this book:

- **New**: a release every six months, close to the newest kernel and graphics drivers — which a gaming PC with a new graphics card needs.
- **Close to upstream**: programs as their authors made them, so their own documentation fits (§2.2).
- **Btrfs by default**: a file system that can take instant snapshots — frozen copies to go back to when an update goes wrong (ch. 37).
- **Red Hat's backing**: paid engineers, careful testing, and SELinux, a strong lock on what each program may touch.

2.5 The whole machine in one picture

Every part of this machine sits in one of four layers. Each layer uses the one below it:

```
layer                 on this machine                 on Windows                  the book
────────────────────  ──────────────────────────────  ──────────────────────────  ──────────
desktop and programs  dwl, waybar · Brave, foot       the desktop · Edge          Parts II–III
services              systemd · NetworkManager        services.msc · Wi-Fi        Part IV
kernel                Linux · its drivers             Windows kernel · drivers    Part IV
hardware              processor · graphics card       the same box                Part IV
```

> **Word — service.** A program that runs in the background with no window, started when the computer starts. Windows lists them in *services.msc*; here **systemd** starts and watches them — the network, the login screen, the snapshot timer.

Read the picture from the bottom: the hardware does the work, the kernel drives it, the services keep things running without a window, and the desktop and programs are what you see.

Part IV climbs the lower layers one at a time: the services in ch. 28 and 33, the kernel and the start in ch. 29, the hardware in ch. 30 and 31.

What you learnt

- An operating system shares the hardware, keeps programs apart, and gives them files, the network and a screen.
- Linux is the kernel; Fedora is a distribution built around it; dwl is the desktop's core.
- A key press travels keyboard → kernel → compositor → the program in focus.
- Tiling is Windows Snap for every window, always.
- The machine is four layers: hardware, kernel, services, desktop and programs.

Check yourself

1. Is Fedora a kernel or a distribution? → *a distribution, built around the Linux kernel*
2. Which program places the windows on this desktop? → *dwl, the compositor*
3. Where does Windows list its services? → *services.msc*

3. Preparing: the computer and two USB sticks

> **In this chapter**
> - check that the PC can run this desktop
> - save everything you want from it before it is erased
> - download Fedora Server and prove the download is intact
> - make an installer stick and a project stick, and start the PC from the first
>
> **You need:** ch. 2.

Everything in this chapter happens on Windows. Each step ends with a check: do not go on until it passes.

3.1 What you need

The project's readiness check, `check.sh`, will test the machine before setup changes anything. These are its limits, so you can check them now, on Windows:

You need	Why	Check on Windows
a 64-bit Intel or AMD PC	`check.sh` stops on anything but x86_64; a Windows-on-Arm laptop (Snapdragon) will not do	*Settings* → *System* → *About*: *System type* says "x64-based processor"
AMD or Intel graphics	the open drivers only; a PC with NVIDIA graphics alone is refused	Task Manager → *Performance* → *GPU*
8 GB of memory or more	under 4 GB setup stops; 4–7 GB works, with a warning	Task Manager → *Performance* → *Memory*
a disk that may be erased completely	the install takes the whole disk (§3.2)	—
40 GB free after the install, much more for games	under 20 GB `check.sh` stops; 20–40 GB is a warning	the disk's size in File Explorer → *This PC*
UEFI start-up	the usual mode for a PC from the last ten years; `check.sh` accepts the older BIOS mode too	`msinfo32` → *BIOS Mode*: "UEFI"
two USB sticks	4 GB or more for the installer; any size for the project	—
a network cable to the router, or a phone that can share its connection over USB	Wi-Fi works only after setup has added it (§4.2)	—
a Windows PC for the preparation	to download and write the sticks — it can be this PC, before it is wiped	—
your GitHub sign-in	the project is private (§3.5)	sign in at github.com

A Bluetooth adapter on a USB plug should be plugged in before setup, so setup sees it.

Check: every row above is true, or you know which warning you will see.

3.2 Save what is on the computer

Installing erases the whole disk you choose: every file, every program, and Windows itself. There is no undo. Copy off everything you want first, to an external disk or a cloud drive:

- your folders: *Documents*, *Desktop*, *Pictures*, *Videos*, *Music*, *Downloads*;
- files kept elsewhere on the disk — a `C:\Projects` folder, say, or a game's saves;
- your browser's bookmarks and passwords (export them, or check that your browser account syncs them);

- licence keys of programs you paid for, and any two-factor recovery codes kept on this PC.

Check: open a few of the copies on another computer, or in the cloud drive's web page. A copy you have not opened is a hope, not a copy.

> **Careful.** Keeping Windows on the **same** disk beside Fedora is not what this book covers or tests. To keep Windows, give Fedora a disk of its own, and choose between them in the firmware's boot menu (§3.6).

If Windows stays on another disk, do three things in Windows first. The safest of all: unplug the Windows disk while you install Fedora, then plug it back in.

> **On Windows.** Turn off *fast startup*: Control Panel → *Power Options* → *Choose what the power buttons do* → *Change settings that are currently unavailable* → untick *Turn on fast startup* → *Save changes*. Then find your 48-digit BitLocker recovery key (aka.ms/myrecoverykey): firmware changes can make Windows ask for it.

Fast startup leaves Windows half asleep instead of shut down, with its disk marked as in use; off, Windows really shuts down. The recovery key is your way back into an encrypted Windows disk: write it down, away from the PC.

3.3 Download Fedora Server, and check it

> **Word — image.** One file holding a whole disc or stick, byte for byte — an `.iso` file. Written onto a stick, it makes a stick the PC can start from.

On Windows, open **fedoraproject.org/server/download** in your browser. Download two files from the *x86_64* row:

File	What it is
`Fedora-Server-dvd-x86_64-44-1.7.iso`	the installer image, the *DVD ISO* — the one with everything on it
`Fedora-Server-44-1.7-x86_64-CHECKSUM`	a small text file of fingerprints, one per image

Both land in your *Downloads* folder. (If the page offers a newer release, take that one and read its number wherever this book says 44.)

> **Word — checksum.** A short fingerprint computed from every byte of a file. Change one byte and the fingerprint changes completely. Like the two check digits in an IBAN, which catch a mistyped number — but far stronger.

The checksum Fedora uses is called **SHA-256**: 64 characters, the digits 0–9 and the letters a–f. If the fingerprint of your download equals Fedora's, every byte arrived intact.

Try this (in PowerShell). Your first commands: go to *Downloads*, then fingerprint the image. The second line takes a minute — it reads the whole file.

```
PS C:\Users\you> cd ~\Downloads
PS C:\Users\you\Downloads> Get-FileHash -Algorithm SHA256 .\Fedora-Server-dvd*.iso | Format-List

Algorithm : SHA256
Hash      : 85837793BFA36DB6BC709B4CECD2EC116951B87D9C53C3D95EB2FAC8DCF7CF1F
Path      : C:\Users\you\Downloads\Fedora-Server-dvd-x86_64-44-1.7.iso
```

You typed	It means
`cd ~\Downloads`	*change directory*: go into the *Downloads* folder of your home (`~`)
`Get-FileHash -Algorithm SHA256 .\Fedora-Server-dvd*.iso`	compute the SHA-256 of the image; `.\` means "here", and `*` stands for the rest of its name — less to type
`| Format-List`	show the result as one line per field, so the hash fits on a line

Now open the CHECKSUM file in Notepad. Among the lines for other images is the one naming yours (one line in the file, broken here to fit the page):

```
SHA256 (Fedora-Server-dvd-x86_64-44-1.7.iso) =
85837793bfa36db6bc709b4cecd2ec116951b87d9c53c3d95eb2fac8dcf7cf1f
```

PowerShell writes the hash in capitals; the file in small letters. They are **the same** hash: case does not matter in these letters. Compare the two, character by character — the first eight, the last eight, and a few in the middle. Fedora's download page shows the same value under the file too.

Check: the two hashes are equal. If they differ, delete the image and download it again; never write a stick from an image that does not match.

> **In SQL terms.** Trino has the same fingerprint: `SELECT to_hex(sha256(to_utf8('Sharjah')))` returns 64 characters. Change one letter — `'sharjah'` — and every character of the result changes. Equal hashes mean equal input.

Part IV explains where else this desktop uses SHA-256: every tool it downloads from GitHub is checked the same way (§27.7).

3.4 Write the installer stick

Fedora Media Writer writes the image onto a stick so the PC can start from it. It is Fedora's own tool, and it runs on Windows.

> **Careful.** Writing the image **erases the whole stick**. Take a stick with nothing you need on it, and unplug every other USB drive first, so you cannot pick the wrong one.

1. Install it. Download it from the same Fedora page (*Fedora Media Writer* → *Windows*), or in PowerShell:

```
winget install -e --id Fedora.FedoraMediaWriter    # Fedora's own package of it
```

2. Plug in the stick and start *Fedora Media Writer* from the Start menu. Windows asks whether it may make changes: *Yes*.

3. The first screen, *Select Image Source*, offers *Download automatically* and *Select .iso file*. Choose **Select .iso file**, then *Next*.

4. On *Write Options*, click *Select…* beside *Selected file* and pick `Fedora-Server-dvd-x86_64-44-1.7.iso` in *Downloads*. Under *USB Drive*, pick your stick — check its name and size.

5. Click **Write**. An *Erase confirmation* asks once more: the entire device will be erased. Confirm with *Write*. The stick is written, then read back and checked.

6. When the window says the image was written successfully, click *Finish*. Eject the stick from the taskbar: the *Safely Remove Hardware and Eject Media* icon → the stick's name.

Check: Media Writer finished without an error. (File Explorer may no longer show the stick's files, or offer to format it: say no — the stick is fine; Windows cannot read it.)

Rufus, a popular Windows tool, works too, in its *DD image* mode. Media Writer is Fedora's own, and needs no choices.

3.5 Put the project on a second stick

The project lives on GitHub as the **private** repository `MoeDia/fedokawaii` — a repository is a project's folder of files, kept with their history. Only you, signed in, can download it. A minimal Fedora has no browser, so you carry the project across on a second stick.

> **Word — file system.** The way a disk's space is organised into files and folders. Windows uses NTFS on its own disk and FAT32 or exFAT on sticks. **exFAT** is read by Windows and by Linux alike — the right choice for a stick that travels between them.

1. Plug the second stick in. In File Explorer, right-click it → *Format…*. Set *File system* to **exFAT** and *Volume label* to **FEDOKAWAII**. Click *Start*, then *OK* to the warning that all data will be erased.

> **Word — archive.** One file that holds a whole folder, often compressed — like a `.zip`. `.tar.gz` is the usual kind on Linux: *tar* packs the folder into one file, *gz* (gzip) makes it smaller.

2. In your browser, sign in to GitHub. Then open this address — the whole project as one archive, as it is now:

```
https://github.com/MoeDia/fedokawaii/archive/refs/heads/main.tar.gz
```

It saves as `fedokawaii-main.tar.gz` in *Downloads*.

3. Copy that file onto the FEDOKAWAII stick **as it is — do not open or unpack it**. Unpacked on Windows, the scripts would lose the mark that lets them run, and could gain Windows line endings that Linux scripts choke on. Fedora unpacks it in §5.1.

4. Eject the stick from the taskbar, as before.

Check: the stick holds one file, `fedokawaii-main.tar.gz`.

3.6 The computer's start-up settings

Before Windows starts, a small program built into the PC runs first: the **firmware**. It decides which disk to start from. To start the installer, you ask it for the USB stick.

Term	Plain meaning
firmware	the program in the PC's own chips that starts first, before any operating system; it has a settings screen of its own
UEFI	the modern kind of firmware, in every PC from the last ten years; the old kind is called BIOS, or *Legacy* / *CSM* in UEFI's settings
boot menu	the firmware's one-time list of disks to start from, opened by a key as the PC starts
Secure Boot	a UEFI setting that starts only signed programs; Fedora's are signed, so it may stay on

1. Plug the installer stick into the PC. Keep the project stick aside for now.

2. Switch the PC on and press the boot-menu key for its maker at once, several times, until a list of disks appears:

Maker	Boot menu (once)	Firmware settings
Acer	F12	F2
ASUS	F8 (desktops) or Esc (laptops)	Del or F2
Dell	F12	F2
HP	Esc, then F9	Esc, then F10
Lenovo	F12	F1 (ThinkPad) or F2
MSI	F11	Del
Gigabyte	F12	Del
ASRock	F11	F2 or Del

If no key works — some PCs start too fast to catch one — go through Windows instead: *Settings → System → Recovery → Advanced startup → Restart now*, then *Troubleshoot → Advanced options → UEFI Firmware Settings → Restart*. In the firmware's settings, find the *Boot* page and put the USB stick first.

3. In the list, choose the stick. If it is listed twice, choose the line that begins with **UEFI:**. Secure Boot may stay on: Fedora's start-up programs are signed (ch. 29 explains how).

What you should see: a dark screen with a short menu — *Install Fedora 44*, *Test this media & install Fedora 44* and *Troubleshooting*. That is the installer's start, and ch. 4 goes on from there.

> **On Windows.** The boot menu is a one-time choice, like *Use a device* on Windows' *Advanced startup* screen. Nothing is changed for good: without the stick, the PC starts from its disk again.

Check: the installer's menu is on the screen.

What you learnt

- The PC needs an Intel or AMD processor, AMD or Intel graphics, 4 GB of memory (8 better) and a disk that may be erased.
- Installing erases the whole disk: copy everything off first, and open the copies.
- A SHA-256 checksum proves a download is intact: `Get-FileHash` must match Fedora's line.
- Fedora Media Writer writes the image; the project goes on an exFAT stick as an unopened `.tar.gz`.
- The firmware's boot menu (F12, F11, F8…) starts the PC from the stick once.

Check yourself

1. Why does the project stay a `.tar.gz` on the stick? → *unpacked on Windows, the scripts lose their run mark and may gain Windows line endings*
2. `Get-FileHash` shows capitals and the CHECKSUM file small letters. Do they differ? → *no: case does not matter; compare the characters*
3. Which line do you choose if the stick is listed twice in the boot menu? → *the one beginning with UEFI:*

4. Installing Fedora Server

> **In this chapter**
> - start the installer and find your way round its summary screen
> - set every installer screen the way the setup needs it
> - log in at the text console and type your first commands
> - use `sudo` once, and switch the computer off from the console
>
> **You need:** ch. 3; the installer's menu on screen.

This is where the PC changes for good. Read each step before you click; nothing is written to the disk until the very last button, *Begin Installation*.

4.1 Starting the installer

The installer stick starts with a short menu:

Entry	What it does
Install Fedora 44	starts the installer at once
Test this media & install Fedora 44	first reads the whole stick and checks it, then starts the installer — chosen by itself after 60 seconds
Troubleshooting -->	more entries, among them *Install Fedora 44 in basic graphics mode*, for a screen that stays black

Choose **Test this media & install Fedora 44** this once (the arrow keys, then **Enter**). A stick that passed Media Writer's check rarely fails, but this check is free.

The first screen says **WELCOME TO FEDORA 44.** and asks *What language would you like to use during the installation process?* Choose *English* and *English (United States)*, then *Continue*.

The next screen is the centre of the installer: **INSTALLATION SUMMARY**.

> **Word — spoke.** One item on the installer's summary screen — *Keyboard*, *Time & Date*, *Installation Destination*… You click it, set it, press *Done* at the top left, and are back on the summary, the **hub**.

The spokes sit in four groups: *LOCALIZATION*, *SOFTWARE*, *SYSTEM* and *USER SETTINGS*. Under each spoke's name a short line says how it is set. A spoke that still needs you is marked with a warning sign, and a bar at the bottom says *Please complete items marked with this icon before continuing to the next step.* Until every mark is gone, *Begin Installation* stays grey — and, as the screen itself says, *We won't touch your disks until you click 'Begin Installation'.*

4.2 The installer, screen by screen

These words come up on the screens below:

Term	Plain meaning
partition	a fixed part of a disk, used as if it were a disk of its own; Windows' *C:* is one
Btrfs	a Linux file system that can take instant snapshots
subvolume	a part of a Btrfs file system with its own folder tree, mounted like a partition and snapshotted on its own: the installer makes `root` (mounted at `/`) and `home` (at `/home`)
snapshot	a frozen copy of the system's files at one moment, to go back to; on Btrfs it is made in an instant and takes space only for what changes after it

Term	Plain meaning
LVM · XFS	the volume manager and file system of Server's automatic layout; the snapshots here are Btrfs ones, so `check.sh` stops on anything else
kdump	memory set aside at every boot to record a kernel crash for later study
root	the administrator account, allowed everything; here it has no password and nobody logs in as it

Set every spoke as below. The order does not matter; *Installation Destination* has a walk of its own after the table.

Spoke	Set	Why
Keyboard	English (US)	the session adds Arabic itself (`KB_LAYOUTS`, §5.2)
Time & Date	your zone (Asia/Dubai), *Network Time* on	
Software Selection	**Minimal Install** (if the list shows only *Fedora Server Edition* and *Fedora Custom Operating System*, pick **Custom Operating System**), nothing ticked on the right	the scripts add exactly what is needed, so `verify.sh` can prove nothing else crept in
Installation Destination	**Custom → Btrfs → create automatically** (the walk below)	Server's *Automatic* is LVM + XFS; snapshots need Btrfs. Result: `/boot/efi`, `/boot`, subvolumes `root` and `home`
Network & Host Name	ethernet **on**; a host name if you like (left empty, setup names the machine `fedokawaii`)	a Minimal Install has no Wi-Fi yet (see below the table)
KDUMP	off	a desktop; saves memory
Root Account	**Disable root account**	everything runs through your own user with `sudo` (§4.3)
User Creation	your name and password; tick the administrator box (below)	setup and every later update run as the administrator, through your user; if the desktop ever fails to start, this user logs in on a text console (§39.4)

Why a cable. A Minimal Install has NetworkManager, but neither its Wi-Fi part nor the firmware many Wi-Fi cards need. Setup adds both in phase 2. Until it has finished, use a cable — or a phone's USB tethering, which counts as a cable. Afterwards, Wi-Fi is in quick settings (§8.2).

> **Careful.** *Accept Changes* plus *Begin Installation* erase **every** disk you ticked. If you keep Windows on another disk, make sure only Fedora's disk has the tick — or unplug the other one (§3.2).

Installation Destination, step by step.

1. Click *Installation Destination*. Under *Local Standard Disks*, click the PC's own disk so it shows a tick. Only that one: the installer stick is listed too — leave it unticked.
2. Under *Storage Configuration*, choose **Custom**. Leave *Encrypt my data.* unticked, unless you want to type a passphrase at every start (`check.sh` accepts both). Click *Done*.
3. **MANUAL PARTITIONING** opens. If the disk held Windows, its old parts are listed under *Unknown*. Click one, click **−** at the bottom, tick *Delete all file systems which are only used by Unknown.*, and click *Delete It*. A small *EFI System Partition* may stay: click it, click **−** again and confirm. Its warning that it *may be a system boot partition* is about Windows, which is going.
4. Above the link in the next step, set *New mount points will use the following partitioning scheme:* to **Btrfs**.
5. Click **Click here to create them automatically.** Four lines appear: `/boot/efi`, `/boot`, `/` and `/home`.
6. Click *Done*. **SUMMARY OF CHANGES** lists everything that will be deleted and made. Read it, then click **Accept Changes**.

Back on the hub, the spoke reads *Custom partitioning selected*.

User Creation, step by step. Type your full name, a user name (short, lower case: `you` in this book's examples) and a password twice. Then tick **Add administrative privileges to this user account (wheel group membership)**. With the root account disabled, the hub will not let you begin without an administrator. Click *Done*.

When no spoke is marked any more, click **Begin Installation**. *INSTALLATION PROGRESS* shows each step; at the end, click **Reboot System** and take the installer stick out as the screen goes dark.

Read more: Btrfs subvolumes.

4.3 Your first login: the text console

The PC restarts. Fedora's boot messages scroll past, and then the screen holds a single line of text. This is the **text console**:

> **Word — text console.** A full screen of text, with no windows and no mouse: you type a line, the computer answers below it. Six of them are always there, on **Ctrl+Alt+F1…F6**. The setup runs here.

Type your user name and press **Enter**; then your password and **Enter**. The password shows nothing as you type — no dots, no stars. That is normal: type it blind.

```
fedora login: you
Password:
[you@fedora ~]$
```

(for example: the name before `login:` is the machine's name)

The last line is the **prompt**: the computer is waiting for a command.

> **Word — prompt.** The text a command line shows when it waits for you. Here it says who you are, on which machine, in which folder — and ends in `$`. You type after it.

Part	It means
`you`	your user name
`fedora`	the machine's name (yours shows the one you chose, or `fedora`)
`~`	the folder you are in: `~` is your home, `/home/you`
`$`	you are an ordinary user; a `#` here would mean root

Try this. Three questions to the computer. Type each line after the `$`, then **Enter**:

```
$ whoami
you
$ pwd
/home/you
$ ls
$
```

You typed	It means
`whoami`	*who am I*: the user you are logged in as
`pwd`	*print working directory*: the folder you are in — your home
`ls`	*list*: the files in that folder — none yet, so it prints nothing

A command that has nothing to say says nothing: silence means it worked. Chapter 11 teaches these commands properly; for now they show the console answers you.

Try this. Is the network cable working? Ask for your addresses, then knock at Fedora's door:

```
$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp5s0           UP             192.168.1.23/24 fe80::…/64
$ ping -c 3 fedoraproject.org
PING fedoraproject.org (…) 56(84) bytes of data.
64 bytes from …: icmp_seq=1 ttl=50 time=182 ms
…
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
```

(for example; shortened)

You typed	It means
`ip -br addr`	the network cards and their addresses, briefly (`-br`). `lo` is the machine talking to itself; `enp5s0` is the cable's card: **UP**, with an address — connected
`ping -c 3 fedoraproject.org`	send three small packets (`-c 3`) to Fedora's server and wait for the answers: 3 received, 0% lost — the internet is reachable

If the card shows `DOWN`, or no address, check the cable and the router; setup cannot start without the network.

Try this. Did the installer make Btrfs, as setup needs?

```
$ findmnt -no FSTYPE /
btrfs
$ findmnt -no OPTIONS / | grep -o 'subvol=[^,]*'
subvol=/root
```

You typed	It means
`findmnt -no FSTYPE /`	the file system of `/` , the top of the system: **btrfs**
`findmnt -no OPTIONS / | grep -o 'subvol=[^,]*'`	of all its options, only the subvolume: **root**. The `|` hands the first command's output to the second, `grep` , which keeps the part that matches

If the first says `xfs` , the installer used its *Automatic* layout: install again, with *Custom → Btrfs* (§4.2).

Some commands change the system, and an ordinary user may not run them. For those, there is one word to put in front:

> **Word — sudo.** *Superuser do*: runs the one command after it as root, the administrator, after asking for **your** password. Only users with administrative privileges — the box you ticked — may use it.

Try this. The same question twice, once as yourself and once through `sudo` :

```
$ whoami
you
$ sudo whoami
[sudo] password for you:
root
```

The first time, `sudo` prints a short warning before it asks — three rules: respect the privacy of others, think before you type, and with great power comes great responsibility. Type your password (invisible again). The answer, `root` , shows what `sudo` does: that one command ran as the administrator. The next command is yours again.

> **On Windows.** `sudo` is the *User Account Control* prompt — "Do you want to allow this app to make changes?" — typed instead of clicked. Like UAC, it covers one command only. For a few minutes afterwards it does not ask for the password again.

Moving about the consoles. Ctrl+Alt+F2 shows the second console, with its own login; **Ctrl+Alt+F1** goes back to the first. Each is separate: log in on two and do two things at once. `exit` logs out of one.

Switching off. `sudo poweroff` switches the computer off; `sudo reboot` restarts it. Leave it on now if you go straight to ch. 5.

What you learnt

- The installer's boot menu: test the stick once; the summary screen is a hub of spokes.
- Installation Destination: *Custom* → *Btrfs* → *Click here to create them automatically* → *Accept Changes*.
- Your user gets administrative privileges; root stays disabled.
- At the console, the prompt `[you@fedora ~]$` says who, where and in which folder.
- `ip -br addr` and `ping` check the network; `findmnt` checks for Btrfs.
- `sudo` runs one command as root, after your own password.

Check yourself

1. Which partitioning scheme do you choose under *Custom*? → *Btrfs*
2. What does `$` at the end of the prompt mean? → *you are an ordinary user (root would be #)*
3. `sudo whoami` prints what, and why? → *root: sudo ran that one command as the administrator*

5. Running the setup

> **In this chapter**
> - unpack the project from its stick into `~/fedokawaii`
> - set your values in `fedokawaii.conf` with `nano`
> - run the readiness check, then **Both**, and follow the three phases
> - read why setup stopped, fix it and run it again
>
> **You need:** §4.3; the project stick (§3.5).

The last rung of Part I. From here, setup does the work: almost everything you see on the desktop is installed and checked by it, through two restarts, while you watch or walk away.

5.1 Getting the project onto the machine

The project is yours alone and private, on GitHub as `MoeDia/fedokawaii`. A Minimal Install has neither `tar` nor `git`, so the first command installs one (the network is up already). There are two routes: the stick you made in §3.5, or straight from GitHub.

A stick's files cannot be read the moment it is plugged in. On Windows, the stick gets a drive letter — *E:*, say. Linux has no drive letters: everything hangs in one tree of folders starting at `/`, and the stick must be hung into it at a folder first.

> **Word — mount.** To attach a disk or stick at a folder, so its files appear there: `sudo mount /dev/sda1 /mnt` puts the stick's files in `/mnt`. **Unmount** (`umount`) takes it away again, safely — like *Eject* on Windows.

Route 1: the USB stick. Plug the FEDOKAWAII stick in. Then:

1. Install `tar` (to unpack the archive) and `nano` (to edit the settings in §5.2):

```
sudo dnf install -y tar nano     # if nano is there already, dnf says so and goes on
```

2. Find the stick. `lsblk` lists every disk and its parts:

```
$ lsblk -o NAME,SIZE,LABEL,FSTYPE
NAME          SIZE LABEL      FSTYPE
sda          28.9G
└─sda1       28.9G FEDOKAWAII exfat
zram0           8G            swap
nvme0n1     931.5G
├─nvme0n1p1   600M            vfat
├─nvme0n1p2     2G            ext4
└─nvme0n1p3 928.9G            btrfs
```

(for example)

The PC's own disk is `nvme0n1` (on a PC with a SATA disk, `sda`), holding the parts the installer just made: `vfat`, `ext4` and `btrfs`. The stick is the small disk with the label **FEDOKAWAII** — here `sda`, and its one part `sda1`, the one to mount. On your PC it may be `sdb1`: read the name off your own list, and use it below. (`zram0` is compressed memory used as swap, not a disk.)

3. Mount it, unpack the project into your home, give the folder its usual name, and unmount:

```
sudo mount /dev/sda1 /mnt                        # the stick's files appear in /mnt
tar -xzf /mnt/fedokawaii-main.tar.gz -C ~ && mv ~/fedokawaii-main ~/fedokawaii
sudo umount /mnt                                 # the stick can come out
cd ~/fedokawaii                                  # into the project
```

In the `tar` line, `-x` extracts, `-z` undoes the gzip compression, `-f` names the archive and `-C ~` puts the result in your home; GitHub names the folder after the branch, so `mv` gives it its usual name. `&&` runs the `mv` only if `tar` succeeded.

Check: `ls` in `~/fedokawaii` lists `fedokawaii.sh`, `setup.sh`, `fedokawaii.conf` and the other files.

Route 2: straight from GitHub. git asks for your GitHub user name, and for a *personal access token* as the password.

> **Word — personal access token.** A long password GitHub makes for one purpose, which you can limit to one repository and to reading, and cancel any day. git takes it where it asks for a password.

Make the token on Windows (or any computer), signed in to github.com: your picture (top right) → *Settings* → *Developer settings* → *Personal access tokens* → *Fine-grained tokens* → *Generate new token*. Give it a name and a short *Expiration* (7 days is plenty). Under *Repository access*, choose *Only select repositories* and pick `MoeDia/fedokawaii`. Under *Permissions*, give *Contents* read-only access. Click *Generate token*, and copy it into a password manager.

> **Careful.** A token is a password. Never put it in a file, a note, a chat or this book's margins. Type it only at git's password prompt, and delete it on GitHub once the clone has worked.

Then, on the console:

```
$ sudo dnf install -y git-core
…
$ git clone https://github.com/MoeDia/fedokawaii ~/fedokawaii
Username for 'https://github.com': your GitHub user name
Password for 'https://…@github.com': the token (invisible as you type)
…
$ cd ~/fedokawaii
```

(shortened)

A fine-grained token is long — about ninety characters, beginning `github_pat_` — and the console cannot paste. That is why the stick is usually quicker.

5.2 Your settings: fedokawaii.conf

`fedokawaii.conf` holds your choices: keyboard, time zone, city, refresh rate and more. Set them **before** the first run: setup reads them as it builds. You edit the file with **nano**, a small text editor that runs in the console:

```
nano fedokawaii.conf      # in ~/fedokawaii
```

nano shows the file, with its keys listed along the bottom: `^O Write Out`, `^X Exit` and more (`^` means **Ctrl**). It needs four of them:

Key	Does
↑ ↓ ← →	move the cursor; type to insert, **Backspace** to delete
Ctrl+O, then **Enter**	save (*Write Out*), under the same name
Ctrl+X	leave; with unsaved changes it asks *Save modified buffer?* — **Y**, then **Enter**

nano lives only until setup starts: its first phase removes it, so the system keeps one editor, neovim.

Most values are right as they stand. These five are the ones people change:

Setting	Default	Change it when
`USERNAME`	empty	never, normally: empty means whoever runs `sudo` — you
`TIMEZONE`	`Asia/Dubai`	you live in another zone (`timedatectl list-timezones` lists the names)
`WEATHER_CITY`	`Sharjah`	the weather line in quick settings should show another city
`NIGHT_LIGHT`	`25.35:55.40`	you live elsewhere: your latitude:longitude, for sunset and sunrise
`KB_LAYOUTS`	`us,ara`	you want other keyboard layouts; the first is used for the shortcuts

For example, for a computer that will live in Dubai, two lines change. Before:

```
WEATHER_CITY="Sharjah"            # bar weather (wttr.in)
NIGHT_LIGHT="25.35:55.40"         # gammastep latitude:longitude
```

After:

```
WEATHER_CITY="Dubai"              # bar weather (wttr.in)
NIGHT_LIGHT="25.20:55.27"         # gammastep latitude:longitude
```

Keep the straight double quotes, and no spaces around `=`.

Every value is validated before anything runs; a wrong value stops the script with a message that names it. Set `GPU="nvidia"`, for example, and setup stops at once with:

```
08:52:07 ✘ fedokawaii.conf:
    GPU must be auto, amd or intel (is 'nvidia')
```

The messages say what to write instead. Type curly quotes — `“Dubai”`, as a word processor would — and the check says `WEATHER_CITY has curly quotes (“ ” ‘ ’) — type straight ones: WEATHER_CITY="Dubai"`.

The file is read as data, never run: only `KEY="value"` lines (single quotes, or none, also do) and `#` comments. Saved on Windows (CRLF line ends, a byte-order mark) it reads the same. A slip is named with the line to write instead — `GPU has spaces around the = — write it GPU="amd"` — and so is a setting written twice or missing. After the install, change it through *settings* in the **Super+U** menu (§35.1), which checks every value before it saves.

Every setting, for reference:

Setting	Default	Meaning
`USERNAME`	empty	the desktop user; empty = whoever ran `sudo`
`GPU`	`auto`	`auto`, `amd` or `intel` — drivers, video decoding, and mpv's scaling. Set it when the kernel does not drive the card yet (too new): setup installs its drivers. NVIDIA alone and virtual machines are not supported; the check says so before anything is installed

Setting	Default	Meaning
`STEAM`	`yes`	Steam, gamescope, gamemode, MangoHud, GE-Proton; switched to `no`, the next update hands those packages back to dnf, and its clean-up removes them (your games stay in your home)
`SSH_SERVER`	`no`	`no` disables sshd and closes ssh/cockpit in the firewall
`MONITOR_HZ`	`144`	the highest refresh rate at or below this is used (30–540)
`KB_LAYOUTS`	`us,ara`	keyboard layouts by their XKB names; the first one is always used for shortcuts. `localectl list-x11-keymap-layouts` lists the names
`KB_TOGGLE`	`grp:alt_shift_toggle`	Alt+Shift switches layout; the bar shows US / AR. `localectl list-x11-keymap-options` lists the others (the switches start with `grp:`)
`TIMEZONE`	`Asia/Dubai`	your time zone, as `timedatectl list-timezones` names it
`WEATHER_CITY`	`Sharjah`	the weather line in quick settings (wttr.in, no key)
`NIGHT_LIGHT`	`25.35:55.40`	latitude:longitude for gammastep, which works out sunset and sunrise from it (north and east are positive)
`THEME`	`dark`	the theme a new install starts in; afterwards it is whatever you last switched to
`WALLPAPER_PACK`	`yes`	fetch the wallpaper pack once (70 pictures, about 210 MB to download; kept redrawn as pixel art, about 4 MB)
`BOOT_MENU_SECONDS`	`2`	how long the themed boot menu shows (0–10); `0` = Fedora's hidden menu
`DNF_PARALLEL`	`10`	parallel downloads (1–20)
`DWL_TAG`, `DWL_COMMIT`, `DWL_WLROOTS`	`v0.9`, pinned, `0.20`	dwl is built from exactly this commit, against wlroots 0.20; should the tag point anywhere else, setup refuses to build
`TFC_TAG`, `TFC_COMMIT`	`v1.4.3`, pinned	the yazi file dialog's portal is built from exactly this commit

Read more: keyboard layout names and switches — localectl(1).

5.3 The menu and its five scripts

The project is a handful of scripts, each with one job, and a menu that runs them. Start the menu from the project folder:

```
$ sudo bash fedokawaii.sh

  F E D O K A W A I I
  setup: not started · theme: not applied

   1  Check    is this machine ready?              read-only
   2  Setup    install the software only
   3  Theme    configs, colours, keybindings only
   4  Both     setup, then the theme automatically
   5  Verify   prove everything is installed right read-only
   6  Update   the season's update, snapshot first
   7  Drift    configs that differ from the repo   read-only
   8  Status   setup progress                      read-only
   9  Repair   find what broke and fix it
   q  Quit

  choose:
```

`bash fedokawaii.sh` runs the script, and `sudo` runs it as root, as installing needs. Type a number and **Enter**. For a first install you need two: **1**, then **4**.

Try this. Choose **1 Check** — the readiness check. It only reads, and changes nothing:

```
── Fedokawaii · readiness check

fedokawaii.conf
  ✔ all values valid

platform
  ✔ x86_64 CPU (x86_64)
  ✔ Fedora 44 (supported)
  ✔ Fedora Server edition
…
✔ READY for setup — 41 checks passed, 1 warning(s)
```

(for example; shortened)

Mark	It means
`✔`	passed
`!`	worth knowing; does not block
`✘`	blocks setup — the line says why, often with the fix

The last line is the verdict. `NOT READY — fix the ✘ items above, then run the check again.` means exactly that.

Two words in the table below need a line each first:

> **Word — repository.** A store things are fetched from. A *package repository* is a server of packages, like a shop's catalogue: Fedora has its own, and setup adds a few (RPM Fusion, Brave's). The project's *git repository*, on GitHub, holds its files and their history.

The setup's work also reaches below the programs, into how the kernel starts:

> **Word — kernel argument.** An option handed to the kernel as it starts, like a switch on a program's shortcut. `cat /proc/cmdline` shows the current ones; setup adds one for AMD graphics.

File	Owns	Touches	Think of it as
`fedokawaii.sh`	the menu for the install	runs the others; nothing of its own	the front desk
`check.sh`	**readiness**	nothing — is this Fedora, online, on Btrfs, with room to work?	the pre-flight check
`setup.sh`	**software**	repositories, packages, drivers, kernel arguments, the dwl binary and its session launcher, services, the performance profile, snapshots — and what software needs before it can run: the LazyVim starter, the Prowlarr container file and key, torrra's settings and its key wrapper	the builder
`theme.sh`	**look and configuration**	every file it ships under `~/.config` and `~/.local` (`bin/kawa-*`, `share/fedokawaii`, `share/applications`, `share/themes`, fonts), `~/.bashrc.d`, `~/.inputrc`, the pixel art, `/etc/dconf` defaults, the boot menu theme and `/etc/default/grub`, the console font and colours, dwl rebuilt from `src/dwl/config.h`, two C helpers built from `src/`, Brave's colours (set once in its profile) and its launcher entries (so a screen share can carry sound, and Brave is listed once)	the decorator

File	Owns	Touches	Think of it as
`verify.sh`	**proof**	nothing of yours — checks against the manifest and the repository; it only wakes Prowlarr to ask it, and writes its result to `/var/lib/fedokawaii/last-verify`	the inspector
`system/fedokawaii`	**upkeep** — the system's own control, installed as the `fedokawaii` command (Super+U)	nothing itself: it reads the records and runs the scripts above through `sudo` (§35.1)	the caretaker

`theme.sh` never installs a package, and `setup.sh` writes no desktop look of its own. The menu offers them separately — **Setup** alone gives you dwl with upstream defaults, **Theme** can be run (and re-run) whenever you like, and **Both** runs them in order. To install, the only file you edit is `fedokawaii.conf`; everything else can be changed afterwards, the chapter 42 way.

Each menu entry is also an argument of the same script, for running one part directly:

```
sudo bash fedokawaii.sh check    # the readiness check alone; changes nothing
sudo bash fedokawaii.sh setup    # Setup: the software only, dwl with upstream defaults
sudo bash fedokawaii.sh theme    # Theme: the look and configuration only (after setup)
sudo bash fedokawaii.sh all      # Both: setup, with the theme applied near the end
```

5.4 What setup does, phase by phase

Choose **4 Both** for a first install. Setup walks through three phases and reboots twice between them. After each reboot it takes the first text console, **tty1** (the one on **Ctrl+Alt+F1**), by itself and carries on, so you can walk away:

```
you     menu → 4 Both
          │
phase 1   check · record · DNF · boot menu · full update · base · tuning · trim · SSH
          │  "rebooting in 10 s → phase 2 continues on this screen"
restart   ─── the first console shows setup again, with no login prompt ───
phase 2   repositories · graphics drivers · this machine's firmware · ffmpeg and media
          │  "rebooting in 10 s → phase 3 continues on this screen"
restart   ───
phase 3   build tools · dwl · login · desktop · fonts · apps · … · theme · verify gate
          │  "Fedokawaii is built — rebooting into the login screen in 15 s"
restart   ───
login     the login screen: user name, password
```

The login screen is where Part II begins (ch. 6).

On screen, each step starts with a line such as `── 1.4 full system update` and ends with `✔ 1.4 full system update`, after the time; the commands it runs scroll by in between. After a restart the first line is the header again, with the new phase: `── Fedokawaii setup · Fedora 44 · phase 2 · you · with theme`.

How long it takes depends mostly on your connection: every phase downloads before it builds, and phase 3 has the most to do. You need not watch: it carries on by itself.

It stops with the failing line and the log (`/var/log/fedokawaii.log`) if anything goes wrong; fix it and run it again — finished steps are recorded in `/var/lib/fedokawaii` and skipped (§5.6). When it is done the login screen comes up, and from then on the system looks after itself through one menu: **Super+U** (§35.1).

The table below names the steps; a few of their words first:

Term	Plain meaning
RPM Fusion	a community repository for software Fedora does not ship itself, such as the full ffmpeg and the *freeworld* video drivers
COPR	Fedora's build service, where anyone can publish a small repository; LACT and lazygit come from two (`dnf copr list` shows them)
pinned commit	one exact version of dwl's source, named by its commit hash; setup refuses to build if the tag `v0.9` points anywhere else
patch	a file of line changes applied to dwl's source before it is built (`src/dwl/dwl.patch`): keys, gaps, frames, centring, menus that close on a click outside, the layout for the bar — and a monitor plugged in later, or back from sleep, set to its best refresh rate and given its wallpaper again (dwl runs `kawa-display --hotplug` for it)
greetd	the login manager: it holds tty1, asks for your user name and password through its plain prompt, agreety, then starts the desktop
verify gate	`verify.sh`, run before the login is switched on; any failure stops setup there — `fedokawaii repair` then puts most of it right, and setup, run again, carries on

The phases, step by step:

Phase	Steps	Why this order
1	readiness check → record what is installed → DNF tuning → boot-menu and two-kernel policy → full update → base tools → the performance profile → remove nano, vim, rescue kernel → SSH policy → **reboot**	the record lets `verify.sh` spot anything extra later; tuning first so the big update downloads ten at a time; update before any third-party repository so RPM Fusion matches the base libraries; SSH closed before the first reboot, so a run that stops early never leaves it open
2	RPM Fusion, Brave, Claude Code (key fingerprint checked), COPRs → GPU drivers → this machine's drivers and firmware → full ffmpeg and media tools → **reboot**	drivers before the programs that use them, so nothing pulls the wrong variant; the reboot activates the GPU kernel argument
3	build tools → dwl (pinned commit + patch) → greetd login → desktop → fonts and toolkit themes → apps → release binaries → the yazi file dialog → Prowlarr + torrra → LazyVim → Steam → groups → theme (if chosen) → **verify gate** → enable the login, snapshots, baseline	the login is enabled last, only after verification, so a failure never leaves you at a black screen

What takes tty1 is `fedokawaii-resume.service`, a *systemd unit*: a small file that tells systemd what to start at boot. While setup holds tty1 there is no login there. To see how far it has got, log in on a second console (**Ctrl**+**Alt**+**F2**) as yourself:

```
sudo bash ~/fedokawaii/fedokawaii.sh status  # the phase, and every finished step
sudo tail -f /var/log/fedokawaii.log         # the log as it grows; Ctrl+C stops watching
```

Halfway through, *status* prints (with your user name in place of *you*):

```
phase 2 · user you · Fedora 44 · theme with setup
  ✔ 1.0 readiness check
  ✔ 1.1 record user-installed packages
  ✔ 1.2 dnf5 tuning
  ✔ 1.3 boot menu
  ✔ 1.4 full system update
  ✔ 1.5 base packages
  ✔ 1.5b performance: tuned profile, Wi-Fi
  ✔ 1.6 remove nano, vim, rescue kernel
  ✔ 1.7 inbound SSH (no)
```

Ctrl+**Alt**+**F1** goes back to setup's screen.

5.5 After setup: the toolkit

At its first reboot, setup copies the project folder for good to `/usr/local/lib/fedokawaii`, owned by root. From then on everything runs from there.

> **Word — toolkit.** The installed copy of the project, `/usr/local/lib/fedokawaii`, which root runs for every update and repair. Nothing running as you can change it — so nothing can slip a change into what root runs.

What that means for you now:

	Until the first reboot	From then on
setup runs from	the folder you unpacked (`~/fedokawaii`)	`/usr/local/lib/fedokawaii`, owned by root
the `fedokawaii` command	not there yet	`/usr/local/bin/fedokawaii`, installed from that copy
`~/fedokawaii`	needed	can be deleted once setup is done

Where this book says *the repository* after the install, it means that copy. You never edit it directly: to change the desktop for good you change your own *working copy* of it and *apply* that (chapter 42) — so every update keeps your change.

5.6 If setup stops

Setup stops at the first thing it cannot do, and says so in red. Nothing is lost: every finished step is recorded, and the next run skips it.

What you see. A line beginning `✘`. When setup itself spots the problem, the line says what to do:

```
09:14:02 ✘ no network — connect ethernet (nmtui) and run again
```

When a command inside a step fails, setup names the line of the script and the command, then points to the log:

```
✘ line 360 in s_update(), called from line 1809: run dnf -y upgrade --refresh (exit 1)
  log: /var/log/fedokawaii.log — fix the cause, then run the same command again (finished
steps are skipped)
```

(for example; the second line is one line on the screen)

Read the log. The lines above the red one usually say why. The log keeps them all:

```
sudo less +G /var/log/fedokawaii.log    # opens at its end; ↑ ↓ to scroll, q quits
```

The two usual causes.

Cause	What it looks like	Fix
the network	`no network`, `unreachable: …`, or a download that fails	check the cable (`ip -br addr`, §4.3), wait a minute, run again
a slip in `fedokawaii.conf`	`✘ fedokawaii.conf:` and a line naming the value (§5.2)	`nano ~/fedokawaii/fedokawaii.conf`, correct it, run again — before phase 1 removes nano

Run it again. Before the first restart: `cd ~/fedokawaii`, `sudo bash fedokawaii.sh` and the **same** entry as before — **4 Both** (choosing *Setup* would drop the theme from the run). Setup prints `skip (done): …` for every finished step and carries on at the one that stopped. **Ctrl+C** stops a run on purpose; it ends with `✘ interrupted — run the same command again to continue`.

After a restart, on tty1. A run that resumed by itself and then stopped ends with these lines, below the reason:

```
Setup stopped — the reason is above. Once it is put right, carry on either way:
  · restart the computer: setup continues by itself
  · or press Alt+F2, log in, and run:  sudo bash /usr/local/lib/fedokawaii/setup.sh
```

There is no login prompt on tty1 itself: **Alt+F2** switches to the second console, where you log in. (On a text console, **Alt+F2** does what **Ctrl+Alt+F2** does.) If the build was already in place and only its last check failed, the lines say instead: *Restart the computer, log in, then run: fedokawaii repair*.

Part I, looking back.

Chapter	You did
1	saw what you are building
2	learnt the words: kernel, distribution, compositor, package
3	checked the PC, saved its files, made two sticks on Windows
4	installed Fedora Server on Btrfs, and typed your first commands
5	set your values, and let setup build the desktop

The login screen is waiting. Part II starts there.

What you learnt

- The project comes over on a stick (`mount`, `tar`, `umount`) or with `git` and a token.
- `fedokawaii.conf` holds your settings; `nano` edits it; every value is checked before use.
- The menu's **1 Check** reads only; **4 Both** builds the software and the look.
- Setup runs in three phases with two restarts, and resumes by itself on tty1.
- When it stops: read the red line and the log, fix, run the same entry again.

Check yourself

1. Which menu entry do you choose for a first install? → *4 Both*
2. Where is setup's log? → `/var/log/fedokawaii.log` — *read with* `sudo less +G`
3. Setup stopped after a restart and tty1 has no login. What now? → *Alt+F2, log in, run* `sudo bash /usr/local/lib/fedokawaii/setup.sh` — *or restart*

Part II — First days on the desktop

Fedora is installed, the setup has built the desktop, and you are at the new machine. Part II is about what you see and touch: the screen, windows, menus, files, the terminal's first steps, copying and drives. There is no theory yet, and every chapter ends with a few minutes of practice. Start with ch. 6, and give ch. 11, the terminal, an unhurried hour of its own.

6. Your first login, and the screen

> **In this chapter**
> - log in and reach the desktop
> - name every part of the screen, and start programs with six keys
> - lock the screen, log out, restart and shut down
>
> **You need:** ch. 5 (the setup has finished and restarted the machine).

These are your first ten minutes on the desktop: the login, a look round the screen, the keys that start things, and the right ways to leave.

6.1 Logging in

After the restart the screen is text only: no pictures, no list of users. You type two lines:

```
fedora login: you
Password:
```

You typed	It means
`you` after `fedora login:`	your user name; `fedora` is the machine's name, its *host name* — yours shows the name you gave the installer, or `fedokawaii`
your password after `Password:`	nothing shows while you type, not even dots — that is normal; **Enter** when done

A few seconds later the desktop appears. A wrong name or password prints `Login incorrect`, and the prompt asks again.

The **login** is greetd's plain text prompt (agreety) on the console: user name, password, nothing else. Above it, it prints a few lines about Fedora and its kernel (the text of `/etc/issue`). Should the desktop's launcher ever be missing, it says so above the prompt, and logging in gives a text console instead: run `fedokawaii repair` there.

> **On Windows.** There is no picture to click away and no list of accounts: you type the user name as well. The password is the one you chose in the installer (§4.2), not a PIN or a Microsoft account's.

How it works: the console's look. That console (tty1) always uses the dark theme's 16 colours (set as the kernel parameters `vt.default_red`, `vt.default_grn` and `vt.default_blu`, sixteen numbers each) and Terminus bold — 12×24 below 1440p, 16×32 from 1440p up. On Fedora 44 the spare consoles (**Ctrl+Alt+F2…F6**) look the same; Fedora 45 runs kmscon there — a console that runs as an ordinary program instead of inside the kernel — which keeps its own colours and font (`/etc/kmscon/kmscon.conf`). To see the palette the kernel was given (the first number is the red of the ink ground, `0f` in hex):

```
grep -o 'vt.default_red=[0-9,]*' /proc/cmdline    # → vt.default_red=15,… (15 = 0x0f)
```

How the machine gets from the power button to this prompt is ch. 29; the colours themselves are ch. 40.

Read more: Fedora's change to kmscon.

6.2 The screen at a glance

The two pictures at the front of the book show the whole screen, dark and light. Three things are on it:

1. **the bar**, a thin strip across the top: workspaces on the left, the song playing in the middle, the clock and the settings on the right (§6.5 walks along it);

2. **windows**, which share the space below the bar without overlapping (ch. 7);
3. **the wallpaper**, behind them, seen wherever no window is.

What is *not* there matters as much. There are no icons on the desktop, no taskbar of open programs, and no buttons on the windows to close or shrink them. You start a program by its name (**Super**+**Space**), move between windows with keys, and close one with **Super**+**Q**.

On Windows	Here
the taskbar and the system tray	the bar; the tray's icons sit near its right end
the Start menu: press Windows, type a name	the launcher: **Super+Space**, type a name
Task View's virtual desktops (**Win+Tab**)	workspaces 1–9, **Super+1…9**
Quick Settings (**Win+A**)	quick settings, **Super+X**
the × in a window's corner	**Super+Q**
the Windows key	**Super** — the same key

6.3 Six keys to start with

A first tour. Super is the Windows key. Six keys carry you a long way:

Press	You get
Super+Space	the launcher: type a few letters of a program's name, then **Enter**
Super+Return	a terminal
Super+B · **Super+E**	the browser (Brave Origin) · the files (yazi)
Super+X, or the ⌄ key at the right end of the bar	quick settings — the theme switch is the first line
Super+/	every key, searchable
Super+V	anything you copied, newest first

Try this. Press each key once, look at what opens, and close it again.

Press	You see	Close it with
Super+Space	a box listing programs; type `bra` and Brave is among the first	**Esc**
Super+Return	a terminal, its prompt waiting	**Super+Q**
Super+E	yazi, the files, in your home folder	**q**
Super+X	quick settings, dropping down from the right end of the bar	**Esc**, a click outside, or **Super+X** again
Super+/	every key in a list; type `lock` and the list narrows to the lines that match	**Esc**
Super+V	the clipboard history: whatever you have copied so far	**Esc**

Each window you open takes a share of the screen. Where each one goes is the next chapter.

6.4 Words for what you see

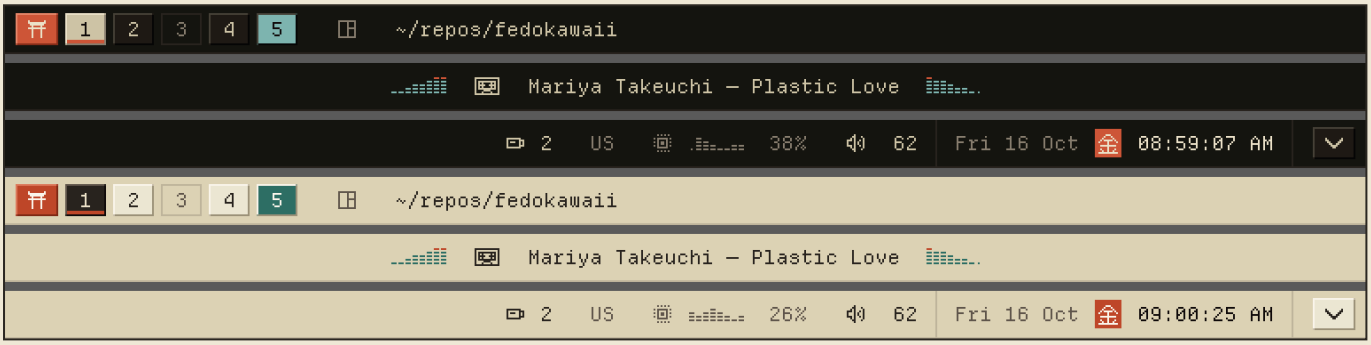
Five words come up on every page from here on.

Terms.

Term	Plain meaning
workspace	a separate set of windows, one set at a time on a screen; dwl calls them *tags*, and a window can be on more than one
tiled · floating · monocle	dwl's three layouts: windows side by side, never overlapping · windows placed freely, by you · one window at a time, at full size
main area	the large left part of a tiled screen; the other windows share the column on the right
launcher	the box that starts a program by its name (fuzzel)
focus	the window that receives what you type; its frame is vermilion

6.5 The bar

The bar runs across the top of every screen. Here it is at twice its size, in three pieces; the table below walks along it from left to right.



The bar, dark and light, at twice its size: the left end, the middle, the right end.

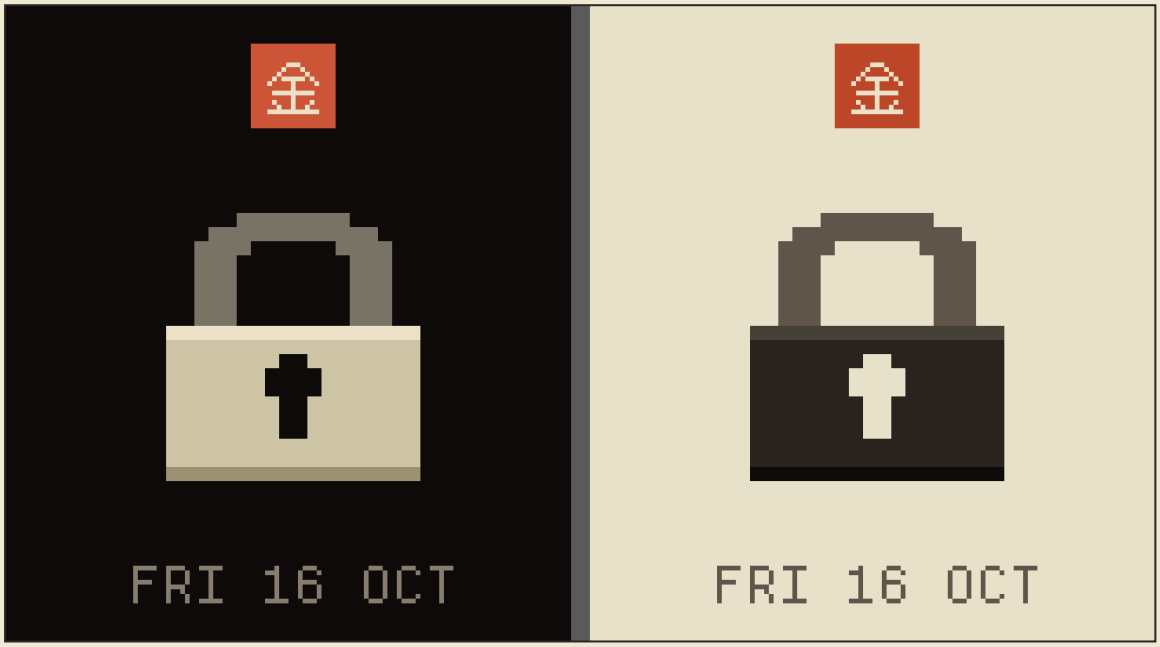
#	Part	What it shows	Click
1	**torii key**	the vermilion key at the far left	opens the launcher
2	**workspaces**	1, 2, 3 always; 4–9 only while they hold a window, are the one you are on or want attention. A raised key = windows on it; the lit key with a vermilion stamp = the one you are on; a teal key = something there wants attention	goes to it
3	**layout icon**	tiled, floating, or monocle (with the number of stacked windows)	
4	**title**	the focused window	
5	**now playing**	artist — title, with a cassette whose reels turn while sound plays, between the two halves of a spectrum meter drawn like an old hi-fi's fluorescent display: lowest notes nearest the title, a full column lights its top segment in vermilion	play/pause · right: next · middle: previous
6	**mode seal**	a small vermilion seal while something is on that you should not forget: **静** (*quiet*) while do-not-disturb holds notices back, **共有** (*sharing*) while the screen is being shared; nothing otherwise	quick settings
7	**tray**	the small icons some programs put there while they run (Steam, Solaar, OpenRGB); nothing otherwise	the program · right: its menu
8	**drive mark**	the drive icon while a USB stick, disk or SD card is plugged in: quiet, then lit once one of them is open; a number beside it for two or more	the drives menu: open or eject (ch. 13)
9	**US / AR**	the keyboard layout (vermilion while it is not the first)	
10	**CPU**	the chip icon, the last eight readings (one every two seconds, the newest on the right) in the same segment style, then the load now in per cent; vermilion above 80 %	opens btop
11	**volume**	speaker icon and level; vermilion when muted	the audio menu · right: mute · scroll: ±2 %

#	Part	What it shows	Click
12	**date, day, time**	`Wed 14 Oct` 水 `08:52:07 PM`: the date, the day of the week as a kanji seal (月 火 水 木 金 土 日, Monday to Sunday), the time to the second	the calendar
13	**⌄ key**	quick settings	

Try this. Click the clock (12): the calendar drops down; **Esc** closes it. Point at the volume (11) and turn the mouse wheel: the number moves 2 at a time. Click the ⌄ key (13): quick settings; **Esc** again.

Which players show in *now playing* is in ch. 15; why the bar costs almost nothing while it waits is in ch. 40.

6.6 Locking the screen



The middle of the lock screen on both themes, at twice its size: the day's kanji seal, the pixel padlock (the typing indicator appears in its keyhole) and the date.

The **lock screen** is the theme's plain ground with a pixel padlock in the exact centre — paper on ink, or ink on paper. Above it the day of the week sits in a small vermilion seal, in kanji (月 Monday, 火 Tuesday, 水 Wednesday, 木 Thursday, 金 Friday, 土 Saturday, 日 Sunday), and below it the date in the bar's pixel letters; both are drawn into the picture as it locks, in a few tenths of a second. Before sleep it locks with the bare padlock at once, so the lid can close.

swaylock's indicator is shrunk to a 15-pixel ring that sits inside the padlock's keyhole, clear of its face: it is invisible until you type, then the keyhole glows — vermilion while typing, teal while checking, red when wrong. It locks before sleep, with **Super+Escape**, and by itself when you are away:

Idle for	What happens
9 min 50 s	a quiet notice, *locking in 10 s* — touch a key and nothing more happens
10 min	the screen locks
15 min	the screens power off (windows stay where they were); the first key wakes them

The password goes in in the keyboard layout that was active (the bar showed it before the lock); if the keyhole turns red although the password is right, press Alt+Shift and type it again.

Try this. Press **Super+Escape**. Type your password — the keyhole glows as you type — and press **Enter**: the desktop is back exactly as it was.

6.7 Logging out, restarting and shutting down

Super+Shift+E opens the power menu: five choices in a small box in the middle of the screen, and above them how long the machine has been up (*up 03h 07m*). Type a few letters or use ↑ ↓, then **Enter**; **Esc** closes it.

Choice	Does	Your open windows
lock	the lock screen, as **Super+Escape**	stay as they are
sleep	locks, then suspends: the machine nearly off, back in seconds at a key	stay as they are
log out	ends the desktop and returns to the text login	close: every program ends
restart	starts the computer again	close
shut down	turns the computer off	close

> **Careful.** Log out, restart and shut down end every program at once, without asking to save. Save your work first: **Ctrl+S** in most programs, `:w` in neovim.

Super+Ctrl+Shift+Q logs out at once, without the menu. It is the one way out should a menu not open: there is no Ctrl+Alt+Backspace. A choice the system refuses — another user still logged in, a program holding it back — says why in a notice: *could not shut down — ….*

Why not the power button? A short press asks the system to shut down, the same clean way as the menu. Holding it down for several seconds is different: the computer cuts its own power at once, like pulling the plug, and whatever was still being written to the disk can be lost. Keep the long press for a machine that answers nothing else (ch. 39).

Try this. Log out and back in.

1. Press **Super+Shift+E**, type `out`, press **Enter**.
2. The screen becomes the text login of §6.1. Log in again.
3. The desktop returns as it starts every time: the bar, the wallpaper, no windows.

What you learnt

- The login is two lines: your user name, then a password that does not show as you type.
- The bar runs along the top; there are no desktop icons and no window buttons.
- **Super+Space** starts any program by name; **Super+/** lists every key; **Super+Q** closes.
- **Super+Escape** locks; the screen also locks itself after 10 minutes away.
- **Super+Shift+E** logs out, restarts or shuts down — save your work first.

Check yourself

1. Nothing appears while you type the password. Is something wrong? → *no: the password never shows, not even as dots*
2. Which key closes a window? → *Super+Q*
3. What happens to an unsaved letter when you choose *restart*? → *its program ends without asking: the changes are lost*

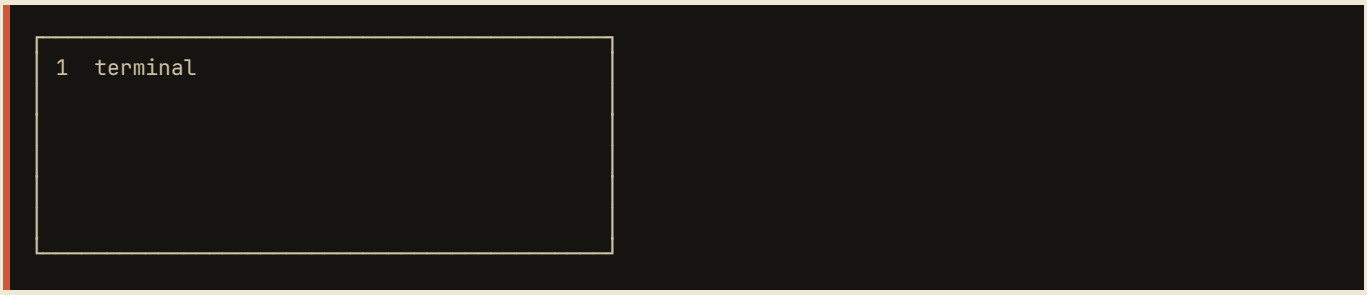
7. Windows, workspaces and keys

> **In this chapter**
> - predict where a new window will go, and rearrange windows with the keyboard
> - find any key, and know the dozen to learn first
> - keep different jobs on different workspaces
>
> **You need:** ch. 6, above all §6.4 (*tiled*, *main area*, *focus*).

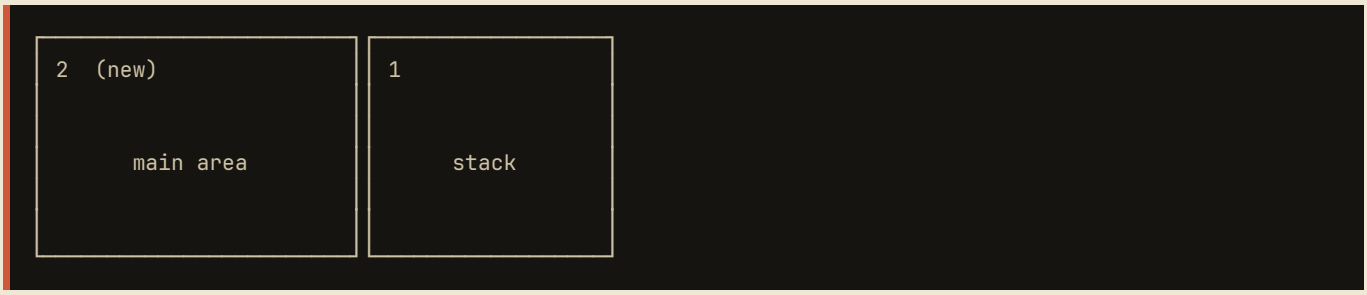
dwl, the part of the desktop that places windows, never lets them overlap unless you ask. That sounds strict, but it means no window is ever lost behind another. This chapter shows the pattern first, then the keys.

7.1 Tiling, in pictures

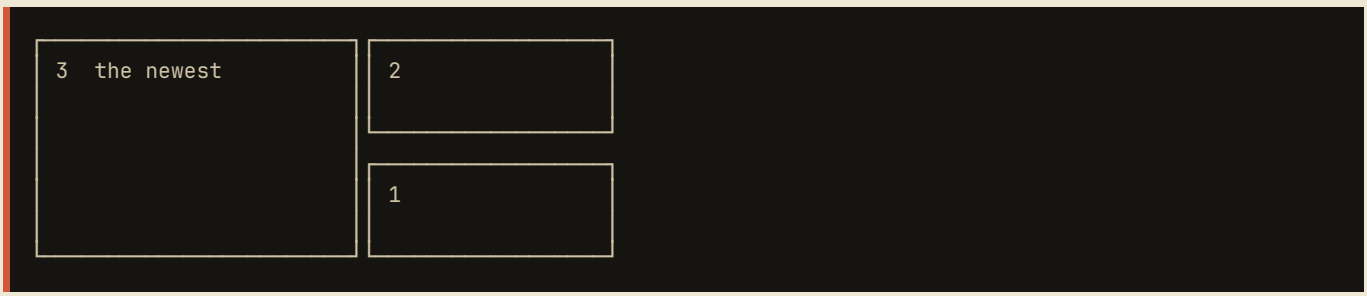
Smallest example first. **One window** takes the whole space below the bar:



Open a **second window** and the screen splits. The new window takes the main area on the left, 55 % of the width; the one before moves into the column on the right, the *stack*:

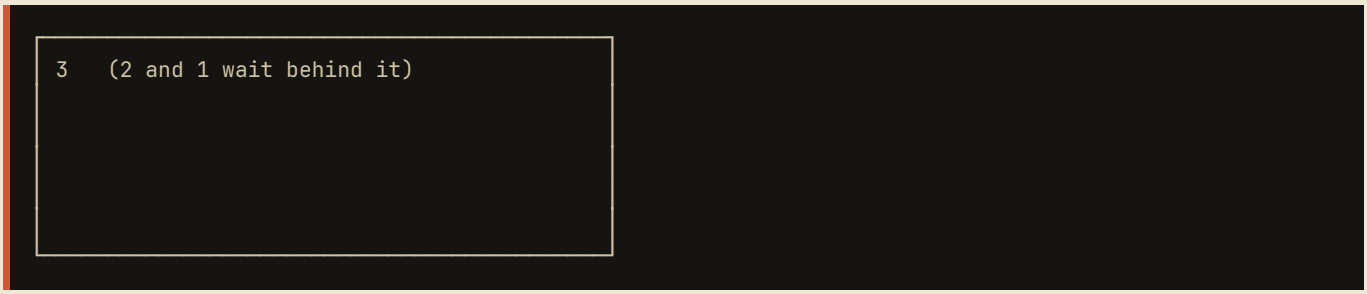


The rule: **the newest window always takes the main area, and the others move down the stack**. A **third window** shows it again — the stack splits its column evenly, newest at the top:

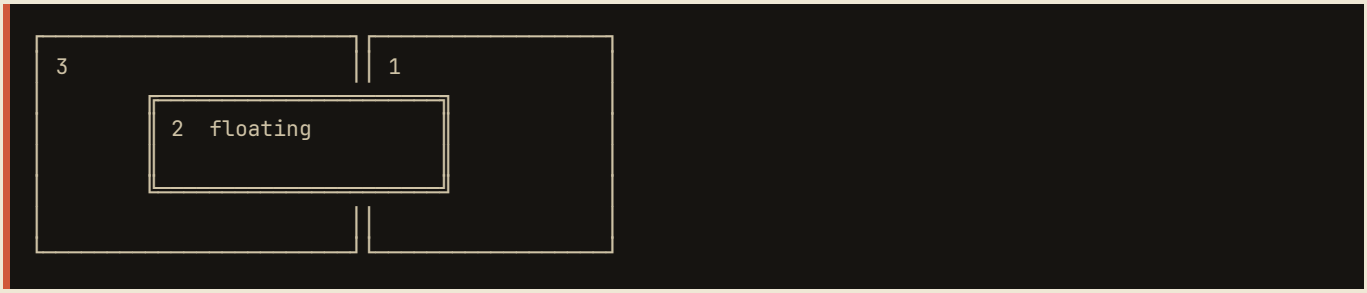


Between and around the windows runs an 8-pixel gap, and each window wears a 2-pixel frame (`gappx` and `borderpx` in `src/dwl/config.h`). The focus follows the mouse: point at a window and it receives your typing.

Monocle (**Super**+**M**) shows one window at a time, filling the whole space under the bar; the others wait behind it, and the bar's layout icon shows how many — `[3]` here. **Super**+**J** and **Super**+**K** bring each to the front in turn:



A **floating** window (**Super**+**Shift**+**Space**) leaves the tiling and lies on top; the others close the gap beneath it. Super+drag moves it, Super+right-drag resizes it. Small tools — the menus' windows, the picture viewer, the file dialog — open floating and centred by themselves:



> **On Windows.** Snap (**Win**+←, **Win**+→) puts two windows side by side when you ask. Here that happens to every window, always, and the main area is the left half you would snap to. Maximising is **Super**+**M** (monocle) or **Super**+**F** (fullscreen, over the bar too).

Try this. Watch the pattern happen. Start on an empty workspace.

Press	What happens
Super+Return three times	three terminals, as in the third picture: the last one in the main area
Super+J, again, again	the vermilion frame moves down the stack, then back to the main area
Super+L, then **Super+H**	the main area grows by 5 % of the width, then shrinks back (it stays between 10 % and 90 %)
Super+K (to terminal 1), then **Super+Shift+Return**	terminal 1 swaps into the main area; on the main window, the same key swaps the top of the stack in
Super+I, then **Super+D**	two windows share the main area, one above the other; then one again
Super+M, **Super+J**, **Super+T**	monocle — one window, the bar shows `[3]` — the next one, then tiled again
Super+Shift+Space, then again	one terminal floats on top, then rejoins the tiling
Super+Q three times	each closes; the others grow to fill the space

7.2 Keybindings

Super is the Windows key. Shortcuts always read the US layout, so they work the same while Arabic is active. The keys in **bold** are the dozen to learn first; the rest can wait until you miss them. **Super**+/ shows this whole list on screen, searchable, and appendix C has a two-page selection to keep beside the keyboard.

Open and close

Keys	Does
Super+Return · Space · B · E	terminal · launcher · Brave Origin · files (yazi)
Super+X · V · /	quick settings · clipboard history · help
Super+A · C · W	audio · calendar · wallpaper
Super+R · Super+N	books (§16.2) · notes, and again to put them away (§16.3)
Super+Q	close the window

Move the focus

Keys	Does
Super+J / K	focus next/previous (or point at a window: the focus follows the mouse)
Super+, / . · Shift+, / .	focus · send to the monitor on the left / right

Arrange

Keys	Does
Super+H / L · I / D	shrink/grow the main area · more/fewer windows in it
Super+Shift+Return · F · Shift+Space	move to the main area · fullscreen · float/unfloat (also Super+middle-click)
Super+drag / right-drag	move / resize
Super+T / Y / M · Shift+T	tiled · floating · monocle · previous layout

Workspaces

Keys	Does
Super+1…9 · Shift+1…9	go to a workspace · send the window to it
Super+Ctrl+1…9 · Ctrl+Shift+1…9	also show a workspace · also show the window there
Super+Tab · Super+0 · Shift+0	previous workspace · all of them · the window on every one

System

Keys	Does
Super+Escape · Super+Shift+E	lock · power menu
Super+Shift+W	**dark ⇄ light theme**
Super+U	the system: update, a new Fedora, repair (§35.1)
Super+Shift+U	change the desktop: keys, bar, colours, … (chapter 42)
Super+Shift+N	dismiss notifications
Print · Shift+Print · Super+Print	screenshot a region · the whole screen · a region, then annotate it (Enter copies)
media keys	volume, mute, microphone mute — the level display shows for a moment, also in fullscreen · play/pause, next, previous
Super+scroll	volume ±2 %, with the level shown like the volume keys (up also unmutes)
Super+] / [· brightness keys	monitor brightness (the same level display)
Alt+Shift	keyboard layout
Right Shift+F12	the game overlay (MangoHud) on / off, in games started with it
Super+Ctrl+Shift+Q · Ctrl+Alt+F2…F6	log out · text consoles (Ctrl+Alt+F1 comes back)

Windows carry no title-bar buttons anywhere — not in GTK apps, not in Brave: dwl tiles them, Super+Q closes them.

> **On Windows.** Some habits carry over: **Win+V** is the clipboard history there too, and here it is **Super+V**. Others change meaning: **Win+1…9** starts taskbar programs, **Super+1…9** goes to a workspace; **Alt+F4** closes a window, **Super+Q** does here.

7.3 Workspaces

A **workspace** is a separate screenful of windows. There are nine, numbered 1–9, and each monitor shows one at a time; dwl calls them *tags*. Press **Super+2** and the screen shows workspace 2: empty at first, with the windows of workspace 1 waiting, untouched, until you come back with **Super+1**.

The rule: **Super+number goes to a workspace; Super+Shift+number sends the focused window there.** A few more keys round it off:

Keys	Does
Super+Tab	back to the workspace you were on before — press again to return
Super+0	every workspace's windows at once, tiled together; **Super+1** ends it
Super+Ctrl+3	also show workspace 3's windows here, for now; again to hide them
Super+Ctrl+Shift+3	put the focused window on workspace 3 *as well* — one window can live on several
Super+Shift+0	the focused window on every workspace (a music player you always want in view)

The bar shows 1, 2 and 3 always, and 4–9 only while they hold a window, are the one you are on, or want attention: a raised key holds windows, the lit key with a vermilion stamp is yours, a teal key wants attention (§6.5).

A layout to copy. Give each kind of work its own number, the same every day, and your hands learn where things are: **1** the browser · **2** terminals · **3** files · **4** reading · **9** Steam. The Steam client opens on 9 by itself (a rule in `src/dwl/config.h`); games open where you are.

> **On Windows.** Task View's virtual desktops are made and closed by hand (**Win+Ctrl+D**) and visited in a row (**Win+Ctrl+←/→**). Here nine always exist, each one key away, and a window can be on two at once.

Try this. Spread three windows over three workspaces.

Press	You see
Super+2, then **Super+Return**	a terminal on workspace 2; in the bar, 2 is lit
Super+3, then **Super+E**	yazi on workspace 3
Super+Tab, **Super+Tab**	2, then 3 again: back and forth between the last two
Super+1, then **Super+B**	Brave on 1; in the bar, 2 and 3 are raised — they hold windows
Super+0	all three windows tiled together; **Super+1** returns
on 1: **Super+Shift+3**	Brave moves to 3, beside yazi; workspace 1 is empty

7.4 Frames: which window has the focus

Everything that floats has the same plain, full border, 2 pixels wide. Windows get theirs from dwl (compiled in, so it suits both themes): vermilion around the focused window, warm grey around the others, teal around one that asks for attention. Menus, notices and the level display are solid cards (nothing of the wallpaper shows through) with a warm-grey border of their own theme; a critical notice's border is vermilion.

Try this. With two terminals open, move the mouse slowly from one to the other: the vermilion frame follows the pointer, and so does your typing.

7.5 Recipe: plug in a second monitor

1. Plug it in. dwl places it beside the first, and the bar appears on it, showing that screen's own workspaces, layout and window title.
2. Within a second or two it moves to the best refresh rate up to `MONITOR_HZ`, the wallpaper is drawn again at its exact size, and the brightness keys reach it too. dwl runs `kawa-display --hotplug` for every monitor that comes, also one that returns after sleeping.
3. **Super+,** and **Super+.** move the focus between screens; **Super+Shift+,** and **Super+Shift+.** send a window across.
4. To put it on the other side: `wlr-randr --output NAME --pos X,Y` (names from `wlr-randr`). This lasts until you log out.

If the new screen is now the one the machine starts on, apply the theme again (**Super+U** → *theme, applied again*) so the boot menu matches its resolution (§29.3).

What you learnt

- A new window takes the main area — the left 55 % — and the others move down the stack.
- **Super+J/K** move the focus; **Super+Shift+Return** swaps a window into the main area.
- **Super+M** shows one window at a time, **Super+T** tiles again; **Super+Shift+Space** floats one.
- Nine workspaces: **Super+1…9** to go, **Super+Shift+1…9** to send a window, **Super+Tab** to go back.
- **Super+/** lists every key; the vermilion frame marks the window you type into.

Check yourself

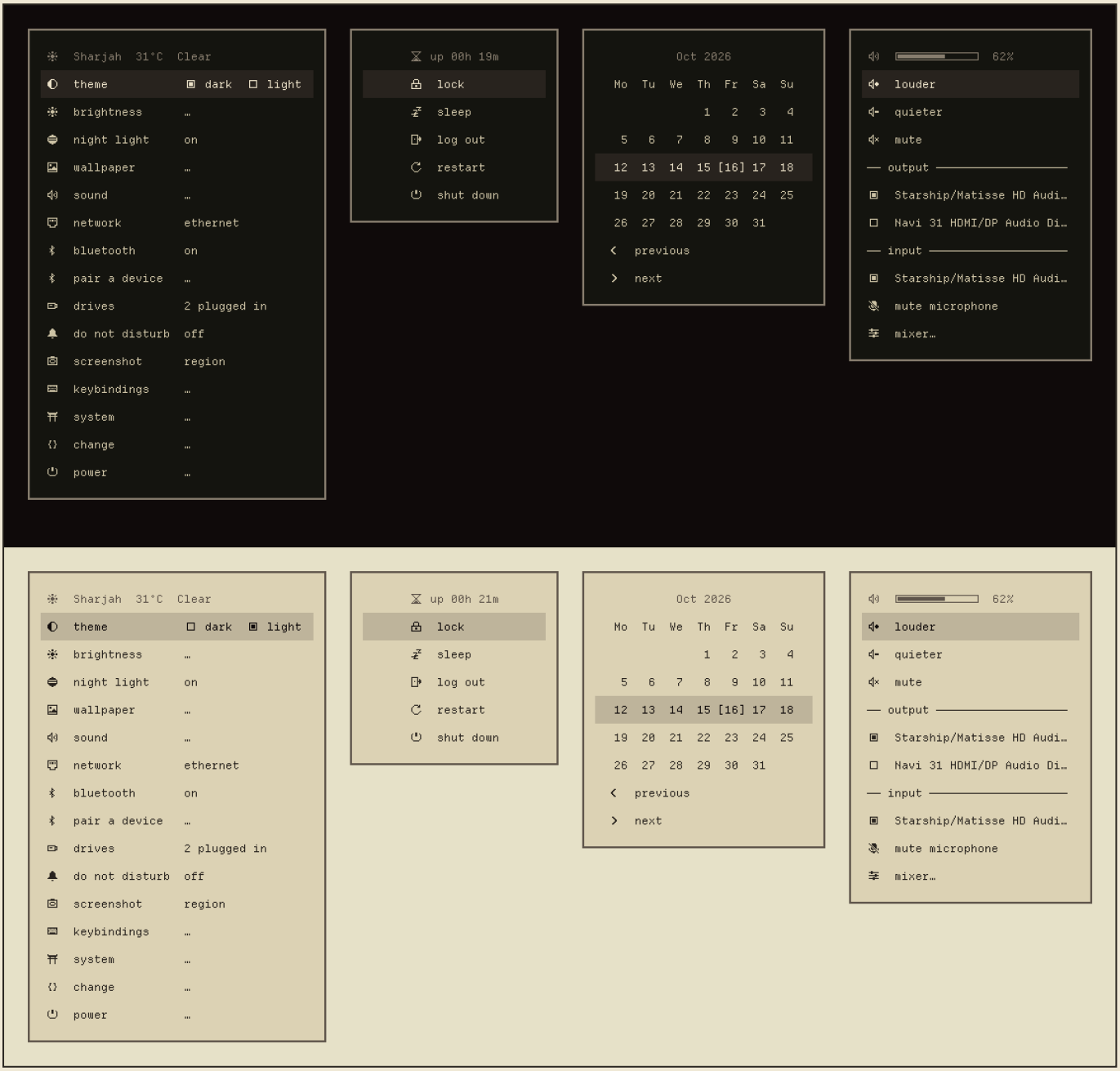
1. Two terminals are open and you start Brave. Where does it go? → *into the main area on the left; the terminals share the right column*
2. How do you put the focused terminal on workspace 4? → *Super+Shift+4*
3. Which key shows one window at a time, full size? → *Super+M (monocle); Super+T tiles again*

8. Menus, settings and notices

> **In this chapter**
> - open every menu, change a setting and close it again
> - join a Wi-Fi network and check you are online
> - pair Bluetooth headphones and send sound to them
> - read, act on and silence notices
>
> **You need:** ch. 7.

What you would look for in Windows' Settings app lives here in a few small menus, each one key or one click on the bar away. They all work the same way, so learning one teaches the rest.

8.1 Menus



Quick settings, the power menu, the calendar and the audio menu, on both themes.

Every menu is the same fuzzel window, so they look and behave alike: type to filter, arrows or mouse to choose. **Esc** closes a menu, and so does a click outside it, or its own key (or its button in the bar) pressed again. Only one menu is open at a time: another menu's key closes the one on screen and opens its own.

Menu	Open with	Inside
quick settings	⌄ key, **Super+X**	**theme** first, as a two-way switch (■ dark □ light) · brightness · night light · wallpaper · sound · network · Bluetooth on/off and pair a device (only when the PC has a Bluetooth adapter) · drives · do not disturb · screenshot · keybindings · **system** (the control, §35.1) · **change** (chapter 42) · power; the weather on top
power	**Super+Shift+E**	lock · sleep · log out · restart · shut down — centred, with your uptime; when the system says no (another user logged in, something holding it off), a notice says why
audio	volume, **Super+A**	the level as a framed bar at the top · louder / quieter / mute (stay open for repeated taps), every output and input (not *Share*, which is picked per program, §14.4) — picking one moves playing sound to it — microphone mute, the full mixer
calendar	the clock, **Super+C**	this month (Monday first, today in brackets), previous / next
brightness	quick settings	the monitor's real backlight over DDC/CI, like Twinkle Tray on Windows — and a laptop's own screen through its backlight — each level drawn as a small framed bar; **Super+]** / **Super+[** (and a laptop's brightness keys) step by 10 %
share what?	a screen share	the screens and windows you can share (§14.4)
drives	the bar's drive mark, quick settings	every file system on a USB stick, disk or SD card — open it in yazi — and **eject** for each drive (ch. 13)
help	**Super+/**	every keybinding, searchable
change	**Super+Shift+U**, quick settings	every part of the desktop by name — keys, bar, bar look, colours, terminal, shell, launcher, notifications, files, drives, autostart, icons, packages, settings — opened in your working copy, then applied; the files that are yours alone; what changed; its history; where to learn each (chapter 42)

Try this. Open each menu once, change one thing, and close it.

Menu	Do this	You see
quick settings, **Super+X**	choose *do not disturb*	the menu opens again, the line now says *on*, and **靜** stands in the bar; choose it again to turn it off
audio, **Super+A**	choose *quieter* twice, then *louder* twice	the bar in the top line shrinks, then grows, 5 % a step; the menu stays open until **Esc**
calendar, **Super+C**	choose *next*, then *back to today*	next month, then this one, today in brackets
power, **Super+Shift+E**	read the top line, then **Esc**	how long the machine has been up: *up 00h 25m*
help, **Super+/**	type `print`	the list narrows to the lines that match: the screenshot keys among them
brightness, from quick settings	choose a level	the monitor itself grows brighter or dimmer (if it answers DDC/CI, below)

> **On Windows.** Quick settings is **Win+A**'s panel, with more in it: the theme, the wallpaper, the drives and the power menu sit beside Wi-Fi, Bluetooth and do not disturb. There is no separate Settings app to look for.

The windows that show more than a list are small terminal windows of their own rather than menus, all in one look — the name in a rule across the top, a search line where there is something to search, the list, and a pane beside or below it: the wallpaper picker (**Super+W**, the wallpaper itself shown as you move — §9.2), the clipboard history (**Super+V** — §12.1), the books (**Super+R** — §16.2) and the system's control (**Super+U** — §35.1). Each opens once: a second press while it is open does nothing.

The volume, microphone and brightness keys show a **level display** for a moment at the bottom centre — also over fullscreen windows and while *do not disturb* is on. It is drawn like the volume scale of an old hi-fi: a wedge

of twenty pixel columns rising left to right, lit in teal with the topmost lit one vermilion (the peak lamp), the number beside it at twice the pixel size; it reads *off* when muted, and holding a key updates it in place. Above 100 % — only the mixer goes there — the wedge stays full and the number shows the level; the volume key never pulls it back down.



The level display, a notice from an app and a critical notice, dark and light.

Aside: brightness over the cable. *DDC/CI* (Display Data Channel Command Interface) is the standard way a computer changes a monitor's own settings — brightness, contrast — over the video cable. Most monitors have it, some with it switched off in their own on-screen menu. `ddcutil`, which the brightness menu uses, shows whether yours answers:

```
ddcutil detect --terse    # one "Display 1" block for each monitor that answers
ddcutil getvcp 10         # feature 10 is brightness: its current and maximum value
```

Read more: ddcutil.com.

8.2 Getting online: cable and Wi-Fi

A **cable** needs nothing: plugged in, it is up. Quick settings' *network* line says what you are on — *ethernet*, *wifi* or *offline*.

Wi-Fi, the first time, takes five steps:

1. **Super+X**, choose *network*. A small window opens with **nmtui**, NetworkManager's menu in the terminal: ↑ ↓ move, **Enter** chooses, **Tab** reaches the buttons, **Esc** goes back.
2. Choose *Activate a connection*.
3. The Wi-Fi networks nearby are listed. Move to yours and press **Enter** (the button beside the list says *Activate*).
4. A box asks for the network's password. Type it, then **Enter**.
5. A `*` before the name now marks the network you are on. Press **Esc** until nmtui closes.

The machine remembers the network and joins it by itself from then on.

The same from a terminal, in two commands — you met the terminal at the console (§4.3), and ch. 11 teaches it properly:

```
nmcli device wifi list                      # the networks nearby, with their signal
nmcli --ask device wifi connect "Home Net"  # join one: it asks for the password
```

`--ask` makes nmcli ask for the password instead of taking it on the line, where it would stay in the terminal's history.

Try this. Check that you are online. **Super**+**Return** opens a terminal; type:

```
$ ping -c 3 fedoraproject.org
PING fedoraproject.org (…) 56(84) bytes of data.
64 bytes from …: icmp_seq=1 ttl=50 time=182 ms
64 bytes from …: icmp_seq=2 ttl=50 time=181 ms
64 bytes from …: icmp_seq=3 ttl=50 time=183 ms

--- fedoraproject.org ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
```

(for example; the addresses left out, and the times vary)

You typed	It means
`ping`	send a tiny message to another machine and time its answer
`-c 3`	three times, then stop — without it, ping goes on until **Ctrl+C**
`fedoraproject.org`	the machine to ask, by its name

3 received is the line that matters: you are online.

If you see `Name or service not known`*:* the name could not be looked up — the machine is not online yet, or has no DNS. Ch. 33 explains the parts, and its checklist (§33.4) finds which one is missing.

8.3 Bluetooth: pairing headphones

Quick settings shows two Bluetooth lines, but only when the computer has a Bluetooth adapter: *bluetooth*, with *on* or *off*, and *pair a device*.

1. **Super**+**X**: if *bluetooth* says *off*, choose it. The menu opens again, now saying *on*.
2. Put the headphones into pairing mode — on most, hold the power button until a light flashes (their manual says how).
3. **Super**+**X** → *pair a device*. Blueman's window, *Bluetooth Devices*, opens floating.
4. Press **Search** in its toolbar. Devices nearby appear in the list.
5. Right-click the headphones and choose *Pair*; when that is done, right-click again and choose *Connect*. *Trust*, in the same menu, lets them connect later without asking.
6. **Super**+**A**: the headphones are a new line under *output*. Choose it. A notice says … *is now the output*, and whatever is playing moves to them.

Next time, switch the headphones on: once paired, most connect by themselves. If not, *pair a device* → right-click → *Connect* again.

When quick settings cannot switch Bluetooth on, a notice says why:

The notice	It means	Do
it is blocked — rfkill unblock bluetooth (or the machine's own switch)	a key or a switch on the machine turned Bluetooth off (a laptop's aeroplane-mode key, say)	turn it back on there, or type `rfkill unblock bluetooth` in a terminal
it did not switch on — is the bluetooth service running?	the Bluetooth service is not answering	`systemctl status bluetooth` shows whether it runs (ch. 28)

How the parts fit — BlueZ underneath, PipeWire for the sound — is §30.3.

8.4 Notices and do not disturb

A **notice** is a small card at the top right, below the bar: a title, a line or two of text, sometimes a picture. The picture of the level display in §8.1 shows two of them — an ordinary one, and a critical one with its vermilion border.

A notice that is	Shows for
ordinary	5 seconds
quiet (a program marks it *low* urgency)	3 seconds
the desktop's own (*Screenshot copied*, *… is now the output*)	2.5 seconds
a drive's (*click to open it in the files*)	8 seconds
critical — the vermilion border	until you click it

A notice can also set its own time: *locking in 10 s* stays 9.5 seconds, nearly to the lock.

A **left click** closes a notice — except a drive's, which opens its files (ch. 13). A **right click** closes every notice on screen, and so does **Super**+**Shift**+**N**. `makoctl restore` brings back the last one that faded by itself.

Do not disturb (quick settings → *do not disturb*) stops notices from showing, and the bar wears the seal **静** while it is on. Two kinds still show, because you would want them: critical notices, and the level display of the volume and brightness keys. While a screen is shared in a call the same rule holds by itself, with the seal **共有**, so nothing private appears on the screen others see.

Try this. Send yourself notices from a terminal (**Super**+**Return**):

```
notify-send "Hello" "my first notice"               # a notice, as any program sends one
notify-send -u critical "Hello" "this one stays"    # critical: a vermilion border
```

The first fades after 5 seconds. The second stays until you click it. Now turn *do not disturb* on, run the first line again — nothing shows — then the second: it still shows. Turn *do not disturb* off again.

> **On Windows.** *Do not disturb* is the same idea as there. The notification centre has no counterpart: there is no list of past notices to open — only `makoctl restore`, for the last.

A terminal can also tell you when a long command has finished: §11.9.

8.5 The system's own control, in one minute

Super+U opens the system's control: a window listing *status*, *update*, the move to the next Fedora, *verify*, *repair*, *snapshots* and a few more, with how each stands beside it. You need one line of it for now.

Every few months — the **update season**, 90 days unless you change it — a notice titled *upkeep* says it is time:

```
the update season is here: 90 days since the last update (Super+U)
```

(for example: the number of days is the real count)

When it comes, press **Super+U**, move to *update* and press **Enter**. That is all it takes: the update downloads and installs everything, checks the result, and tells you how it went. Ch. 35 explains the control, and ch. 36 what an update does.

> **On Windows.** Windows Update installs when it decides to and restarts when it must. Here an update runs only when you start it, and the notice is the only reminder: nothing changes under you between seasons.

What you learnt

- Every menu is the same box: type to filter, **Enter** to choose; **Esc**, a click outside or its own key again closes it.
- Quick settings (**Super**+**X**) holds the theme, brightness, network, Bluetooth, drives and do not disturb.
- Wi-Fi: *network* → *Activate a connection*; `ping -c 3 fedoraproject.org` checks you are online.
- Pair in Blueman (*Search*, *Pair*, *Connect*), then choose the headphones in **Super**+**A**.
- A click closes a notice, **Super**+**Shift**+**N** closes all; do not disturb (**静**) lets only critical ones through.
- When the *upkeep* notice says the update season is here: **Super**+**U** → *update*.

Check yourself

1. How do you leave any menu without changing anything? → *Esc, a click outside it, or its own key again*
2. The headphones are paired, but the sound still comes from the speakers. What next? → *Super+A, and choose the headphones under output*
3. Do not disturb is on. Which notices still show? → *critical ones, and the level display of the volume and brightness keys*

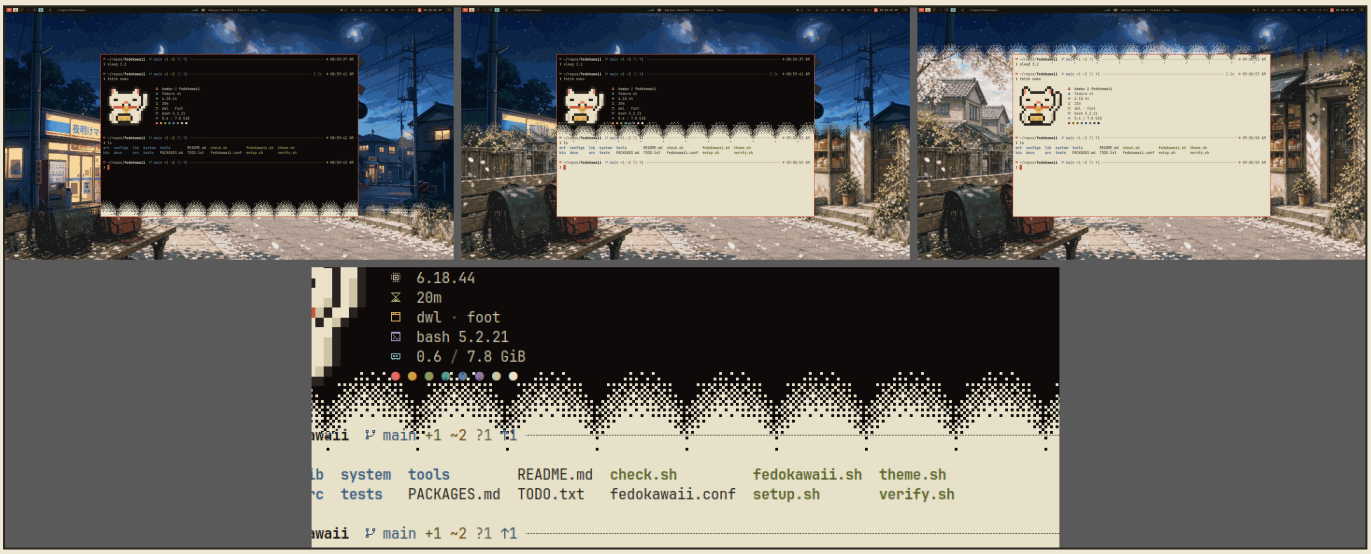
9. Theme and wallpaper

> **In this chapter**
> - switch between the dark and the light theme
> - choose a wallpaper for each theme, from the pack or your own pictures
>
> **You need:** §8.1 (quick settings).

The desktop has two themes, sumi ink by night and washi paper by day, and every program follows the one you choose. This short chapter is the switch, and the pictures behind your windows.

9.1 Dark and light

Press **Super+Shift+W** (or quick settings → *theme*). The screen holds still for a moment while the terminals and everything in them, the bar, menus, notifications, GTK and Qt apps, Brave, mpv, btop, zathura and the wallpaper all change underneath; then the new theme arrives as a row of seigaiha wave crests with a dithered fringe — rising from the bottom like dawn for the light theme, falling from the top like nightfall for the dark one. The lock screen and the next menus follow from then on.



A switch to the light theme, three moments of it: the crests rise from the bottom edge.

The theme you leave on is the one the next login starts in. From a terminal, `kawa-theme light` or `kawa-theme dark` chooses one, `kawa-theme` alone switches, and `kawa-theme get` names the one in use. The two palettes, and how each program gets its colours, are in ch. 40.

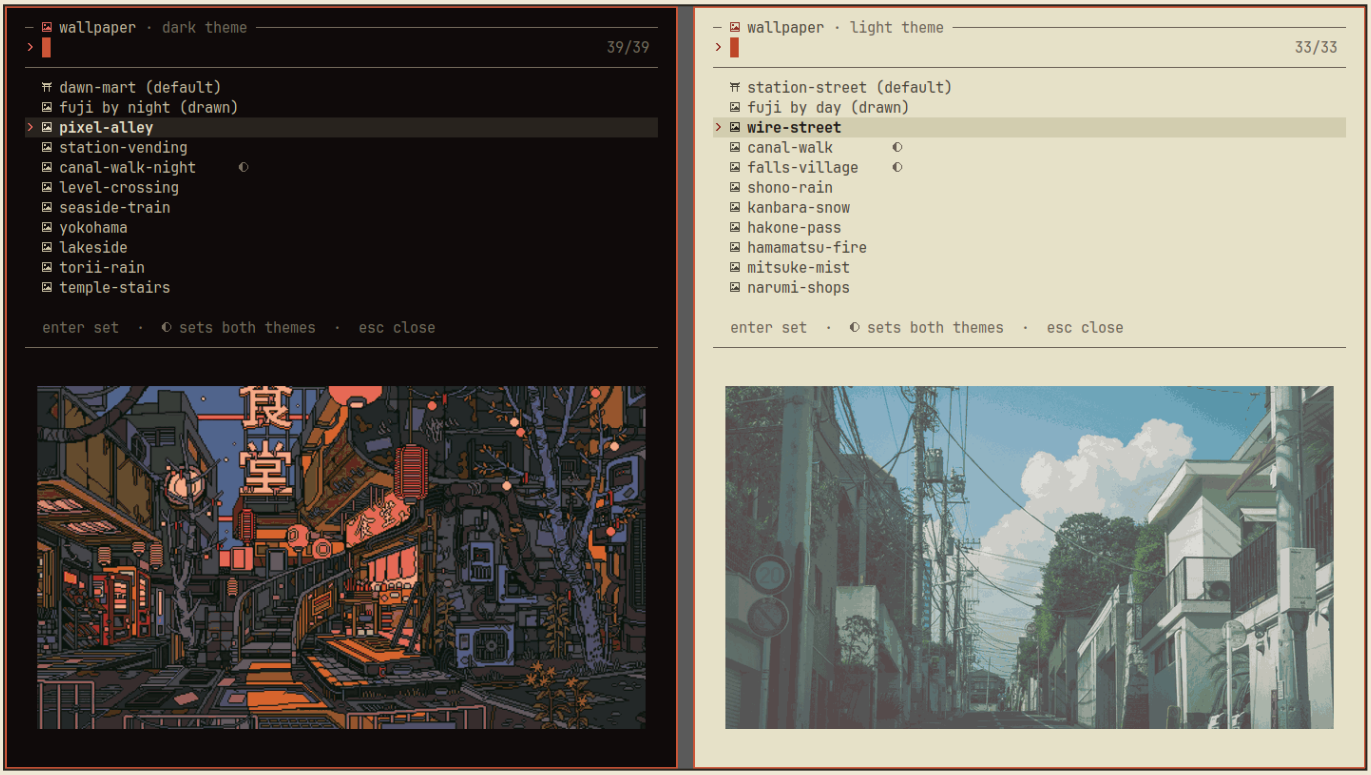
Try this. Open a terminal and Brave side by side, then press **Super+Shift+W**. Watch the crests rise; both windows are light now, and so is the bar. Press it again for the night.

9.2 Wallpapers



The two default wallpapers, redrawn as pixel art: dawn-mart on the dark theme, station-street on the light.

Super+W (or *wallpaper* in quick settings) opens the picker: a small window with the current theme's wallpapers listed on top and, underneath, the one under the cursor — the picture itself, as your screen will show it, scaled down by a whole number so the pixel art stays crisp. Move with ↑/↓ or the mouse, type to search; **Enter** sets it, **Esc** leaves everything as it was. A ◐ after a name means the place also comes in the other theme's light (see the pairs below), and choosing it sets both.



The wallpaper picker (Super+W) on both themes: the list on top, the picture under the cursor as the screen will show it.

Try this. Press **Super+W** and walk down the list with ↓: each picture appears underneath as you reach it. Press **Esc** — nothing has changed. Open it again, choose one with **Enter**, and watch it dissolve in over the old one.

Each theme remembers its own wallpaper. The choices:

- **default** — a retro railway town from the pack: **dawn-mart** on the dark theme (a konbini glowing by a level crossing under the stars) and **station-street** on the light one (the same kind of crossing on a shop street under cherry blossom). Without the pack, the default is *fuji*.
- **fuji** — drawn here: the seigaiha (blue-sea wave) pattern tiled 1:1, with Mt Fuji, a torii in the water, birds and a woodblock-print cartouche along the bottom; by night on the dark theme, by day on the light one.
- **the pack** (`WALLPAPER_PACK="yes"`) — 70 pictures (38 for the dark theme, 32 for the light) fetched once from GitHub at pinned commits and checked by SHA-256, then redrawn as 640×360 pixel art with 32 colours.

Offline, the fetch stops after three pictures that could not reach GitHub and leaves the rest for the next run. Every one is Japanese and retro, and sits in the themes' own colours — ink and paper, vermilion, teal and indigo (each was measured against the two palettes; nothing neon, nothing off-key) — and no two look alike. The picker lists them grouped by kind:

  - *dark*: towns and streets by night (the konbini at the crossing, a lantern-lit pixel alley, a station's vending machines, a two-car train by the sea, Yokohama's Meiji harbour); shrines, gardens, rain and snow (a torii in the rain, a castle in the pines, a stone lantern by a moonlit pond, an onsen in snow); rooms and baths (a tatami room, a sentō, a CRT by a rainy window); ink and woodblock (Hokusai's waves and Lake Suwa, an ensō, a plum branch at night, a ronin); and anime and film (Jujutsu Kaisen, My Hero Academia, Hunter × Hunter, Final Fantasy VII, Totoro, Your Name, Girls' Last Tour, Chainsaw Man).
  - *light*: streets and villages by day; Hiroshige's Tōkaidō road (driving rain at Shōno, night snow at Kanbara, the Hakone pass, the mist at Mitsuke, the crests at Seki, the crowd fording the Ōi); Hokusai (the Great Wave, the fisherman at Kajikazawa, the gust at Ejiri, Mannen bridge, the roofers at Mitsui); ukiyo style, ink and brush (kimono walkers under cherry trees, an ink dragon, a plum branch, an ensō, koi, a tiger, Musashi); and anime (Demon Slayer, Frieren and three illustrations).

  Five come in pairs — the same picture by day and by night (a canal walk, a waterfall village, ink ridges, the ensō, the plum branch) — and choosing one sets the other theme's too. Sources and licences, one line each: `~/.local/share/fedokawaii/wallpapers.list`. Nothing is stored in the repository; the files are fetched for your own use. A picture dropped from a later list is deleted from your pack the next time it is fetched.

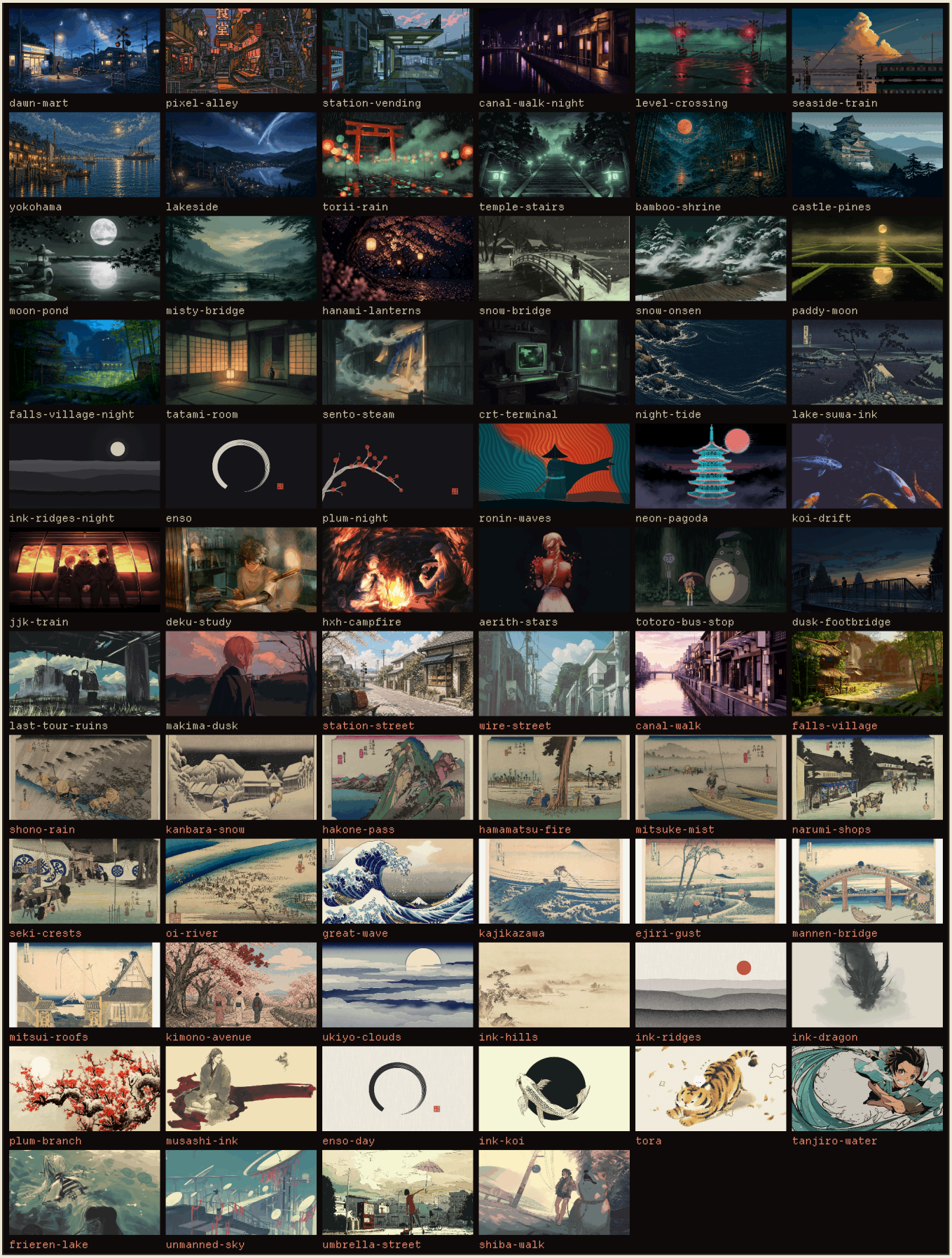
- **your own** — anything in `~/Pictures/Wallpapers`, shown as it is on either theme. One that is not there at login (deleted, or on a drive not plugged in yet) leaves the theme's default up meanwhile; the choice is kept and comes back with the picture.

Pixel art is enlarged by a whole number to cover each monitor, so it never blurs: every pixel of the 640×360 picture becomes a square block of screen pixels.

Screen	Each art pixel becomes	640×360 becomes
1080p	3 × 3	1920×1080
1440p	4 × 4	2560×1440
4K	6 × 6	3840×2160

The results — and the picker's previews — are cached in `~/.cache/kawa-wallpaper`. A new choice dissolves in over the old one in an ordered dither whose grain is exactly one pixel of that art.

Tip: `~/.cache/kawa-wallpaper` can be deleted at any time; `kawa-wallpaper --restore` draws the current wallpaper again.



The whole pack: the dark theme's pictures first, then the light theme's.

9.3 Recipe: switch the theme and the wallpaper

1. **Super+Shift+W** switches dark ⇄ light (or quick settings → *theme*, or `kawa-theme light` in a terminal).

2. **Super+W** chooses a wallpaper **for the theme you are on**; each theme remembers its own. Pictures of a day / night pair set the other theme too.
3. For your own pictures, put them in `~/Pictures/Wallpapers`; they appear in the same menu, on both themes.

What you learnt

- **Super+Shift+W** switches every program between dark and light at once; the choice outlasts a restart.
- **Super+W** picks a wallpaper for the theme you are on; **Esc** leaves everything as it was.
- Each theme remembers its own wallpaper; a ◐ day / night pair sets both.
- Your own pictures go in `~/Pictures/Wallpapers`, and show on both themes.

Check yourself

1. You chose a wallpaper on the dark theme, then switch to light. Does it follow? → *no: each theme keeps its own — unless it is one of a ◐ pair*
2. Where do your own wallpapers go? → *`~/Pictures/Wallpapers`*
3. Why does the pixel art stay sharp on a 4K screen? → *each pixel of the 640×360 picture becomes an exact 6 × 6 block*

10. Files and folders

> **In this chapter**
> - read and write a path, and know where your own files belong
> - make, copy, rename, delete and restore files in yazi
> - open, upload and save files from any program
> - choose which program opens a kind of file
>
> **You need:** ch. 6 (**Super+E** opens the files).

Everything you keep is a file in a folder. This chapter gives you the map — paths — then the file manager, yazi, and the dialog every program uses to open and save. The terminal in the next chapter uses the same map.

10.1 Folders, paths and your home

Here is a file:

```
/home/you/Documents/tenancy.pdf
```

Read it from the left: start at the top of everything (`/`), go into the folder `home`, then `you`, then `Documents`; the last part is the file itself.

> **Word — path.** The address of a file or folder: the folders to pass through, each followed by `/`, then the name. `/home/you/Documents/tenancy.pdf` is a path.

The rule: on Linux, everything sits in **one tree** that starts at `/`, called *root*. There are no drive letters. A USB stick appears as one more folder inside the tree (ch. 13), and so does everything else:

```
/                    the top of the tree: "root"
├── etc/             the system's settings
├── usr/             the programs
├── home/
│   └── you/         your home: ~
│       ├── Documents/
│       ├── Downloads/
│       └── Pictures/
└── run/media/you/   where USB sticks open
```

Part IV draws the whole tree (§23.2). One folder in it is yours:

> **Word — home folder.** Your own folder, `/home/you`, written `~` for short. Everything you keep goes inside it. Outside it you can read most things and change almost nothing.

`~` is the path you will type most: `~/Documents`, `~/Downloads`, `~/Pictures/Wallpapers`.

> **On Windows.** `C:\Users\you` is `/home/you` here. Paths use `/`, not `\`, and there is no `C:` or `E:`: one tree from `/`, sticks included. Names are case-sensitive — `Notes` and `notes` are two different folders.

A path that starts with `/` (or `~`) is **absolute**: it means the same place wherever you are. One without is **relative**: it starts from the folder you are in. Four examples, read from inside `/home/you`:

Path	Kind	Means
`/home/you/Documents`	absolute	your Documents folder, from anywhere
`~/Documents`	absolute: `~` is `/home/you`	the same folder, shorter
`Documents`	relative	`Documents` inside the folder you are in — here, the same again
`..`	relative	the folder one level up: from `~/Documents`, that is `~`

Three more things about names:

- A name that starts with a dot — `.bashrc`, `.config` — is **hidden**: lists leave it out unless you ask. Settings live in hidden files and folders.
- **Case matters**: `Notes`, `notes` and `NOTES` are three different names.
- In a terminal, a space separates words, so a name with a space needs quotes: `'bank statements'`. A `-` instead — `bank-statements` — saves typing later.

Try this. Walk the tree in yazi.

Press	You see	It means
Super+E	three columns: `/home` on the left, your home in the middle, a preview on the right	yazi starts in `~`
.	new names starting with a dot appear: `.bashrc`, `.config`, …	the hidden files, shown
. again	they disappear	hidden again
h	the middle column is `/home` now, with `you` in it	one level up: `..`
h again	`/`: `etc`, `home`, `usr` and the rest	the top of the tree
g h	your home again	`~`, from anywhere
q	yazi closes	

10.2 Where your own files go

A short, fixed layout makes them quick to find, to back up and to clean:

Folder	For	Why
`~/repos/`	every git repository, one folder each	the prompt shows the branch and changes there; lazygit and neovim find the project from the folder. The Fedokawaii clone can stay at `~/fedokawaii` (§5.1)
`~/Documents/`	papers, letters, spreadsheets, PDFs you keep	one place for everything you would hate to lose
`~/Pictures/` · `~/Videos/` · `~/Music/`	what you keep of each kind	the standard folders; file dialogs and yazi know them
`~/Downloads/`	arrivals only	not in snapshots — move what you keep into one of the folders above, delete the rest
`~/Books/`	books and comics, in folders of your own if you like	the books window reads it (§16.2); its Ctrl+A moves books here from `~/Downloads`, `~/Documents` and `~/Desktop`
`~/Notes/`	`notes.md` (Super+N), and any other notes you start	plain Markdown: searchable with `rg`, readable anywhere (§16.3)

Snapshots are not backups: they live on the same disk (ch. 37). Copy the folders in this table to another disk or machine now and then.

10.3 yazi, the file manager



yazi: the parent folder, the current folder, and a preview of the file under the cursor.

Super+E opens yazi in its own terminal window, starting in your home folder; `q` quits and closes the window. Typed in a terminal, `yazi` starts in the folder you are in. Three columns: the parent folder, the current one, a preview of the file under the cursor. Its colours are the terminal's, so it switches with the theme. The file under the cursor is the *hovered* one; files marked with Space are *selected*. Most keys act on the selected files, or on the hovered one when none are. Press **~** or **F1** for the complete key list of the installed version.

Try this: your first files in yazi. Seven steps, a few minutes. The folder you make here is the one the terminal uses in ch. 11.

1. Press **Super+E**. yazi opens in your home.

2. Make a folder: press **a**, type `practice/` and press **Enter**. A box titled *Create:* takes the name; the `/` at its end makes a folder rather than a file. `practice` appears in the list. Move onto it with **j** / **k** and press **l** to go in: it is empty.

3. Copy a file into it. Press **h** to go back to your home, then **.** to show the hidden files. Move to `.bashrc` — your shell's settings — and press **y** (copy). Move back to `practice`, go in (**l**) and press **p** (paste). A copy of `.bashrc` is there now; the original is untouched.

4. Rename the copy: press **r**. A box titled *Rename:* holds the name. **Ctrl+E**, then **Ctrl+U** empties it; type `bashrc.txt` and press **Enter**. Without its leading dot the file is no longer hidden.

5. Send it to the trash: press **d**. yazi asks *Trash 1 selected file?*; press **y**. The file is gone from the list.

6. Bring it back: **g t** opens the trash. Move to `bashrc.txt`, press **O** and choose *Restore selected files*. Press **g h**, go into `practice`: it is back.

7. Press **.** to hide the hidden files again, and **q** to close yazi. Leave `~/practice` as it is: the terminal starts there.

If the trash is empty in step 6: you pressed **D**, not **d**, and answered **y** — see below.

> **Careful. D** (with Shift) deletes for good, past the trash. yazi asks *Permanently delete 1 selected file?* first — answer **n** unless you mean it.

Those seven keys are most of daily use. The whole list follows; **~** shows it inside yazi too.

> **On Windows.** yazi is File Explorer with the mouse put away, though the mouse works too: **y** / **p** are Ctrl+C / Ctrl+V, **r** is F2, **d** is Delete (to the trash, the Recycle Bin), **D** is Shift+Delete.

Keys	Does
h j k l (or arrows)	parent folder · down · up · open folder
gg · G · H · L	top · bottom · back · forward through visited folders
g h · g d · g c · g t	home · Downloads · `~/.config` · the trash
M · g m	the drives: open or eject a USB stick, disk or SD card · go where they open (ch. 13)
g Space	type a path and jump there
Enter · O	open the file (a folder: go into it) · choose what to open it with
Space · v · Ctrl+A	select a file · select a range · select all
y · x · p · P	copy · cut · paste · paste, overwriting
d · D	move to the trash · **delete for good**
a	create a file; end the name with `/` for a folder
r	rename; with several files selected, rename them all in neovim
.	show / hide hidden files
s · S	search file names (fd) · file contents (ripgrep) below this folder
f · / · n · N	filter the list as you type · find · next · previous match
z · Z	jump to any folder or file below this one by typing part of its name (fzf) · jump to a folder you often use, from anywhere, by a word (zoxide — the same list as the shell's `cd`)
c c · c f · c d	copy the file's full path · its name · the folder's path
t t · 1...9 · [ ]	new tab · go to a tab · previous / next tab
Tab	details of the file (type, size, picture dimensions)
, then a letter	sort: `m` modified, `s` size, `n` natural, `a` alphabetical (capital = reversed)
K · J	scroll the preview
; · :	run a shell command in the background · run one in this window (yazi returns when it ends)
w	running tasks (copies, moves)

Opening

On a folder, Enter goes into it (yazi's own Enter would open the folder in neovim: `~/.config/yazi/plugins/smart-enter.yazi`). With files selected (Space), Enter opens all of them; otherwise the one under the cursor. A Xournal++ note (`.xopp`) opens in Xournal++ — yazi would otherwise take it for a gzip archive. On a file, Enter uses the first *opener* — a command yazi runs to open a file — for the file's type (its MIME type, such as `image/png`, §10.5): text and JSON open in neovim inside the same window (`:q` brings you back to yazi); an archive is unpacked next to itself (yazi's *Extract here*, with 7-Zip); pictures, PDFs, videos and the rest go to `xdg-open`, which hands them to the default program for that type (§10.5). **O** lists every choice for the hovered file. Two of yazi's own choices there, *Show EXIF* and *Show media info*, need `exiftool` and `mediainfo`, which this desktop does not install: they answer "command not found". mpv's **i** shows a video's details instead (§15.2).

Previews that work here: text and code (in the terminal's colours), JSON (jq), pictures (drawn in foot as Sixel), the first page of PDFs (pdftoppm), video stills (ffmpeg), fonts and HEIC/AVIF/JXL pictures (ImageMagick), and the contents of archives (7-Zip). SVG pictures show no preview, because resvg is not packaged for Fedora.



yazi's preview column showing a picture, drawn in the terminal as Sixel by foot.

Bulk rename

Select the files (Space or v), press **r**: neovim opens with one name per line. Change the lines, save and quit (`:wq`), and yazi asks before renaming. Keep the lines in the same order and do not add or remove any. Every neovim edit works on the list: `:%s/^IMG_/trip_/` turns `IMG_0412.jpg` into `trip_0412.jpg`, and every other line alike (ch. 17, where a macro also numbers them).

Trash

d moves files to the desktop trash (`~/.local/share/Trash`). **g t** opens it; there, **O** on a file offers *Restore selected files* and *Empty trash bin*. **D** skips the trash entirely.

Tip: files moved with **d** still take their space until the trash is emptied; make it part of the seasonal routine (§38.4).

Commands on files

This part uses the terminal's ways — commands, quoting, `%` names — so it reads best after ch. 11. Come back to it then.

In `;` and `:` commands, `%h` stands for the hovered file and `%s` for the selected ones (the hovered one when none are), each already quoted, so names with spaces are safe. The command runs in the current folder.

Type	Does
`;` then `chmod +x %h`	makes the hovered script executable
`;` then `7z a backup.7z %s`	packs the selected files into `backup.7z`, here
`:` then `du -sh %s; read`	shows their sizes until you press Enter (without the `read`, yazi comes back before you can read them)

Recommended: yazi, neovim and lazygit together. Work in one terminal: browse in yazi, press Enter on a file to edit it in neovim, `:q` to return; when the work is done, `:` then `lazygit` to stage and commit, and `q` in lazygit brings you back to yazi. Uploading a file in Brave opens yazi too, as the file dialog (§10.4, §14.3).

Pitfalls.

- Input boxes (rename, create, filter) work like a tiny vi: Esc first leaves typing, a second Esc — or Ctrl+C — cancels.
- Three files are the desktop's: `theme.toml` (colours, the file-kind icons — the desktop's own pixel icons), `init.lua` (the frame: one free column at each edge, as every popup has, and the file dialog's title rule and key line) and `keymap.toml` (**M** and **g m**, the drives), with the drives plugin in `plugins/`. Your own openers go in `yazi.toml`, which you create; your own keys go in `keymap.toml` in your working copy (chapter 42), so the next Theme keeps them (an example follows this list). yazi itself is held at the version these files are written for (§27.7).
- Search results (s, S) are a temporary list; Esc leaves it, Ctrl+S stops a search still running.

A key of your own, as an example — `g` then `r` to go to `~/repos`. Add its line to the list in `configs/home/.config/yazi/keymap.toml` in your working copy (Super+Shift+U → *files*), then apply:

```
# keymap.toml — these keys are tried before yazi's own
[mgr]
prepend_keymap = [
  { on = "M",          run = "plugin drives",          desc = "Drives: open or eject one" },
  { on = [ "g", "m" ], run = "plugin drives -- media", desc = "Go to the open drives" },
  { on = [ "g", "r" ], run = "cd ~/repos",             desc = "Go to ~/repos" },   # yours
]
```

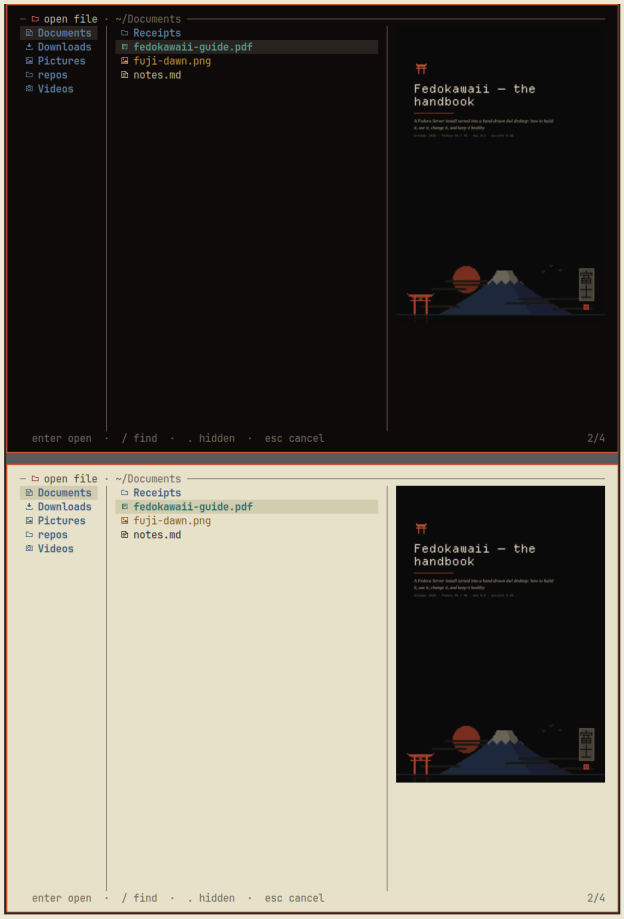
yazi reads it when it starts; `~` then lists the new key with its *desc*. The file dialog reads its own list instead, `configs/home/.config/fedokawaii/yazi-pick/keymap.toml` (the menu's *files* opens it beside the others): a key you want there too goes in both.

Read more: yazi-rs.github.io/docs/quick-start for the keys, and the *Configuration* pages beside it for `keymap.toml` and `yazi.toml`. The site describes the newest yazi; `~` lists the keys of the version installed here.

10.4 Open, upload and save: the file dialog

Every program's *Open*, *Upload* and *Save as* — Brave's uploads and downloads, every GTK program's dialogs — opens **yazi** (§10.3) in a small floating terminal: the same file manager, keys and previews you already use, instead of a separate dialog with its own ways. It is dressed like the other popups — what it is for in a rule along the top (*open file*, *open files*, *open folder*, *save file*, with the folder you are in), the keys that matter along the bottom. It starts in the folder the program suggests (Brave: where you last saved or uploaded), else your home.

Folders opened from anywhere — a download's *Show in folder*, `xdg-open ~/Pictures` — open in yazi too, in a window of its own, as **Super+E** does. How the dialog is wired is §32.3.



The file dialog an upload opens: yazi in a floating window, the PDF under the cursor previewed.

Try this. In Brave, press **Ctrl+O** (open a file). The dialog opens, *open file* in its top rule. Go into `practice`, put the cursor on `bashrc.txt` and press **Enter**: Brave shows the file's text. **Ctrl+W** closes that tab.

Keys in the dialog

In the dialog	Open a file	Open a folder	Save
move · into a folder · back	↑ ↓ (or j k) · → or Enter (l) · ← (h)	the same	the same
choose	**Enter** on a file	**Enter** on a folder, or **q** in the one you are in	**Enter** on the file the program suggested (it is already there, under its name; **r** renames it first)
jump	**z** a few letters of a name below here · **Z** a folder you use often · **g d** Downloads	the same	the same
a USB stick, disk or SD card	**M**: the drives; **Enter** on one goes there (ch. 13)	the same	the same
several at once	**Space** marks each, then **Enter** (where the program accepts several)	Space, then Enter	—
save into another folder	—	—	go there and press **Enter** on the folder
replace a file	—	—	**Enter** on it; it asks first
search · hidden files	`/` · `.`	the same	the same
cancel	**Esc**	**Esc**	**Esc** (the suggested file goes again)

Saving shows the file before it is written: the program's suggested name is waiting in the folder, standing in for the file until you choose — its preview says *the file being saved* — and Enter keeps it there. Is a file of that name there already, the stand-in is *name (1).ext* beside it (as Brave names a second download) — Enter on the old one replaces it instead, after asking. Cancel, and the stand-in is removed again: nothing is left behind.

10.5 Which program opens a file

A file's name may end in `.pdf`, but programs go by something more exact: its MIME type.

> **Word — MIME type.** The standard name for a kind of file, in two parts: `application/pdf` for a PDF, `image/png` for a PNG picture, `text/plain` for plain text. Programs choose how to open a file by it.

Default programs come from `~/.config/mimeapps.list` (set by Theme), so `xdg-open`, yazi, Brave's downloads and every other program agree: the list names one program for each MIME type. `xdg-mime` asks it:

```
xdg-mime query default text/plain        # → kawa-edit.desktop (neovim in a terminal)
xdg-mime query default inode/directory   # → kawa-files.desktop (yazi in a terminal)
```

The whole list:

Type	Opens in
web pages, links	Brave Origin
folders	yazi (in a terminal window)
text, code, config files, Markdown	neovim (in a terminal window)
pictures	imv
video, music, playlists	mpv
PDF, EPUB, XPS, FictionBook	zathura
comic books (CBZ/CBR)	zathura
Word / OpenDocument text, RTF	AbiWord
spreadsheets, CSV	Gnumeric
Xournal++ notes	Xournal++ (to `file` a note is gzip, so opened from elsewhere it shows in yazi first — Enter there opens Xournal++)
archives (`.zip`, `.tar.gz`, `.7z`, `.rar` …)	yazi, on the archive — Enter unpacks it beside itself
magnet links, `.torrent` files	torrra (ch. 20)

On this desktop `xdg-open` types a file with `file`, which has names of its own for some kinds (Python is `text/x-script.python`, C/C++/Go/Rust `text/x-c`); the list names those too, so code opens in neovim whichever way it is opened.

Programs that ask for a terminal (yazi's own launcher entry, say) get foot.

10.6 Recipe: choose which program opens a type of file

The defaults are in `~/.config/mimeapps.list`, which Theme manages (§10.5).

1. Find the type — its *MIME type*, the standard name for a kind of file: `xdg-mime query filetype book.pdf` → `application/pdf`.
2. See what is chosen now: `xdg-mime query default application/pdf` → `org.pwmt.zathura-pdf-mupdf.desktop` (zathura).
3. Find the candidates: `grep -l 'application/pdf' /usr/share/applications/*.desktop`.

4. For good: ch. 42. Change the line in `configs/home/.config/mimeapps.list` in the repository — your working copy, which that chapter sets up — and apply it. To try one first, `xdg-mime default NAME.desktop application/pdf` changes it at once — until the theme is next applied, which puts the repository's choice back.

What you learnt

- A path names a file from the top of the tree: `/home/you/Documents/tenancy.pdf`; `~` is your home, `..` the folder above.
- Names that start with a dot are hidden; names are case-sensitive.
- In yazi: **a** makes, **y** / **p** copy and paste, **r** renames, **d** sends to the trash, **g t** opens it.
- Every program's *Open*, *Upload* and *Save as* is yazi too.
- Each MIME type has one default program, listed in `~/.config/mimeapps.list`.

Check yourself

1. What does `~/Downloads` stand for? → */home/you/Downloads*
2. How do you see the hidden files in yazi? → *press* `.` *— again to hide them*
3. Where does a file go after **d** in yazi, and after **D**? → *d: to the trash, where O → Restore brings it back · D: nowhere — it is gone*

11. The terminal: first steps

> **In this chapter**
> - open a terminal and read its prompt
> - move between folders, and make, copy, move and remove files
> - take any command apart, and get help on it
> - get out of any program, and read the three common errors
>
> **You need:** ch. 10 (folders and paths) and §4.3 (the console).

This chapter is the pivot of the book. From here on, many things are quicker typed than clicked, and every later chapter shows commands. Give it an unhurried hour. Type every example yourself — do not paste — and look at what comes back.

11.1 Terminal, shell and console: five words

Press **Super+Return**. A window opens with a line of text across the top and a `❯` below it, the cursor waiting: a terminal. Five words describe what you are looking at, and they are often mixed up:

Word	Is	Here
console	the screen and keyboard attached to the machine, in text mode, before any desktop	tty1 (the login) and tty2–tty6 (**Ctrl+Alt+F2…F6**)
TTY	a terminal device in the kernel: `/dev/tty1` is the first console	`tty` prints the one you are on
PTY (pseudo-terminal)	a terminal device made for a program that draws a terminal in a window; it appears as `/dev/pts/N`	every foot window
terminal emulator	the program that draws the text, sends your keys and owns the PTY	foot (§11.9)
shell	the program in the terminal that reads commands and runs them	bash (§18.1)

Try this. Ask the terminal which device it is.

```
$ tty            # in a foot window
/dev/pts/3
```

(for example; on a text console it prints `/dev/tty2`)

`tty` names the terminal device you are typing into: a PTY, as this is a window. In this book a command is written after `$`; on your screen you type it after `❯`. Do not type the `$`.

> **On Windows.** Windows Terminal is the window and PowerShell the shell inside it — the same split as foot and bash here. The console, full-screen text with no windows at all, has no everyday Windows counterpart: you met it at the login.

On Fedora 44 the consoles are drawn by the kernel itself. Fedora 45 moves them to **kmscon**, a console that runs as an ordinary program and draws through KMS (§31.1), with its own font and colours; greetd keeps tty1 for itself (its drop-in declares a conflict with `kmsconvt@tty1.service`), so the login looks the same on both.

11.2 The shell's words

Two more words come up as soon as you type anything:

Term	In one line	Here
variable · environment variable	a named value, read as `$NAME` · one marked with `export`, so every program started from this shell gets a copy	`$HOME` · `EDITOR=nvim`
exit status	the number a command ends with: 0 is success, anything else a failure	`$?`; the prompt's seal

Try this.

```
$ echo $HOME
/home/you
$ echo $?
0
```

You typed	It means
`echo $HOME`	`echo` prints what follows it; the shell first puts the value of `$HOME`, your home, in its place
`echo $?`	the exit status of the command before: `0`, it succeeded

The rest of the shell's words you will meet bit by bit; each is explained in ch. 25:

Term	In one line	Here
readline	bash's line editor: what Tab, Up and the word keys do while you type	`~/.inputrc`
`PATH`	the folders searched, in order, for a command typed by its name	`~/.local/bin` first
alias · function	a word typed in place of a longer command · a named piece of shell code that takes arguments and can decide	`ll` · `prowlarr`, `prowlarr stop`
pipe · redirection	one command's output into the next command · output into a file, or input from one	§11.6; *By example*, §25.1
glob	a pattern that bash turns into the matching file names before the command runs	`*.pdf`
quoting	`'…'` keeps text exactly as typed; `"…"` keeps it one word but still expands `$NAME`	`"$file"`

11.3 The prompt

You met the **prompt** at the console (§4.3): the text the shell prints when it is ready for your next command. In a terminal on the desktop it is two lines: a ruled header that says where you are, then `❱`, after which you type.

Three prompts, and what each says. **In your home**, in a new terminal:

```
⛩ ~ ···· ☾ 08:52:07 PM
❱
```

The torii starts the entry; `~` is the folder you are in; the moon says it is night, then the time to the second.

After a command that failed, in `~/practice`:

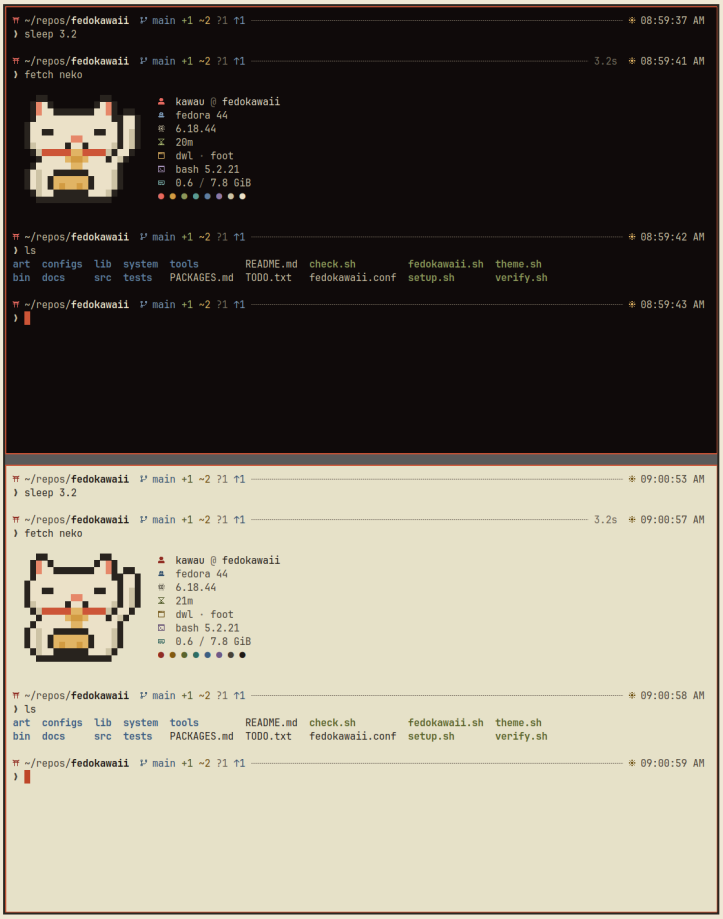
```
⛩ ~/practice ···· × 1  ☾ 08:53:10 PM
❱
```

The seal `× 1` says the last command ended with exit status 1 — it failed — and the `❱` below turns vermilion.

In a git repository with changes, after a command that took 4.2 seconds and failed:

```
⛩ ~/repos/site   main +2 ~1 ····· 4.2s   × 2   ☾ 08:52:07 PM
❱
```

You are in `~/repos/site`, on the branch `main`, with two changes staged and one modified (git is ch. 41); the command took 4.2 s and ended with status 2.



A real terminal: the prompt's ledger line per command, a command that took over three seconds, fetch, and ls in a git repository with changes.

Part	Shows
torii	where every entry starts
path	the folders above in the text colour, the one you are in bright and bold (the last three; `…/` for the rest); `user@host` first over SSH
git	branch, then only what is non-zero: `+` staged, `~` modified, `?` untracked, `!` conflicted, `↑` ahead, `↓` behind — one `git status` call, parsed in bash
rule	a dotted line to the right edge, so each command reads as a new entry
right end	time taken (3 s or more) · the exit code in a vermilion seal when the command failed · sun by day, moon by night · the time to the second
`❱`	the text colour; vermilion after a failure

(The terminal's "vermilion" and "paper" are its ANSI red and white, so they follow the theme like everything else in it.) Nothing on the header line has a background, so the cursor below never touches it. Outside a git repository the prompt runs entirely inside bash — it starts no process to draw itself (Fedora's own systemd hook, which tells terminals where each command starts and ends, adds one small one).

Feature	How
new terminal in the current folder (Ctrl+Shift+N)	the prompt reports the folder (OSC 7)
jump between prompts (Ctrl+Shift+Z / X)	the prompt marks itself (OSC 133)
last command's output in neovim (Ctrl+Shift+G)	OSC 133 marks around the output + foot's `pipe-command-output`
history	50 000 entries, duplicates erased, each command saved as it runs, so every new window has it; Up/Down search by what you typed
instant terminals	`foot --server` at login; Super+Return opens a client

11.4 Your first commands

A guided quarter of an hour, in two parts: this section moves around and makes things, the next copies, moves and removes them. Open a terminal and work through the steps in order. You met `pwd` and `ls` at the console (§4.3); now they get their full story.

> **Word — command.** A line you type at the prompt and send with **Enter**. Its first word names the program to run: in `ls -l`, the program is `ls`.

Try this — step 1. Ask where you are.

```
$ pwd
/home/you
```

`pwd` — *print working directory* — names the folder the terminal is in. A new terminal starts in your home.

> **Word — working folder.** The folder a terminal is "in". A name typed without a full path is looked for there. The prompt shows it.

Try this — step 2. See what is here.

```
$ ls
Desktop  Documents  Downloads  fedokawaii  Music  Pictures  practice  Public  Templates  Videos
```

(for example)

`ls` lists a folder: the folder you are in when you name none. Folders come first, then files, in the theme's colours. Here `ls` is **eza**, a modern `ls` with the same name, and `ll` shows more; both are in ch. 18. Everything below works the same with a plain `ls`.

> **On Windows.** PowerShell has `ls`, `cd`, `pwd` and `cat` too, but there they are other names for PowerShell's own commands, with other options. The ones here are the originals: the same on every Linux machine, and on a Mac.

Try this — step 3. Go into your practice folder.

```
$ cd practice
$ pwd
/home/you/practice
$ ls
bashrc.txt
```

You typed	It means
`cd practice`	*change directory*: into `practice` — a relative path, looked for in the working folder
`pwd`	you are in `/home/you/practice`; the prompt now shows `~/practice`
`ls`	the one file in it: the copy you made in yazi (§10.3)

If you see `zoxide: no match found`: there is no `practice` here — you skipped the yazi steps. `mkdir -p ~/practice` makes it (step 4 explains `-p`), and `cd practice` works; `ls` then shows nothing.

Here `cd` does one more thing: it remembers the folders you visit, and later takes you to one from anywhere by a word of its name. That is **zoxide**, built into `cd` itself — there is no separate `z` command on this desktop (ch. 18). For now it works like any `cd`.

Try this — step 4. Make two folders, one inside the other.

```
$ mkdir -p trips/japan
$ ls
trips  bashrc.txt
```

`mkdir` — *make directory* — makes `trips/japan`. With `-p` it makes `trips` first, because `japan` goes inside it, and it says nothing if they exist already. Without `-p`, `mkdir trips/japan` fails while `trips` is missing.

Try this — step 5. Make two files, and read one.

```
$ echo 'Tokyo, Kyoto, Osaka' > trips/japan/plan.txt
$ touch trips/japan/packing.txt
$ cat trips/japan/plan.txt
Tokyo, Kyoto, Osaka
```

You typed	It means
`echo '…' > trips/japan/plan.txt`	`echo` prints its text; `>` sends it into a file instead of the screen — making the file, or replacing what was in it
`touch trips/japan/packing.txt`	an empty file (on a file that exists, only its date changes)
`cat trips/japan/plan.txt`	print the file's contents

The quotes keep `Tokyo, Kyoto, Osaka` together as one piece of text; §11.6 says why.

Try this — step 6. Move around.

```
$ cd trips/japan
$ pwd
/home/you/practice/trips/japan
$ cd ..
$ pwd
/home/you/practice/trips
$ cd -
$ pwd
/home/you/practice/trips/japan
```

You typed	It means
`cd trips/japan`	two levels down in one step: a path can pass through several folders
`cd ..`	one level up: `..` is the folder above
`cd -`	back to where you were before the last `cd`

`cd` with nothing after it goes home, from anywhere.

11.5 Copy, move and remove

The second half of the quarter hour. You are in `~/practice/trips/japan`; if not, `cd ~/practice/trips/japan` takes you there.

Try this — step 7. Copy a file, then rename the copy.

```
$ cp plan.txt plan-v2.txt
$ ls
packing.txt  plan-v2.txt  plan.txt
$ mv plan-v2.txt draft.txt
$ ls
draft.txt  packing.txt  plan.txt
```

You typed	It means
`cp plan.txt plan-v2.txt`	*copy*: from the first name to the second
`mv plan-v2.txt draft.txt`	*move*: here, a new name in the same folder — that is how you rename

`mv` moves too: `mv draft.txt ..` would put the file one folder up, in `trips`.

Try this — step 8. Look at the details.

```
$ ll
.rw-rw-r--@ 20 you 2026-10-16 09:14 draft.txt
.rw-rw-r--@  0 you 2026-10-16 09:13 packing.txt
.rw-rw-r--@ 20 you 2026-10-16 09:12 plan.txt
```

(for example: the times are yours)

`ll` lists every file with its details, hidden ones included: who may read and write it (`.rw-rw-r--`, ch. 24), an `@` for the security label every file carries here (§34.2), the size in bytes, the owner, and when it last changed. `plan.txt` is 20 bytes — 19 letters, spaces and commas, and the end of the line; `packing.txt` is empty.

> **Careful.** `rm` deletes at once: a terminal has no trash, and nothing asks "are you sure?". To keep a way back, use `gio trash FILE` instead.

Try this — step 9. Remove one file for good, and send one to the trash.

```
$ rm draft.txt
$ gio trash packing.txt
$ ls
plan.txt
```

You typed	It means
`rm draft.txt`	*remove*: gone for good
`gio trash packing.txt`	moved to the trash — the same trash as yazi's **d**, where **g t** brings it back

`rm -r FOLDER` removes a folder and everything in it. It is the most dangerous command in this chapter: read the name twice before you press Enter.

Try this — step 10. Read a long file a page at a time.

```
less ~/.bashrc.d/fedokawaii.sh   # your shell's settings: about 300 lines
```

`cat` would pour all of it past you at once; `less` shows one screenful and waits, `:` at the bottom. These keys move in it:

Key	Does
Space · **b**	the next page · the page before (or the mouse wheel, PageDown, PageUp)
`/history`, then **Enter**	find *history*; **n** the next match, **N** the one before
q	quit: back at the prompt

Try this — step 11. Go home.

```
$ cd
$ pwd
/home/you
```

Leave `~/practice` as it is: Part IV practises there too.

11.6 The anatomy of a command

Every command line you will meet has the same parts. Take this one:

```
ls -l --sort=size ~/Downloads    # your downloads, one per line, smallest first
```

Part	Kind	Here
`ls`	the command: the program to run	list a folder
`-l`	a short option: one letter after one `-`	the long form: one line per file, with details
`--sort=size`	a long option: a word after `--`, here with a value after `=`	sort by size
`~/Downloads`	an argument: what to act on	the folder to list

> **Word — option.** A word starting with `-` or `--` that changes how a command works: `-l`, `--sort=size`. Short ones can be joined: `-la` is `-l` and `-a` together.

Many options come in both forms, one letter and one word: `ls -a` and `ls --all` are the same. Joined short options save typing; long ones read better in a script.

> **Word — argument.** What a command acts on, usually a file or a folder: `~/Downloads` in `ls ~/Downloads`. Most commands take several, separated by spaces.

That last point — *separated by spaces* — is why a name with a space needs quotes.

Try this. Make folders with and without quotes, then tidy up.

```
$ cd ~/practice
$ mkdir bank statements
$ ls
bank  statements  trips  bashrc.txt
$ mkdir 'bank statements'
$ ls
bank  'bank statements'  statements  trips  bashrc.txt
$ rmdir bank statements 'bank statements'
$ ls
trips  bashrc.txt
```

You typed	It means
`mkdir bank statements`	two arguments: two folders, `bank` and `statements`
`mkdir 'bank statements'`	one argument in quotes: one folder with a space in its name — `ls` shows it in quotes too
`rmdir bank statements 'bank statements'`	three arguments: remove three folders. `rmdir` removes only empty ones, so it is safe to tidy with

One more piece joins commands together: the **pipe**, `|`. It hands one command's output to the next command as its input:

```
$ ls /usr/bin | grep ^git
git
git-receive-pack
git-shell
git-upload-archive
git-upload-pack
```

(for example)

`ls /usr/bin` lists the folder of programs — over two thousand names. `grep ^git` keeps only the lines that start (`^`) with *git*.

> **In SQL terms.** `ls -l --sort=size ~/Downloads` reads like `SELECT * FROM downloads ORDER BY size`: the command is the verb, options shape the result, the argument names the table. A pipe chains steps: `ls /usr/bin | grep ^git` is `… WHERE name LIKE 'git%'`.

Pipes and the `>` of step 5 are explained properly in §25.2.

11.7 Getting help: tldr, man and --help

Three tools on this desktop answer most small questions:

```
tldr tar          # a few common examples of a command
man 1 tar         # its whole manual (/word searches, n the next match, q quits)
tar --help | less # the short list of its options
```

Start with `tldr`: it shows the handful of things people do with a command, as examples. The first time, it fetches the page, so it needs the network.

Try this.

```
$ tldr tar

  tar

  Archiving utility.

  - [c]reate an archive and write it to a [f]ile:
    tar cf path/to/target.tar path/to/file1 path/to/file2 ...

  - E[x]tract a (compressed) archive [f]ile into the current directory [v]erbosely:
    tar xvf path/to/source.tar.ext
…
```

(for example; shortened — the pages are updated now and then)

Each example is a sentence, then the command; the letters in brackets show which option stands for which word — `c` create, `f` file, `x` extract, `v` verbose. The `path/to/…` parts are yours to fill in.

The whole family:

Command	Does	Example
`man`	the manual (opens in neovim here; `gO` lists sections, `q` closes)	`man 5 systemd.timer`
`man -k` · `apropos`	search the manuals' one-line descriptions	`apropos -s 1 snapshot`
`whatis`	one command's one-line description	`whatis findmnt`
`tldr`	a few common examples	`tldr ss`
`--help`	a program's own short list of options	`nmcli --help`
`help`	bash's own commands	`help jobs`

A manual opens in neovim here, so it moves a little differently from `less`:

To	In `less`	In a man page
go down · up a page	**Space** · **b**	**PageDown** · **PageUp** (or the mouse wheel)
find a word	`/word`, then **Enter**	`/word`, then **Enter**
the next · the previous match	**n** · **N**	**n** · **N**
the list of sections	—	**gO**
leave	**q**	**q**

In a man page, **Space** opens neovim's own key menu instead of turning the page (ch. 17); **Esc** closes it. A man page's number says which part of the manual: 1 commands, 5 file formats, 8 system administration — `man 5 systemd.timer` is the file format, not a command.

11.8 Typing less: Tab, history and Ctrl+R

Four keys save most of your typing. Try each in `~/practice`.

Tab completes names. Type the start of a file or folder name and press **Tab**: the shell finishes it.

You type	Tab gives
`cat tr`, Tab	`cat trips/` — the only name starting with *tr*
`ja`, Tab	`cat trips/japan/`
`p`, Tab	`cat trips/japan/plan.txt` — the only file left; **Enter** runs it
`ls ~/D`, Tab	three names fit: `Desktop/ Documents/ Downloads/` are listed under the line, at the first Tab
`ow`, Tab	`ls ~/Downloads/`; **Enter** lists it

Completion ignores case here: `ls ~/d` and Tab finds the same three. Tab also completes the names of commands, and the options of many of them after `--`.

Up brings back earlier commands. On an empty line, **Up** goes back through what you typed, newest first; **Down** comes forward again. Type the start first, and Up offers only commands that begin with it: type `cat`, press Up, and `cat trips/japan/plan.txt` comes back. **Enter** runs it again.

Some commands are not kept, so Up skips them: a bare `ls`, `ll`, `cd`, `pwd`, `clear` and a few more, and anything typed with a space in front (§25.7).

Ctrl+R searches the whole history. Press **Ctrl+R**: a list of earlier commands opens, titled *history*. Type `plan`: the list narrows to the commands that match, the best match marked. **Enter** puts the chosen line at the prompt

— Enter again runs it — and **Esc** gives up. fzf matches loosely, letters in order rather than side by side; §18.2 has the marks that make it exact.

Alt+. repeats the last argument. After `mkdir -p trips/kyoto`, type `cd` and press **Alt+.**: the line becomes `cd trips/kyoto`.

Try this.

```
$ mkdir -p trips/kyoto
$ cd trips/kyoto
$ pwd
/home/you/practice/trips/kyoto
```

Type the second line as `cd` and **Alt+.** — not by hand. Then `cd ..`, **Ctrl+R**, type `kyoto`, **Enter** twice: you are back in `kyoto`. `cd` takes you home.

The full set — **Ctrl+T** to insert a path, **Alt+C** to jump into a folder, `**` and Tab, and the rest — is in §18.1.

11.9 foot, the terminal

Super+Return opens a terminal: the window that shows text and passes your keys to the program running in it, usually the shell (§18.1). Each one is a window of the foot *server* — one foot process, started at login, that draws every terminal window — which is why it appears at once. **Ctrl+Shift+N** inside a terminal opens another in the same folder. The prompt and its marks are described in §11.3.

Keys	Does
Ctrl+Shift+C · Ctrl+Shift+V	copy · paste. Selecting text with the mouse already copies it
Shift+PageUp / PageDown	scroll back through the last 20 000 lines (also the mouse wheel)
Ctrl+Shift+R	search the scrollback: type, then Ctrl+R for the older match, Ctrl+S for the newer; Ctrl+W widens the match by a word; Enter ends the search with the match selected; Esc cancels
Ctrl+Shift+O	URL mode: every link on screen gets a short label; type the label to open it in Brave; Esc leaves
Ctrl+Shift+Y	copy mode: every commit hash, IP address and file path on screen gets a label; type it and that text is on the clipboard — no careful selecting. Type the label in capitals and it is pasted where you are typing as well
Ctrl+Shift+Z / X	jump to the previous / next prompt
Ctrl+Shift+G	the last command's output in neovim (§11.3)
Ctrl+Shift+U	type a character by its Unicode number, in hexadecimal: Ctrl+Shift+U, `e9`, Enter types *é*. Nothing shows while you type the number; Esc gives up
Ctrl++ · Ctrl+- · Ctrl+0	larger · smaller · normal text (Ctrl+wheel too)
double · triple · four clicks	select a word · the text inside the quotes around it (the whole line where there are none) · the whole line
Ctrl+drag	select a rectangle, such as one column of `ll`'s output

Recommended. A terminal started as `footclient -a float` opens floating and centred, like the network window from quick settings, btop from the bar and the clipboard history; good for a quick look at something without disturbing the tiling. In programs that use the mouse themselves (btop, yazi, lazygit), hold **Shift** while dragging to select text anyway.

Tip: to keep a long output, press Ctrl+Shift+G and save it from neovim with `:w ~/Documents/output.txt`.

A notice when a long command ends

A command that runs 10 seconds or more sends a notice when it finishes — *done in 2m14s*, or *failed (1) after 40.3s*, with the folder it ran in — but only while its terminal is not the window you are looking at. Start an upgrade, go elsewhere, and hear when it is done.

Safe clipboard

Programs in the terminal may *put* text on the clipboard (neovim over SSH does that with OSC 52 — an *escape sequence*, an instruction a program prints to the terminal in place of text), but never *read* it: a file you `cat` or a server you `ssh` into cannot see what you copied (`[security] osc52=copy-enabled`).

Pitfalls.

- Every selection replaces the clipboard (`selection-target=clipboard`) and lands in the clipboard history (Super+V). Middle-click pastes the *primary* selection — a second, older clipboard that holds whatever was last selected — which foot itself no longer fills; use Ctrl+Shift+V.
- All Super+Return windows live in one server process. If it ever stops, they all close together; a new server starts within a second, but the windows do not come back. For something that must not be cut off — a long copy, an upgrade — start the plain **Foot** entry from the launcher, which runs as a process of its own.
- The server reads `foot.ini` once, when it starts: any change — the font, a key, the scrollback — reaches new windows only after the server restarts, which in practice means the next login (§39.2). The theme's colours are the exception: they switch in every open window.
- Right-to-left text is shown but not reordered (§39.2, known issues).

Read more: `man foot` and `man foot.ini` describe every key and option of the installed version; the project lives at codeberg.org/dnkl/foot.

11.10 When the terminal seems stuck

Sooner or later the prompt does not come back. Nothing is broken: a program is still running, or waiting for you. Find what you see in the first column:

You see	It means	Do
a program runs and does not end (`ping fedoraproject.org`, without `-c`)	it runs until told to stop	**Ctrl+C**
a page of text, with `:` or `(END)` at the bottom	a pager, `less`	**q**
a man page	the manual, in neovim	**q**
`~` down the left edge and a status line at the bottom	neovim, the editor	**Esc**, then `:q!` and **Enter** — leaves without saving
`>` at the start of a new line	the shell waits for the rest: a quote was opened and not closed	**Ctrl+C**, then type the line again
no prompt and no output	a program reads what you type (`cat` with no file)	**Ctrl+D** — the end of input — or **Ctrl+C**
`[1]+ Stopped`	**Ctrl+Z** paused the program	`fg` brings it back to the front
nothing moves, keys seem dead	**Ctrl+S** paused the output	**Ctrl+Q**

Ctrl+D on an empty prompt ends the shell, and the terminal window closes with it.

Try this. Stop a program and read its exit status.

```
$ sleep 60          # press Ctrl+C
^C
$ echo $?
130
```

`sleep 60` waits a minute; **Ctrl+C** ends it early, and the shell prints `^C`. The prompt shows the seal `× 130`, and `echo $?` prints the same number: 130 means *ended by Ctrl+C* (§26.2 explains the number). Pausing and resuming is in §26.3.

Three error messages cover most of what goes wrong:

```
$ pwdd
bash: pwdd: command not found
$ cat notes.txt
cat: notes.txt: No such file or directory
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

Message	Means	Try
`command not found`	no program has that name — a typo, or not installed	check the spelling; Tab completes command names
`No such file or directory`	nothing of that name in that place	`pwd` — are you where you think? `ls` — what is the name exactly? Case counts
`Permission denied`	it is there, but your user may not read or change it	leave it: most such files are the system's (ch. 24 explains, and `sudo`)

Two commands say the same in their own words. For a name that is not there, `ls` (eza) prints `"notes.txt": No such file or directory (os error 2)`, and `cd` (zoxide) prints `zoxide: no match found`.

What you learnt

- `pwd` names the folder you are in; `cd` changes it — alone, it goes home; `..` is up, `-` is back.
- `ls` and `ll` list; `mkdir -p`, `touch`, `cp` and `mv` make, copy, move and rename; `cat` and `less` read.
- `rm` removes for good, past the trash; `gio trash` keeps a way back.
- A command is a program, its options (`-l`, `--sort=size`) and its arguments; quotes keep a name with spaces together.
- Tab completes, Up recalls, Ctrl+R searches; `tldr`, `man` and `--help` explain.
- Ctrl+C stops a program; `q` leaves a pager or a man page.

Check yourself

1. Which command names the folder you are in? → *`pwd`*
2. What does `-p` add to `mkdir`? → *no complaint if the folder exists, and missing parent folders are made too*
3. A page of text fills the terminal and the prompt is gone. How do you get out? → *`q` — and if that does nothing, Ctrl+C*

12. Copy, paste and screenshots

> **In this chapter**
> - copy and paste between any two programs, the terminal included
> - bring back something you copied earlier
> - take a screenshot of a region or the whole screen, and mark it up
>
> **You need:** ch. 11, and §11.9 for the terminal's keys.

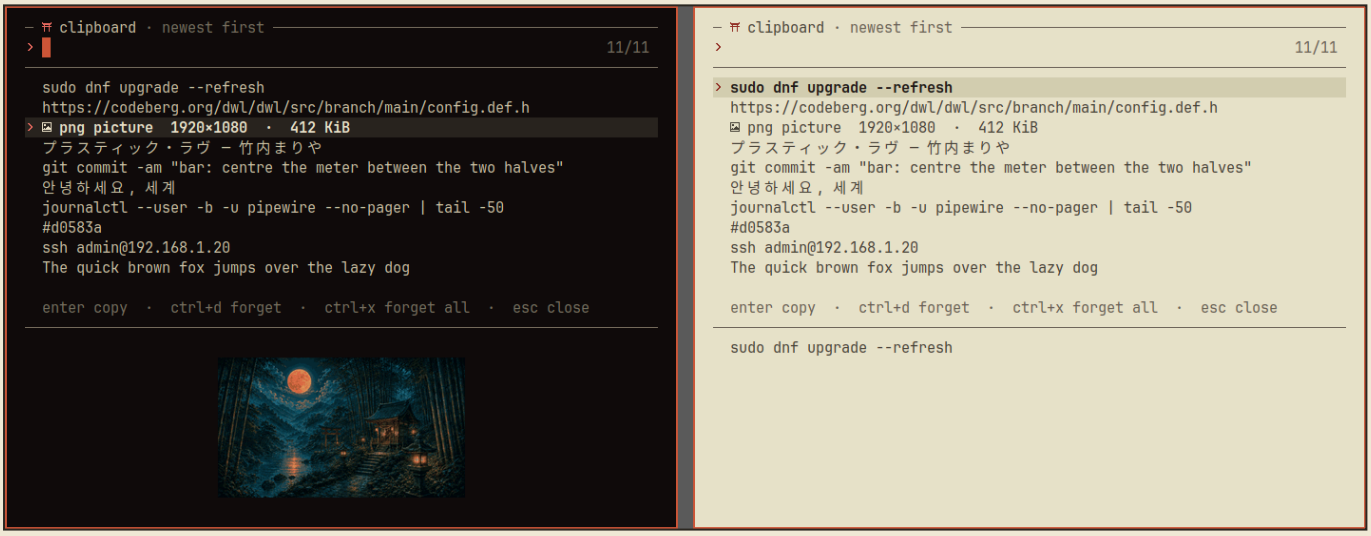
Copying is the glue between programs. Here it works a little better than you may be used to: a copy survives its program closing, and every copy is kept in a history.

12.1 Copy and paste

Three keys cover it:

Where	Copy	Paste
every program	**Ctrl+C**	**Ctrl+V**
the terminal	**Ctrl+Shift+C** — or select the text	**Ctrl+Shift+V**
anything copied earlier	—	**Super+V**, choose it, **Enter**; then paste as usual

The terminal needs Shift because **Ctrl+C** there stops the running program (§11.10).



The clipboard history (Super+V) on both themes: newest first, the full text or the picture itself underneath.

> **On Windows. Win+V** is **Super+V** here: the clipboard history, pictures included. In Windows Terminal, too, Ctrl+C stops a program and copying needs Ctrl+Shift+C.

Try this. Copy from the terminal, without the mouse.

```
$ echo 'Sharjah, 34 °C' | wl-copy
$ wl-paste
Sharjah, 34 °C
```

`wl-copy` puts what it is given on the clipboard; `wl-paste` prints what the clipboard holds. Press **Super+V**: the line is at the top of the history. Press **Esc**, click into Brave's address bar and press **Ctrl+V**: there it is.

Copying works the same way in every program: Ctrl+C / Ctrl+V (Ctrl+Shift+C / V in the terminal), and selecting text in the terminal copies it as well. Two things make it dependable:

- **A copy outlives its program.** Closing the program you copied from does not empty the clipboard, as it would on a plain Wayland desktop. Copies a password manager marks as secret are never kept, nor copies over 64 MiB, nor the primary selection (select + middle-click).
- **Between Wayland and X11 programs** (Steam, older games) the compositor passes copies both ways by itself. (*X11* is the older display system Wayland replaces; programs written only for it run through *XWayland*, a compatibility layer dwl starts when one needs it.)

How it works: the promise. On Wayland a copy is only a promise — the program that copied hands the data over when something pastes — so normally, closing it empties the clipboard. Here `kawa-clipkeep` (C, in `src/kawa-clipkeep/`, built by **Theme**) reads every format of each copy the moment it is made, and when the clipboard is then emptied because its owner closed, it offers the same copy again, in all its formats. While the program that copied is open nothing changes: it keeps serving its own copy. One limit, from Wayland itself: clearing the clipboard on purpose while the program that copied is still open looks exactly like that program closing, so the copy comes back once — clear it again, or close that program first.

Two commands show what the clipboard holds right now:

```
wl-paste --list-types    # the formats the current copy is offered in, one per line
wl-paste                 # the copy itself, as text
```

Super+V opens the clipboard history in a small floating window: the last copies, newest first, pictures included. Type to search; ↑/↓ or the mouse move; the full text — or the picture itself — shows underneath. A copy that is neither text nor a picture is listed as *data · not text*; its kind and size show underneath.

Keys	Does
Enter	put that entry back on the clipboard and close; then paste as usual
Ctrl+D	forget that entry
Ctrl+X	forget them all (it asks first; the clipboard itself stays)
Esc	close, the clipboard exactly as it was

The history is cliphist's (`~/.cache/cliphist`), fed by `wl-paste --watch` from the moment you log in; it keeps the last 750 copies. Its window uses the terminal's own colours, so it follows dark and light like foot.

Tip: copied a password by hand from somewhere that does not mark it secret? Open Super+V and press Ctrl+D on it, so it does not stay in the history.

12.2 Screenshots

Three keys, one for each kind of picture:

Press	Takes	Then
Print	a region: drag a box with the mouse; **Esc** gives up	saved and copied
Shift+Print	the whole screen — every monitor, in one picture	saved and copied
Super+Print	a region, opened in satty to mark it up	**Enter** copies, **Ctrl+S** saves (§12.3)

While you drag, everything outside the box is dimmed and the box is edged in vermilion. Print and Shift+Print do two things at once: they save the picture in `~/Pictures/Screenshots/`, and put it on the clipboard, ready to paste. A notice, *Screenshot copied*, shows a small square cut from the picture and the file's name.

Each file is named after the moment it was taken, `2026-10-16_09-12-03.png`; two in the same second get `-2`, `-3` after the time. The folder is set as `SCREENSHOT_DIR` in `~/.config/fedokawaii/env`.

> **On Windows. Win+Shift+S** (the Snipping Tool's region) is **Print** here, and **Win+PrtScn** (the whole screen, saved) is **Shift+Print**. Both save *and* copy, so there is nothing to choose afterwards.

Try this. Press **Print** and drag a box round the clock in the bar. Read the notice, then look for the file:

```
$ ls ~/Pictures/Screenshots
2026-10-16_09-12-03.png
```

(for example)

Then open a chat or a mail in Brave and press **Ctrl+V**: the picture is pasted.

12.3 Marking up a screenshot: satty

Super+Print: drag a region (the selection frame is in the theme's vermilion), and satty opens with it, the arrow tool ready (Print and Shift+Print in §7.2 save and copy without annotating).

Keys	Does
z · i · r · e	arrow · line · rectangle · ellipse
t	text: type, Shift+Enter for a new line, Enter to place it, Esc to drop it
m	numbered markers (1, 2, 3 … with each click)
g · u · b	highlighter (hold Ctrl for freehand) · blur · brush
c · p	crop · pointer (select and move)
1 … 5	colour: vermilion, yellow, green, indigo, paper
Ctrl+Z · Ctrl+Y	undo · redo
Enter	**copy to the clipboard and close**
Ctrl+S	**save to `~/Pictures/Screenshots/` and close**
Ctrl+Shift+S	save under a name you choose, and close
Esc	close without keeping anything

Pitfall. Satty closes after the first copy or save (`early-exit`). Enter copies but does **not** save a file — unlike Print and Shift+Print, which do both. Pick the one you need. The copy stays on the clipboard after satty closes (§12.1). If satty is not installed (it is fetched from GitHub, so an install made offline may lack it), Super+Print saves and copies a plain shot, and its notice says so (*not annotated: satty is missing*). Once online, **Super+U** → *repair* fetches it.

Tip: screenshots pile up; `fd -e png --changed-before 30d . ~/Pictures/Screenshots` lists those older than a month, and adding `-X rm` deletes them.

Read more: Satty's README, for version 0.22, the one installed here.

12.4 Recipe: take, annotate and paste a screenshot

1. **Super+Print**, drag a region.
2. In satty, the arrow is ready: drag to draw. **t** for text, **m** for numbered markers, **1…5** for colours, Ctrl+Z to undo.
3. Press **Enter**: the picture is copied and satty closes. (Ctrl+S saves it to `~/Pictures/Screenshots/` instead.)
4. Paste with **Ctrl+V** in Brave, a chat, or Claude Code. **Super+V** brings back earlier copies, pictures included.

For a plain shot, **Print** (region) or **Shift+Print** (whole screen) both saves and copies.

What you learnt

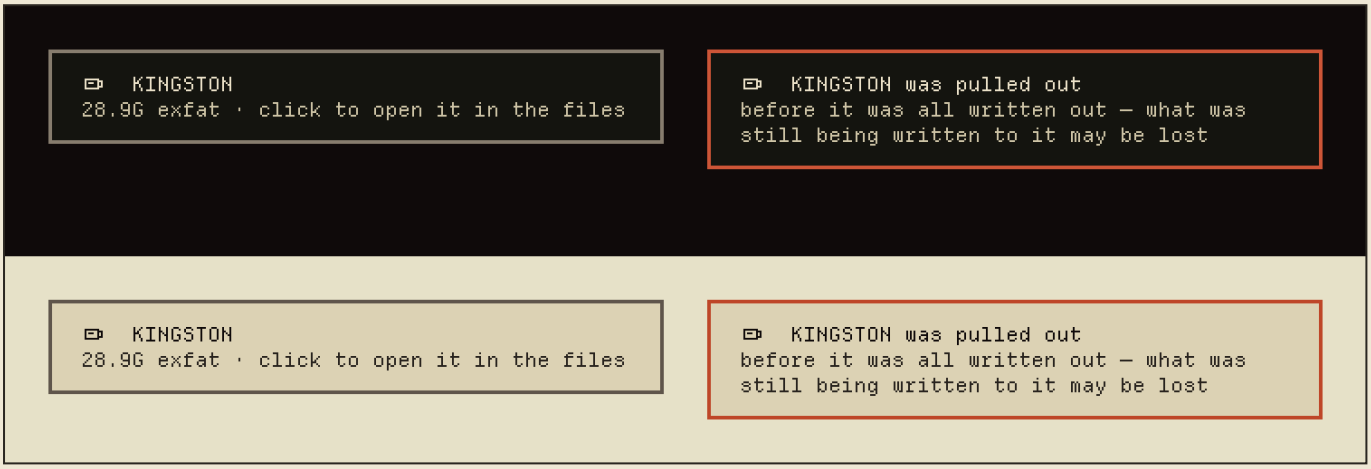
- **Ctrl+C** / **Ctrl+V** everywhere; **Ctrl+Shift+C** / **V** in the terminal, where selecting also copies.
- A copy outlives the program it came from, and **Super+V** brings back earlier ones.
- **Print** takes a region, **Shift+Print** the whole screen: saved in `~/Pictures/Screenshots` and copied.
- **Super+Print** opens satty: **Enter** copies, **Ctrl+S** saves.

Check yourself

1. Why does Ctrl+C not copy in the terminal? → *there it stops the running program; copy with Ctrl+Shift+C, or by selecting*
2. Where do Print's screenshots go? → *into `~/Pictures/Screenshots`, and onto the clipboard*
3. You pressed Enter in satty. Is there a file? → *no: Enter only copies; Ctrl+S saves*

13. USB sticks, disks and SD cards

> **In this chapter**
> - open a stick, a disk or an SD card, and find its files
> - eject safely, and know what to do when a drive is *in use*
> - reach drives from yazi, any file dialog or a terminal
> - check, rename or erase a stick
>
> **You need:** ch. 10 (yazi) and ch. 11.



A stick that opened (click the notice for its files), then eject's "safe to unplug" — dark and light.

Plug a drive in and it opens by itself: a notice names it, the bar shows the drive mark, and its files are one click away. **Eject** before you pull it out; the notice **safe to unplug** says when you can.

13.1 What happens when you plug one in

A few words first. A disk is divided into **partitions**, and each partition holds a **file system**. You met both, and *mount*, while installing (§4.2, §3.5, §5.1); here is what they mean for a stick:

Terms.

Term	Plain meaning
partition	a separate part of a disk with a file system of its own: a stick usually has one, a Windows disk several. `sdb` is a disk, `sdb1` its first partition
file system	how a drive keeps files: *exFAT* (most sticks and SD cards), *FAT32* (small or old ones), *NTFS* (Windows disks), *ext4* or *Btrfs* (Linux disks)
mount · mount point	to open a file system so its files show in a folder · that folder: `/run/media/YOU/LABEL` (a file system with no label: its UUID instead)
label	the name a file system carries (`KINGSTON`, `CANON EOS`); a drive with none is shown by its maker and model (a card in the machine's own reader: *SD card*)
eject	write out what is still on its way to the drive, close every file system on it, then switch the drive off — or, when it says its media come out (a card in a reader, and many sticks say so of themselves), eject the media. Either way it is safe to unplug
udisks · udiskie	the system service that mounts drives for the person at the screen, without a password · the small program the session runs that asks it to mount whatever you plug in
LUKS	Linux's standard disk encryption: an encrypted drive opens only with its passphrase

What happens, step by step:

You	You see
plug a stick in	a notice: **KINGSTON** — *28.9G vfat · click to open it in the files*; the drive mark in the bar
click the notice	yazi opens on the stick, in a window of its own
plug in an encrypted drive	a small box asks for its passphrase, your typing masked; **Esc** leaves it locked
click the drive mark (or quick settings → **drives**)	the drives menu: each file system with its size, *open* or *closed*, then **eject** for each drive
a drive that will not open (a file system Linux cannot read, a wrong passphrase)	a notice under the drive's name: *not opened*, with the reason

The notice stays 8 seconds; after that the drive mark is the way in. The mark sits in the bar only while a drive is plugged in: quiet while every drive is closed, lit once one is open, and with a number beside it for two drives or more.



The drives menu, from the bar's drive mark (the 2: two drives plugged in): one open, one closed, each to eject.

The **drives menu**, titled *drives*, has a line for each file system — its name, its size, *open* or *closed* — then one *eject …* line for each drive. Choose a file system and it opens, with yazi on it; choose *eject* and the drive is ejected (next section). With nothing plugged in, it says *nothing plugged in*.

> **On Windows.** Windows gives a stick a letter — `E:` — and an AutoPlay box. Here it becomes a folder, `/run/media/you/KINGSTON`, and the notice offers its files. *Safely Remove Hardware* is *eject*.

Try this. Plug in a USB stick. Read the notice — the stick's name, size and file system — and click it: yazi opens on the stick. Press **q**. Now click the drive mark in the bar: the menu lists the stick as *open*. Press **Esc**; ejecting is next.

13.2 Ejecting

Eject from wherever you are:

- the drives menu (the drive mark, or quick settings → *drives*): *eject KINGSTON*;
- yazi, or any file dialog: **M**, then **e** on the drive (§13.3);
- a terminal: `kawa-drives eject /dev/sdb1` (§13.5).

Eject first **writes everything out**: copies that looked finished may still be on their way to a slow stick. If that takes more than a moment, a notice says *writing what is still on its way to it — wait for "safe to unplug"*, and stays. Then eject closes every file system on the drive (an encrypted one is locked again) and switches the drive off, or ejects the card from its reader. Last comes the notice that matters, under the drive's name: **safe to unplug**. A disk image you opened (§13.4) says **closed** instead: its file is free to move or delete.

> **Careful.** Pull a drive out only after **safe to unplug**. Until then, files you just copied may be only half on it, even though the copy looked finished.

When eject cannot finish, the notice says why, and the drive stays open and safe:

The notice	It means	Do
KINGSTON is in use — *still open in: bash — close it there, then eject again*	a program of yours has a file or its folder on the drive, and the notice names it: `bash` is a terminal standing in a folder there	`cd` out of the drive (or close that terminal, or the named program), then eject again
KINGSTON is still being written to — *eject it again in a moment*	the system gave up waiting for a slow write	wait a little, then eject again
KINGSTON was pulled out while open (critical)	the stick went before it was ejected	read it — *what was still being written to it may be lost* — and click it away

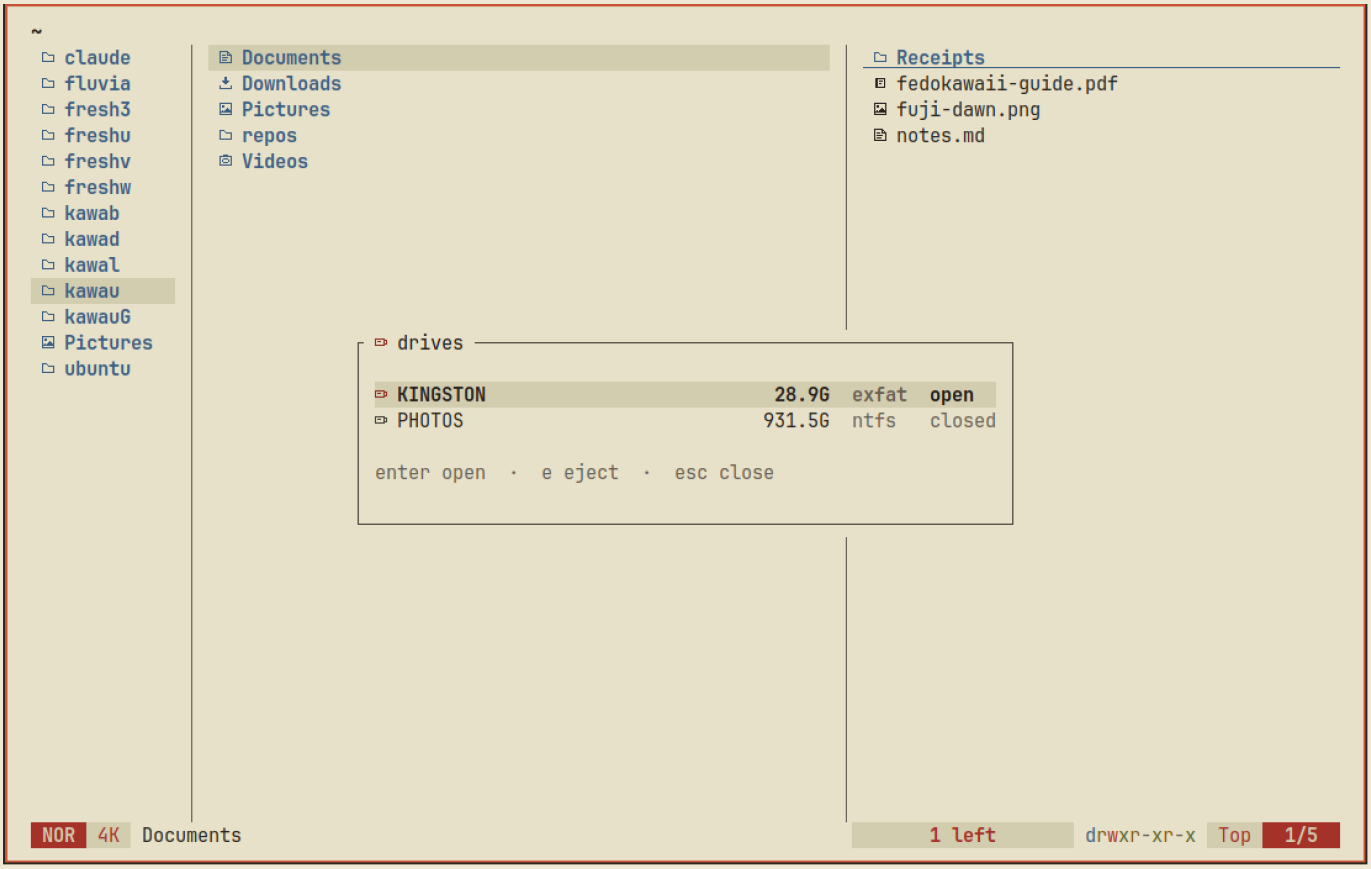
The last one has a vermilion border and stays until you click it, with a reminder: *eject first next time (the bar's drive mark, or M in yazi)*. A drive pulled out while eject was still writing gets a critical notice of its own: **KINGSTON was pulled out** — *before it was all written out*.

Try this. Eject the stick from the menu: click the drive mark, choose *eject KINGSTON* (your stick's name). Wait for **safe to unplug**; then pull it out. The drive mark leaves the bar.

13.3 In yazi and every file dialog

In yazi and in every file dialog (uploads in Brave included), **M** shows the drives:

Key	Does
M	the drives: each with its size, file system, open or closed
j k · ↑ ↓	move through the list
Enter · l · →	open it (mounted first if need be; an encrypted one asks for its passphrase) and go there
e	eject its drive: yazi steps out of it first, the list closes, and a notice says when it is safe to unplug. Standing in a drive, the list opens on it — **M** then **e** ejects the drive you are looking at
Esc · q · M	close the list
g m	go to `/run/media/YOU`, where every open drive is (no drive open: a note says so)



M in yazi: the drives, the open one lit; Enter goes there, e ejects it.

The list is a small box titled *drives*: the open drives' icons lit, the closed ones grey, and along the bottom *enter open · e eject · esc close*. With nothing plugged in it says *nothing plugged in*; **g m** with no drive open says *no drive is open (plug one in, or M)*.

So to attach a photo from an SD card in Brave: **Upload** → the dialog opens → **M** → Enter on the card → Enter on the photo.

13.4 Encrypted drives and disk images

An encrypted drive (LUKS) asks for its passphrase as soon as it is plugged in: a small box with the drive's name, *passphrase*, and your typing masked. **Enter** opens it; **Esc** leaves it locked, and it waits in the drives menu and under **M** as *closed* — opening it there asks again. A wrong passphrase gives the *not opened* notice. Eject locks it before switching the drive off.

A disk image — an `.iso`, an `.img` — is a whole drive kept in one file. It opens the same way as a stick, once you tell the system to treat the file as a drive:

```
udisksctl loop-setup -f FILE.iso   # the file becomes a drive of its own, and opens
```

It opens with a notice, and the drives menu and **M** list it under its label (an image with none: under its file's name); eject closes it and frees the file, with the notice **closed**.

13.5 From a terminal

The same things, by hand:

```
lsblk -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINTS     # every disk and what is on it; sdb, sdc… are drives
kawa-drives list                                # only the drives: a line per file system
udisksctl mount -b /dev/sdb1                    # open one → "Mounted /dev/sdb1 at /run/media/…"
kawa-drives eject /dev/sdb1                     # write out, close and switch off that drive
```

Try this. With a stick plugged in and open, draw the map:

```
$ lsblk -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINTS
NAME          SIZE FSTYPE LABEL    MOUNTPOINTS
nvme0n1     931.5G
├─nvme0n1p1 …
…
sdb          28.9G
└─sdb1       28.9G vfat   KINGSTON /run/media/you/KINGSTON
```

(for example; shortened)

`lsblk` is the map: a whole disk (`sdb`), its partitions (`sdb1`), and for each its size, file system, label and where it is open. The system's own disk is `nvme0n1` (or `sda` on SATA); the drives menu never lists it, and nothing on the machine's own disks opens by itself — a Windows partition or a second internal drive stays closed until you open it (`udisksctl mount -b …`).

13.6 Looking after a drive

The tools for the usual file systems are installed. Each works on a *closed* file system: close it with `udisksctl unmount -b /dev/sdb1`, and the drive stays plugged in and on. (Ejecting does not help here: the drive is switched off, and plugged back in it opens again by itself.) Read the name with `lsblk` twice: these act on whatever you name.

> **Careful.** `mkfs` erases the whole file system you name, at once. Name the wrong one — `nvme0n1p3` instead of `sdb1` — and your own files are gone. Check with `lsblk` first.

To	exFAT	FAT32
check and repair	`sudo fsck.exfat /dev/sdb1`	`sudo fsck.vfat -a /dev/sdb1`
rename (the label)	`sudo exfatlabel /dev/sdb1 TRIP`	`sudo fatlabel /dev/sdb1 TRIP`
erase and start over	`sudo mkfs.exfat -L TRIP /dev/sdb1` — **everything on it is gone**	`sudo mkfs.vfat -F 32 -n TRIP /dev/sdb1` — the same

An NTFS disk is Windows' own: check and repair it there (`chkdsk`); here it opens as it is.

exFAT is the one to choose for a stick shared with Windows, macOS, cameras and TVs: every one of them reads and writes it, and a file may be larger than 4 GB, which FAT32 refuses.

13.7 Pitfalls, and where it lives

Pitfalls.

- An NTFS disk from a Windows PC that was not fully shut down — *Fast Startup* or hibernation leaves it marked as in use — opens read-only or not at all. Shut Windows down with **Shift** held on *Shut down*, or turn Fast Startup off in Windows' power settings.
- "still open in: bash" (eject's notice) means a terminal stands in a folder on the drive: `cd` out of it (or close that terminal), then eject.
- A drive with nothing Linux can read on it — brand new, or a file system it does not know — shows no mark and opens nothing; `lsblk` still lists it.

What	Where
what is mounted, and how	udiskie: `~/.config/udiskie/config.yml` (started by the session)
the notices, the mark, the menu, eject, the passphrase box	`~/.local/bin/kawa-drives` (`kawa-drives --help`)
the drives in yazi	`~/.config/yazi/plugins/drives.yazi/`, the keys in `~/.config/yazi/keymap.toml` (the file dialog's: `~/.config/fedokawaii/yazi-pick/keymap.toml`)
where drives open	`/run/media/YOU/LABEL`

How udisks lets you mount without a password is §32.6.

Read more: udisks · udiskie · `man lsblk` · `man udisksctl`.

13.8 Part II, looking back

A week ago this was a blank machine. Now you can do everything below without looking anything up — and when you cannot, **Super**+/ and `tldr` are one key away.

You can	With	Taught in
log in, lock, log out and shut down	the login prompt · **Super+Escape** · **Super+Shift+E**	ch. 6
open, arrange and close windows	**Super+Return**, **Super+Space** · **Super+J/K**, **Super+Shift+Return** · **Super+Q**	ch. 7
keep each job on its own workspace	**Super+1…9** · **Super+Shift+1…9**	§7.3
change settings, get online, pair headphones	**Super+X** · **Super+A**	ch. 8
switch the theme and the wallpaper	**Super+Shift+W** · **Super+W**	ch. 9
find, copy, rename and restore files	yazi, **Super+E**	ch. 10
move around and handle files by typing	`pwd`, `ls`, `cd`, `mkdir`, `cp`, `mv`, `rm`, `less`	ch. 11
copy, paste and take screenshots	**Ctrl+C/V**, **Super+V** · **Print**	ch. 12
use and eject drives	the drive mark · **M** in yazi	this chapter

Part III turns to the programs you will use every day, starting with the launcher and the browser (ch. 14).

What you learnt

- A drive opens by itself: click its notice for its files; the bar's drive mark opens the drives menu.
- Eject before you pull a drive out, and wait for *safe to unplug*.
- *still open in: …* names the program that holds the drive: close it (or `cd` out), then eject again.
- In yazi and every file dialog, **M** lists the drives and **g m** goes where they open.
- `lsblk` maps disks and partitions; read the name twice before `mkfs`.

Check yourself

1. A copy to a stick has just finished. Can you pull the stick out? → *not yet: eject it, and wait for "safe to unplug"*
2. Eject says *still open in: bash*. What does that mean? → *a terminal stands in a folder on the drive: `cd` out of it (or close it), then eject again*
3. Which file system suits a stick shared with Windows and a TV? → *exFAT*

Part III — Everyday work

You can find your way round the desktop now, and you have typed your first commands. Part III is the programs you will use every day, from the gentlest — the launcher and the browser — to the most rewarding: neovim, the shell's helpers, and the do-it-yourself recipes that need both. Start with ch. 14, and give ch. 17 (neovim) a slow week of its own.

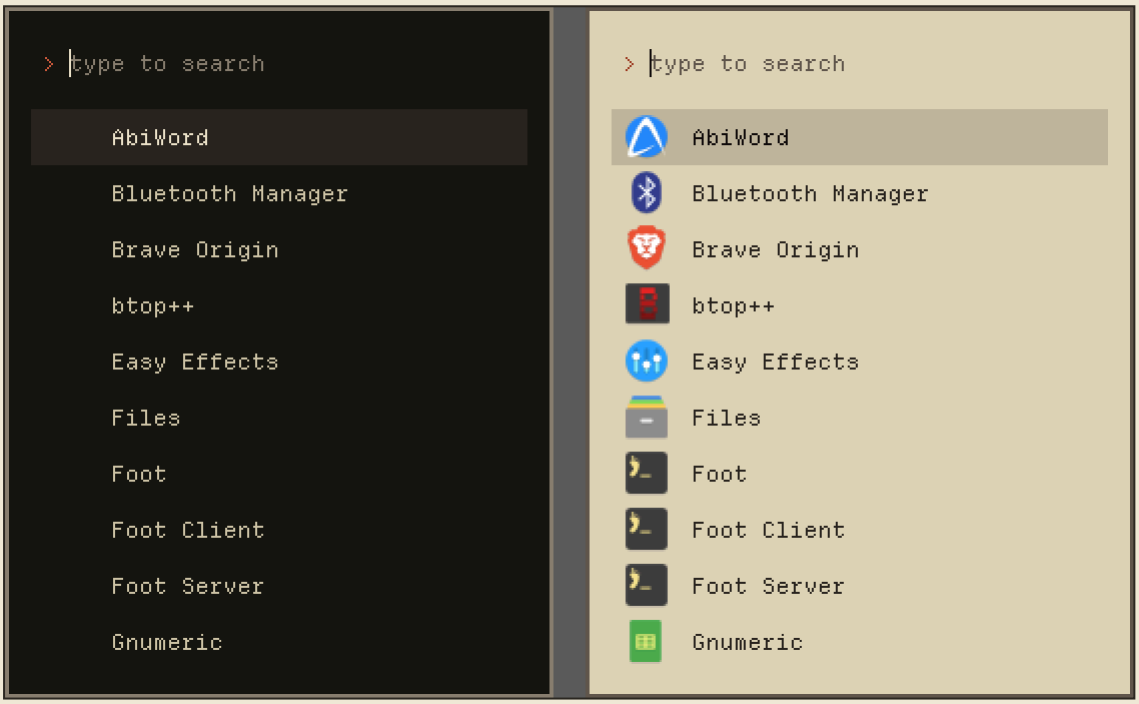
Each program gets the same treatment: what it is for here, how you open it, one thing to try at once, the handful of things worth knowing, and where it trips people up. Keys are given as this desktop sets them, or as the defaults of the version installed here — and each program documents itself for that version: `man fuzzel`, `~` *in yazi,* `:help` *in neovim, and* `tldr` *and* `man` *for any command (§11.7).*

14. Programs and the web

> **In this chapter**
> - start any program by typing part of its name
> - browse with Brave, and upload a file from it
> - share a screen, a window or a tab in a call, with the right sound
>
> **You need:** §6.3 (the first six keys) and §10.4 (the file dialog).

The launcher and the browser are the two programs you will open most. Neither needs much learning: this chapter shows the few things this desktop changes in them, and the one setting in a call that decides what the others hear.

14.1 The launcher: fuzzel



The launcher (Super+Space): Papirus icons and names, on both themes.

Super+Space, or the torii key at the left of the bar. Type part of a name, Enter starts it, Esc closes.

> **On Windows.** This is the Start menu's search box: press the key, type, Enter. It looks at program names only — no files, no settings, no web results.

Try this. Start the file manager through the launcher, close it, and see the launcher remember it.

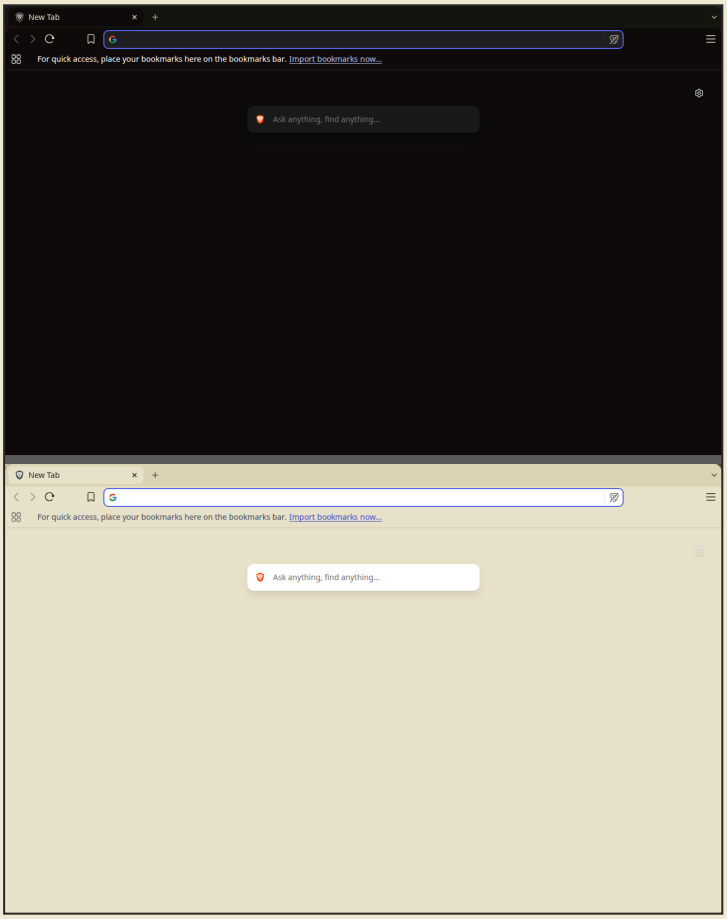
1. Press **Super+Space**. A small panel opens in the middle of the screen: `>` and *type to search* on its first line, programs below.
2. Type `fil`. The list shrinks to the names that match, **Files** among them; move to it with ↓ if it is not the first.
3. Press **Enter**. **Files** is yazi, the file manager (§10.3), in a terminal window.
4. Press **Super+Q**: the window closes.
5. Press **Super+Space** again and look before you type: **Files** has moved up the list. **Esc** closes the launcher.
6. It matches application **names only**: *brave* finds Brave Origin, *browser* finds nothing.
7. The programs you start most often move to the top (the counts live in `~/.cache/fuzzel`).

8. Terminal programs with a launcher entry, such as Neovim and btop, open in a terminal window. **Files** (yazi) and **Torrents** (torrra) are entries of this desktop's own.
9. The other tools installed as release binaries — presenterm, spotify_player, satty — have no launcher entry; start them in a terminal.

Keys	Does
Up · Down (or Ctrl+P · Ctrl+N)	move in the list
Enter	start the highlighted program
Tab	start it when only one match is left; otherwise move down the list
Shift+Delete	forget how often you started the highlighted program, so it loses its place near the top
Esc	close

Read more: `man fuzzel` and `man fuzzel.ini`; codeberg.org/dnkl/fuzzel.

14.2 Brave Origin, the browser



Brave Origin's new-tab page on both themes: the tab strip and toolbar in the palette, no window buttons; the address bar, focused, keeps Brave's own grey (white on paper) and blue ring.

Super+B. Brave Origin is Brave with Rewards, Wallet, VPN, Leo, News and the Tor windows removed from the build — Talk, Playlist, Speedreader, the Web Discovery Project and the usage and crash reports go too; the Shields (ad and tracker blocking) remain. It is free on Linux and the only browser here (`verify` fails if another appears).

Try this. Open a tab, lose it, and get it back.

1. **Super+B** opens Brave. **Ctrl+T** opens a new tab with the cursor in the address bar: type `fedoraproject.org` and press Enter.

2. **Ctrl+W** closes that tab. **Ctrl+Shift+T** brings it back, on the same page.
3. **Super+Q** closes the whole window, every tab in it — the one key here to treat with care.

The keys table below has the rest; the two short sections before it are what this desktop changes in Brave, and what to look at once.

What this desktop sets up

Very little, on purpose:

- **The desktop's colours.** Brave runs in its GTK mode (Theme sets it in the profile while Brave is closed): the tab strip, toolbar, menus and dialogs take the palette from the GTK theme, the new-tab page is the theme's ground, and both follow a theme switch live. Web pages that offer a dark version follow the portal's dark / light. The address bar and Brave's own pages (settings, Shields) keep Brave's greys.
- **No window buttons** — the tab strip ends at the tab list; Super+Q closes the window.
- **File dialogs.** Upload buttons and *Save as* open the desktop's file dialog, served by the portal (§10.4).
- **Screen sharing** goes through the portal and the *share what?* menu; a tab's sound always travels, and *Also share system audio* is turned on for screens and windows (§14.4).
- **No flags file** — Chromium 140 and later choose Wayland by themselves. Whether videos are decoded by the GPU is Brave's own default; `brave://gpu` → *Video Decode* shows it.
- Links opened elsewhere — foot's URL mode, lazygit, spotify_player's sign-in — open in Brave, and magnet links clicked in Brave open in torrra (§20).

Worth checking once

These live in Brave's own settings, are not managed here, and their names can move between versions:

- *Settings → Shields*: the defaults for blocking trackers and ads and for fingerprinting. The lion icon in the address bar changes them for one site — the first thing to try when a site misbehaves.
- *Settings → Privacy and security*: the data-collection switches. Origin removes most reporting; look rather than assume.
- *Settings → Search engine*, the password manager and sync, to taste.

Keys	Does
Ctrl+T · Ctrl+W · Ctrl+Shift+T	new tab · close tab · reopen the last closed tab
Ctrl+L · Ctrl+Tab · Ctrl+1…8 · Ctrl+9	address bar · next tab · that tab · the last tab
Ctrl+Shift+N	private window
Ctrl+J · Ctrl+H · Ctrl+F	downloads · history · find in page
F11 or Super+F	fullscreen

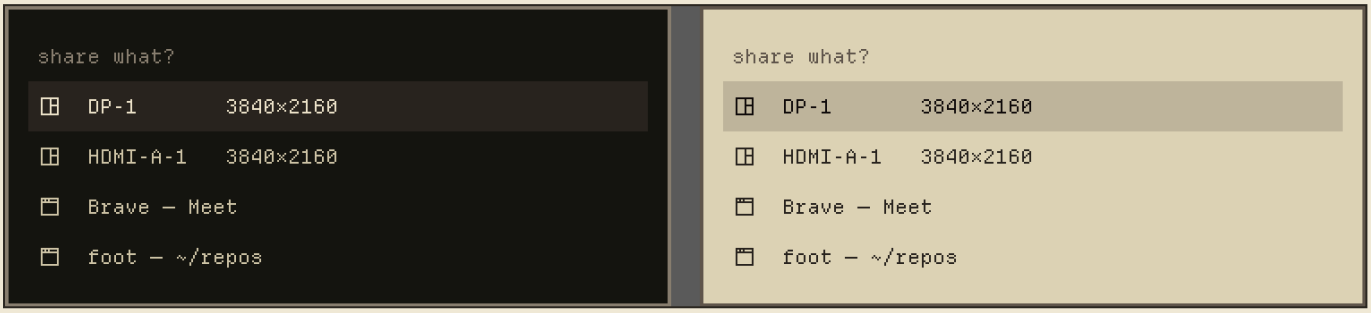
Pitfalls. Super+Q closes the whole window with all its tabs; Ctrl+W closes one, and Ctrl+Shift+T brings tabs back. Choosing a theme from the Chrome Web Store replaces GTK mode (Theme then leaves it be); *Settings → Appearance → Theme → GTK* brings the desktop's colours back.

14.3 Recipe: upload a file from Brave

1. In Brave, click the site's upload button. The file dialog opens: yazi, floating, in the folder you last uploaded from (§10.4).
2. Find the file: ↑ ↓ and → ← through the folders — or **z** and a few letters of its name to jump to it below this folder, **Z** and a word to jump to a folder you use often, **g d** for Downloads.
3. Press **Enter** on it. For several files (where the site takes several), **Space** on each, then Enter. **Esc** cancels.

14.4 Screen sharing — picture and sound

A call in Brave (or any program) that shares a screen asks the desktop portal, which asks what to share with the **share what?** menu: every screen with its size, and every open window by its title (a window without one is listed as *untitled window* and the start of its id). With only one screen and no choice to make, it is taken without asking. Sharing runs at up to 60 frames a second. While anything is shared, notifications are held back so they never appear in the call — critical ones and the level display still show.



"share what?" — two screens with their sizes, and the open windows by title.

Sound has three ways in, depending on what the call needs:

You want the others to hear	Do
the sound of a **Brave tab** (a video you are showing)	share the tab and keep *Also share tab audio* ticked — Brave captures it itself
the **computer's sound** with a whole screen or a window	tick *Also share system audio* in Brave's picker (Theme turned this Brave feature on). It carries everything you hear — including the call itself, so others may hear an echo of their own voices. Headphones do not help; use it for a moment, or use the next row instead
only certain programs (a game, a player), in any call program	in pavucontrol (audio menu → mixer → *Playback*) move those programs to the **Share** output — you still hear them — then pick **Share (as microphone)** as the microphone, or as a second audio input, in the call program. Only what you moved there is sent, never the call

> **Word — PipeWire.** The sound server: the program that carries every other program's sound to the speakers or headphones — and the pictures of a screen share to the call.

Share is a pair of PipeWire devices of this desktop's own, set up in `~/.config/pipewire/pipewire.conf.d/60-share.conf`: an output you can send programs to, and *Share (as microphone)*, which a call can record from. `wpctl status` lists both under *Filters* (WirePlumber shows loopbacks there).

Your microphone itself travels as usual; check it in the audio menu (Super+A → input).

14.5 Recipe: share your screen, with sound, in a call

1. In the call, choose to share. In Brave's picker:
   - a **tab** — its sound comes along when *Also share tab audio* is ticked;
   - a **window** or the **entire screen** — tick *Also share system audio* to send what you hear (it includes the call itself; see §14.4 for the quieter way).
2. For a window or a screen, the desktop's *share what?* menu appears; pick one. With only one choice it is taken without asking.
3. Notifications are held back while you share; critical ones still show.
4. Your microphone travels as usual; check it in the audio menu (Super+A → input).

What you learnt

- **Super+Space** starts a program by part of its name; it matches names only.
- **Super+B** opens Brave. **Ctrl+W** closes a tab and **Ctrl+Shift+T** brings it back; **Super+Q** closes the whole window.
- An upload button opens yazi as the file dialog: **Enter** picks a file, **Esc** cancels.
- A shared tab brings its own sound; a screen or window needs *Also share system audio*, or the **Share** output for chosen programs only.

Check yourself

1. You type `browser` in the launcher and nothing appears. Why? → *it matches program names only: type `brave`*
2. Which key closes one tab, and which closes them all? → *Ctrl+W one tab; Super+Q the whole window*
3. How can a call hear a game but not the call itself? → *move the game to Share in pavucontrol, and choose Share (as microphone) in the call*

15. Pictures, video, music and sound

> **In this chapter**
> - look through pictures, and play any video file or web link
> - keep a copy of a web video, or only its sound
> - play Spotify in a terminal, and pause any player with the media keys
> - send each program's sound where you want it, and clean your microphone
>
> **You need:** §10.3 (yazi) and §8.1 (the audio menu).

Every program in this chapter opens from yazi with **Enter** on a file: the desktop knows which one fits each kind (§10.5). What follows is what to press once it is open, and how sound finds its way to the right speaker.

15.1 Pictures: imv

Open a picture from yazi or Brave's downloads (imv is the default for pictures), or type `imv picture.jpg`, `imv folder/` (every picture in it) or `imv -r folder/` (subfolders too). It opens **floating**, on the theme's ground colour.

Try this. Look through the screenshots you took in ch. 12.

```
imv ~/Pictures/Screenshots/      # every picture in the folder
```

The first picture opens in a floating window. **→** shows the next one, **+** zooms in, **a** shows it at its actual size, **r** fits it to the window again, and **q** closes imv: the terminal is yours again.

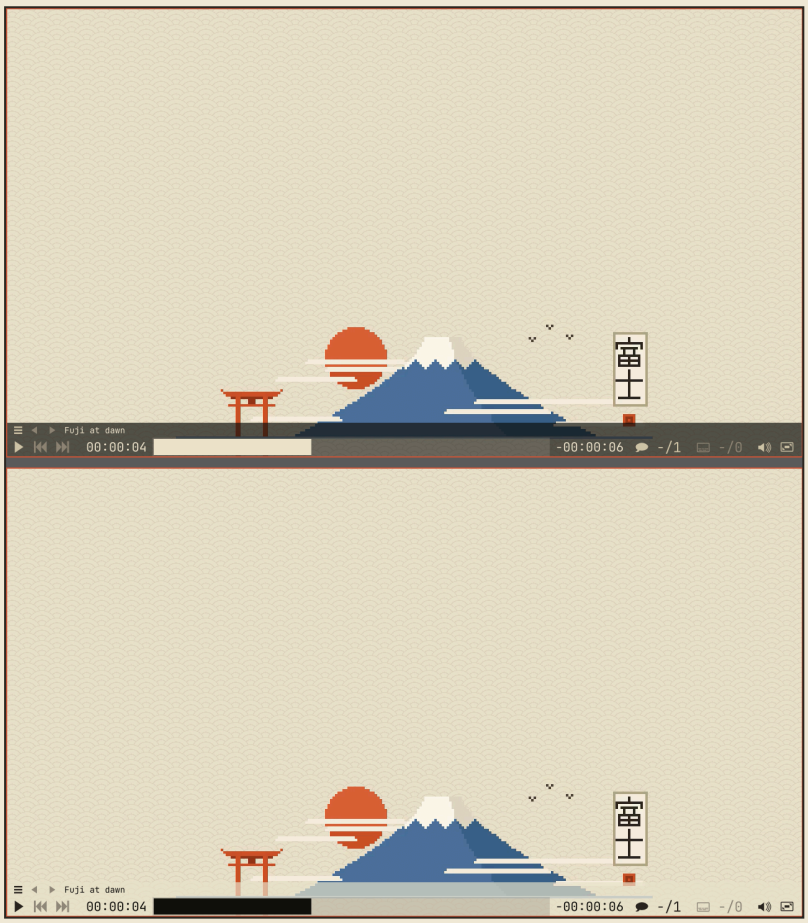
Keys	Does
← → or [ ]	previous · next picture
gg · G	first · last
+ or i or ↑ · - or o or ↓ · a · r · c	zoom in · out · actual size · fit again · centre
h j k l	move around a zoomed picture
s	cycle the scaling mode
f · d	fullscreen · name, size and position overlay
t · T	slideshow: one second more · less per picture (0 = off)
Space · .	play / pause an animation · next frame
x · q	take the picture off the list · quit

Pitfalls. x only removes the picture from imv's list; the file stays. imv cannot edit: to mark up a picture, `satty --filename picture.png --output-filename marked.png` (§12.3). Its colours are read at start, so an open imv keeps the old theme.

15.2 Video: mpv and yt-dlp

Term	In one line
container format	the file type that holds the streams together: `.mkv`, `.mp4`, `.webm`
codec	how one stream is compressed: H.264, HEVC, VP9 or AV1 for video, AAC or Opus for sound. One container can hold different codecs
bitrate	how many bits one second of a stream takes: more is sharper and larger (**i** shows it)
hardware decoding (VA-API)	the graphics card unpacks the video instead of the CPU, which stays cool and quiet
MPRIS	the standard way for a player to say what it plays and to take play and pause from the bar and the media keys (§15.5)

So a `.mkv` file (container) may hold HEVC video and Opus sound (codecs); **i** in mpv shows which, and at what bitrate.



mpv with its on-screen controller in the palette.

Open a video from yazi or Brave's downloads (mpv is the default for video and sound), or type `mpv file`, `mpv folder/` or `mpv 'https://…'`.

Try this. Play a video, stop half-way, and come back to it.

1. In yazi, go to a video — one copied from your phone, or a screen recording (§19.9) — and press **Enter**. mpv opens and plays it.
2. **Space** pauses and plays again; **→** jumps 5 seconds on.
3. **i** shows, for a few seconds, what the file holds: its container, codecs and bitrate.
4. **q** quits half-way through. Open the same file again: it starts where you stopped.

At the end of a file the picture stays on its last frame; **q** closes the window.

What the configuration does for you: it decodes on the GPU (VA-API) and leaves the output to mpv's own probing (Vulkan, falling back to OpenGL, so a broken driver still shows a picture); on a discrete AMD card (2 GB of video memory or more) it scales with the high-quality filters (`profile=high-quality`). It smooths motion to the monitor's refresh rate, starts at 80 % volume (up to 150 %), loads subtitles whose names resemble the video's, prefers English then Arabic for sound and subtitles, and saves screenshots as PNG in `~/Pictures/Screenshots`.

It **remembers where you stopped** — and a film watched to the end opens at the beginning next time, not on its last frame (`scripts/kawa-resume.lua`). Its on-screen text, controller and menus wear the theme's colours and switch live with it (`scripts/kawa-ipc.lua` lets `kawa-theme` reach every open player); subtitles keep paper on ink, which reads on any picture. It shows in the bar's *now playing* and answers the media keys (mpv-mpris).

Keys	Does
Space · p	pause
← → · ↑ ↓	back / forward 5 s · 1 min
Shift+← →	exactly 1 s
9 0 · m	quieter / louder · mute
f · Esc	fullscreen · leave it
v · j / J · #	subtitles on / off · next / previous subtitle track · next audio track
z · Z	subtitles 0.1 s earlier · later
[ ] · Backspace	slower / faster · normal speed
s · S · Ctrl+S	screenshot · without subtitles · of the window as you see it
g p · g s · g a · g c · g w	pick from the playlist · subtitles · audio tracks · chapters · the files with a saved position
Ctrl+P	menu of everything
i · ?	playback statistics · the key list
Ctrl+H	GPU decoding off / on
q · Q	quit (the position is kept either way)

yt-dlp

mpv plays web videos through yt-dlp, at up to 2160p. On its own, `yt-dlp URL` downloads into the current folder; `yt-dlp -F URL` lists the formats; `yt-dlp -x --audio-format mp3 URL` keeps only the sound (ffmpeg does the conversion).

Try this. Keep only the sound of a talk you are allowed to keep, as an MP3 in your *Music* folder. Copy the video's address in Brave (Ctrl+L, Ctrl+C), then:

```
cd ~/Music
yt-dlp -x --audio-format mp3 'LINK'    # LINK: paste it with Ctrl+Shift+V
```

yt-dlp prints its progress line by line and leaves one `.mp3` file, named after the video. `mpv` and the file's name plays it; the recipe below does the same for the whole video.

YouTube hides its video addresses behind JavaScript challenges: yt-dlp solves them in Node.js (`~/.config/yt-dlp/config`), which it starts with Node's `--permission` flag. On the Node 24 installed here that blocks file access, child programs and native add-ons — not the network, which Node restricts only from version 25. The code Node runs is yt-dlp's own solver (EJS).

An aside: how the config chooses what to play — `ytdl-format=bestvideo[height<=?2160]+bestaudio/best`:

Part	Means
`bestvideo[height<=?2160]`	the best video-only stream at most 2160 lines high (`?`: or one whose height is unknown)
`+bestaudio`	together with the best sound-only stream
`/best`	failing that, the best single file holding both

The same syntax works with `-f`: `yt-dlp -f 'bv*[height<=1080]+ba/b' URL` downloads at most 1080p (`bv`, `ba` and `b` are short for best video, best audio, best). `yt-dlp -F URL` lists every stream with its container, codecs and bitrate.

Read more: the mpv manual (also `man mpv`) and yt-dlp's format selection.

Pitfalls. Japanese, Chinese and Korean subtitles come out in Noto Sans CJK, Arabic in Noto Naskh. If `i` shows dropped frames on a weak GPU, try `mpv --interpolation=no file` first.

15.3 Recipe: watch or keep a video from the web

1. Copy the link in Brave.
2. In a terminal: `mpv 'LINK'` to watch it in mpv, or `yt-dlp 'LINK'` to download it into the current folder.
3. Keep the quotes: links often contain `&`, which the shell would otherwise act on.

15.4 Music: spotify_player and cava

spotify_player is Spotify in the terminal: type `spotify_player`. It needs a Premium account. The first start opens Spotify's sign-in page in Brave, usually twice: once for the Web API (lists, search) and once for the built-in player. Approve each, and both are cached in `~/.cache/spotify-player` (`spotify_player authenticate` signs in again from scratch). It plays through its own built-in client, appears in the bar's *now playing*, and answers the media keys.

Keys	Does
Space · n · p	play / pause · next · previous
+ - · _	louder · quieter · mute
> · < · ^	forward · back · to the start of the track
Ctrl+S · Ctrl+R	shuffle · repeat mode
g s · g l · g y · z	search page · library · liked tracks · queue
/	search in the current page
Enter · a	play the selection · actions on the current track
Tab · Backspace	next panel · previous page
D	choose the playback device: this computer, or a phone or speaker signed in to the same account (Spotify Connect)
?	every key
q	quit

Its colours are the terminal's own with vermilion accents (the `sumi` theme in `theme.toml`), so it switches with the desktop.

cava draws the bar's music meter (§6.5) with its own config in `~/.config/fedokawaii`. Type `cava` in a terminal for the classic full-window bars: Up / Down sensitivity, Left / Right bar width, `f` / `b` colours, `r` reloads the config after an edit, `q` quits. That copy reads `~/.config/cava/config`, which it creates on first run and which is yours to change.

Try this (with a Premium account). Play a song, and watch it.

1. Type `spotify_player`. The first time, sign in twice in Brave, as described above.
2. **g s** opens the search page: type a song's name, Enter, then **Enter** on a track. It plays, and the bar's *now playing* shows it with the meter moving.
3. Open a second terminal (**Super**+**Return**) and type `cava`: the same music, as bars across the window. **q** closes cava.
4. Back in spotify_player, **Space** pauses and **q** quits; the music stops with it.

Read more: spotify_player's README (every key and command) and cava's.

15.5 Now playing, and the media keys

> **Word — D-Bus.** The desktop's message system: programs send each other short messages over it — "what are you playing?", "pause". Chapter 32 shows how it works.

Every player that speaks MPRIS shows in *now playing* and answers the media keys: spotify_player, Brave (a video or a tab playing sound), and mpv (through the mpv-mpris plugin). *MPRIS* is the standard D-Bus interface through which a player says what it plays and takes play, pause and skip from anyone.

To	Press or click
play or pause	the **Play/Pause** key, or click the title in the bar
the next track	the **Next** key, or right-click the title
the previous track	the **Previous** key, or middle-click the title

With several players at once — music in spotify_player and a paused video in a Brave tab — the bar shows the one playing (the last to start, if more than one does), else the last one paused. The keys and the clicks run `kawa-nowplaying play-pause`, `next` or `previous`, which act on the player that is **playing**: the music pauses, and the paused video stays as it was. Plain `playerctl` would pick the player that started first, and could start the video on top of the music.

Try this. Play a video in mpv (§15.2), then ask from a terminal:

```
$ playerctl -l
mpv
$ playerctl status
Playing
```

(for example)

Press the **Play/Pause** key, or click the title in the bar, and ask again:

```
$ playerctl status
Paused
```

Command	Does
`playerctl -l`	every player that is up, one per line
`playerctl status`	`Playing`, `Paused` or `Stopped`
`kawa-nowplaying play-pause`	what the Play/Pause key does: the player that is playing (`next`, `previous` too)
`playerctl play-pause`	plain playerctl: the first player that started

Read more: the MPRIS specification.

15.6 Sound: the mixer and effects

The audio menu (**Super+A**, §8.1) covers everyday needs: volume, mute, the output and the input. Two programs go further.

Above 100 %. The volume keys, Super+scroll and the audio menu's *louder* all stop at 100 %. Only the mixer goes further: its sliders reach 153 %. A level set there is left alone — *louder*, the volume key and Super+scroll up do nothing above 100 % — while *quieter*, the key and Super+scroll down still lower it.

pavucontrol, the full mixer: audio menu → *mixer…*. It opens floating. Its tabs: *Playback* — a volume and an output device for **each program** (send a call to headphones and music to speakers, or a game to the **Share** output for a call, §14.4); *Recording* — which microphone each program listens to, shown while it records; *Output* and *Input Devices* — levels and ports of each device; *Configuration* — the card's profile (for example HDMI against analogue, or turning a card off).

> **On Windows.** The *Playback* tab is the volume mixer of Windows' sound settings: one slider and one output per program. The audio menu is the speaker icon on the taskbar.

Try this. Make one program quieter than the rest, and send it elsewhere.

1. Play something: music in spotify_player, or a video in mpv.
2. **Super+A**, then *mixer…* at the foot of the menu. pavucontrol opens on *Playback*, and the player has a row of its own.
3. Move that row's slider to the left: only that program gets quieter.
4. The button at the right of the row names its output. Choose another one (headphones, a monitor's speakers) and the program's sound moves there alone.
5. **Super+Q** closes the mixer; your choices stay.

EasyEffects (from the launcher; on Fedora 44 it is a Qt 6 program): effects in real time. *Output* effects change what you hear — an equaliser, bass, loudness. *Input* effects change your microphone — noise reduction, a gate (it mutes everything below a set level, so the room's hum drops out between words), a compressor (it evens out loud and quiet) — and every program that records hears the cleaned sound. Save a chain as a preset; `easyeffects -p` lists the presets, `easyeffects -l name` loads one, `easyeffects -b 1` / `-b 2` bypasses all effects / restores them.

Try this. Play music, open **EasyEffects** from the launcher, and add an equaliser to its *Output* effects: lowering the bands at the left — the bass — changes what you hear at once. Save the chain as a preset, and `easyeffects -p` in a terminal lists it by name.

Pitfalls. EasyEffects works **only while it runs**, and nothing starts it at login. Its *Autostart on login* switch writes an autostart file that only full desktops read; dwl does not. To keep it running, put one line in `~/.config/dwl/autostart.local.sh`, the login file that is yours alone (§42.1: the theme never writes it):

```
easyeffects --service-mode --hide-window &   # at every login, window hidden; & keeps it running
```

It takes effect at the next login. A program you have moved to a device in pavucontrol stays there; choosing another output in the audio menu no longer moves it.

What you learnt

- **Enter** on a file in yazi opens the right program: imv for pictures, mpv for video and sound.
- mpv remembers where you stopped; **i** shows a file's codecs; **q** quits.
- `yt-dlp 'LINK'` keeps a web video; `-x --audio-format mp3` keeps only its sound.
- The media keys and clicks on *now playing* act on the player that is playing.
- **Super+A** for everyday sound; *mixer…* (pavucontrol) for each program's own level and output; only the mixer goes above 100 %.

Check yourself

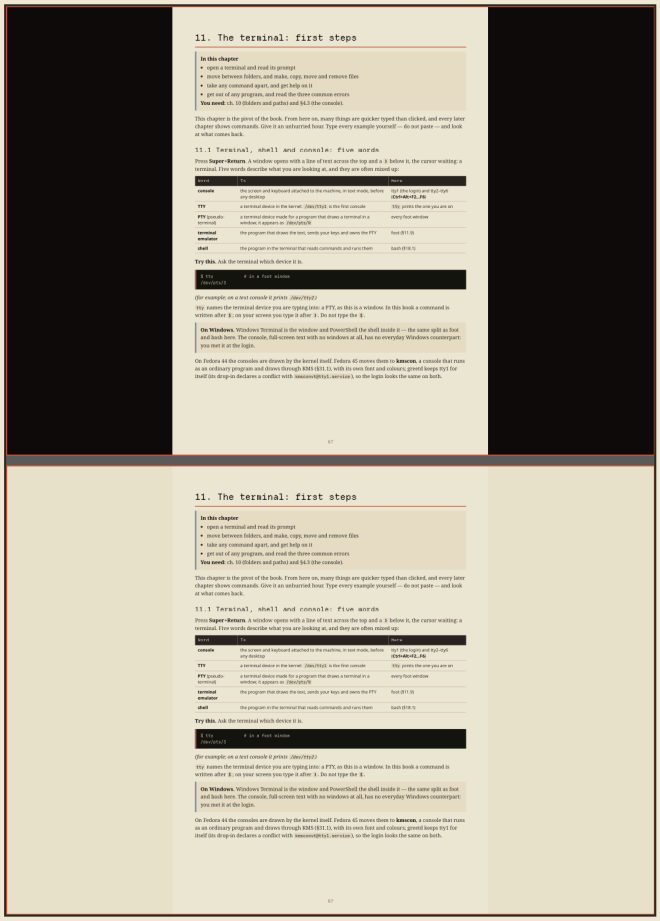
1. A file ends in `.mkv`. Does that tell you its video codec? → *no: `.mkv` is the container; i in mpv shows the codecs inside*
2. Music plays in spotify_player while a video waits, paused, in Brave. What does Play/Pause do? → *it pauses the music: the keys act on the player that is playing*
3. How do you get the volume above 100 %? → *only in the mixer, pavucontrol, up to 153 %; the keys stop at 100 %*

16. Reading, documents and notes

> **In this chapter**
> - read and search PDFs and e-books, in a library that remembers your page
> - keep a diary of notes that saves itself as you type
> - convert office files and PDFs, and present slides from a text file
> - calculate with units, money and dates
>
> **You need:** §10.3 (yazi) and §11.4 (the first commands).

Reading and writing ordinary documents is where most days go. Each section here starts with one small thing to do, then lists the keys and the details.

16.1 PDFs, EPUBs and comics: zathura



A PDF in zathura, in the palette.

Open a document from yazi or Brave's downloads (zathura is the default for PDF, EPUB and comic books), or type `zathura --fork file.pdf` — `--fork` gives the terminal back. zathura reads **PDF**, **EPUB** and **XPS** (through mupdf) and **CBZ/CBR** comic archives. Pages open fitted to the window, one per row; the status and command bars are hidden until needed.

Try this. Open a PDF, find a word in it, and read it in the theme's colours.

1. In yazi, go to a PDF in `~/Downloads` or `~/Documents` — a bank statement, say — and press **Enter**. zathura opens it, the page fitted to the window.
2. Type `/`, a word you know is in it, and Enter: the first match is marked. **n** goes to the next one.
3. **Ctrl+R** turns the page into the theme's colours; **Ctrl+R** again turns it back.

4. **J** turns to the next page. **q** closes zathura, and the next time the file opens on this page.

Keys	Does
j k · h l	scroll down / up · left / right
J K · Space · Shift+Space	next / previous page · a screen down / up
gg · G · 42G	first page · last page · page 42
a · s	fit the page · fit the width
+ - =	zoom in · out · original size
d	two pages side by side
/ · ? · n · N	search forward · backward · next · previous result
Tab	table of contents (Enter opens an entry)
f · F · c	follow a link by its label · show where it leads · copy its address
Ctrl+O · Ctrl+I	back · forward through your jumps
m*x* · '*x*	set mark *x* · go back to it
Ctrl+R	**recolour the page in the theme's colours** — paper text on ink, or ink on paper
r	reload the file (set here; zathura's own `r` would rotate)
p	print (set here)
Ctrl+N · Ctrl+M	show / hide the status bar (page number) · the command bar
F5 · F11	presentation · fullscreen
q	quit

Selecting text with the mouse copies it to the clipboard. Commands start with `:`, as in neovim:

```
:bmark ch3     save a bookmark called ch3 at this page
:blist         list the bookmarks
:bjump ch3     go back to it
:info          the document's details: title, author and so on
```

Read more: `man zathura` (keys and commands) and `man zathurarc` (every setting).

Pitfalls. Rotation has no key, because `r` reloads here. A theme switch makes open zathura windows re-read their config (§40.2), and the config says `recolor false`, so a recoloured page returns to normal: press Ctrl+R again. Printing to paper needs a print service; none is in the manifest, so the print dialog can save to a file but may list no printers.

16.2 Your library: books (Super+R)



The books window (Super+R). Dark: the ones being read first, each with its page and a bar, the cover of the one under the cursor beside. Light: adding books (Ctrl+A) from Downloads, Documents and Desktop.

Super+R opens your library: every book in `~/Books` (and its folders) — PDF, EPUB, MOBI, FB2, XPS and comic books (CBZ, CBR, CB7) — in a small floating window, the ones you are reading at the top, the most recent first. Each line shows how far in you are — a bar (vermilion while reading, teal once finished) and, at the right, one of four marks:

Mark	Means
`87/120`	reading: on page 87 of 120 (the page zathura remembered when you last closed it)
`p. 37`	reading a book without fixed pages — EPUB, MOBI, FB2 flow their text to the window's size, so there is a page but no total (its bar shows one step: begun)
`done`	finished: you closed it on its last page
`new`	not opened yet (its kind, page count and size show under its cover)

The cover of the book under the cursor shows beside the list, centred in its pane, with the title, where you are and when you last read it under it.

Keys	Does
type	search by title
Enter	read it in zathura — it opens on the page where you stopped (§16.1)
Ctrl+A	add books: every book in `~/Downloads`, `~/Documents` and `~/Desktop` is listed; **Tab** marks several, **Enter** moves them into `~/Books`, **Esc** goes back
Ctrl+D	remove it: the file goes to the trash (yazi: **g t** brings it back)
Esc	close

Try this. Put a PDF in your library, read a little, and come back to your page.

1. **Super+R**, then **Ctrl+A**: the books in `~/Downloads`, `~/Documents` and `~/Desktop` are listed. Move to one and press **Enter**: it moves into `~/Books`, and the library comes back with the book in it, marked `new`.
2. **Enter** on it opens it in zathura. Type `5G` to go to page 5, then **q**.
3. **Super+R** again: the book is at the top, marked `5/120` for a book of 120 pages, with its bar a little way along. **Esc** closes the library.

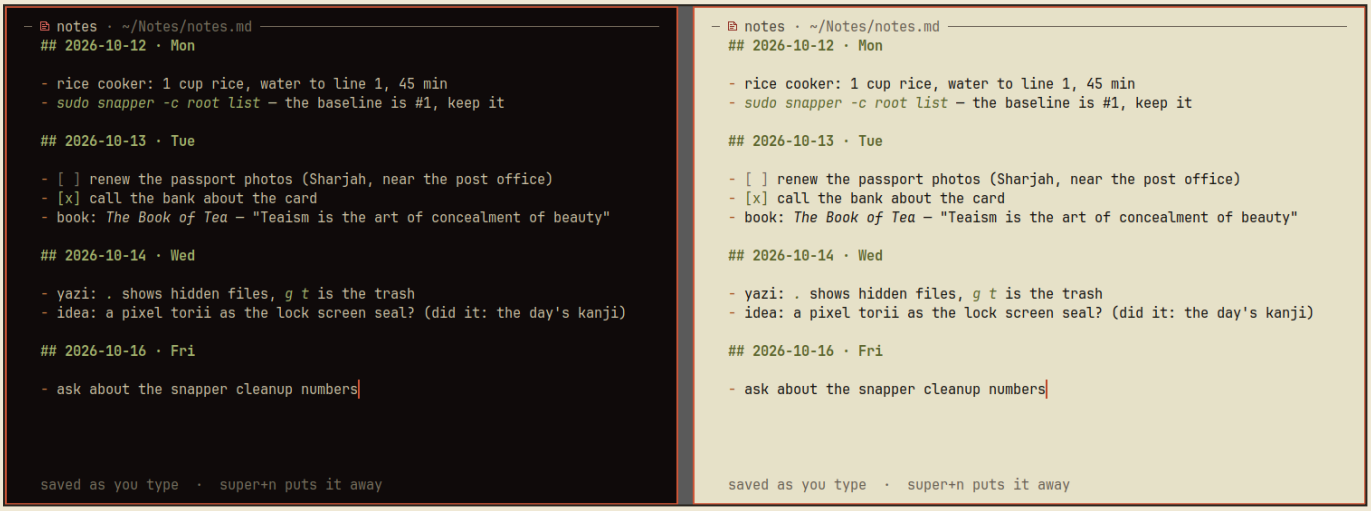
How it keeps track

Nothing new runs in the background and nothing is stored twice: the page comes from zathura's own memory, which zathura writes when you close a book; the window only reads it. Page counts and covers are worked out once per book and kept.

What	Where
your books	`~/Books/` — folders inside it are fine
the page you are on, per book	`~/.local/share/zathura/bookmarks.sqlite` (zathura's; read, never written)
covers and page counts	`~/.cache/kawa-books/` — safe to delete, drawn again when needed
the window	`~/.local/bin/kawa-books` (Super+R)

Tips. Folders inside `~/Books` (`~/Books/Japanese`, `~/Books/Work`) keep a large library tidy; the search matches titles anywhere in it. A book moved to another folder keeps its page: the window finds it by its name and opens it there. In zathura, **Ctrl+r** turns the pages into the theme's colours for reading at night (§16.1).

16.3 Notes (Super+N)



The notes window (Super+N): one file, a heading per day, open at the end and ready to type.

Super+N opens your notes in a small floating window; **Super+N again** puts them away. It is one plain file, `~/Notes/notes.md`, that reads top to bottom like a diary: the first time you open it on a day, that day's date becomes a heading, and the window opens at the end of the file. **You can type at once** — there is no key to press first. Write anything — a phone number, a command, a thought — and press Super+N; it is saved as you type, so closing the window any other way (Super+Q included) loses nothing. A day you opened but wrote nothing under leaves no empty heading behind.

Try this. Write a first note, then read it from a terminal.

1. **Super+N**. The window opens with today's date as a heading and the cursor under it; the line along the bottom says *saved as you type · super+n puts it away*.
2. Type `buy a SIM card for Japan`.
3. **Super+N** puts the window away. There is nothing to save: it was saved as you typed.

```
$ tail -3 ~/Notes/notes.md
## 2026-10-16 · Fri

buy a SIM card for Japan
```

(for example: the heading is today's date and day)

`tail -3` prints the last three lines of a file: the heading, an empty line, your note.

What	Where
your notes	**`~/Notes/notes.md`** — plain Markdown, one file
the window	`~/.local/bin/kawa-notes` (Super+N): neovim, the desktop's editor, dressed as a page — the name in a rule across the top and a quiet line of keys along the bottom, like every popup; no line numbers, the text three columns in (`~/.config/fedokawaii/notes.lua`)

Underneath, the window is neovim, the desktop's editor. You need none of its keys to write notes; once ch. 17 has taught them, they work here too: **Esc** then `/word` finds something, `gg` goes to the top, `u` undoes.

> **Word — Markdown.** Plain text with a few marks for structure: `##` before a heading, `-` before an item in a list. Any editor reads it as it is; some programs draw it as formatting.

Markdown is optional in your notes, but it makes them quicker to scan later:

You type	It means
`## Japan trip`	a heading
`- milk`	an item in a list
`- [ ] call the bank`	a to-do; `- [x]` once it is done
`**bold**`	what matters

Tips. To find something from any terminal: `rg -i rice ~/Notes`. Notes are plain text in your home, so the snapshots of `/home` keep their history too (§37.2). Want a separate file for a project? Start one in `~/Notes` from neovim (`:e ~/Notes/japan-trip.md`); Super+N always opens `notes.md`.

16.4 Documents: Xournal++, AbiWord, Gnumeric and the PDF tools

Xournal++ (launcher) writes on PDFs: *File → Annotate PDF*, then the pen (Ctrl+Shift+P), highlighter (Ctrl+Shift+H), text (Ctrl+Shift+T) and eraser (Ctrl+Shift+E). It saves your marks in a separate `.xopp` file and never changes the PDF; *File → Export as PDF* writes a new PDF with the marks in it — that is the file to send.

AbiWord for letters and simple documents, **Gnumeric** for spreadsheets (both from the launcher, both the defaults for their file types). Both open Microsoft and OpenDocument files; *File → Save As* lists what each can write. The Liberation fonts stand in for Arial, Times New Roman and Courier New with the same widths, so pages break where their authors meant. From a terminal: `abiword --to=pdf letter.docx` and `ssconvert sheet.xlsx sheet.csv` convert without opening a window. Neither is a full office suite: a document with tracked changes or a complex layout is better opened in the web version of the program it came from.

> **On Windows.** AbiWord stands in for Word and Gnumeric for Excel: both open `.docx` and `.xlsx`, and save back to them. They are much smaller programs — keep Office on the web for the hard cases.

Try this. Turn a Word file into a PDF without opening it, then look at the result:

```
cd ~/Documents
abiword --to=pdf "Offer letter.docx"    # writes Offer letter.pdf beside it
zathura --fork "Offer letter.pdf"       # the PDF; q closes it
```

PDF tools in a terminal: `qpdf --empty --pages a.pdf b.pdf -- both.pdf` joins files, `qpdf in.pdf --pages . 1-3 -- first-three.pdf` takes pages out, `pdftotext file.pdf -` prints the text, `pdfunite a.pdf b.pdf both.pdf` is the other way to join.

In `--pages`, each file name is followed by the pages to take (`.` stands for the input file):

Range	Takes
`1-3`	pages 1 to 3
`1,4,7`	pages 1, 4 and 7
`5-z`	page 5 to the end: `z` is the last page
`z-1`	every page, in reverse order

Try this. Take the first three pages out of a long PDF, and count them.

```
$ qpdf --show-npages statement.pdf
12
$ qpdf statement.pdf --pages . 1-3 -- first-three.pdf
$ qpdf --show-npages first-three.pdf
3
```

(for example: your statement has its own number of pages)

You typed	It means
`qpdf --show-npages statement.pdf`	how many pages the file has
`qpdf statement.pdf --pages . 1-3 -- first-three.pdf`	from this file (`.`), pages 1 to 3, into a new file; qpdf says nothing when all is well
`qpdf --show-npages first-three.pdf`	the new file has three pages; `statement.pdf` is unchanged

Ch. 19 has many more recipes like this one, for PDFs, pictures, video and sound.

Read more: the qpdf manual and Xournal++'s guide.

16.5 Slides in the terminal: presenterm

A presentation is a Markdown file. `presenterm talk.md` shows it full-window and **reloads it every time you save**, so you can edit in neovim beside it; `presenterm --present talk.md` turns that off for the real thing. Super+F makes the window fullscreen; Ctrl++ in foot makes the text larger. Colours follow the theme at start (`terminal-dark` or `terminal-light`).

```
---
title: Fedokawaii
author: me
---

First slide

<!-- pause -->
This line appears on the next key press.

<!-- end_slide -->

Second slide
<!-- speaker_note: say hello -->
```

Keys	Does
l j → ↓ PageDown Space	next
h k ← ↑ PageUp	previous
n · p	next · previous slide at once, skipping its pauses
gg · G · 7G	first · last · slide 7
Ctrl+P	slide index
Ctrl+E	run the code block on screen (only when started with `-x`)
? · q	keys · quit

Try this. Save the example above as `~/talk.md` — `nvim ~/talk.md`, press `i`, type it in, then Esc and `:wq` (ch. 17 explains these keys) — and show it:

```
presenterm ~/talk.md      # the talk, full-window
```

Each **Space** steps on, and the line after `<!-- pause -->` comes on a press of its own. **q** ends the show.

Pictures show in foot. `presenterm --export-html talk.md` writes one self-contained HTML file; PDF export needs weasyprint, which is not installed. For speaker notes, run `presenterm talk.md --publish-speaker-notes` in one window and `presenterm talk.md --listen-speaker-notes` in another.

Read more: presenterm's documentation — pauses, columns, code and exports, for the version installed here (0.16).

16.6 The calculator: qalc

Try this. Ask two questions, one in each form.

```
$ qalc '3.5 h to min'
3.5 hours = 210 min
$ qalc -t '1500 sqft to m^2'
139.35456 m²
```

You typed	It means
`qalc '3.5 h to min'`	convert; the answer repeats the question as qalc understood it
`qalc -t '1500 sqft to m^2'`	`-t`: the result alone — a flat's size from a listing, in square metres

`qalc` in a terminal, or `qalc 'expression'` for one answer (`-t` prints only the result). It knows units, currencies, bases and functions.

```
qalc '2^64'
qalc '75 km/h to mph'
qalc '255 to hex'
qalc '3.5 h to min'
qalc -e '100 USD to AED'      # -e fetches current exchange rates first
```

In an answer that repeats the question, `=` marks an exact result and `≈` a rounded one:

```
$ qalc -t '255 to hex'
0xFF
$ qalc '75 km/h to mph'
75 kilometers/hour ≈ 46.60283942 mph
```

Inside it: `ans` is the last result, `to unit` converts it, `exrates` updates the rates, `help`, `quit`. Up and Down recall earlier lines. For a quick sum without leaving your work, `footclient -a float qalc` opens it in a small floating window.

Read more: `man qalc`, and the project's site for the full list of units and functions.

What you learnt

- **Enter** on a PDF or e-book opens zathura: `/` searches, **Ctrl+R** recolours, **q** closes and the page is remembered.
- **Super+R** is your library in `~/Books`; **Ctrl+A** brings books in from Downloads.
- **Super+N** is one diary file, `~/Notes/notes.md`: type at once, and it saves itself.
- `abiword --to=pdf` and `ssconvert` convert office files; `qpdf` picks, joins and counts pages.
- `presenterm talk.md` shows a Markdown file as slides; `qalc` calculates with units and money.

Check yourself

1. Where do your notes live, and how do you save them? → *in `~/Notes/notes.md`; they save themselves as you type*
2. Which command makes a new PDF of pages 1 to 3 of `report.pdf`? → *`qpdf report.pdf --pages . 1-3 -- part.pdf`*
3. What does `-t` change in `qalc -t '75 km/h to mph'`? → *only the result is printed, not the question*

17. Writing text: neovim

> **In this chapter**
>
> - open, change, save and quit a file without getting stuck
> - edit with a small grammar: a verb, a count, a motion
> - find files and text, and work on several files at once
> - let LazyVim's language tools check and tidy your code
>
> **You need:** ch. 11 (the terminal) and §16.3 (notes: neovim already).

Every text file on this machine — a note, a setting, a script, a commit message — opens in neovim. It deserves a slow week. After that it is quicker than any editor you have used, and its basic keys work in the Vim found on nearly every Linux server you will ever reach.

Why learn it. In Notepad, every key types a letter, and everything else is the mouse and the menus. In neovim, most of the time the keys you press are **commands**, not text: `dd` deletes a line, `ciw` changes a word, `.` does the last change again. That is baffling for the first ten minutes and fast for good after: your hands stay on the keys, and every command is a few letters you can count, repeat and combine.

> **On Windows.** Notepad is always typing. Neovim starts in a mode for commands: **i** starts typing, **Esc** always comes back. Ctrl+S saves as you expect, but Ctrl+C and Ctrl+V do not copy and paste: `y` and `p` do (§17.8), and Ctrl+Shift+V pastes while you type.



Neovim with LazyVim, the palette's gruvbox, on a shell script.

Learn in this order. The first sections are a course; the rest is reference to come back to. Read them with neovim open beside the book, one step a day.

Step	Learn	In
1	the modes, and typing a file	§17.1
2	getting out of any trouble	§17.2
3	moving, typing, deleting, searching, replacing	§17.3
4	the grammar — `dw`, `cw`, `ciw` — and the dot	§17.4 · §17.5
5	text objects, and selecting first	§17.6 · §17.7
6	the clipboard, then macros	§17.8 · §17.9
7	files, the picker and the Space menu	§17.10 to §17.13
8	code: replacing across files, language servers, formatting	the rest, when you need it

The last section, §17.22, spreads the same steps over a month and starts with neovim's own tutor.

The *Try this* sections share one small file, `/tmp/practice.txt`, which you type in the first of them. Each later one ends with `:q!` — quit without saving — so the file is the same every time. `/tmp` is emptied when the computer restarts: if the file has gone, type it again.

Neovim is the only editor here (`nvim`, `vi`, `$EDITOR`, the man pages, yazi's Enter). It runs **LazyVim**, a ready-made set of add-ons that setup put in `~/.config/nvim` (§17.19). Started without a file, it shows a start screen with one-key actions: `f` find file, `n` new file, `g` find text, `r` recent, `c` config, `s` restore the last session in this folder, `x` extras, `l` plugins, `q` quit.

Terms. You need only *mode* today. The others come back where they are used; this table is here to look back at.

Term	In one line
mode	what keys do at the moment: give commands (Normal), type text (Insert), select (Visual)
buffer	a file's text held in memory while you edit it; the file on disk changes only when you save
window	a view onto a buffer: two windows can show two files, or two places in one
tab	a whole layout of windows — not one file each, as in a browser
operator · motion · text object	a verb (`d` delete) · a move (`w` to the next word) · a named piece of text (`iw` this word)
register	one of neovim's clipboards, named by a character: `a` … `z`, and `+` for the desktop's
leader	the key that starts LazyVim's own commands: Space
plugin	an add-on written in Lua; LazyVim is a collection of them, installed and updated by lazy.nvim
picker	a search window: type, choose from the list, see a preview
language server · LSP	a separate program that understands one language and answers neovim's questions — where is this defined, what is wrong here · the protocol they speak (*Language Server Protocol*)
treesitter	a parser in neovim that reads the structure of code — functions, strings, comments — for colours, folds and text objects; LazyVim installs the parsers for common languages
formatter	a program that rewrites a file's layout (indentation, spacing) without changing what it does
quickfix list	a list of places — file and line — to step through, such as search results

17.1 Modes

Neovim is always in one *mode*. Its name stands in the box at the left end of the status line, each in a colour of its own. Esc leaves every other mode for Normal.

Mode	The box	You are	Get there with
Normal	NORMAL, vermilion	giving commands: moving, deleting, copying. Neovim starts here	Esc
Insert	INSERT, teal	typing text, as in any editor	`i`, `a`, `o` and a few others (below)
Visual	VISUAL, yellow	selecting text, to act on it	`v`, `V`, Ctrl+V
Command-line	COMMAND, blue	typing a command after `:`, or a search after `/`; Enter runs it, Esc drops it	`:` or `/`
Replace	REPLACE, red	typing over the existing text	`R`
Terminal	TERMINAL, teal	in a shell inside neovim; Esc Esc, quickly, stops typing into it	Ctrl+/ (again to hide it)

`:` opens a small box near the top of the window with suggestions under it (Tab completes); a search with `/` stays at the bottom.

Try this. Type the practice file. In a terminal:

```
nvim /tmp/practice.txt      # a new, empty file
```

The box at the left of the status line says NORMAL. Press **i**: it says INSERT, and what you type now goes into the file. Type these five lines, with Enter after each:

```
Trip to Japan
tea, rice and green tea
train pass for 7 days
mv old_name.txt "new name.txt"
call the bank
```

Press **Esc** — NORMAL again — then type `:wq` and press Enter. Back in the shell, look:

```
$ cat /tmp/practice.txt
Trip to Japan
tea, rice and green tea
train pass for 7 days
mv old_name.txt "new name.txt"
call the bank
```

You typed	It means
i	Insert mode: from now on, keys type text
Esc	back to Normal mode: keys are commands again
`:wq` Enter	`:` opens the command line, `w` writes the file, `q` quits

If a small menu of words pops up while you type, **Enter** takes a word from it instead of starting a new line: press **Ctrl+E** to close the menu first (§17.16). If neovim adds a closing `"` as you type the first one, type on: your own closing `"` steps over it.

17.2 Never stuck

Whatever happened, one of these gets you out of it:

Situation	Do
you do not know which mode you are in, or keys do odd things	**Esc** — twice if unsure. You are in Normal mode (Ctrl+[is the same key)
save and quit	`:wq` Enter, or `ZZ`. Ctrl+S saves, also while typing
quit without saving	`:q!` Enter; `:qa!` closes every file. Plain `:q` on a changed file asks *Save changes?* — Yes, No or Cancel
undo · redo	`u` · Ctrl+R, as many times as needed
throw away everything since the last save	`:e!` Enter
the status line says *recording @q*	you pressed `q` and a letter: press `q` once more
a window of old commands opened at the bottom	you typed `q:` : `:q` Enter closes it. (Ctrl+C there copies the line under the cursor to the command line; Esc then drops it)
letters come out in Arabic, or Normal-mode keys do nothing	the keyboard layout changed (the bar shows AR): Alt+Shift. The desktop's shortcuts always read the US layout; neovim reads the letters you type
neovim vanished and the shell says *Stopped*	Ctrl+Z put it to sleep: type `fg`
Found a swap file on opening a file	a neovim crashed while editing it: `R` recovers the unsaved text — save it, and answer `D` (delete the old swap file) the next time the question comes. (A file open in another running neovim opens anyway, with the warning *W325*: save it in one place only)
readonly or *Permission denied* on a system file	`:q!`, then `sudoedit /etc/the-file` in the shell: your own neovim edits a copy, and sudo puts it in place (§24.3)

Tip: undo survives closing the file — LazyVim keeps the undo history on disk, so `u` the next morning still takes back yesterday's edits. Space s u lists the whole history to pick a point from.

Try this. Make trouble on purpose, and get out of it. `nvim /tmp/practice.txt`, then:

1. Press `dd` three times: three lines vanish.
2. Press `u` three times: they come back, one at a time.
3. Press `q`, then `w`: the status line says *recording @w*. Press `q` once more: it goes.
4. Press `x` a few times anywhere — each deletes a letter — then quit **without saving**: `:q!` and Enter.

```
$ cat /tmp/practice.txt
Trip to Japan
tea, rice and green tea
train pass for 7 days
mv old_name.txt "new name.txt"
call the bank
```

The file is exactly as you typed it: `:q!` threw the deleted letters away.

17.3 The first hour

Moving is best practised on a long file you cannot damage. A copy of this book's own text does well; the toolkit keeps one (§5.5):

```
cp /usr/local/lib/fedokawaii/docs/fedokawaii-guide.md /tmp/long.txt   # a copy to play with
nvim /tmp/long.txt
```

Spend ten minutes on each block below, in order, and do every key at least five times. Plain text files wrap long lines and underline spelling mistakes in red (Space u s hides them).

Moving. Arrows work, but h j k l sit under your right hand, and the hand never leaves the keys.

Keys	Move
`h` `j` `k` `l`	left · down · up · right
`w` · `b` · `e`	to the next word · back a word · to the end of the word
`W` · `B` · `E`	the same, counting only spaces as gaps (`~/.config/nvim` is one word)
`0` · `^` · `$`	start of the line · its first character after the indentation · its end
`gg` · `G` · `42G` (or `:42`)	first line · last line · line 42
Ctrl+D · Ctrl+U	half a screen down · up
`}` · `{`	to the next · previous blank line: a paragraph at a time
`%`	to the matching bracket
Ctrl+O · Ctrl+I	back · forward to where you were before a jump
`s`, a letter or two, a label	**flash**: every visible place with those letters gets a one-letter label; type it to land there

The numbers in the left margin count lines **from the cursor**: the line you are on shows its real number, every other line how far away it is. Any move takes a count in front — `5j` goes five lines down, and the 5 can be read off the margin.

Typing.

Keys	Start typing
`i` · `a`	before · after the cursor
`I` · `A`	at the start · at the end of the line
`o` · `O`	on a new line below · above
Esc	stop typing: back to Normal mode

Changing.

Keys	Do
`x`	delete the character under the cursor
`r`, then a character	replace one character
`dd` · `yy`	cut · copy (*yank*) the line
`p` · `P`	paste after · before the cursor; a whole line goes below · above
`J`	join the next line onto this one
`~`	switch the case of a letter
`.`	**repeat the last change** — the most useful key in neovim
`u` · Ctrl+R	undo · redo

Searching. Searches ignore case unless you type a capital letter.

Keys	Do
`/word` Enter · `?word` Enter	search forward · backward
`n` · `N`	next · previous match
`*` · `#`	the word under the cursor, forward · backward
Esc	clear the highlighting

Replacing. While you type a replacement, the file shows the result live.

Command	Replaces
`:s/old/new/`	the first *old* on this line
`:s/old/new/g`	every *old* on this line
`:%s/old/new/g`	every *old* in the file (`%` means all lines)
`:%s/old/new/gc`	the same, asking at each: `y` yes, `n` no, `a` all the rest, `q` stop
`:%s/\<old\>/new/g`	whole words only — not the *old* in *golden*
`:'<,'>s/old/new/g`	in the selected lines: select, press `:`, and the `'<,'>` is filled in

The *old* is a **regular expression**: a pattern in which some characters stand for others. The few worth knowing first:

Pattern	Matches	For example
`.`	any one character	`c.t`: *cat*, *cut*
`^` · `$`	the start · the end of the line	`^#`: a `#` at the start
`\<` · `\>`	the start · the end of a word	`\<old\>` (above)
`\d` · `\s`	a digit · a space or a tab	`\d\d`: two digits
`*` · `\+`	the thing before it, any number of times · at least once	`\s\+$`: spaces at a line's end
`[…]`	one of the characters in the brackets	`[0-9]`, `[aeiou]`
`\.`	a real dot: the backslash takes the special meaning away	`\.sh$`

So `:%s/\s\+$//e` removes the spaces at the ends of all lines (the `e` keeps it quiet when there are none). ripgrep, behind Space / and yazi's **S**, writes the same ideas a little differently: `+` without the backslash, `\b` for the edge of a word.

Finally `:w` to save and `:q` to quit. Delete `/tmp/long.txt` whenever you like.

Try this. One pass through the hour on the small file: `nvim /tmp/practice.txt`, then

You type	It does
`G`, `o`, `buy a SIM card`, Esc	a new line under the last one
`gg`, `A`, a space and `in April`, Esc	adds to the end of line 1
`:%s/tea/coffee/g` Enter	every *tea* in the file becomes *coffee*

The screen now shows:

```
Trip to Japan in April
coffee, rice and green coffee
train pass for 7 days
mv old_name.txt "new name.txt"
call the bank
buy a SIM card
```

`:q!` throws it all away, ready for the next section.

17.4 The grammar: verb, count, motion

Most of neovim is three parts said in order: *what to do* (a verb — neovim calls it an operator), *how often* (a count, optional), and *over what* (a motion, or a text object). `d` deletes and `w` moves a word, so `dw` deletes a word, `d3w` three words, `d$` to the end of the line. Every verb works with every motion, so each new motion gives you a new command with every verb. A verb typed twice acts on the whole line.

Verb	Does	Twice: the whole line
`d`	delete (a cut: the text goes to the clipboard)	`dd`
`c`	change: delete, then type in its place	`cc`
`y`	yank: copy	`yy`
`>` · `<`	indent · outdent	`>>` · `<<`
`gu` · `gU` · `g~`	lower case · UPPER CASE · switch case	`guu` · `gUU` · `g~~`
`=`	re-indent	`==`
`gc`	comment out · back in	`gcc`

Any move from the first hour works as the motion — `w`, `b`, `e`, `0`, `$`, `G`, `gg`, `}`, a `/search` — and `j` or `k` make the verb take whole lines. Two more are made for verbs: `fx` reaches onto the next *x* on the way, `tx` up to just before it (any character in place of *x*; `F` and `T` look backwards). Text objects (next block) work as motions too.

You type	It does
`dw` · `d3w`	delete to the next word · three words
`D` (the same as `d$`)	delete to the end of the line
`dt)` · `df,`	delete up to the next `)` · up to and including the next comma
`5dd` · `dG`	delete five lines · everything to the end of the file
`cw` · `ciw`	change to the end of the word · the whole word, wherever the cursor is in it
`ci"` · `ci(`	change what is inside the quotes · the brackets
`yip` · `>ip` · `gcip`	copy · indent · comment out the paragraph
`gUiw`	the word in capitals
`d/end` Enter	delete up to the next *end*

Try this. `nvim /tmp/practice.txt`, and put the cursor on the *o* of `old_name`: `4G` (line 4), then `w` (the next word). Type each command below and compare the line; then `u` undoes it before the next one (after `ci"`, press Esc first). `:q!` when you are done.

You type	The line becomes
(nothing yet)	`mv old_name.txt "new name.txt"`
`dw`	`mv .txt "new name.txt"` — `w` stops at the dot
`dW`	`mv "new name.txt"` — `W` runs on to the next space
`dt.` · `df.`	`mv .txt "new name.txt"` · `mv txt "new name.txt"` — up to the dot · the dot too
`ci"`	`mv old_name.txt ""`, typing between the quotes: the object finds the next quotes on the line
`d$`	`mv` and the space after it

17.5 The dot

`.` repeats the last change, verb and typed text together. To rename a word in a few chosen places:

1. `*` on the word: neovim searches for it, highlights every copy and moves to the next one.
2. `ciw`, the new word, Esc: this copy is changed.
3. `n` to the next copy and `.` to change it the same way — or `n` again to leave one as it is.

Try this. `nvim /tmp/practice.txt`, then `2G` — the cursor on the first *tea* of line 2:

You type	Line 2 becomes
`*`	unchanged: the cursor jumps to the second *tea*
`ciw`, `coffee`, Esc	`tea, rice and green coffee`
`n`	unchanged: the search wraps round to the first *tea*
`.`	`coffee, rice and green coffee`

Two changes, but `coffee` typed once. `:q!` when you are done.

Tip: `f` and `t` are LazyVim's (flash): they look past the end of the line too, highlight every possible target, and pressing `f` again moves on to the next one.

17.6 Text objects

A text object names a piece of text by what it is rather than where it ends, so it works wherever the cursor sits inside it. `i` takes the inside, `a` takes it *around*, with its surroundings: `iw` is a word, `aw` the word and the space after it. They follow a verb, or `v` to select:

Object	`i…` (inside)	`a…` (around)
`w` · `W`	the word	the word and a space
`s` · `p`	the sentence · the paragraph	with the space · the blank line after it
`"` · `'`	inside the quotes	with the quotes
`(` `[` `{` `<`, or `b` for the nearest	inside the brackets	with the brackets
`q`	inside the nearest quotes of any kind	with them
`t`	inside an HTML or XML tag	with the tags
`a`	an argument between commas	with its comma

Object	`i…` (inside)	`a…` (around)
`f` · `c`	the body of a function · a class (in code that treesitter can read)	the whole function · class
`g`	the whole file	the whole file

`daw` deletes a word and its space; `yi"` copies a quoted path without its quotes; `dap` deletes a paragraph; `yag` copies the whole file. LazyVim's mini.ai adds `q`, `a`, `f`, `c` and `g`, and makes all of them look ahead: `ci"` works from the start of the line, before the quotes. It adds a few more as well — `u` a function call, `d` a number, `o` a block, an *if* or a loop; pause after `vi` and the key panel lists them all.

Try this. `nvim /tmp/practice.txt`, then:

You type	You see
`/rice` Enter	the cursor on *rice*, in line 2
`daw`	`tea, and green tea` — the word and its space are gone
`4G`, then `di"`	`mv old_name.txt ""` — from the start of the line, the object found the quotes
`u`, `u`	both lines as they were

`:q!` when you are done.

Surround comes with an extra, `coding.mini-surround` (switch it on in `:LazyExtras`, below). Then `gsaiw"` puts quotes round a word — `gsa`, an object, the character — and `gsa` works on a selection too; `gsd"` removes the quotes round the cursor; `gsr"'` turns double quotes into single ones.

17.7 Visual mode and block mode

Select first, then act — the way to go when the piece has no name.

Keys	Does
`v` · `V` · Ctrl+V	select characters · whole lines · a rectangle (a block)
any move or object	extends the selection: `vip` a paragraph, `v$` to the end of the line
`o` · `gv`	jump to the other end · select the last selection again
`d` · `y` · `c`	delete · copy · change the selection
`>` · `<`	indent · outdent; the selection stays, so press again for more
`u` · `U` · `~`	lower case · upper case · switch case
`J` · `gc`	join the lines · comment them out or back in
Alt+J · Alt+K	move the selected lines down · up (a single line too, in Normal mode)
`p` · `P`	replace the selection with the clipboard; `P` leaves the clipboard as it was
Ctrl+A · `g` Ctrl+A	add one to every number · count up down the selection: a column of `0`s becomes 1, 2, 3 …
`:`	a command for the selected lines (`:'<,'>` appears)

Block mode selects a rectangle, which is how columns are edited. Press Ctrl+V, then move down (`j`, or `3j`) and across. With a block selected:

- `I`, the text, Esc — the text goes in at the left edge of the block on **every** line. It shows on the first line while you type and on the others when you press Esc: Ctrl+V, `3j`, `I`, then `- [ ]` and a space, then Esc turns four lines into a checklist.
- `$`, then `A`, the text, Esc — appended at the end of every line, whatever its length.
- `c` replaces the block's contents on every line; `d` deletes the column; `r` and a character fills it.

LazyVim lets the cursor go past the end of short lines in block mode, so a block can be as wide as the longest line. (foot's own paste is Ctrl+Shift+V, so Ctrl+V reaches neovim.)

Try this. Turn the practice file into a checklist. `nvim /tmp/practice.txt`, then `gg`, **Ctrl+V**, `4j` — a thin block down all five lines — `I`, type `- [ ]` (a space at the end), and Esc. The screen shows:

```
- [ ] Trip to Japan
- [ ] tea, rice and green tea
- [ ] train pass for 7 days
- [ ] mv old_name.txt "new name.txt"
- [ ] call the bank
```

One `u` takes all five back. `:q!` when you are done.

17.8 Registers and the clipboard

Neovim keeps several clipboards, called *registers*, each named by one character. To use one, type `"` and its name before the command: `"ayy` copies the line into register `a`, `"ap` pastes it back. Typing `"` alone — or Ctrl+R in Insert mode — shows a popup of what each register holds; Space s " lists them in the picker, and `:reg a0` prints only `a` and `0`.

Here, the plain commands use the desktop's clipboard. LazyVim sets `clipboard=unnamedplus` (except in an SSH session), so `y`, `d`, `c`, `x` and `p` all go through the system clipboard: a line yanked in neovim pastes into Brave with Ctrl+V, a link copied in Brave pastes in neovim with `p`, and every yank lands in the clipboard history (Super+V, §12.1) — and, thanks to `kawa-clipkeep`, stays on the clipboard after neovim closes. `"+y` and `"+p`, the way plain Vim reaches the clipboard, work too and do exactly the same.

Register	Holds
`"` (unnamed) and `+`	the last yank or delete — here, the system clipboard
`0`	the last **yank** only; deletes leave it alone, so `"0p` pastes what you copied even after deleting since
`1` … `9`	the last nine deletions of a line or more, newest first
`-`	the last deletion within a line (`dw`, `x`)
`a` … `z`	your own: `"ay` stores, `"Ay` (the capital) adds to the end
`_`	the black hole: `"_dd` deletes a line without touching the clipboard
`*`	the primary selection (select, then middle-click)
`/` · `:` · `.` · `%`	the last search · the last command · the last text typed · this file's name

Tip: since deletes go to the clipboard, a `dd` replaces the link you copied in Brave a minute ago. It is still in Super+V; to keep it on the clipboard, delete with `"_d`, or paste your last yank with `"0p`.

Try this. A register of your own, then the desktop's clipboard. `nvim /tmp/practice.txt`:

1. `gg`, `"ayy` copies line 1 into register `a`; `G`, `"ap` pastes it under the last line. The file now ends with `call the bank`, then `Trip to Japan`.
2. `3G`, `yy` copies line 3 — to the desktop's clipboard, where a plain `y` goes here.
3. `:q!`, and in the terminal:

```
$ wl-paste
train pass for 7 days
```

`wl-paste` prints what the clipboard holds: the line you copied, still there after neovim has closed.

17.9 Macros

A macro is a recording of keys that neovim plays back: any edit you can do once by hand, it does a hundred times. The recording goes into a register, like a copy.

Keys	Does
`qa`	start recording into register `a` (any letter); the status line shows *recording @a*
`q`	stop recording
`@a` · `@@`	play it once · play the last played macro again
`10@a`	play it ten times in a row
`qA` … `q`	record more onto the end of `a`
`:5,20normal @a`	play it once on each of lines 5 to 20
`:'<,'>normal @a`	play it once on each selected line
`:g/pattern/normal @a`	play it once on each line containing *pattern*

Three habits make a macro reliable:

- **Start from a known place.** Begin with a move that does not depend on where the cursor happened to be — `0` or `^` — and end by moving to where the next run should start, usually `j`, the next line.
- **Move by meaning, not by counting.** `t.`, `f,`, `w`, `/word` find their place on lines of any length; `llll` works only on lines exactly like the first.
- **A macro stops at the first command that fails.** On the last line `j` fails, and playback ends there — so `100@a` means "to the end".

A worked example: numbering photos. Select some pictures in yazi and press **r** (bulk rename, §10.3): neovim opens with one name per line.

```
IMG_20261003_101512.jpg
IMG_20261003_101530.jpg
IMG_20261003_102244.jpg
IMG_20261003_103901.jpg
```

They should become `beach_01.jpg`, `beach_02.jpg` and so on.

1. Do the first line by hand: `gg`, then `0ct.` (change everything up to the dot), type `beach_01`, Esc. It reads `beach_01.jpg`.
2. Record how to get from one line to the next. On line 1, `qa` starts the recording, then: `0yt.` copies `beach_01` (everything before the dot); `j0ct.` goes down and changes that line up to its dot; **Ctrl+R** then `0` types the text just copied; Esc; **Ctrl+A** adds one to the number under the cursor; `q` stops. Line 2 reads `beach_02.jpg`.
3. `100@a` does the rest: each run copies the line it is on, moves down, writes the name and counts up, until the last line stops it.
4. `:wq` — and yazi asks once more before renaming.

Ctrl+A keeps the leading zero (`beach_09` becomes `beach_10`). It reads a `-` just before a number as a minus sign, which is why the example uses `_`: from `beach-01`, Ctrl+A makes `beach00`.

Editing a macro. A macro is only text in a register, so it can be corrected instead of recorded again. On an empty line, `"ap` pastes it, with Esc shown as `^[` and Ctrl+A as `^A`. Edit it like any text — to type a special key into it, press Ctrl+V first in Insert mode, then the key — then `0"ay$` copies the line back into `a` (`$` rather than `yy`, so no line break comes along) and `dd` removes the line.

Without a recording: `:normal` and `:g`. `:normal` runs Normal-mode keys on every line of a range, as if you typed them there with the cursor at the start of each — often all a macro would have done. Insert mode ends by itself at the end of each line's keys.

Command	Does
`:%normal A;`	adds `;` to the end of every line
`:'<,'>normal I#`	puts `#` at the start of each selected line
`:g/^TODO /normal 0cw- [ ]`	turns every line starting `TODO` into a checklist item, `- [ ] …`
`:g/^$/d`	deletes every empty line
`:v/keep/d`	deletes every line that does **not** contain *keep*
`:sort` · `:sort u`	sorts the lines of the file (or of a selection) · and drops repeated ones

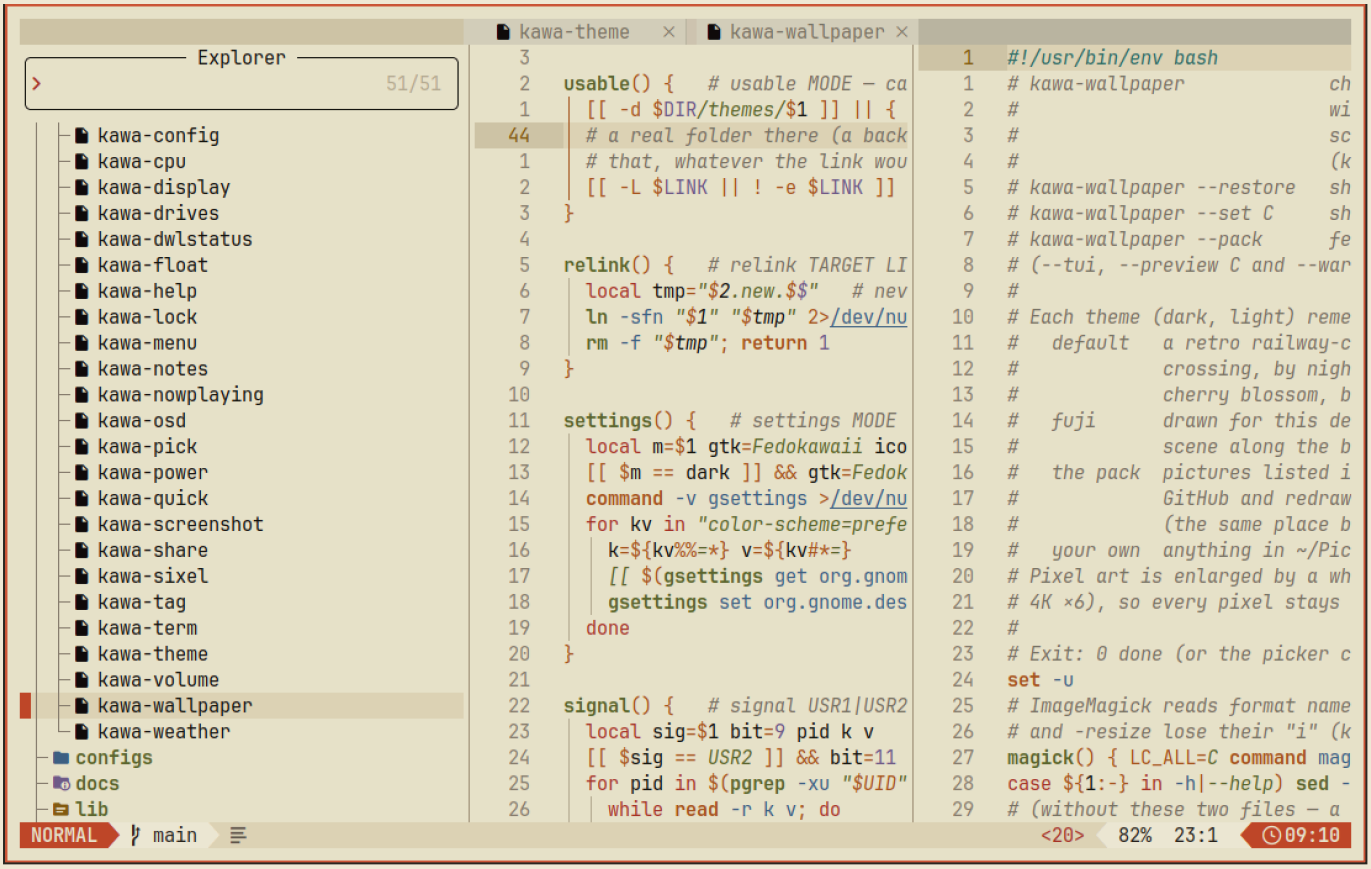
`:g/pattern/` picks lines by what they contain and runs the command after it on each — `d` to delete, `normal` and any keys, `normal @a` a macro; `:v` picks the lines that do not match.

17.10 Files, buffers and windows

Every file you open becomes a *buffer* and stays open until you close it; once there are two or more, LazyVim lists them along the top. A *window* is a view onto a buffer: splitting the screen shows two files side by side, or two places in one.

To see the difference: `:e notes.md`, then `:e todo.md` — two buffers in one window; Shift+H brings *notes* back. Space | splits the window, and both halves show *notes* (two places in one file); Shift+L in one half shows *todo* there. Space w d closes a window, but its buffer stays open until Space b d.

Keys	Does
Space Space · Space f r	find a file in the project · a recent file
Space , · Space b b	the open buffers · back to the one before
Shift+H · Shift+L	previous · next buffer
Space b d · Space b o	close this buffer · close all the others
`:e path`	open a file by its path; Tab completes
Space - · Space |	split: a new window below · to the right
Ctrl+H · J · K · L	to the window on the left · below · above · on the right
Ctrl+↑ · ↓ · ← · →	make the window taller · shorter · narrower · wider
Space w d · Space w m	close the window (the file stays open) · zoom it to the whole screen and back
Ctrl+W `=`	make all windows the same size
Space Tab Tab · Space Tab d	a new tab, a whole new layout of windows · close it

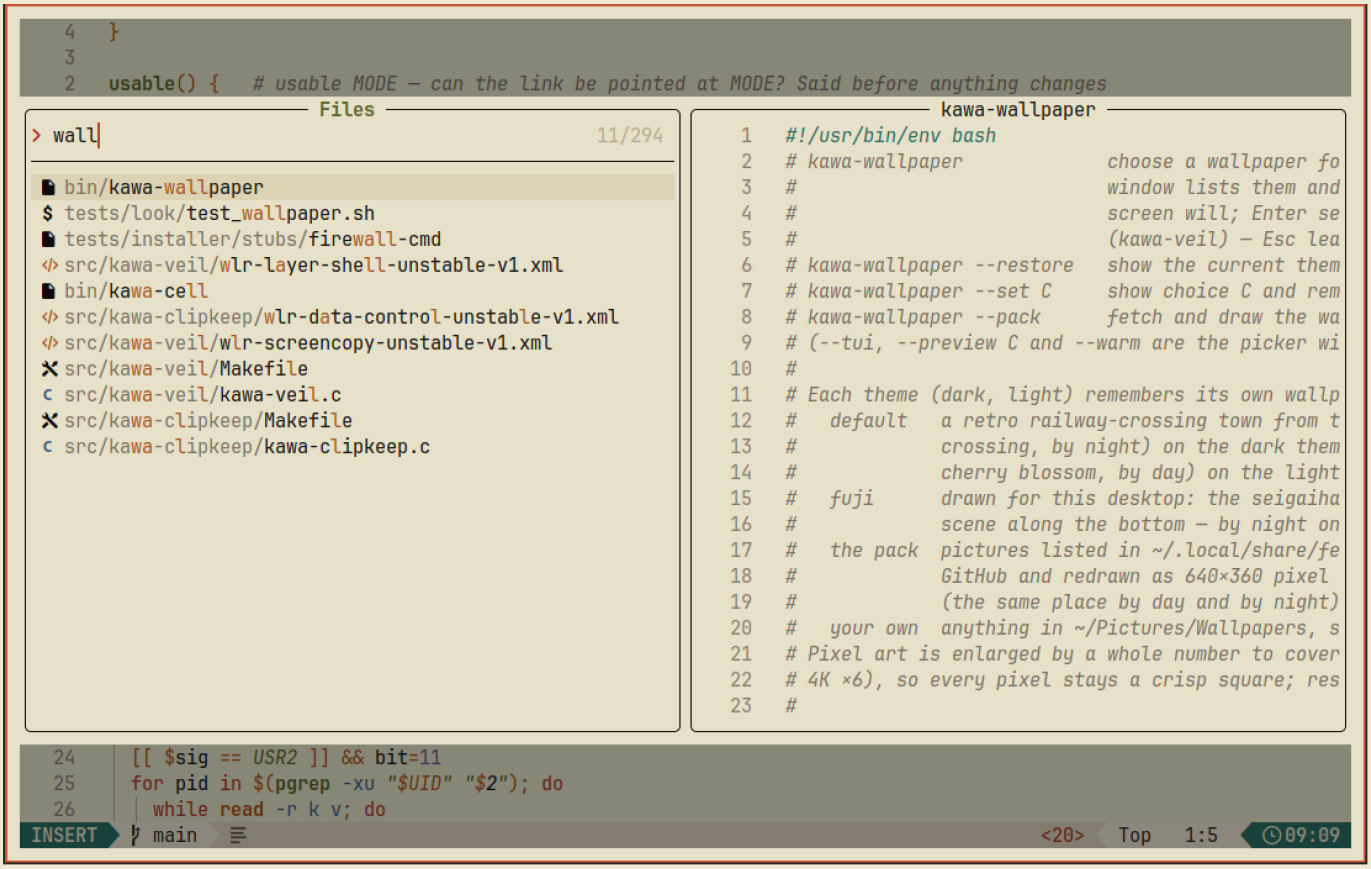


Two windows side by side, two shell scripts, the file explorer on the left and the open buffers listed along the top.

17.11 Finding

Space Space opens the *picker*, LazyVim's search window: a line to type in, the matches under it, a preview of the file beside them. It matches loosely — `brc` finds `bashrc`. The same window serves every search in LazyVim (files, text, buffers, help, keys), with the same keys:

Keys in a picker	Does
typing	filters the list
↑ ↓ or Ctrl+J · Ctrl+K	move in the list
Enter	open
Ctrl+V · Ctrl+S · Ctrl+T	open in a window to the right · below · in a new tab
Tab	mark several; Enter then opens them all
Alt+H · Alt+I	include hidden files (`.bashrc`, a dotfiles folder) · files git ignores
Ctrl+Q	put every result in the quickfix list (*Search and replace*, below)
Alt+P · Ctrl+F · Ctrl+B	preview on or off · scroll it down · up
Esc Esc, or Ctrl+C	close. The first Esc only stops the typing; `?` then lists every key



The file picker (Space Space): the typed letters, the matching files, a preview of the one selected.

The "project" (LazyVim calls it the *root*) is the folder the language server works in, or else the nearest folder above the file that holds `.git` (a git repository) or `lua`, or else the folder neovim was started in.

17.12 The file tree

Space e opens the project's folders at the left; Enter (or `l`) on a file opens it and the tree stays, `h` closes a folder. In the tree, `a` makes a new file (end the name with `/` for a folder), `r` renames, `d` deletes — to the desktop trash, after asking — `c` and `m` copy and move, `H` shows hidden files and `I` the ones git ignores, `o` opens a file with the desktop's program for it, `q` closes the tree and `?` lists the rest.

17.13 The leader key

Most of LazyVim's own commands start with **Space**, the *leader*. Press it and wait a moment: a panel in the lower right lists what can follow, in groups — `f` files, `s` search, `g` git, `c` code, `b` buffers, `w` windows, `u` interface switches, `x` problem lists, `q` quitting and sessions. Press a letter to go down a level, Backspace to come back up, Esc to give up. The same panel helps after `g`, `z`, `[`, `]` and `"`, and Space s k searches every key by its description — type *split* or *comment*.

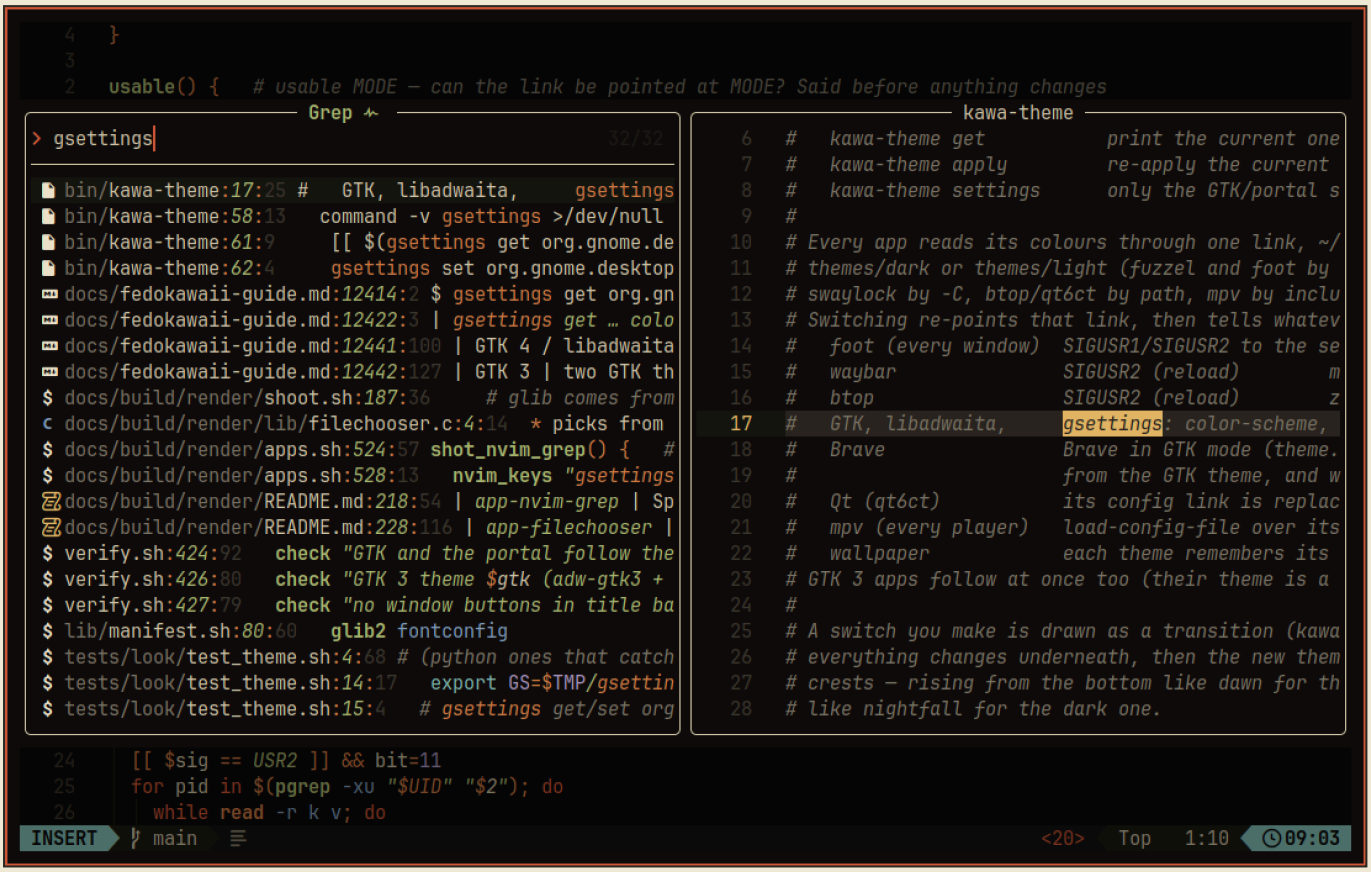


The key panel after Space: the groups that can follow, in the palette.

Keys	Does
Space Space · Space f f	find a file in the project
Space / · Space s w	search the project's text · the word under the cursor (or the selection) everywhere
Space , · Space f r	the open buffers · recent files
Space f c · Space f n	a file of neovim's own config · a new empty file
Space e	the file tree
Space g g	lazygit, in a floating window (§41)
Space g b · Space g f · Space g l	the history of this line · of this file · of the project
Space b d	close the buffer
Space - · Space | · Space w d	split below · split right · close the window
Space c f	format the file, or the selection
Space c a · Space c r	code actions · rename a symbol everywhere
Space c d · Space x x	the problems on this line · all problems, in a list
Space s r	search and replace across files
Space s k · Space s h	search the keys · the help pages
Space s " · Space s u	the registers · the undo history
Space u w · Space u s · Space u l	line wrap · spelling · line numbers, on or off
Space u f · Space u F	formatting on save, on or off: everywhere · this file
Space q s · Space q q	restore the last session in this folder · quit everything
Space l · Space c m	plugins (Lazy) · tools (Mason)
Space ?	the keys that belong to this buffer only

17.14 Search and replace across a project

Space / searches the text of every file in the project as you type (with ripgrep: what `.gitignore` excludes and hidden files are skipped — Alt+H and Alt+I bring them in). Each result shows the file, the line and the match; the preview shows the place around it, and Enter goes there.



Space /: every line in the project containing the typed text, the place previewed beside it.

To change text in many files, Space s r opens **grug-far**, a replace form in a window of its own:

1. Type what to find on the *Search* line. Matches from every file appear underneath as you type.
2. Move down to *Replace* (Esc, then `j`, or Tab in Normal mode) and type the new text. The results show each line as it will be — nothing is written yet.
3. *Files Filter* starts with the current file's extension (`*.sh` when you came from `setup.sh`; empty when you came from a script without one, such as those in `bin/`): clear it to search every file, or write your own (`*.md`). *Flags* takes ripgrep's options: `-i` to ignore case, `-w` for whole words, `--hidden` for hidden files.
4. In Normal mode, `\r` — backslash, then r; backslash is LazyVim's *local leader* — writes every replacement into the files. `g?` lists the other keys, `\c` closes the form.

The classic way works too: Space /, then Ctrl+Q puts every match in the *quickfix list* — neovim's list of places to step through — and `:cdo s/old/new/g | update` replaces in each listed line and saves each file. `]q` and `[q` step through the list; Space x q shows it.

Tip: run a project-wide replace in a git repository with everything committed first: lazygit then shows exactly what changed, and discarding takes it all back (§41).

17.15 Language servers

A *language server* is a helper program that understands one language: where a name is defined, where it is used, what is wrong. Neovim talks to it and shows the answers — a problem is underlined, with a short note at the end of the line and a sign in the margin. LazyVim brings one for Lua (the language of neovim's config) from the start; others come with *extras* and Mason (below). Space c l shows which servers serve the current file.



A language server at work: a problem underlined with its note, and the documentation that K shows.

Keys	Does
`gd` · `gr`	go to where the name under the cursor is defined · list every place it is used
`K`	the documentation of the name under the cursor (Ctrl+F · Ctrl+B scroll it)
`gK` · Ctrl+K in Insert mode	the parameters of the function being called
Ctrl+O	back to where you were before `gd`
Space c a	code actions: the fixes the server offers here
Space c r	rename the symbol everywhere it is used
`]d` · `[d`	next · previous problem (`]e` `[e` errors only, `]w` `[w` warnings)
Space c d	the full text of the problems on this line
Space x x · Space s d	every problem, in a list · in the picker
Space s s	jump to a function, heading or other symbol in this file
Space u h	the grey hints the server writes into the code (types, parameter names), on or off

For this desktop's shell scripts:

1. `:Mason`, type `/bash` and Enter to find **bash-language-server**, press `i`. (It is an npm package; `:checkhealth mason` says whether npm is found.)
2. Open a shell script. The server checks it with ShellCheck (installed with the desktop: the server finds it by itself), completes variable names and finds a function's definition with `gd`.
3. Try it: a line `echo $1` gets a note — *Double quote to prevent globbing and word splitting*, ShellCheck's SC2086, the quoting rule of §25.4; `echo "$1"` clears it. Every finding has a page that explains it: shellcheck.net/wiki/SC2086.

17.16 Completion

While you type, a menu offers what could come next: names from the language server, file paths (after `./`, `~/` or `/`), snippets (small templates with blanks to fill in, such as a whole `for` loop), and words from the open files. A box beside it shows the documentation of the highlighted entry.

Keys in Insert mode	Does
↑ ↓ or Ctrl+P · Ctrl+N	move in the menu
Enter · Ctrl+Y	take the highlighted entry
Ctrl+E	close the menu and keep typing
Ctrl+Space	open the menu yourself; again for the documentation
Tab · Shift+Tab	in an inserted snippet: to its next · previous blank to fill in
Ctrl+B · Ctrl+F	scroll the documentation

On the command line (after `:`), Tab completes commands and file names.

17.17 Formatting

LazyVim tidies a file's layout — indentation, spacing, line breaks — **each time you save it**, when it has a formatter for the type (conform.nvim runs them; stylua, for Lua, comes installed). Shell scripts are the exception here: `after/ftplugin/sh.lua` turns formatting on save off for them, because shfmt would re-indent and re-align them, this desktop's own scripts among them.

Keys or command	Does
Space c f	format now: the file, or the selection — a shell script too, when you ask
Space u f	formatting on save on or off, everywhere
Space u F	the same for this file only (shell scripts start at off)
`:ConformInfo`	which formatters apply to this file, and whether they are installed

17.18 Extras and Mason: adding a language

Language support comes in *extras*: `:LazyExtras` (or `x` on the start screen) lists them; move to one, press `x` to switch it on or off, and restart neovim. The choice is kept in `~/.config/nvim/lazyvim.json`, which is yours.

Extra	Brings
`lang.markdown`	for notes: a language server for links and headings, Markdown checks, headings and tables drawn in place (Space u m), a live preview in the browser (Space c p)
`lang.python` · `lang.json` · `lang.yaml` · `lang.toml`	each language's server, and its formatter where it has one
`coding.mini-surround`	the `gs` keys for quotes and brackets (above)

The servers and tools then install through **Mason** (`:Mason`, or Space c m), which downloads them into neovim's data folder — binaries, PyPI and npm packages alike (Node.js is installed). In the Mason window: `i` install, `u` update, `U` update all, `X` remove, Ctrl+F show one language's tools only, `g?` help.

17.19 How LazyVim is set up here

LazyVim is a ready-made configuration for neovim: a collection of plugins and settings, installed and updated by lazy.nvim. Setup copied LazyVim's *starter* into `~/.config/nvim`. The repository adds three files to it, and Theme manages only those:

File	Does
`lua/config/lazy.lua`	the starter's own, with the background check for new plugin versions switched off, so plugins move only with `update`
`lua/plugins/colorscheme.lua`	the remapped gruvbox, which follows the dark and light theme by itself (§40.2)
`after/ftplugin/sh.lua`	keeps shell scripts from being reformatted on save (§17.17)

Everything else in the folder is yours: your own plugins and settings go beside them (*Pitfalls*, in §17.22).

17.20 Keeping it updated and healthy

`update` already runs `Lazy! sync`, which brings every plugin to its newest version and records the versions in `~/.config/nvim/lazy-lock.json`. Inside neovim, `:Lazy` (Space l) shows them: `U` update, `S` sync, `R` restore the versions in the lock file — the way back when an update breaks something. Mason's tools are **not** part of `update`: open `:Mason` and press `U` now and then. Space L shows LazyVim's changelog. `:checkhealth` explains anything that does not work: every plugin and tool reports OK, WARNING or ERROR, with advice. `:LazyHealth` loads every plugin first, so none is skipped.

17.21 A daily workflow

1. **Super**+**Return**, the project's folder (`~/repos/notes` — a folder name alone changes into it, §18.1), then `nvim`.
2. On the start screen, `s` brings back the files you had open here last time; or Space Space for a file, `r` for a recent one.
3. Work. Ctrl+S saves; Space / finds anything in the project; Shift+H and Shift+L move between the open files; `gd` and `K` explain code.
4. Space g g: read the diff, stage, commit, push (§41); `q` returns to the editor.
5. Space q q quits. The session of this folder is saved by itself, ready for step 2 tomorrow.

17.22 A learning path

Fifteen minutes a day beats an afternoon a month: the keys move into your hands only by use.

When	Do	Why
day 1	`nvim --clean +Tutor`, about 30 minutes; then, in the same neovim, `:Tutor vim-02-beginner` (neovim 0.11.3 and later; Fedora 44 has 0.12). Inside LazyVim `:Tutor` is switched off	the official lessons, in plain neovim without LazyVim's changes, exactly as written (`vimtutor` belongs to Vim, which is not installed)
week 1	the first hour above, again each day: moving, typing, `x`, `dd`, `u`, `:w`, `:q`	until you stop reaching for the mouse and the arrows
week 2	the grammar: `dw`, `cw`, `ciw`, `ci"`, `dt)` — and `.`	every verb you know multiplies with every motion
week 3	Space Space, Space /, Space e, buffers and splits	moving between files as quickly as within one
week 4	visual block, `:%s`, one macro a day on real work	the keys that save hours
afterwards	one new key a week, from the Space panel or Space s k	small steps stay learnt

Pitfalls.

- LazyVim changes a few keys of plain Vim: `s` and `S` are flash (Vim's `s` is `cl`), Shift+H and Shift+L switch buffers (in Vim they move to the top and bottom of the screen), Ctrl+H/J/K/L move between windows, and `K` shows documentation. Tutorials written for Vim mean the plain keys; `nvim --clean` has them.
- With the completion menu open, **Enter takes the highlighted entry** instead of starting a new line. Ctrl+E closes the menu first.
- Markdown, plain text and commit messages wrap long lines — `j` and `k` then move by screen line — and check spelling: `]s` · `[s` jump between mistakes, `z=` offers corrections, `zg` adds the word to your list, Space u s turns the checking off.
- Hidden files do not show in Space Space or Space / — the dotfiles of a dotfiles repository included. Alt+H in the picker, `H` in the tree.
- `lang.markdown` also corrects Markdown files on save where its checker finds problems. For a file whose layout must stay as it is — this handbook's source, say — Space u F turns it off.
- Do not set `background` yourself, and leave Space u b alone: neovim learns light or dark from foot and changes with the theme.
- Your own plugins and settings go in `lua/plugins/*.lua` and `lua/config/*.lua`; only the three files named in §17.19 are managed by Theme (`lua/config/lazy.lua` among them, so leave that one to the repository). A setting of your own, for example:

```
-- ~/.config/nvim/lua/config/options.lua — yours; LazyVim reads it at start
vim.opt.relativenumber = false   -- the margin shows plain line numbers
vim.opt.wrap = true              -- long lines wrap in every file, not only in text
```

Cheat sheet.

To	Keys
get back to Normal mode	Esc
save · quit · both · quit without saving	Ctrl+S · `:q` · `:wq` · `:q!`
undo · redo · repeat the last change	`u` · Ctrl+R · `.`
move by character · word · to the line's start, end	`h` `j` `k` `l` · `w` `b` `e` · `0` `$`
top · bottom of the file · line 42	`gg` · `G` · `42G`
half a screen down · up	Ctrl+D · Ctrl+U
jump to anything on screen	`s`, a letter or two, the label
back · forward along your jumps	Ctrl+O · Ctrl+I
type before · after · at the line's start · end · on a new line	`i` · `a` · `I` · `A` · `o` `O`
delete a character · a line · to the line's end	`x` · `dd` · `D`
change a word · inside quotes · inside brackets	`ciw` · `ci"` · `ci(`
copy a line · a paragraph · paste	`yy` · `yip` · `p` `P`
indent · comment out a line	`>>` · `gcc`
move a line down · up	Alt+J · Alt+K
select characters · lines · a block	`v` · `V` · Ctrl+V
search · next · previous · the word under the cursor	`/text` · `n` · `N` · `*`
replace in the whole file	`:%s/old/new/g` (`gc` to confirm each)
record · stop · play a macro	`qa` · `q` · `@a`, `@@`, `10@a`
open the link under the cursor in Brave	`gx`

To	Keys
fold · unfold · open every fold	`zc` · `zo` · `zR`
find a file · text · an open buffer	Space Space · Space / · Space ,
the file tree · lazygit	Space e · Space g g
previous · next buffer · close it	Shift+H · Shift+L · Space b d
split below · right · move between · close	Space - · Space | · Ctrl+H/J/K/L · Space w d
definition · uses · documentation · rename	`gd` · `gr` · `K` · Space c r
next problem · all problems	`]d` · Space x x
format · code action	Space c f · Space c a
replace across the project	Space s r
every key	Space, then wait · Space s k
plugins · tools · diagnosis	`:Lazy` · `:Mason` · `:checkhealth`

Read more: inside neovim, `:help` — `:help usr_toc` is the user manual chapter by chapter, and `:help text-objects`, `:help registers`, `:help pattern` and `:help lsp` go straight to a topic (the same pages are at neovim.io/doc/user). LazyVim's keys, plugin by plugin: lazyvim.org/keymaps, with the extras under *Extras* on the same site. Mason: github.com/mason-org/mason.nvim.

What you learnt

- **Esc** always returns to Normal mode; `u` undoes; `:q!` leaves without saving.
- **i** starts typing before the cursor, **o** on a new line; `:wq` saves and quits, Ctrl+S saves.
- A verb and a motion make a command — `dw`, `ciw`, `d$` — and `.` repeats the last change.
- Text objects name a piece by what it is: `iw` a word, `i"` inside quotes, `ip` a paragraph.
- Plain `y` and `p` use the desktop's clipboard.
- Space Space finds a file, Space / finds text, and Space alone lists everything else.

Check yourself

1. Keys do strange things and you do not know which mode you are in. What do you press? → *Esc, twice if unsure*
2. What does `ciw` do, wherever the cursor is in the word? → *deletes the whole word and starts typing in its place*
3. You changed a file by mistake and want it back as it was saved. How? → *`:q!` to quit without saving, or `:e!` to reload it*

18. Finding anything: the shell's helpers

> **In this chapter**
> - find an earlier command, a file or a folder by typing a few letters of it
> - jump to a folder you use by one word of its name
> - read, search and list files with `bat`, `rg`, `fd` and `eza`
>
> **You need:** §11.8 (Tab, history and Ctrl+R) and ch. 17 (to edit what you find).

The shell you met in ch. 11 comes with a handful of small helpers, each doing one job. Together they answer the question you will ask most often at a computer: *where is it?*

18.1 bash on this desktop, and its helpers

Bash, with Fedora's own `~/.bashrc` reading `~/.bashrc.d/fedokawaii.sh` (should a `~/.bashrc` not read that folder, Theme adds Fedora's few lines for it). The prompt and the history size are in §11.3, the invisible OSC 7 / 133 marks in §40.7; this is the rest of what the file and `~/.inputrc` set. A handful of small tools make it quicker, each doing one job:

Tool	Job	Replaces
fzf	find by typing a few letters of anything: history, files, folders	scrolling, `Ctrl+R`'s one-match-at-a-time search, long paths
zoxide	`cd` remembers the folders you use and jumps to them by a word	typing `cd ../../repos/project`
eza	`ls`, with folders first, sizes you can read, each file's git state, trees	`ls -la`, `tree`
tldr	the five commands you actually wanted, with examples	reading a whole man page
bat, **ripgrep**, **fd**	reading a file, searching inside files, finding files (§18.2)	`cat`, `grep -r`, `find`

Keys or input	Does
Up / Down	earlier / later commands that start with what you have already typed: `git c`, then Up, offers `git commit …`, `git checkout …` in turn
Ctrl+R	search the whole history in fzf: type any part of a command, Enter puts it on the line. Ctrl+R again lists in time order instead of best match first; Shift+Delete removes the highlighted command from the history (a password typed by mistake)
Ctrl+T	pick a file or folder below here (previewed: a file's start, a folder's tree) and insert its path where you are typing
Alt+C	pick a folder below here and go into it
`nvim **`, then Tab	fuzzy completion for any command: `**` + Tab opens fzf for that argument
`cd proj`	jumps to the folder you use most whose name matches *proj*, from anywhere (`~/repos/project`); `cd re pro` matches both words in order; a real path still works exactly as always
`cdi`	the folders zoxide knows, in fzf
Tab	complete, ignoring case; one press lists every choice when there are several
Ctrl+Left / Right	move by word
Ctrl+Backspace · Ctrl+Delete	delete a word backwards · forwards
Alt+.	insert the last argument of the previous command (press again for the one before): after `mkdir ~/repos/notes`, typing `cd` and Alt+. gives `cd ~/repos/notes`

Keys or input	Does
a folder name on its own	changes into it (`autocd`)
`**` in a path	any depth of folders: `ls **/*.pdf` (`globstar`)

In every fzf list — Ctrl+R, Ctrl+T, Alt+C, the clipboard history — each word you type narrows the list, matched loosely:

Type	Keeps
`dnf inst`	lines that contain both words, in any order
`'ssh`	lines with exactly *ssh* (a leading `'` turns the loose match off)
`^bin` · `.pdf$`	lines that start with *bin* · end in *.pdf*
`!sudo`	lines without *sudo*

Alias or setting	Does
`ls` · `ll` · `lt`	eza: names, folders first · everything, with sizes, dates (`2026-10-15 13:04`) and each file's git state (`N` new, `M` modified) · a tree, two levels deep
`tldr tar`	examples for `tar` (any command); the pages are fetched on first use and refreshed at most once a quarter, when you ask
`update` · `verify` · `repair`	§35.1
`snaps`	the snapshot list (§37.4)
`dwl-log`	the compositor's log, opened at the end
`torrra` · `prowlarr`	torrra through its key wrapper · Prowlarr's web page, started first (`prowlarr stop` when done) (§20)
`ssh`	always reaches the server as `xterm-256color` : few servers know foot's own terminal type
`history`	shows the date and time of each command
`EDITOR` , `VISUAL` , `vi`	all neovim; `man` pages open in neovim too (`q` closes; `gO` lists the sections)
`less`	colours kept, mouse wheel works, searches ignore case unless you type a capital

Try this. Pick a file without typing its path, then jump to a folder by a word.

1. In your home, type `bat` — with a space — and press **Ctrl+T**. A list of the files and folders below your home opens, with a preview of the highlighted one beside it. Type `notes` , move to `Notes/notes.md` , and press **Enter**: the line now reads `bat Notes/notes.md` . Enter once more, and bat shows your notes with line numbers.

2. Go to your notes folder once, then somewhere else, then back by a word:

```
$ cd ~/Notes
$ cd /tmp
$ cd not
$ pwd
/home/you/Notes
```

You typed	It means
`cd ~/Notes`	an ordinary `cd` ; zoxide notes the folder as you arrive
`cd /tmp`	anywhere else
`cd not`	there is no `not` in `/tmp` , so zoxide jumps to the folder you have used whose name matches

Pitfalls. `cd` here is zoxide's: a name that is not a folder where you are is looked up among the folders you have used, so `cd notes` typed in the wrong place lands in `~/repos/notes` if that is the *notes* you use most. Check the prompt before running something destructive there. A misspelt name is not corrected either: `cd Dcouments` answers *zoxide: no match found* (`builtin cd Dcouments`, bash's own `cd`, does correct it, through `cdspell`). Outside foot — the text consoles (Ctrl+Alt+F2…F6), a login from another machine — the prompt uses plain characters, because only foot has the icon font.

18.2 bat, ripgrep, fd, fzf, eza, zoxide and tldr

Small command-line tools, each for one job; ripgrep and fd also power neovim's finders and yazi's search, fzf the clipboard window, the shell's Ctrl+R / Ctrl+T / Alt+C and yazi's **z**, zoxide the shell's `cd` and yazi's **Z**. Their keys in the shell are in §18.1.

Command	Does
`bat file`	the file with line numbers, git changes in the margin and syntax colours from the terminal's palette (so it follows the theme)
`bat -p file` · `bat -A file`	plain, no decorations · show tabs, spaces and line ends
`rg text`	search below the current folder, skipping what `.gitignore` excludes
`rg -i text` · `rg -l text` · `rg -t py text`	ignore case · only file names · only Python files
`rg -uu text`	also hidden and ignored files
`fd name`	find files whose name contains *name*
`fd -e pdf` · `fd -t d conf`	by extension · folders only
`fd -u -S +1g . ~`	files over 1 GB anywhere in your home, hidden and ignored ones included (`-u`; `-H` alone still skips what a `.gitignore` names)
`fd -e jpg -x magick {} {.}.webp`	run a command on every match
`nvim "$(fzf)"`	pick any file below here by typing part of its name, and edit it
`ll` · `lt` · `eza -lT -L 3 --git`	the folder in detail · as a tree · a deeper tree with sizes and git state
`ll -s modified` · `ll -s size -r`	newest last · biggest first
`cd proj` · `cdi` · `zoxide query -l`	jump by a word · pick in fzf · every folder zoxide knows, best first
`zoxide remove ~/old/folder`	forget a folder
`tldr ffmpeg` · `tldr --update`	examples · refresh the pages now

On Fedora the fd package is called `fd-find`, but the command is `fd`.

Tips. fzf matches *fuzzily*: the letters you type must appear in that order, not side by side. In Ctrl+R and every other fzf list, words separated by a space may come in any order, and a few marks narrow the search:

You type	Finds
`ssh 192`	lines holding both, in any order — `ssh admin@192.168.1.20`
`'word`	*word* exactly, its letters side by side
`^git` · `.md$`	lines that start with `git` · that end in `.md`
`!test`	everything that does **not** contain `test`
`.sh$ | .py$`	either one: `|` means *or*

zoxide ranks by *frecency* — a score of how often and how recently you went to a folder — so the folder you live in wins after a day or two; `zoxide query -ls` lists the scores.

Read more: `man rg`, `man fd` and `man fzf` (they open in neovim here), or `tldr` with any of these names.

18.3 Practice: find a file, a word, a folder

Five everyday "where is it?" questions, each answered from a terminal. Your files are your own, so your lines will differ: what matters is their shape.

1. Every PDF in your home, wherever it is.

```
$ fd -e pdf . ~
/home/you/Documents/Tenancy contract.pdf
/home/you/Documents/Bank statement October.pdf
/home/you/Books/statement.pdf
```

(for example) `-e pdf` keeps one extension, `.` matches any name, and `~` says where to look. fd skips hidden folders and whatever a `.gitignore` leaves out.

If you see nothing: no PDF is in your home yet — try `fd -e png . ~` for your screenshots.

2. Where a word appears in your notes.

```
$ rg -i rice ~/Notes
/home/you/Notes/notes.md
3:rice cooker: compare prices
8:Rice, two kilos
```

(for example) The file's name, then each line that holds the word, after its line number. `-i` ignores case, so *Rice* is found too.

If you see nothing: no note holds that word — search for one you wrote in §16.3.

3. Jump to a folder by one word of its name, from anywhere.

```
$ cd ~/Books
$ cd /etc
$ cd boo
$ pwd
/home/you/Books
```

zoxide learnt `~/Books` when you went there; from `/etc`, `boo` is not a folder, so it jumps. In yazi, **Z** and the same word does the same (§10.3).

If you see `zoxide: no match found`: you have not been to that folder yet in a terminal — `cd` to it once by its path.

4. Pick a file to edit by typing part of its name.

```
cd ~
nvim "$(fzf)"      # a list of every file below here; type, then Enter
```

fzf lists the files below your home; type `notes`, choose `Notes/notes.md` and press Enter: neovim opens it. `:q` closes it. `$( … )` is replaced by what the command inside it prints — here the name you chose — and the quotes round it keep a name with spaces whole (quoting: ch. 25).

If neovim opens with no file in it: you left the list with Esc — `:q`, and try again.

5. The biggest folders in your home.

```
$ du -xh -d1 ~ | sort -h | tail -5
1.4G    /home/you/Videos
2.2G    /home/you/.cache
3.1G    /home/you/Documents
19G /home/you/.local
26G /home/you
```

(for example)

Part	Does
`du -xh -d1 ~`	the size of each folder in your home: `-h` in K, M and G, `-d1` one level down, `-x` on this subvolume only
`| sort -h`	`|` hands those lines to `sort`, which puts them smallest first, reading `G` as more than `M`
`| tail -5`	keeps the last five: the biggest

The last line is your whole home. `~/Downloads` is missing: it is a subvolume of its own, which `-x` steps over (§38.2) — `du -sh ~/Downloads` measures it alone. Steam's games live in `~/.local/share`, which is why `.local` is often the largest.

> **In SQL terms.** The pipeline reads like `SELECT size, folder FROM du ORDER BY size LIMIT 5` — except that `sort` goes smallest first, so `tail` keeps the last five rows instead of the first.

If you see `du: invalid maximum depth 'l'`*:* the option is `-d1`, with the digit one.

What you learnt

- **Ctrl+R** searches your history, **Ctrl+T** inserts a path, **Alt+C** goes into a folder — each in fzf.
- `cd` and one word jumps to a folder you have used; check the prompt before acting there.
- `fd` finds files by name, `rg` finds text inside files, `bat` shows a file with line numbers.
- `du -xh -d1 ~ | sort -h | tail -5` lists the biggest folders; `|` passes lines along.

Check yourself

1. Which command lists every PDF below your home? → `fd -e pdf . ~`
2. How do you find which note mentions *visa*, whatever its case? → `rg -i visa ~/Notes`
3. Why can `cd notes` land in a folder you did not mean? → *zoxide jumps to the matching folder you use most when there is no* `notes` *where you are*

19. Do it yourself: what websites charge for

> **In this chapter**
> - join, split, shrink, lock, sign and search PDFs
> - convert, shrink and clean pictures, video and sound
> - download, record, pack and check files, and convert units and money
>
> **You need:** ch. 11 (commands, folders, quotes) and ch. 18 (finding the file).

Joining two PDFs, shrinking a video for WhatsApp, turning an iPhone photo into a JPEG: the usual answer is a website that wants the file uploaded, stops at a size or a page count, stamps a watermark on the result or asks for an account, and keeps a copy on a server you know nothing about. Every recipe in this chapter does the same job on this computer, with programs setup has already installed (`lib/manifest.sh`). Nothing leaves the machine; there are no size limits, no daily quotas, no watermarks and no sign-ups; a contract, a passport scan or a bank statement stays yours. Most recipes are one command and finish in a second.

Term	In one line
PDF · picture	a PDF page keeps text, drawings and pictures apart, so text stays sharp at any size; a picture (JPEG, PNG, HEIC) is only a grid of pixels
lossless · lossy	lossless keeps every pixel or sound exactly (PNG, zip); lossy drops detail you are unlikely to miss, for a much smaller file (JPEG, WebP, MP3, H.264)
resolution, dpi	dots (pixels) per inch of paper: a scan 2480 pixels wide is a sharp A4 page at 300 dpi
OCR	*optical character recognition*: a program reads the letters in a picture and turns them into text
metadata	facts about a file kept inside it: author, dates, and in a photo (its *EXIF*) the phone and the place
codec · container	how a stream is compressed (H.264, HEVC, AAC) · the file that holds the streams (`.mp4`, `.mov`); see §15.2
checksum	a fingerprint computed from every byte of a file: change one byte and it comes out completely different (§3.3)
encoding	how letters are stored as bytes: UTF-8 today, Windows-1256 in many older Arabic files
regular expression	a pattern for text: in `s/\r$//`, `\r$` means "a carriage return at the end of a line"

19.1 Before you start

- Open a terminal (**Super**+**Return**) and go to the folder with the files: `cd ~/Documents`, or, once you have been there, a word of its name (`cd docs`, §18.2). In yazi, **c d** copies the folder's path; paste it after `cd` with Ctrl+Shift+V.
- Names with spaces go in quotes, as in the examples: `"Tenancy contract.pdf"`. Tab completes a name and adds the quoting itself.
- Every recipe writes a **new** file and leaves the original alone, unless it says otherwise. Open the result first; only then send the original to the trash (§19.10).
- A password typed into a command is kept in the shell history unless the line starts with a **space** (§25.7). The code blocks below that hold a password start with one.

Try this. Every recipe has the same shape: go to the folder, run one command that writes a new file, and check the result. Here, a tenancy contract and its Ejari certificate become one PDF:

```
$ cd ~/Documents
$ qpdf --empty --pages "Tenancy contract.pdf" "Ejari certificate.pdf" -- "Tenancy, complete.pdf"
$ qpdf --show-npages "Tenancy, complete.pdf"
14
```

(for example: 12 pages and 2)

You typed	It means
`cd ~/Documents`	go where the files are
`qpdf --empty --pages A B -- C`	start from nothing, take every page of A, then of B, and write C; the originals stay as they were
`qpdf --show-npages C`	check: the new file has the pages of both

Then open it — `zathura --fork "Tenancy, complete.pdf"` — before you send it anywhere.

Tip: `tldr qpdf`, `tldr magick` and `tldr ffmpeg` show a page of further examples for each program (§18.2).

19.2 PDFs: join, split, pick and turn pages

qpdf moves whole pages around without touching what is on them, so nothing is lost, and it is instant even on a book.

Want	Command
join files, in this order	`qpdf --empty --pages cover.pdf "Annual report.pdf" annex.pdf -- complete.pdf`
join every PDF in a folder, by name	`qpdf --empty --pages scans/*.pdf -- scans.pdf`
pages from several files	`qpdf --empty --pages report.pdf 1-3 annex.pdf 2 -- summary.pdf`
pages 3 to 7	`qpdf report.pdf --pages . 3-7 -- part.pdf`
everything but pages 2 and 5	`qpdf report.pdf --pages . 1-z,x2,x5 -- trimmed.pdf`
a new order · back to front	`qpdf report.pdf --pages . 3,1,2,4-z -- reordered.pdf` · `--pages . z-1`
odd pages · even pages	`--pages . 1-z:odd` · `--pages . 1-z:even`
one file per page	`qpdf --split-pages report.pdf page.pdf` → `page-01.pdf`, `page-02.pdf` … (as many digits as the last page needs)
files of ten pages each	`qpdf --split-pages=10 book.pdf part.pdf` → `part-01-10.pdf`, `part-11-20.pdf` …
turn every page a quarter right	`qpdf scan.pdf --rotate=+90 scan-turned.pdf`
turn some pages (`-90` turns left)	`qpdf scan.pdf --rotate=180:2,4 scan-fixed.pdf`
turn page 1 in the file itself	`qpdf --replace-input --rotate=+90:1 scan.pdf`
how many pages	`qpdf --show-npages report.pdf`

How a page list reads, on a 12-page file:

Page list	Gives
`3-7`	pages 3 to 7
`z` · `z-1`	the last page · every page, last first
`1-z,x2,x5`	1, 3, 4, 6 … 12: `x` takes pages out of the range before it
`3,1,2,4-z`	3, 1, 2, then 4 to 12
`2-z:odd`	2, 4, 6 … 12: `:odd` and `:even` count positions in the list, not page numbers

`.` stands for the file named before `--pages`, and a file given without a list comes in whole. Turning a page only changes a flag, so it costs nothing either.

Pitfalls. A wildcard sorts by name, so `scan-10.pdf` comes before `scan-2.pdf`: name files `scan-02.pdf`, or list them in the order you want. A wildcard also matches the file you made last time — keep the output outside that folder, as above. `--replace-input` is the one recipe here that changes the original.

Read more: qpdf's manual, at `--pages`, `--split-pages` and `--rotate`.

19.3 PDFs and pictures

Pictures out of a PDF. `pdfimages -list brochure.pdf` shows what is inside, one line per picture:

```
page   num  type   width height color comp bpc  enc interp  object ID x-ppi y-ppi size ratio
--------------------------------------------------------------------------------------------
   1     0 image    2480  3508  rgb     3   8  jpeg   no         8  0   300   300 3493K  14%
```

`width` and `height` are its pixels, `enc` how it is stored (`jpeg`; `image` for one kept without loss) and `x-ppi` its resolution on the page. `-all` writes each picture in the format it is stored in, so a JPEG comes out byte for byte as it went in, at its full size — which no screenshot gives you:

```
mkdir brochure-images
pdfimages -all brochure.pdf brochure-images/img           # img-000.jpg, img-001.png …
pdfimages -all -f 3 -l 3 brochure.pdf brochure-images/p3  # page 3 only
```

Black-and-white scans can come out in fax formats (`.ccitt`, `.jb2e`) that few programs open; `-png` in place of `-all` turns everything into PNG.

Pages into pictures. `pdftoppm` draws each page; `-r` is the resolution — 150 for a screen or a message, 300 for printing or OCR.

Want	Command
every page as PNG	`pdftoppm -r 150 -png report.pdf page` → `page-1.png` … (`page-01.png` from ten pages up)
JPEG, smaller files	`pdftoppm -r 150 -jpeg -jpegopt quality=85 report.pdf page`
only page 2, one file	`pdftoppm -r 300 -png -f 2 -l 2 -singlefile report.pdf page2` → `page2.png`
a thumbnail, 300 px on its long side	`pdftoppm -png -scale-to 300 -f 1 -singlefile report.pdf cover` → `cover.png`

Pictures into a PDF. ImageMagick puts one picture on each page. `-density` sets the page size, because pixels ÷ density = inches. Left out, the picture's own value is used — 72 on a phone. For an A4-wide page, use the picture's width ÷ 8.3:

Picture	`-density`	Page width
phone photo, 4032 px wide	left out (72)	56 in — 1.42 m
phone photo, 4032 px wide	480	8.4 in — 21.3 cm, A4
scan, 1240 px wide	150	8.3 in — 21 cm, A4

```
magick receipt-1.jpg receipt-2.jpg receipt-3.jpg -units PixelsPerInch -density 480 receipts.pdf
magick id-front.jpg id-back.jpg -append +repage -units PixelsPerInch -density 300 "ID card.pdf"
```

The second puts both sides of a card on one page, front above back; without `+repage` the page keeps the size of the first picture. PNGs go in without loss. JPEGs are decoded and saved again at their own quality — hard to see, but a second generation. To put JPEGs in exactly as they are, let tesseract build the PDF: it copies JPEG data untouched, and makes the text searchable as a bonus (it reads every page, so allow a few seconds each):

```
ls receipt-*.jpg > list.txt
tesseract list.txt receipts --dpi 480 -l eng+ara pdf      # writes receipts.pdf
```

Read more: `man pdfimages`, `man pdftoppm` and `man tesseract`; every ImageMagick option is on its options page.

19.4 PDFs: smaller, locked, unlocked

Shrink. Ghostscript writes the file anew and scales down every picture above the target resolution, saving it as JPEG. Text and drawings stay sharp, because they are not pictures.

```
gs -q -sDEVICE=pdfwrite -dPDFSETTINGS=/ebook -o "Scanned bill-small.pdf" "Scanned bill.pdf"
```

Setting	Pictures scaled to	For
`/screen`	72 dpi	the smallest file; small print in scans becomes hard to read
`/ebook`	150 dpi	email and upload forms — the usual choice
`/printer` · `/prepress`	300 dpi	printing; little smaller than before

What is lost is detail in the pictures, and only there. A picture is scaled only when it is more than 1.5 times the target: `/ebook` takes a 300 dpi scan down to 150, but leaves a 200 dpi one as it is. Pure black-and-white scans (1 bit per pixel) are never taken below 300 dpi by `/screen` or `/ebook`, so small print stays readable. In a test, two pages of 300 dpi photographs (7.8 MB) came out at 390 KB with `/ebook` and 115 KB with `/screen`; a PDF that is mostly text hardly changes.

For a middle way, give a target of 200 and lower that 1.5 to 1 — otherwise a 300 dpi scan stays untouched, since 300 is not more than 1.5 × 200:

```
gs -q -sDEVICE=pdfwrite -dPDFSETTINGS=/ebook \
   -dColorImageResolution=200 -dGrayImageResolution=200 \
   -dColorImageDownsampleThreshold=1.0 -dGrayImageDownsampleThreshold=1.0 \
   -o "Scanned bill-200.pdf" "Scanned bill.pdf"
```

Resolution is counted against the page: a phone photo on a page 1.4 metres wide (§19.3) is only 72 dpi there, so no setting touches it — make such PDFs at A4 width first.

Smaller without any loss: `qpdf --recompress-flate --compression-level=9 --object-streams=generate report.pdf report-small.pdf` packs the file's structure tighter without touching a picture or a letter: anything from a few per cent to a third on text-heavy files, nothing on scans.

Lock with a password.

```
qpdf --encrypt --user-password='open sesame' --owner-password='something else' --bits=256 -- \
     contract.pdf contract-locked.pdf
```

The user password opens the file. The owner password governs printing, copying and changes; qpdf insists on one, and it should differ from the user password — if they are the same, whoever can open the file can also lift its limits. AES-256 opens in every current PDF reader, phones included. To let anyone open a file but ask readers not to print or copy it, give only the owner password, with the limits after `--bits` (qpdf refuses them before it):

```
qpdf --encrypt --owner-password='something else' --bits=256 --print=none --extract=n -- \
     contract.pdf contract-noprint.pdf
```

Not every reader obeys these limits.

Remove a password you know.

```
qpdf --password='open sesame' --decrypt contract-locked.pdf contract.pdf
```

A file that opens without a password but refuses printing or copying needs no `--password`: `qpdf --decrypt` alone lifts those limits.

Look before you open, clean before you send. `pdfinfo invoice.pdf` shows the page count and size, the program that made it, and whether it is encrypted, holds a form or carries JavaScript — which an invoice or a CV has no business carrying. The lines to read:

```
Form:           none
JavaScript:     no
Pages:          2
Encrypted:      no
```

`qpdf --list-attachments invoice.pdf` lists files tucked inside it, and `qpdf --show-attachment=NAME invoice.pdf > NAME` takes one out. Before passing a file on, `qpdf --remove-info --remove-metadata draft.pdf final.pdf` drops the author, title, creating program and creation date recorded in it (the date of the last change stays).

Pitfall. A PDF with a certificate-based digital signature — the kind a reader can verify — stops validating once any program here rewrites it. Keep and send it as you received it.

Read more: Ghostscript's PDF settings (*-dPDFSETTINGS* and the image parameters) and qpdf's encryption options.

19.5 PDFs: text, scans and OCR

Text out. `pdftotext -layout statement.pdf` writes `statement.txt`, columns where they were; `-f 2 -l 3` limits it to pages 2–3, and `-` in place of a file name prints it (`pdftotext -layout statement.pdf - | less`). If nothing comes out, the PDF is a scan — pictures of pages — and needs OCR. There is no faithful PDF-to-Word converter here: the text is what you get, ready to paste into AbiWord.

Make a scan searchable. Tesseract reads the pages and writes an invisible text layer; qpdf lays it over the original, whose pages stay exactly as they were:

```
mkdir ocr
# every page as a 300 dpi picture: ocr/page-01.png, ocr/page-02.png …
pdftoppm -r 300 -png "Tenancy contract.pdf" ocr/page
# their names, in page order (the padded numbers keep page-10 after page-09)
ls ocr/page-*.png > ocr/list.txt
# tesseract reads them and writes ocr/text.pdf: the text alone, invisible
tesseract ocr/list.txt ocr/text --dpi 300 -l eng+ara -c textonly_pdf=1 pdf
# qpdf lays that text over the original pages
qpdf "Tenancy contract.pdf" --overlay ocr/text.pdf -- "Tenancy contract (searchable).pdf"
```

Check it: `pdftotext "Tenancy contract (searchable).pdf" - | head` prints the first lines it recognised.

Afterwards zathura's `/` finds words in it and text can be selected and copied; the file grows only a little. Then `gio trash ocr`. `-l eng+ara` reads English and Arabic; name only the languages on the page, since each one adds time. `tesseract --list-langs` shows the ones installed; others are packages (`dnf list 'tesseract-langpack-*'`) and go into the manifest like any program (§42.12). Turn sideways pages upright first (§19.2): OCR reads upright text best.

A photo or a picture.

Want	Command
its text, printed in the terminal	`tesseract receipt.jpg - -l eng+ara`
its text, into `receipt.txt`	`tesseract receipt.jpg receipt -l eng+ara`
a searchable PDF, `receipt.pdf`	`tesseract receipt.jpg receipt -l eng+ara pdf`

Tip: to read the text of anything on screen, press **Print** and drag around it — the shot is copied (§12.2) — then `wl-paste -t image/png | tesseract - - -l eng` prints its text in the terminal, where selecting it copies it.

19.6 Fill in and sign a PDF

Xournal++ (§16.4) writes on top of a PDF and exports a new one — the job of the "sign a PDF" websites.

1. Once: turn your signature into a picture. Sign plain white paper with a dark pen, photograph it close and square-on, and cut it out with a transparent background:

```
magick signature-photo.jpg -colorspace gray -level 25%,75% -fuzz 30% -transparent white \
  -trim +repage signature.png
```

If grey specks of paper remain, raise `-fuzz` to 40%; if the strokes break up, lower it. Keep `signature.png` somewhere private, such as `~/Documents`.

2. Start Xournal++ and choose **File → Annotate PDF**.

3. Fill in: the text tool (Ctrl+Shift+T); click where an answer goes and type. The toolbar changes the font and colour. The text sits on top of the page, not inside the form's own fields.

4. Sign: the image tool (Ctrl+Shift+I); click where the signature goes and choose `signature.png`. Drag its corners to size it; to move it later, select it with Select Rectangle (Ctrl+Shift+R).

5. File → Export as PDF, under a new name such as `contract-signed.pdf`: that is the file to send. Ctrl+S keeps your marks in a separate `.xopp` file, so you can change them and export again — from a terminal too: `xournalpp -p contract-signed.pdf contract.xopp`.

Pitfall. A picture of your signature is what most forms ask for, but it is not a certificate-based digital signature, which a reader can verify and which a government or bank signing service adds.

19.7 Pictures

ImageMagick's `magick` does what the picture websites do. The pattern is always `magick input [what to do] output`, and the output's extension chooses the format. Instructions on the web often write `convert`, ImageMagick 6's name; Fedora has ImageMagick 7, where the command is `magick` (and `magick identify`, `magick mogrify` for the others).

Want	Command
iPhone photo (HEIC) → JPEG	`magick IMG_2041.HEIC IMG_2041.jpg`
WebP → PNG or JPEG	`magick picture.webp picture.png`
PNG → WebP, without loss	`magick chart.png -define webp:lossless=true chart.webp`
JPEG → WebP	`magick photo.jpg -quality 80 photo.webp`
half the size	`magick photo.jpg -resize 50% photo-half.jpg`
fit inside 1920 × 1920, proportions kept	`magick photo.jpg -resize 1920x1920 photo-1920.jpg` (`'1920x1920>'` only ever shrinks)
a smaller JPEG file	`magick photo.jpg -quality 75 photo-q75.jpg`
under a size limit, for upload forms	`magick photo.jpg -resize 600x600 -define jpeg:extent=200kb photo-200k.jpg`
crop a box: width × height + left + top	`magick shot.png -crop 800x600+100+50 +repage part.png`
a square from the centre	`magick photo.jpg -gravity center -crop 1:1 +repage square.jpg`
a border	`magick chart.png -bordercolor white -border 40 framed.png`
side by side · one above the other	`magick a.jpg b.jpg +append +repage both.jpg` · `-append` in place of `+append`

Want	Command
a "copy for …" mark across an ID scan	`magick id-front.jpg -gravity center -pointsize 48 -fill 'rgba(200,0,0,0.45)' -annotate -25x-25+0+0 'COPY FOR VISA APPLICATION ONLY' id-front-marked.jpg`
an animated GIF	`magick -delay 50 -loop 0 frame-*.png slideshow.gif` (delay in hundredths of a second)

JPEG quality 90 looks like the original; 75 is about half the file; below 60, blocks start to show. `jpeg:extent` tries qualities until the file fits.

Try this. Make a smaller copy of a phone photo, and compare the two.

```
$ magick identify photo.jpg
photo.jpg JPEG 4032x3024 4032x3024+0+0 8-bit sRGB 3.72988MiB 0.000u 0:00.000
$ magick photo.jpg -resize 1920x1920 photo-1920.jpg
$ magick identify photo-1920.jpg
photo-1920.jpg JPEG 1920x1440 1920x1440+0+0 8-bit sRGB 878442B 0.000u 0:00.000
```

(for example) `identify` names the format, the size in pixels and the file's size: here 4032 × 3024 pixels and 3.7 MiB before, 1920 × 1440 pixels and 878,442 bytes after — about a quarter. `-resize 1920x1920` fitted the picture inside that square and kept its proportions; `photo.jpg` is untouched.

What a photo gives away. A phone photo records where it was taken (GPS, to a few metres), when, and on which phone, and the file carries all of it wherever it goes. To see the position:

```
$ magick identify -format '%[EXIF:GPS*]' photo.jpg
…
exif:GPSLatitude=25/1,12/1,432/25
exif:GPSLatitudeRef=N
exif:GPSLongitude=55/1,16/1,372/25
exif:GPSLongitudeRef=E
…
```

The fractions are degrees, minutes and seconds: 25° 12′ 17.28″ N, 55° 16′ 14.88″ E. No output means the photo holds no position. To send a photo without any of it:

```
magick photo.jpg -auto-orient -strip photo-clean.jpg
```

`-strip` removes everything that is not the picture. `-auto-orient` comes first because "which way up" is stored the same way: stripped alone, a portrait photo turns on its side.

Many at once. Write the results into a new folder, so the originals stay and a second run does not pick up the first one's output:

```
mkdir -p small
for f in *.jpg; do magick "$f" -auto-orient -resize '1600x1600>' -quality 82 -strip "small/$f"; done
fd -d 1 -e heic -x magick {} {.}.jpg      # every HEIC here → a JPEG beside it
```

The second line is a **loop**: one command, run once for each file. Read it in three parts:

Part	Means
`for f in *.jpg;`	for each name here ending in `.jpg`, in turn, call it `f`
`do magick "$f" … "small/$f";`	run this, with `$f` standing for that name; the quotes keep a name with spaces whole
`done`	the end of what is repeated

Ch. 25 explains variables such as `$f`, and why the quotes matter.

fd matches `.HEIC` and `.heic` alike and converts several files at once; `-d 1` keeps it to this folder (without it, subfolders too). Avoid `magick mogrify` unless you add `-path other-folder`: on its own it overwrites the originals.

Tip: `magick identify photo.jpg` prints the format, the size in pixels and the file size — check before and after.

Read more: ImageMagick's options page explains each option used here; `tldr magick` gives short examples.

QR codes.

Want	Command
a link as a QR code, in the terminal	`qrencode -t utf8 'https://example.org/menu'` (`-t utf8i` on the light theme)
the same as a picture	`qrencode -s 10 -o menu-qr.png 'https://example.org/menu'`
your Wi-Fi: guests join by scanning	`qrencode -s 10 -o wifi.png 'WIFI:S:Home Net;T:WPA;P:correct horse 42;;'`

`-s` is the size of one square in pixels. In the Wi-Fi text, `S:` is the network name, `T:` the security (`WPA` for a network with a password, `nopass` for an open one) and `P:` the password; a `;`, `,`, `:`, `\` or `"` inside a name or password needs a `\` in front. Start that line with a space, since it holds a password (§19.1). In the terminal, `utf8` draws the light squares in the text colour, which is light only on the dark theme; `utf8i` swaps them for the light theme.

19.8 Video and sound

ffmpeg — the full RPM Fusion build — converts nearly anything to anything. `ffprobe -hide_banner clip.mkv` first shows what a file holds; its video and audio codecs decide which recipe is needed. The lines to read look like this, for an iPhone video:

```
Stream #0:0[0x1](und): Video: hevc (Main 10) (hvc1 / 0x31637668), …, 1920x1080, …
Stream #0:1[0x2](und): Audio: aac (LC) (mp4a / 0x6134706D), …, stereo, …
```

The word after `Video:` is the video codec (`hevc`), the word after `Audio:` the sound's (`aac`).

An MP4 that plays everywhere (H.264 video, AAC sound) — from an iPhone's HEVC `.MOV`, a `.webm` or a `.mkv`:

```
ffmpeg -i IMG_4471.MOV -c:v libx264 -crf 23 -pix_fmt yuv420p -c:a aac -b:a 160k \
  -movflags +faststart IMG_4471.mp4
```

If ffprobe already shows `h264` and `aac`, only the container changes — instant, and nothing is lost: `ffmpeg -i "Phone video.mkv" -c copy "Phone video.mp4"`.

Shrink for WhatsApp or email.

```
ffmpeg -i "Phone video.mp4" \
  -vf scale=1280:1280:force_original_aspect_ratio=decrease:force_divisible_by=2 \
  -c:v libx264 -crf 28 -preset slow -pix_fmt yuv420p -c:a aac -b:a 96k \
  -movflags +faststart "Phone video-small.mp4"
```

Each part, in order:

Part	Does
`scale=1280:1280:…`	fits the picture within 1280 pixels on its long side, upright or sideways; `force_divisible_by=2` keeps both sizes even, which `yuv420p` needs
`-crf 28`	the quality: *constant rate factor*, steady quality with the size following it (below)
`-preset slow`	spends more time for a smaller file
`-pix_fmt yuv420p`	the colour format every phone plays
`-b:a 96k`	the sound at 96 kbit/s
`-movflags +faststart`	lets it start playing before it has fully arrived

`-crf`	Gives
18	looks like the original
23	ffmpeg's default
28	small
32	smaller still

Each 6 more roughly halves the file; 0 is lossless and 51 the worst.

Want	Command
cut 1:30–2:10, no re-encoding	`ffmpeg -ss 00:01:30 -to 00:02:10 -i talk.mp4 -c copy talk-cut.mp4`
the same, cut to the frame	`ffmpeg -ss 00:01:30 -to 00:02:10 -i talk.mp4 -c:v libx264 -crf 18 -c:a aac talk-cut.mp4`
the sound alone, untouched	`ffmpeg -i talk.mp4 -vn -c:a copy talk.m4a` (`.opus` when ffprobe says `opus`)
the sound as MP3	`ffmpeg -i talk.mp4 -vn -c:a libmp3lame -q:a 2 talk.mp3`
a WhatsApp voice note as MP3	`ffmpeg -i PTT-20261015-WA0003.opus -c:a libmp3lame -q:a 4 voice.mp3`
without sound	`ffmpeg -i clip.mp4 -c copy -an clip-silent.mp4`
one frame as a picture	`ffmpeg -ss 00:00:12 -i clip.mp4 -frames:v 1 still.png`

The keyframe catch. A copied cut can only begin on a keyframe — a complete picture, which videos carry every few seconds. ffmpeg starts at the one before your time and tells the player to skip ahead; most players start cleanly, some show a still picture for a moment. When the cut must land on the exact frame, use the second row, which re-encodes.

A GIF from a clip — four seconds from 0:05, 12 frames a second, 480 pixels wide:

```
ffmpeg -ss 5 -t 4 -i clip.mp4 \
  -vf "fps=12,scale=480:-1:flags=lanczos,split[a][b];[a]palettegen[p];[b][p]paletteuse" \
  -loop 0 clip.gif
```

`palettegen` picks the GIF's 256 colours from the clip itself, so it does not come out grainy. GIFs are large: keep them short and narrow.

Join clips without re-encoding:

```
printf "file '%s'\n" "part 1.mp4" "part 2.mp4" "part 3.mp4" > list.txt
ffmpeg -f concat -safe 0 -i list.txt -c copy joined.mp4
```

The clips must share codec, picture size and frame rate, as clips from one phone or camera do; convert mixed clips with the MP4 recipe at one size first. `-safe 0` allows spaces and full paths in the list; rename files whose names contain an apostrophe.

Read more: FFmpeg's H.264 encoding guide — CRF, presets and compatibility.

19.9 Download and record

Video and sound from the web, with yt-dlp (§15.2) — only what you have the right to keep:

Want	Command
the best version, into this folder	`yt-dlp 'LINK'`
into `~/Videos`	`yt-dlp -P ~/Videos 'LINK'`
no larger than 1080p	`yt-dlp -S res:1080 'LINK'`
only the sound, as MP3	`yt-dlp -x --audio-format mp3 'LINK'`
only 1:30–2:10 of it	`yt-dlp --download-sections '*1:30-2:10' 'LINK'`
one video from a playlist link	`yt-dlp --no-playlist 'LINK'`
which formats exist	`yt-dlp -F 'LINK'`

Keep the link in single quotes (§15.3). When YouTube changes its site, downloads fail until a fixed yt-dlp reaches Fedora; `sudo dnf upgrade --refresh yt-dlp` fetches exactly that, without a full `update`.

Read more: yt-dlp's README, under *Usage and Options* and *Sorting Formats*.

Record the screen with wf-recorder. **Ctrl+C** in its terminal stops it and finishes the file; keep recordings in `~/Videos`:

```
wf-recorder -f ~/Videos/screen-$(date +%F-%H%M).mp4                    # the whole screen
wf-recorder -g "$(slurp)" -f ~/Videos/clip.mp4                         # drag a region first
wf-recorder -g "$(slurp)" -a -f ~/Videos/howto.mp4                     # … with your microphone
wf-recorder -g "$(slurp)" --audio=@DEFAULT_MONITOR@ -f ~/Videos/demo.mp4 # … with what you hear
```

With two monitors and no region, it asks which one to record; `-g "$(slurp -o)"` lets you click one instead. For a file meant for a phone or a website, add `-x yuv420p`. A recording of the whole screen includes the terminal you started it from: start it, switch workspace (Super+2) and work there. An existing file is not overwritten without asking — `Overwrite? Y/n`, where Enter alone means yes — which the date in the first name avoids.

Read more: `man wf-recorder`.

Tip: for a short animation of something on screen, record an MP4 and make the GIF from it (§19.8) — smaller and smoother than recording a GIF directly.

19.10 Files: archives, checksums, trash, renaming, text

Archives, with 7-Zip (`7z`):

Want	Command
zip a folder (opens anywhere)	`7z a "Tax 2026.zip" "Tax 2026"`
7z with a password, names hidden too	`7z a "Tax 2026.7z" "Tax 2026" -p -mhe=on`
zip with a password (AES-256)	`7z a -tzip "Tax 2026.zip" "Tax 2026" -p -mem=AES256`
look inside · test it	`7z l archive.7z` · `7z t archive.7z`
unpack here · into a folder	`7z x archive.zip` · `7z x archive.zip -o"Tax 2026"` (no space after `-o`)
a `.tar.gz` or `.tar.xz`	`tar xf archive.tar.gz` (7z would only peel off the outer layer)
a `.rar` · look inside it	`bsdtar -xf archive.rar` · `bsdtar -tf archive.rar`

`-p` with nothing after it asks for the password, so it never reaches the history. A password-protected zip shows its file names to anyone, and its AES encryption may need 7-Zip on the other side; a `.7z` with `-mhe=on` hides the names as well. 7z also opens ISO images and most other formats, but not RAR: Fedora's 7-Zip leaves RAR out for licence reasons, so RAR files open with `bsdtar` (libarchive, which the books window and zathura use for comics; it unpacks RAR, though it cannot make one). In yazi, `;` then `7z x %h` unpacks the file under the cursor (§10.3).

> **On Windows.** This is the same 7-Zip, by the same author, without its window: `7z a` is *Add to archive*, `7z x` is *Extract Here*, and the archives open on either system.

Checksums. To check a download against the checksum its website publishes:

```
sha256sum Fedora-Server-dvd-x86_64-44-1.7.iso                              # prints it
echo 'PASTED-CHECKSUM  Fedora-Server-dvd-x86_64-44-1.7.iso' | sha256sum -c  # OK or FAILED
sha256sum -c --ignore-missing Fedora-Server-44-1.7-x86_64-CHECKSUM          # a checksum file
```

Two spaces go between the checksum and the name. A signed checksum file adds a warning about "improperly formatted lines" — its signature — which is harmless; the `OK` is what counts:

```
Fedora-Server-dvd-x86_64-44-1.7.iso: OK
sha256sum: WARNING: … lines are improperly formatted
```

A checksum only proves the file matches the list it came with. Fedora also signs that list; its download page shows how to check the signature with `gpgv` (part of gnupg2, installed).

Delete safely. `gio trash old-draft.docx` (or **d** in yazi) moves a file to the trash instead of deleting it; **g t** in yazi shows the trash, restores from it and empties it. Only emptying gives the space back (§10.3).

What fills the disk. `fd -u -S +500m . ~ -X du -h | sort -h` lists every file over 500 MB in your home, the largest last (`-u` takes in hidden folders and what a `.gitignore` leaves out; `-H` alone would skip the latter); `du -xh -d1 ~ | sort -h | tail` shows the largest folders (§38.3).

Rename many files. In yazi, select them (Space, or v for a range) and press **r**: neovim opens with one name per line. Edit them as text, then `:wq`, and yazi asks before renaming. Keep the order and the number of lines.

Typed in neovim	Does
`:%s/IMG_/Holiday-/`	on every line (`%`), `IMG_2041.jpg` → `Holiday-2041.jpg`
`:%s/^/2026-10 /`	`IMG_2041.jpg` → `2026-10 IMG_2041.jpg` (`^` is the start of a line)
Ctrl+V, move down and across, then `d`	deletes the same block of characters from every line

Text files.

Want	Command
Arabic subtitles show as gibberish	`iconv -f WINDOWS-1256 -t UTF-8 film.ar.srt > film.ar.utf8.srt`
… or play them right as they are	`mpv --sub-codepage=cp1256 film.mkv`
which encoding is it	`file notes.txt`
Windows line endings (`^M`) out, in place	`sed -i 's/\r$//' notes.txt`
compare two texts side by side	`nvim -d old.txt new.txt`
compare two PDFs	`pdftotext -layout old.pdf; pdftotext -layout new.pdf; nvim -d old.txt new.txt`

`file` reports old Arabic files as "ISO-8859" or "Non-ISO extended-ASCII"; if the letters are garbled, Windows-1256 is almost always the answer.

19.11 Office documents and spreadsheets

AbiWord and Gnumeric (§16.4) convert without opening a window. AbiWord writes its result next to the original; `ssconvert` writes the name you give.

Want	Command
Word → PDF	`abiword --to=pdf "Offer letter.docx"` → `Offer letter.pdf`
Word → OpenDocument · and back	`abiword --to=odt "Offer letter.docx"` · `abiword --to=docx "Offer letter.odt"`
Word → plain text	`abiword --to=txt "Offer letter.docx"`
every Word file here → PDF	`for f in *.docx; do abiword --to=pdf "$f"; done`
Excel → CSV (the sheet that was open when it was saved)	`ssconvert "Expenses October.xlsx" "Expenses October.csv"`
one sheet, by its name	`ssconvert -O 'sheet=October' budget.xlsx october.csv`
every sheet to its own CSV	`ssconvert -S budget.xlsx 'budget-%s.csv'` (`%s` is the sheet's name)
CSV → Excel	`ssconvert "Expenses October.csv" "Expenses October.xlsx"`
Excel → PDF · → OpenDocument	`ssconvert budget.xlsx budget.pdf` · `ssconvert budget.xlsx budget.ods`

> **In SQL terms.** A CSV from `ssconvert` is a table ready to load anywhere: Power BI's *Get data → Text/CSV*, or a database's import. With `-S`, each sheet becomes a CSV of its own — one table per sheet.

Pitfalls. Ordinary letters and sheets convert well; tracked changes, text boxes, macros and elaborate layouts do not survive. Gnumeric writes dates and numbers into a CSV the way it displays them, so `2026-10-01` can come back as `2026/10/01` and `120.50` as `120.5`: check a CSV before you feed it to something strict.

Read more: `man ssconvert` (every option) and `abiword --help`.

19.12 Quick conversions: units, money, time

qalc (§16.6) answers in one line, repeating the question (`75 kilometers/hour ≈ 46.60283942 mph`); `-t` prints the result alone.

Want	Command	Answer
speed	`qalc -t '75 km/h to mph'`	`46.60283942 mph`
height	`qalc -t '5 ft 11 in to cm'`	`180.34 cm`
temperature	`qalc -t '30 °C to °F'`	`86 °F`
area	`qalc -t '1500 sqft to m^2'`	`139.35456 m²`
20 % of a sum	`qalc -t '4500 * 20%'`	`900`
a sum plus 5 %	`qalc -t '4500 + 5%'`	`4725`
money, at today's rate	`qalc -e '100 USD to AED'`	about 367 AED (the dirham is pegged to the dollar)
days until a date	`qalc -t 'days(today; 2026-12-25)'`	`71`, asked on 15 October 2026
data sizes	`qalc -t '3 GiB to MB'`	`3221.225472 MB`

`-e` (long form `-exrates`) fetches current exchange rates first; without it, qalc uses the rates it fetched last, and offline it says so and carries on with those. Write "20 % of" as a multiplication: qalc misreads `of`.

Time zones. `date` knows every zone and its summer time:

```
TZ=Asia/Tokyo date                                    # the time in Tokyo now
date -d 'TZ="Europe/London" 2026-10-20 15:00'         # 15:00 in London, in your time
TZ=Europe/London date -d 'TZ="Asia/Dubai" 2026-10-20 15:00' # 15:00 in Dubai, in London's time
```

On a computer set to Dubai time the second line answers 18:00, and the third 12:00 BST: London keeps summer time until 25 October 2026, and `date` knows it.

Zone names come from `timedatectl list-timezones | rg -i york`.

Tip: the password-generator websites, offline: `tr -dc 'A-Za-z0-9' < /dev/urandom | head -c 24; echo` prints 24 random letters and digits.

What you learnt

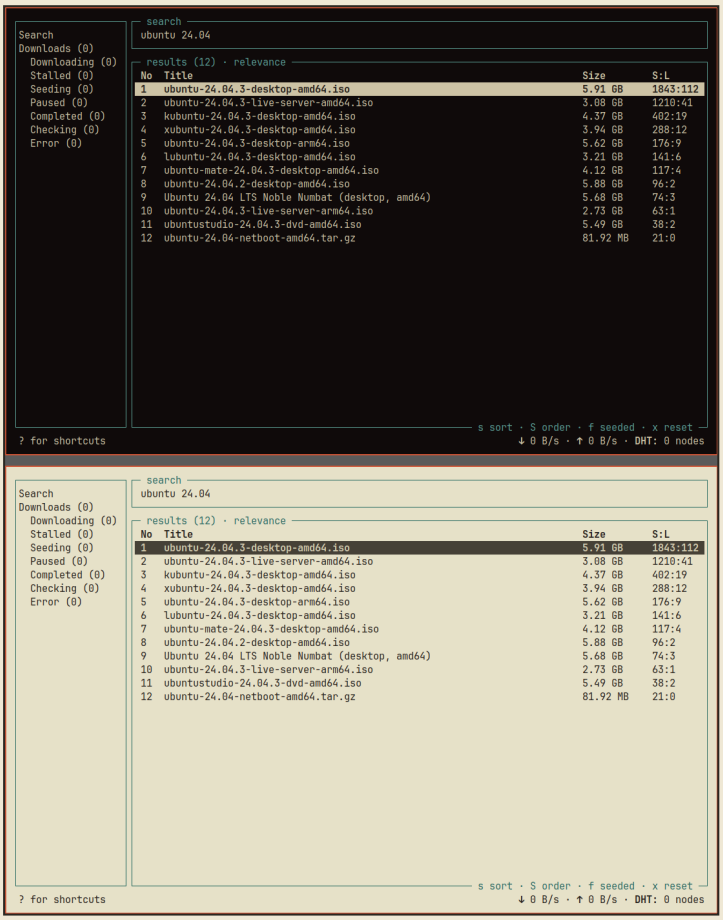
- Every recipe writes a **new** file: check it before you trash the original.
- `qpdf` joins, splits and turns pages without loss; `gs` shrinks a PDF; `tesseract` reads scans.
- `magick input [what to do] output` converts pictures; `-strip` removes what a photo gives away.
- `ffprobe` shows what a video holds; `ffmpeg` converts, cuts and shrinks it.
- `for f in *.jpg; do …; done` runs one command for every file.
- `7z`, `sha256sum` and `qalc` replace the archive, checksum and converter websites.

Check yourself

1. What does `qpdf --replace-input` do that most recipes here do not? → *it changes the original file instead of writing a new one*
2. How do you send a photo without its position and phone? → *`magick photo.jpg -auto-orient -strip photo-clean.jpg`*
3. Why can a copied cut (`-c copy`) start a moment early? → *it can only begin on a keyframe*

20. Torrents: searching and downloading

> **In this chapter**
>
> - set up Prowlarr once, with the torrent sites you choose
> - search, download and seed with torrra
> - open magnet links from Brave, and act on torrra's messages when something is wrong
>
> **You need:** ch. 14 (Brave) and ch. 11 (the terminal).



torrra after a search: twelve results with their sizes and seeders:leechers, in the terminal's colours.

Two pieces work together. **Prowlarr** (a small service in a container that runs only while it is needed, §34.5) knows how to search hundreds of torrent sites — *indexers* — and answers one search for all of them.

> **Word — container.** A program run fenced off from the rest of the system: it sees only its own files and the folders it is given. Chapter 34 tells the whole story.

torrra is the program you use: it sends your search to Prowlarr, shows the results, and downloads what you pick with libtorrent, the engine behind most torrent clients. One API key connects them. Setup made it, and its one master copy is `~/.config/fedokawaii/secrets.sh`: Prowlarr receives it as a podman *secret* (podman keeps that copy in `~/.local/share/containers/storage/secrets/`, readable by you alone, and `update` renews it from secrets.sh should you change the key there), torrra receives it from its wrapper each time it starts, and neither program writes it into its own settings.

Term	In one line
torrent	a set of files shared piece by piece among many computers; a small `.torrent` file describes it: names, sizes and a checksum for every piece
peer · seeder · leecher	any computer sharing the same torrent · one that has all of it · one still downloading
`S:L`	torrra's column of seeders:leechers for each result, such as `412:37`; more seeders means a faster, surer download
magnet link	a link that names a torrent by its fingerprint, the *info-hash*, instead of carrying the `.torrent` file; the rest comes from peers
indexer	a website that lists torrents and answers searches; Prowlarr asks all of yours at once
seeding · ratio	sharing a finished download · uploaded ÷ downloaded: a ratio of 2 means you gave back twice what you took
API key	a long password one program shows another; here 32 hexadecimal characters in `secrets.sh`

A magnet link, taken apart:

Part	Says
`magnet:?`	this is a magnet link
`xt=urn:btih:` and 40 hexadecimal characters	which torrent: its info-hash
`&dn=debian-13.1.0-amd64-netinst.iso`	a name to show while the rest arrives
`&tr=udp://…`	a *tracker*, a server that introduces peers to each other (optional)

Download only what you have the right to: Linux and BSD images, public-domain films and books, Creative Commons music, your own backups. Torrenting shares what you download with others while it runs, and your address is visible to every peer.

20.1 The first time: Prowlarr

1. Type `prowlarr` in a terminal: it starts Prowlarr and opens http://localhost:9696 in Brave. Prowlarr asks you to **create a login** — a user name and password for this page (it asks even on this computer, so no web page can reach it behind your back). Brave can remember it.
2. **Indexers → Add Indexer.** Prowlarr lists every site it knows, public and private; filter by language or type, pick the ones you use, and **Test** each one before saving. Sites marked as needing *FlareSolverr* (a browser that gets past Cloudflare checks) will not work here — none is installed — so choose others. In each indexer's settings, tick **Prefer Magnet URL** (an advanced setting: **Show Advanced** reveals it); torrra's own documentation asks for it.
3. Optional: **Settings → Apps** is for connecting other programs (Sonarr, Radarr); torrra needs nothing there — it already holds the key.
4. Done with the page: `prowlarr stop`. A search in torrra starts Prowlarr by itself and stops it again afterwards.

That is all. `verify` warns while Prowlarr has no indexers, and torrra says so plainly instead of showing an empty list.

20.2 Searching and downloading with torrra

Open **Torrents** from the launcher, or type `torrra` in a terminal (`torrra search 'debian 13'` searches straight away; `torrra downloads` shows only the downloads). Its colours are the terminal's, so it follows the theme. When Prowlarr is not running, a search starts it first — `torrra: starting Prowlarr…` — which takes up to a minute the first time after the computer starts.

Keys	Does
type, then Enter	search all your indexers
j k · ↑ ↓ · Ctrl+D / Ctrl+U · gg · G	move · a page · top · bottom
Enter (or l) · Esc	the torrent's details · close them
Enter, in the details	its file list, every file ticked; **Enter** there starts the download (Space unticks a file; **d** takes all without waiting for the list)
s · S · f · x	sort menu · reverse the order · hide results without seeders · back to your defaults
Tab	the next panel (search, results, downloads)
p	pause or resume a download
f (on a download)	choose which files of it to download (Space selects, Enter confirms)
o	its options: speed limits, ratio, sequential download (for watching while it loads)
d · D	remove the torrent · remove it **and delete its files**
t	"turtle mode": the slow speed limits on / off
Ctrl+T · ? · Ctrl+Q	another colour theme · every key · quit

Results are sorted by **seeders** — the number of people sharing the whole file — highest first; more seeders means faster and more certain. Downloads go to `~/Downloads`, which is its own Btrfs subvolume, so a film you delete gives its space back at once rather than living on in snapshots.

Tip: when you are done with a torrent, **D** removes it and deletes its files in one step. Move anything you keep out of `~/Downloads` — nothing there is in a snapshot.

Try this (once Prowlarr has an indexer, §20.1). Download a Linux image — the legal way to watch torrents work.

1. Open **Torrents** from the launcher. If Prowlarr is not running yet, the window first says `torrra: starting Prowlarr…` — up to a minute after the computer starts.
2. Type `debian 13` and press Enter. The results arrive, most seeders first.
3. **Enter** on one shows its details; **Enter** again lists its files, every one ticked; **Enter** once more starts the download.
4. **Tab** moves to the next panel, as far as the downloads; there the download grows. **p** pauses it, **p** again resumes it.
5. **Ctrl+Q** quits, and the download pauses with it. Open **Torrents** again: it carries on where it was.

Magnet links in Brave

The first magnet link you click makes Brave ask whether to open it with *xdg-open*; tick *Always allow* and say yes. From then on a magnet link — or a `.torrent` file opened from the downloads (a `file://` link to one too) — starts torrra in its own window at that torrent's file list; **Enter** starts the download (pressed before the list has loaded, it takes every file). torrra runs one window at a time: if it is already open, a notice says so and the link waits on your clipboard — close torrra and click the link again (or, in a terminal, `torrra download 'PASTE'`).

20.3 How it behaves

Downloads run only while torrra's window is open. Close it and they pause; open torrra again and they continue where they were. There is no background service.

Seeding. A finished download keeps sharing while torrra is open, without a limit by default. To stop at a ratio or a time, set it per torrent (**o**), or for all; the torrent pauses when it gets there, and `0` removes a limit again:

```
torrra config set seeding.default_max_ratio 2   # once it has shared twice its size
torrra config set seeding.default_max_time 3d   # or after 3 days (also 12h, 90m)
```

Speed. Unlimited by default. **t** turns *turtle mode* on and off: the slow limits, 10 KB/s each way until you set your own, with `TURTLE` in the status bar while it is on. `torrra config set speed_limit.download_limit 5M` sets one (`500K`, `2M`; `unlimited` turns one off), and `speed_limit.upload_limit` the other.

Ports. libtorrent listens on **6881** (TCP and UDP, opened in the firewall by setup) and asks your router to forward it (UPnP / NAT-PMP). If 6881 is busy, it quietly uses a random port the firewall does not open, and fewer peers can reach you.

One torrra at a time — a second window would share the same downloads, so it says torrra is already open.

Settings live in `~/.config/torrra/config.toml` (yours); `torrra config list` shows them, `torrra config edit` opens the file in neovim.

20.4 When something is wrong

torrra itself exits quietly on every failure; its wrapper (`kawa-torrra`) checks Prowlarr first and says what is missing — in the terminal, or as a notice when torrra was opened from the launcher or a link:

Message	Do
Prowlarr is not answering on 127.0.0.1:9696	`systemctl --user status prowlarr`; `journalctl --user -u prowlarr -e`
Prowlarr refused the key in secrets.sh	`fedokawaii update` hands Prowlarr the key again
Prowlarr has no indexers yet	§20.1, step 2
no PROWLARR_API_KEY in secrets.sh	`fedokawaii update` makes one (only a search needs it: magnet links and .torrent files still download)
secrets.sh is missing	`fedokawaii update` makes it
secrets.sh must be yours and private	`chmod 600 ~/.config/fedokawaii/secrets.sh`
Prowlarr did not start	`journalctl --user -u prowlarr -e` says why
Prowlarr answered HTTP … (a number)	the same: `journalctl --user -u prowlarr -e`
not installed … · cannot start …	`fedokawaii update` reinstalls torrra
Prowlarr is not set up here	`fedokawaii repair` (podman or the container file is missing)
port 9696, Prowlarr's, is taken by another program	`ss -ltnp 'sport = :9696'` shows which; close it
run torrra inside your session	a terminal of the desktop, or an SSH login — not `su` or a cron job
torrra is already open in another window	close that one first; a link you clicked waits on the clipboard
no such file: …	the .torrent file was moved or deleted before torrra opened it — download it again
not a magnet link or a .torrent file: …	only magnet links, http(s) links to a torrent, and files ending in .torrent open in torrra
a search finds nothing although indexers are set	test them in Prowlarr (**Indexers → Test All**); one may be down or need FlareSolverr

Read more: torrra's usage guide (for version 2.5.1, the one installed), Prowlarr's indexer pages and the magnet link format.

What you learnt

- Prowlarr searches your indexers; torrra searches through it, downloads and seeds.
- `prowlarr` opens its web page once, to add indexers; `prowlarr stop` when done.
- Downloads go to `~/Downloads` and run only while torrra's window is open.
- **D** removes a torrent and its files; a ratio or time limit stops seeding.
- When something is wrong, torrra's wrapper says what is missing.

Check yourself

1. torrra is closed. Are your downloads still running? → *no: they pause, and continue when torrra opens again*
2. What does the `S:L` column `412:37` mean? → *412 seeders and 37 leechers: plenty of people have the whole file*
3. A search shows *Prowlarr has no indexers yet*. What do you do? → *open `prowlarr`, Indexers → Add Indexer, test and save one*

21. Games and hardware

> **In this chapter**
> - install and play Windows games through Steam and Proton
> - add the performance overlay and gamemode to a game
> - control RGB lighting, Logitech devices, coolers and an AMD graphics card
>
> **You need:** §14.1 (the launcher) and §7.3 (workspaces).

The last chapter of Part III is for play, and for the hardware that likes to be tuned: the lights, the mouse, the cooler and the graphics card.

21.1 Steam — games

Start **Steam** from the launcher. The client's main window always goes to **workspace 9** (a rule in `config.h`); press **Super+9** to reach it. Games open on whatever workspace you are on. Steam runs through XWayland, and its own interface ignores the desktop's themes (§39.2).

> **On Windows.** It is the same Steam, signed in to the same account, with the same library. A Windows game runs here through Proton, which answers its Windows calls with Linux ones.

Try this. Start Steam, and give one game the overlay.

1. **Super+Space**, type `steam`, Enter. Steam opens on workspace 9: **Super+9** goes there.
2. Open a game's *Properties → General*, and put `mangohud %command%` in its launch options.
3. Start the game. **Right Shift+F12** shows or hides the overlay in its corner: frames per second, the CPU's and the GPU's load and temperature.
4. Quit the game; **Super+Tab** takes you back to the workspace you were on before.

Term	In one line
XWayland	the bridge that lets an older X11 program, such as Steam, show on a Wayland desktop
Proton	Valve's compatibility layer, built on Wine: it runs Windows games on Linux
GE-Proton	a community build of Proton that carries fixes and media patches ahead of Valve's
`%command%`	in launch options, the place where Steam puts the game's own command line

Each game's **launch options** (in Steam: the game's *Properties → General*) take the tools installed with it:

Launch options	Does
`gamemoderun %command%`	gamemode: the CPU and scheduler tuned for the game while it runs (`gamemoded -s` reports whether it is active)
`mangohud %command%`	the performance overlay, in the desktop's ink and Departure Mono; **Right Shift+F12** shows or hides it
`gamemoderun mangohud %command%`	both
`gamescope -W 2560 -H 1440 -r 120 -f -- %command%`	the game inside its own small compositor at a fixed size and refresh rate; add `-w 1920 -h 1080 -F fsr` to render lower and upscale
`gamescope --mangoapp -f -- %command%`	gamescope with the overlay — inside gamescope, use `--mangoapp` rather than `mangohud`

GE-Proton

Setup installs the newest GE-Proton, and `update` adds new releases, keeping the newest two so a game pinned to the older one still starts. Restart Steam to see a new one. Choose it per game in *Properties → Compatibility* (*Force the use of a specific Steam Play compatibility tool*), or for every game in Steam's *Settings → Compatibility*. GE-Proton uses the kernel's `ntsync` driver (loaded at boot) by itself: the waiting and signalling between a Windows game's threads is done by the kernel instead of being imitated, which is faster.

```
ls -l /dev/ntsync      # the device is there: the driver is loaded
```

Should one game misbehave with it, the launch option `PROTON_NO_NTSYNC=1 %command%` runs that game without it.

Pitfalls. The overlay stays off until a game's launch options ask for it, and outside gamescope `mangohud` is the switch; inside it, `--mangoapp`. MangoHud's settings are a managed file: change them in the repository's `MangoHud.conf`. `update` keeps only the newest two GE-Proton builds, so a game forced to an older one needs a new choice under *Compatibility*.

Read more: the READMEs of GE-Proton (its launch options), gamescope and MangoHud.

21.2 Hardware — OpenRGB, Solaar, liquidctl, LACT

Four programs, each for one kind of device. Try the ones for the hardware you have; the others find nothing to control.

Try this. Look before you change anything.

1. **Super+Space**, `lact`, Enter (AMD graphics cards only): LACT shows the card's clocks, temperatures and power. Read them, change nothing, and close it with **Super+Q**.
2. In a terminal, `solaar show` lists a Logitech receiver and what is paired to it, with the battery levels; `liquidctl list` lists the coolers and fan hubs it can drive. Each lists only what is plugged in.

OpenRGB (launcher) sets the lighting of RGB devices from one window: find the devices, choose colours and effects, save a *profile*. Nothing starts it at login, so the lighting falls back to the devices' own defaults after a power cut; `openrgb --profile NAME` applies a saved profile, and that command can go in `~/.config/dwl/autostart.local.sh` (§42.1) as `openrgb --profile NAME &`, like the EasyEffects line in §15.6. Memory and motherboard lighting is reached over *SMBus*, a small bus on the motherboard that chips such as the RAM's lighting controllers hang on; if those devices do not appear, see OpenRGB's own SMBus notes — this desktop sets up nothing beyond loading `i2c-dev` (for monitor brightness, which uses the same kind of bus). Its window takes the Qt palette, so it follows the theme.

Solaar (launcher) manages Logitech receivers and their devices: pair a mouse or keyboard to a receiver, read battery levels, change buttons and pointer settings. `solaar show` lists everything in a terminal. Like EasyEffects, its autostart entry is for full desktops, so it runs only when you start it.

liquidctl controls AIO coolers, fan hubs and some power supplies from the terminal: `liquidctl list` finds them, `liquidctl initialize all` prepares them (needed after every boot on many devices), `liquidctl status` shows temperatures and speeds, `liquidctl set pump speed 70` and `liquidctl set fan speed 20 30 40 60 50 100` set them. The fan line is a curve, read in pairs of temperature and duty (the share of full speed):

Liquid at	Fans at
20 °C	30 %
40 °C	60 %
50 °C	100 %

On liquid coolers the temperature is the coolant's, from the cooler's own probe, not the CPU's. Most devices forget these settings when powered off, and nothing here applies them again at boot; liquidctl's README shows a small systemd service that does.

LACT (launcher; AMD only) shows the graphics card — clocks, temperatures, power — and changes it: fan curve, power limit, performance level, and, thanks to the `amdgpu.ppfeaturemask` kernel argument setup added, clocks and undervolting (running the chip at a lower voltage for the same clock: cooler and quieter, until it is too low to be stable). Its service, `lactd`, applies the settings in `/etc/lact/config.yaml` at every boot. Change one thing at a time and test it: an unstable clock or voltage can freeze the machine, and LACT's wiki has a page on recovering from a bad overclock.

Part III, looking back. You can now do a day's work on this desktop with its own programs:

Chapter	You can
14	start any program by name, browse, upload a file, share a screen with sound
15	view pictures, play and keep video, play music, send sound where you want
16	read and search documents, keep notes, convert office files, calculate
17	edit any text file in neovim without getting stuck
18	find any file, word, folder or earlier command in seconds
19	do on your own computer what websites charge for
20	search, download and seed torrents
21	play Windows games, and tune the lights, mouse, cooler and graphics card

Part IV turns from using the machine to understanding it: what happens underneath each of these programs, one layer at a time, from the files on the disk to the network.

What you learnt

- Steam opens on workspace 9 (**Super+9**); Proton runs Windows games, GE-Proton when a game needs it.
- A game's launch options take `gamemoderun`, `mangohud` and `gamescope` before `%command%`.
- **Right Shift+F12** shows or hides the overlay.
- OpenRGB, Solaar, liquidctl and LACT each control one kind of device; only LACT re-applies its settings at boot by itself.

Check yourself

1. Where does Steam's window go, and how do you reach it? → *workspace 9: Super+9*
2. What do you type in a game's launch options to get the overlay? → *`mangohud %command%`*
3. Your RGB lights forget their colours after a power cut. Why? → *nothing starts OpenRGB at login: add `openrgb --profile NAME &` to `autostart.local.sh`*

Part IV — Understanding Linux

You use the desktop every day now; Part IV explains why it works. Each chapter takes one part of the system: the idea in plain words, then the same idea on this machine, then a few minutes of practice. Start with ch. 22 and read in order: each chapter builds on the ones before.

22. A map of the system

> **In this chapter**
> - place every program you know on one of eight layers
> - read Part IV's examples: `$` lines, *for example*, `…`
> - answer five questions about any Linux machine on your own
>
> **You need:** §2.5 (the machine in one picture) · ch. 11 (the terminal).

Part IV explains what happens underneath: how the machine starts, where its files live, who may do what, how programs run and talk to each other, and how the packages fit together into this desktop. Each idea comes with something you can type on this machine to see it for yourself.

How to read the examples.

- A line that starts with `$` is a command you type; the lines under it are what it prints. Lines without `$` in a `sh` block are commands too, with a `#` comment after them.
- Output marked *for example* varies from machine to machine: process numbers, sizes, versions, dates. The shape is what matters.
- `…` marks lines or columns left out.
- `ls` on this desktop is eza (§18.1). Where an example needs the classic `ls` output, it uses `command ls`, which skips the alias and runs coreutils' own `ls`.
- `sudo` asks for your password; everything else here only reads, unless it says so.
- The machine is called `fedora` in the examples. Yours shows the name you gave it in the installer — or `fedokawaii`, the name setup gives a machine left without one.

Terms. The words Part IV leans on most. Each is taught in the place named; the full glossary is appendix E.

Term	Plain meaning	Taught in
kernel	the core program that drives the hardware and shares it out between all other programs	§2.2
userspace	everything that is not the kernel: systemd, the shell, the desktop, your programs	§22.1
distribution	a kernel plus thousands of userspace programs, built, tested and updated together: Fedora is one	§2.2
package	one piece of a distribution in a single file (`.rpm`): programs, libraries, settings, and what it needs	ch. 27
process	a program while it runs, with its own number (PID)	ch. 26
unit	something systemd manages: a service, a timer, a mount, a target	ch. 28
upstream	the project that writes a piece of software; Fedora is *downstream* of it	§2.2

22.1 The layers of this desktop

A computer is built in layers, and each layer uses only the one below it: a program asks a library, the library asks the kernel, the kernel drives the hardware. One key press, from the bottom up:

```
hardware      the keyboard sends a key code
kernel        its input driver turns it into an event
the session   dwl decides which window has the focus
your program  foot shows the letter
```

§2.5 drew the machine in four layers. On this desktop there are eight, each with the programs that live on it:

```
your programs      Brave Origin · foot + bash · mpv · yazi · neovim · Steam
the session        dwl (on wlroots) · waybar · mako · fuzzel · swayidle · swaylock · kawa-*
desktop services   PipeWire + WirePlumber · xdg-desktop-portal · D-Bus · polkit agent · udiskie
system services    systemd · greetd · NetworkManager · resolved · BlueZ · firewalld · tuned
libraries          glibc · Mesa · libva · ffmpeg · GTK · Qt · fontconfig · FreeType · HarfBuzz
kernel             Linux: amdgpu or i915/xe · Btrfs · zram · SELinux · cgroups · the network stack
firmware           UEFI · shim and GRUB2 (loaded by it) · CPU microcode · GPU firmware
hardware           CPU · GPU · NVMe disk · network · sound · USB
```

Everything above the kernel line is *userspace*: ordinary programs, which reach the hardware only by asking the kernel.

Try this. Ask two layers for their names.

```
$ uname -r
7.2.8-200.fc44.x86_64
$ echo "$XDG_CURRENT_DESKTOP"
wlroots
```

(for example)

You typed	It means
`uname -r`	the running kernel's version: Linux 7.2.8, Fedora's build 200, for Fedora 44
`echo "$XDG_CURRENT_DESKTOP"`	the session's name for itself, set when you log in (§29.8)

> **On Windows.** The layers are the same with other names: the firmware, the NT kernel and its drivers, the services in `services.msc`, the desktop (Explorer and `dwm.exe`), then your programs. What differs is that here every layer is a separate, replaceable project.

Part IV walks these layers one subject at a time, from files (ch. 23) upwards; ch. 29 follows them in time, as the machine starts.

Read more: Fedora's editions · the release life cycle · Fedora Changes.

22.2 How Fedora got here: the changes that matter

Fedora changes by *Changes*: a proposal, accepted for one release, which is the default from then on. Much of what Part IV describes started that way. The ones that matter here:

Fedora	Became the default
17	the `/usr` merge: `/bin`, `/sbin` and `/lib` became links into `/usr` (§23.2)
18	`/tmp` in memory (tmpfs)
30	dbus-broker as the D-Bus message bus (§32.1) · boot entries as Boot Loader Specification files (§29.2)
31	cgroups version 2 (§28.6)
33	Btrfs for the desktop editions (Server keeps XFS on LVM, hence §4.2's custom layout) · swap on zram · systemd-resolved for DNS (§33.2)
34	PipeWire for all sound, in place of PulseAudio and JACK (§31.8) · zram as large as memory, at most 8 GiB
35	WirePlumber as PipeWire's session manager
41	DNF 5 as `dnf` (§27.2)
42	`/usr/sbin` merged into `/usr/bin` (§23.2)

Fedora	Became the default
43	RPM 6.0 (§27.1)
45	kmscon for the text consoles (§11.1) · RPM refuses unsigned packages by default (ch. 34)

22.3 How to explore any Linux machine

Five questions answer most of *what is going on here?* on any Linux machine. Each has a command; the chapters that follow explain the answers.

Question	Ask with	Explained in
What is this command?	`type` · `command -v`	ch. 25
Which package owns this file?	`rpm -qf`	ch. 27
What does this setting say?	`cat` · `less`	ch. 23
Is it running?	`pgrep -a` · `systemctl status`	ch. 26 · ch. 28
What did it say?	`journalctl`	ch. 28

1. What is this command? A word you type may be an alias, a function, a command built into bash, or a program on disk.

```
$ type ll
ll is aliased to `eza -la --group-directories-first --git --time-style=long-iso'
$ command -v nvim
/usr/bin/nvim
```

`type` names what a word is; `command -v` prints the program's path.

2. Which package owns this file? Every file under `/usr` belongs to a package; files the desktop built itself, under `/usr/local`, belong to none.

```
$ rpm -qf /usr/bin/foot
foot-…fc44.x86_64
$ rpm -qf /usr/local/bin/dwl
file /usr/local/bin/dwl is not owned by any package
```

3. What does this setting say? Settings are plain text files. `cat` prints a short one; `less FILE` pages through a long one (**q** quits).

```
$ cat /etc/greetd/fedokawaii.toml
# /etc/greetd/fedokawaii.toml — installed by setup.sh (greetd reads it: greetd.service.d/
# fedokawaii.conf). The greeter is agreety (part of greetd): a plain user name + password
# prompt on tty1.
[terminal]
vt = 1

[default_session]
command = "/usr/local/bin/kawa-greeter"
user = "greetd"
```

> **On Windows.** Windows keeps most settings in the registry, a database you open with `regedit`. Linux has no registry: system settings are text files in `/etc`, yours are text files in `~/.config`. Anything that reads text — `cat`, `rg`, neovim, git — can read, search, compare and keep them.

4. Is it running? `pgrep -a NAME` lists running programs by name, with their numbers; `systemctl status` describes a service.

```
$ pgrep -a waybar
1340 waybar
$ systemctl status greetd
● greetd.service - Greeter daemon
     Loaded: loaded (/usr/lib/systemd/system/greetd.service; enabled; preset: disabled)
    Drop-In: /etc/systemd/system/greetd.service.d
             └─fedokawaii.conf
     Active: active (running) since Fri 2026-10-16 08:40:12 +04; 41min ago
…
```

(for example, shortened) **q** leaves the status view when it fills the screen.

5. What did it say? Programs and services write to the journal, and `journalctl` reads it. `-t` picks one program's lines:

```
$ journalctl -t kawa-snap
-- No entries --
```

`kawa-snap` writes only when a snapshot around a DNF run could not be taken, so *no entries* is good news. `journalctl -b -p err` lists this boot's errors.

What you learnt

- The machine is built in layers; each uses only the one below it, and everything above the kernel is userspace.
- Fedora brings its big changes one release at a time; Btrfs, PipeWire and DNF 5 all came that way.
- Five questions explore any Linux machine: `type`, `rpm -qf`, `cat`, `pgrep -a` or `systemctl status`, and `journalctl`.

Check yourself

1. On which layer does dwl live, and on which NetworkManager? → *the session · system services*
2. What does `rpm -qf` answer? → *which package owns a file — or that none does*
3. Where would you look for a system setting on this machine instead of a registry? → *in a text file under `/etc`*

23. Files and the file system

> **In this chapter**
> - read kernel settings and device facts as ordinary files
> - say what each top folder is for, and where programs keep their files
> - see how disks and Btrfs subvolumes are attached to the one tree
> - tell a file's name from the file, and a hard link from a symbolic one
>
> **You need:** ch. 22 · §10.1 (folders, paths and your home).

On Windows each disk has its own letter. Here everything — disks, USB sticks, even the kernel's settings — hangs from one tree that starts at `/`. This chapter walks that tree: what is in it, how disks are attached to it, what Btrfs adds, and what a file really is.

23.1 Everything is a file — nearly

Unix's oldest idea: anything you might read or write is reached through a path and the same few operations — open, read, write, close. The simplest case is one kernel setting, read like a text file:

```
$ cat /proc/sys/vm/swappiness
100
```

So programs that only know how to read text can read the kernel's state, a device or a setting. That `100` is set by the performance profile (§30.4). Two more on this machine:

```
$ cat /sys/class/drm/card1/device/vendor # the GPU's maker: 0x1002 is AMD, 0x8086 Intel
0x1002
$ cat /proc/asound/cards                 # the sound cards the kernel found
 0 [Generic        ]: HDA-Intel - HD-Audio Generic
 …
```

(for example; the card number in `/sys/class/drm` varies) The idea has limits, and knowing them saves confusion:

Not a file, or not only a file	How it is reached instead
network cards	by name through the kernel's network interface (`ip link`); there is no `/dev/eth0`
the GPU's real work	`/dev/dri/*` is opened like a file, but drawing goes through special calls (`ioctl`) made by Mesa, not `read` and `write`
messages between programs	sockets (`/run/user/1000/bus` is one) and D-Bus (§32.1): a path to connect to, then a conversation
a process	it appears as a folder, `/proc/PID`, but you start and stop it with `fork`, `exec` and signals (§26.1)

23.2 The hierarchy

Every file hangs from one tree whose top is `/`, and each top folder has one job. Look at the top of the tree:

```
$ ls /
afs  boot  etc   lib    media  opt   root  sbin  sys  usr
bin  dev   home  lib64  mnt    proc  run   srv   tmp  var
```

The folders that matter, each with an example from this machine:

Path	Holds	Here, for example
`/usr`	the installed software: programs in `/usr/bin`, libraries in `/usr/lib64`, data in `/usr/share`. Owned by packages; never edited by hand	`/usr/bin/foot`
`/usr/local`	software installed by this machine's owner, outside any package	`/usr/local/bin/dwl`, `/usr/local/lib/fedokawaii` (the toolkit)
`/etc`	system settings, as text	`/etc/greetd/fedokawaii.toml`, `/etc/dnf/libdnf5.conf.d/`
`/var`	data that changes as the system runs: logs, caches, databases	`/var/log/journal`, `/var/lib/fedokawaii`, `/var/cache/libdnf5`
`/home`	each user's own folder	`/home/you` (`~`)
`/boot`	the kernels, their initramfs images (the start-up's first stage, ch. 29) and GRUB; `/boot/efi` is the EFI partition	`/boot/vmlinuz-7.2.8-200.fc44.x86_64`
`/run`	runtime state, in memory, gone at shutdown	`/run/user/1000` (your `$XDG_RUNTIME_DIR`: the Wayland and D-Bus sockets)
`/tmp`	temporary files, in memory (tmpfs), emptied at each start	yazi's previews
`/proc`	the kernel's view of processes and settings, made up as you read it	`/proc/cmdline`, `/proc/1234/status`
`/sys`	the kernel's view of devices and drivers	`/sys/class/drm`, `/sys/firmware/efi`
`/dev`	device files, made by the kernel and udev (§30.2)	`/dev/nvme0n1`, `/dev/dri/renderD128`, `/dev/zram0`
`/opt`	large self-contained programs from outside the distribution	Brave's package puts the browser under `/opt/brave.com/`

> **Word — device file.** A file in `/dev` that stands for a piece of hardware: reading or writing it talks to a driver, not to a stored file. `/dev/nvme0n1` is the whole SSD; `/dev/nvme0n1p3`, one partition on it.

`/proc` and `/sys` hold no stored files at all: the kernel makes up their contents each time you read them.

> **On Windows.** `C:\Program Files` and `C:\Windows` correspond to `/usr`, `C:\Users\you` to `/home/you`, and a second disk is not `D:` but a folder in the same tree. Paths use `/`, not `\`, and names are case-sensitive: `Notes` and `notes` are two folders.

The `/usr` merge. Long ago `/bin` and `/sbin` held the programs needed before `/usr` could be mounted. Fedora 17 moved everything into `/usr` and left links behind, and Fedora 42 went one step further: `/usr/sbin` (programs meant for the administrator) merged into `/usr/bin`. On a system installed with Fedora 42 or later all four names lead to one folder:

```
$ command ls -ld /bin /sbin /usr/sbin
lrwxrwxrwx. 1 root root 7 … /bin -> usr/bin
lrwxrwxrwx. 1 root root 8 … /sbin -> usr/sbin
lrwxrwxrwx. 1 root root 3 … /usr/sbin -> bin
```

Old paths such as `/usr/sbin/sshd` keep working; they end in `/usr/bin`.

23.3 Where programs keep their files

Each kind of file has one place. Knowing which places belong to the scripts, which to you, and which can be thrown away is most of keeping a desktop clean.

Take one program, mpv. Its settings are in `~/.config/mpv/mpv.conf`; where each video stopped is in `~/.local/state/mpv/`. Most of your home follows that rule, the XDG Base Directory specification: a program splits its files into settings, data, state and cache, and each kind has its own folder. No `XDG_*` variable is set on this desktop, so every program uses the default folders:

Kind	Folder (variable)	For example
settings	`~/.config` (`XDG_CONFIG_HOME`)	`~/.config/foot/foot.ini`
data: what you would miss	`~/.local/share` (`XDG_DATA_HOME`)	`~/.local/share/zathura/bookmarks.sqlite`: the page you reached in each book
state: worth keeping, not precious	`~/.local/state` (`XDG_STATE_HOME`)	`~/.local/state/mpv/`: where each video stopped
cache: can be made again	`~/.cache` (`XDG_CACHE_HOME`)	`~/.cache/kawa-wallpaper/`: the scaled wallpapers
your own programs	`~/.local/bin`	the `kawa-*` helpers

Read more: XDG Base Directory Specification.

> **On Windows.** `AppData\Roaming` is the nearest thing to `~/.config` and `~/.local/share` together, and `AppData\Local` to `~/.cache`. The difference: here the split is kept by nearly every program, so a whole kind — every cache — can be cleared at once.

The whole picture, place by place, with who writes there and what may go:

Place	Holds	Who writes it	Safe to delete?
`/etc`	system-wide settings: DNF (`/etc/dnf/libdnf5.conf.d/90-fedokawaii.conf`, the snapshot hook in `/etc/dnf/libdnf5-plugins/actions.d/`), the tuned profile, NetworkManager, `/etc/dconf`, `/etc/default/grub`, snapper's configs	Setup and Theme (as root), and Fedora's packages	no. Change Fedokawaii's own files through `fedokawaii.conf` and the toolkit (§5.5); anything else with `sudoedit FILE`, as a drop-in where the program reads one (§28.3)
`~/.config`	every program's settings	Theme owns the files listed in §42.2; the rest are yours	no — but a managed file edited here is replaced at the next Theme run
`~/.config/fedokawaii/`	`env`, `themes/`, `current`, `icons.sh`, `secrets.sh`	Theme (`env` is written from `fedokawaii.conf`); `secrets.sh` is yours	no
`~/.local/bin`	`kawa-*` helpers, `fetch`, `brave-origin`, `kawa-veil`, `kawa-clipkeep`, the release binaries (`yazi`, `ya`, `satty`, `presenterm`, `spotify_player`), `torrra`	Theme, `update` and pipx	no. Your own scripts may live here too, under names the desktop does not use there
`~/.local/share`	program data: Steam and its games, pipx's torrra, podman's images, the trash, neovim's plugins and Mason tools, the pixel art and the wallpaper pack	the programs themselves	no, not as a whole; see §38.1 and §38.2 for the parts you can trim

Place	Holds	Who writes it	Safe to delete?
`~/.local/state/fedokawaii/`	theme stamps, each theme's wallpaper choice, `backup/<time>/`, `lazy-lock/` (neovim's plugin versions before each update)	Theme, `update` and the session	the old folders in `backup/` — the theme itself keeps only its newest 20
`~/.cache`	logs, previews, the scaled wallpapers, the clipboard history, launcher counts	the programs	yes, piece by piece; every program rebuilds its part (§38.2)
`~/Downloads`	what Brave, torrra and yt-dlp fetch	the programs	yes — and its space comes back at once: it is its own Btrfs subvolume, outside every snapshot
`~/Pictures/Screenshots`	Print, Shift+Print, satty's Ctrl+S, mpv's `s`	the screenshot tools	yes, whenever you like
`~/Pictures/Wallpapers`	your own wallpapers, shown on both themes	you	yes
`~/Books`	your library: what the books window (Super+R) lists	you (and its Ctrl+A)	yes, book by book — Ctrl+D in the window sends one to the trash
`~/Notes/notes.md`	your notes (Super+N)	you	only if you mean it: it is your writing

23.4 Mounts

You met the word in Part I (§5.1): to **mount** a file system — a partition, a USB stick, a Btrfs subvolume — is to attach it to a folder of the tree, where its files then appear. That folder is its **mount point**. A stick Windows would call `E:` appears as `/run/media/you/STICK`.

Try this. Ask which kind of file system holds two folders.

```
$ findmnt -no FSTYPE /
btrfs
$ findmnt -no FSTYPE /tmp
tmpfs
```

You typed	It means
`findmnt -no FSTYPE /`	the top of the tree is mounted from the SSD, in Btrfs
`findmnt -no FSTYPE /tmp`	`/tmp` is a separate mount, of a file system that lives in memory

On this machine, `findmnt` draws the whole tree of mounts:

```
$ findmnt -t btrfs,ext4,vfat -o TARGET,SOURCE,FSTYPE
TARGET          SOURCE                FSTYPE
/               /dev/nvme0n1p3[/root]  btrfs
├─/boot         /dev/nvme0n1p2         ext4
│ └─/boot/efi   /dev/nvme0n1p1         vfat
└─/home         /dev/nvme0n1p3[/home]  btrfs
```

(for example) Both `/` and `/home` come from the same partition, `nvme0n1p3`: they are two Btrfs *subvolumes* of one file system (§4.2), shown in brackets. `findmnt -no OPTIONS /` lists the options, among them `compress=zstd:1` (every file is compressed as it is written) and `subvol=/root`. `/proc`, `/sys`, `/run` and `/tmp` are mounts too, of file systems that live in memory: `findmnt /tmp` shows `tmpfs`.

23.5 Btrfs: subvolumes, snapshots and copy-on-write

Most file systems overwrite a changed block where it lies. Btrfs writes the new version somewhere free and then points the file at it.

> **Word — copy-on-write.** Writing a changed block to a new place, then pointing the file at it, instead of overwriting the old block. Like saving an edited spreadsheet under the same name while the old version stays on the disk until nothing needs it.

For example: copy a 4 GB film on Btrfs and `cp` returns at once, because the copy shares the film's blocks until one of them changes. Three things follow from copy-on-write:

Because of copy-on-write	You get	Here
old blocks stay valid until nothing points at them	a **snapshot** is a second set of pointers to the same blocks: instant, and free until something changes	snapper's snapshots of `/` and `/home` (§37.2)
a file can share blocks with another	a copy that costs nothing until you edit it: `cp` makes these by itself on Btrfs (GNU coreutils 9 and later)	`cp big.iso copy.iso` returns at once
every block is checksummed	a disk that returns wrong data is noticed	the monthly scrub (§38.2)

> **Word — snapshot.** A frozen view of a subvolume at one moment, made by keeping the old blocks' pointers. It takes no time and no space at first; it grows only as files change after it. Not a backup: it lives on the same disk.

A **subvolume** is a folder with its own tree, snapshotted as a whole; a snapshot never reaches into a subvolume nested inside it. That is why `~/Downloads` is one (§37.2): it is left out of `/home`'s snapshots, so a deleted download frees its space at once. The top folder of every subvolume has inode number 256 (inodes below):

```
sudo btrfs subvolume list /    # root, home, home/you/Downloads, and the snapshots
stat -c %i ~/Downloads         # 256
```

Ch. 37 puts snapshots to work: what is snapshotted, when, and how to undo with one.

23.6 Inodes and links

A file's name is not part of the file.

> **Word — inode.** The record a file system keeps for one file: type, permissions, owner, size, times and where its data lies — everything but its name. A folder is a list of names, each pointing at an inode, like shelf labels pointing at catalogue cards.

That makes two kinds of link possible. Both at once, in a fresh, empty folder under `/tmp`: `mktemp -d` makes one, and `cd "$(mktemp -d)"` goes into it (`$( … )` puts a command's output in its place, §25.1).

```
$ cd "$(mktemp -d)"
$ echo hello > a.txt
$ ln a.txt b.txt          # a hard link: a second name for the same inode
$ ln -s a.txt c.txt       # a symbolic link: a tiny file holding the path "a.txt"
$ stat -c '%i %h %F %N' a.txt b.txt c.txt
1052114 2 regular file 'a.txt'
1052114 2 regular file 'b.txt'
1052131 1 symbolic link 'c.txt' -> 'a.txt'
```

(for example: the inode numbers vary) `a.txt` and `b.txt` share one inode, so its link count (`%h`) is 2; `c.txt` has an inode of its own.

	Hard link	Symbolic link
is	another name for the same inode (`%h`, the link count, says how many)	a small file that holds a path
when the original name is deleted	the data stays until the last name goes	the link dangles: it points at nothing
across file systems or subvolumes	no	yes
to a folder	no	yes

> **Word — symbolic link.** A tiny file that holds the path of another file or folder; opening the link opens what it points at. Like a Windows shortcut, except that every program follows it, not only File Explorer.

Symbolic links hold this desktop together in several places:

Link	Points at	Why
`~/.config/fedokawaii/current`	`themes/dark` or `themes/light`	every program reads its colours through it; switching the theme re-points one link (§9.1)
`/etc/resolv.conf`	`../run/systemd/resolve/stub-resolv.conf`	DNS goes through systemd-resolved (§33.2)
`/usr/local/bin/vi`	`/usr/bin/nvim`	`vi` opens neovim
`/etc/systemd/system/display-manager.service`	greetd's unit	which login manager starts (`fedokawaii repair` checks it)

Read more: file-hierarchy(7) · inode(7) · Btrfs subvolumes · Unify /usr/bin and /usr/sbin.

23.7 Practice: a walk through the tree

Six short tasks check each idea of this chapter on your own machine. They only read, except in `~/practice`, the folder from ch. 11, where one link is made.

1. Look at the top of the tree.

```
$ ls /
afs  boot  etc   lib    media  opt   root  sbin  sys  usr
bin  dev   home  lib64  mnt    proc  run   srv   tmp  var
```

You should see about twenty names, among them `etc`, `home` and `usr`.

If you see your own folders (Documents, Downloads…): you typed `ls` without the `/`.

2. Ask which system this is.

```
$ cat /etc/os-release
NAME="Fedora Linux"
VERSION="44 (Server Edition)"
…
PRETTY_NAME="Fedora Linux 44 (Server Edition)"
…
```

(shortened) The installer was Fedora Server's, so the file still says *Server Edition*; the desktop was added by setup.

If you see `No such file or directory:` `os-release` has a hyphen, not a dot or a space.

3. See where `/` and `/home` come from.

```
$ findmnt /
TARGET SOURCE                FSTYPE OPTIONS
/      /dev/nvme0n1p3[/root] btrfs  rw,relatime,seclabel,compress=zstd:1,ssd,…
$ findmnt /home
TARGET SOURCE                FSTYPE OPTIONS
/home  /dev/nvme0n1p3[/home] btrfs  rw,relatime,seclabel,compress=zstd:1,ssd,…
```

(for example, shortened) One partition, two subvolumes: `[/root]` and `[/home]`.

If you see nothing at all: the path is not a mount point (`findmnt ~` prints nothing); `findmnt -T ~` finds the mount that holds a folder.

4. Show the hidden files in your home.

```
$ ls -a ~
.bashrc.d  .local   Documents   Music     practice   Videos         .bash_profile
.cache     Books    Downloads   Notes     Public     .bash_history  .bashrc
.config    Desktop  fedokawaii  Pictures  Templates  .bash_logout   .inputrc
```

(for example) Every name that starts with a dot is hidden from a plain `ls`: settings, caches, the shell's history. `practice` is the folder from ch. 11; `fedokawaii`, the project you installed from (§5.5).

If you see no dot names: the `-a` (*all*) is missing.

5. Make a symbolic link to the trip plan from ch. 11, then break it, and mend it.

```
$ cd ~/practice
$ ln -s trips/japan/plan.txt japan.txt
$ cat japan.txt
Tokyo, Kyoto, Osaka
$ mv trips/japan/plan.txt trips/japan/plan-old.txt
$ cat japan.txt
cat: japan.txt: No such file or directory
$ command ls -l japan.txt
lrwxrwxrwx. 1 you you 20 … japan.txt -> trips/japan/plan.txt
$ mv trips/japan/plan-old.txt trips/japan/plan.txt
```

The link holds only a path, 20 characters long. While nothing has that name, it dangles; the last `mv` gives the name back, and the link works again.

If the first `cat` *already says* `No such file or directory:` the plan is not there; make it with `mkdir -p trips/japan` and `echo 'Tokyo, Kyoto, Osaka' > trips/japan/plan.txt`.

6. Read the file's inode, and a link the desktop relies on.

```
$ stat trips/japan/plan.txt
  File: trips/japan/plan.txt
  Size: 20          Blocks: 8          IO Block: 4096   regular file
Device: 0,41    Inode: 1052377     Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/     you)   Gid: ( 1000/     you)
Context: unconfined_u:object_r:user_home_t:s0
…
$ readlink ~/.config/fedokawaii/current
themes/dark
```

(for example, shortened) `stat` prints the inode: its number, its link count, its owner and permissions (ch. 24), its SELinux label (§34.2) and its times — but no name. `readlink` prints where a symbolic link points.

If you see `themes/light`*:* the light theme is on — a right answer, not a mistake.

What you learnt

- Nearly everything is reached as a file in one tree from `/`; `/proc` and `/sys` are the kernel's own view, made up as you read them.
- Each top folder has one job: software in `/usr`, settings in `/etc`, changing data in `/var`, your files in `/home`.
- In your home, settings go in `~/.config`, data in `~/.local/share`, caches in `~/.cache`.
- A file system is mounted at a folder; `/` and `/home` are two Btrfs subvolumes of one partition.
- Copy-on-write makes snapshots and copies instant; an inode is the file, a name only points at it.

Check yourself

1. Where would you find the system's DNF settings, and where your own foot settings? → `/etc/dnf/…` · `~/.config/foot/foot.ini`
2. Why does deleting a download free its space at once, despite the snapshots? → `~/Downloads` *is its own subvolume, left out of* `/home`*'s snapshots*
3. You rename the file a symbolic link points at. What happens to the link? → *it dangles: it still holds the old path*

24. Users, permissions and sudo

> **In this chapter**
> - read your user and group entries, and what each group gives
> - read and set a file's permissions, by letters and by numbers
> - run one command as root with `sudo`, and know who asks when there is no `sudo`
> - say why the desktop never runs as root
>
> **You need:** ch. 23 (files and the tree).

Every file in the tree has an owner, and every program runs as some user. This chapter shows who you are to the kernel, what a file's permissions allow, and how an administrator borrows root's power for one command at a time.

24.1 Users and groups

The kernel knows users and groups only as numbers. Names are for people.

> **Word — UID.** *User ID*: the number the kernel knows a user by; the name is looked up from it. Like a phone's contacts: the call goes to the number, the name is what the list shows. Yours is 1000; root's is 0.

Try this. Ask the kernel for your number, then for your name.

```
$ id -u
1000
$ id -un
you
```

You typed	It means
`id -u`	your UID, the number every file you own carries
`id -un`	the name looked up for that number

The names come from two text files. Your own line in the first of them:

```
$ getent passwd you
you:x:1000:1000:You:/home/you:/bin/bash
```

Field	Value	Means
1	`you`	the user name
2	`x`	the password lives elsewhere: hashed, in `/etc/shadow`, readable by root only
3, 4	`1000`, `1000`	the UID and the primary GID. Fedora numbers people from 1000; lower numbers are system users
5	`You`	a description, usually the full name
6, 7	`/home/you`, `/bin/bash`	the home folder and the login shell

> **In SQL terms.** `/etc/passwd` is a table: one row per user, seven columns, `:` between them. `getent passwd you` is `SELECT * FROM passwd WHERE name = 'you'`, and the UID is the key that every file's owner column refers to.

root is UID 0: the kernel lets it past every permission check (SELinux still applies). On this machine root has no password and cannot log in (§4.2); administrators borrow its power one command at a time. **System users** such as `greetd` own a service's files and run it with less power than root.

> **Word — group.** A named set of users given access together; its number is the **GID**. Each user has one primary group, here of the same name, and may join more. Like a mailing list: grant something to it, and every member has it.

```
$ id -nG
you wheel video input render gamemode
```

(for example)

Group	Gives	Added by
`wheel`	administrator: `sudo`, and polkit's password prompts accept your password	the installer (*Add administrative privileges to this user account* ticked)
`video`, `render`	the GPU's device files, also outside a graphical session	setup
`input`	input devices directly	setup
`gamemode`	gamemode's CPU and GPU boosts	setup, with Steam

Group changes reach a session only at the next login.

24.2 Permissions

Each file has an owner, a group, and three sets of three permissions. The shortest way to see them is one line of `ls -l`:

```
$ command ls -l ~/.config/fedokawaii/secrets.sh /usr/local/bin/start-dwl
-rw-------. 1 you  you   120 … /home/you/.config/fedokawaii/secrets.sh
-rwxr-xr-x. 1 root root 2875 … /usr/local/bin/start-dwl
```

(for example) Read the first ten characters of the first line from the left (the SELinux label is §34.2):

```
-  rw-  ---  ---  .
│  │    │    │    └─ "." an SELinux label (Part IV's last chapter); "+" would mean an ACL as well
│  │    │    └────── others: no access
│  │    └─────────── the group: no access
│  └──────────────── the owner (you): read and write
└─────────────────── the type: - file, d folder, l link, c character device, s socket
```

Letter	On a file	On a folder	Octal
`r`	read its contents	list the names in it	4
`w`	change its contents	create, rename and delete names in it	2
`x`	run it as a program	enter it and reach what is inside	1

Changing them, by letters. `chmod` takes *who* (`u` the owner, `g` the group, `o` others, `a` all), then `+` or `-`, then the letters. A worked example in `~/practice`, with `ls -l` after each step:

```
$ cd ~/practice
$ echo hello > note.txt
$ command ls -l note.txt
-rw-rw-r--. 1 you you 6 … note.txt
$ chmod o-r note.txt
$ command ls -l note.txt
-rw-rw----. 1 you you 6 … note.txt
$ chmod g-w,u+x note.txt
$ command ls -l note.txt
-rwxr-----. 1 you you 6 … note.txt
```

You typed	It means
`echo hello > note.txt`	a new file: you and your group may read and write it, others read it (why: *umask*, below)
`chmod o-r note.txt`	others lose *read*: the last set becomes `---`
`chmod g-w,u+x note.txt`	the group loses *write*, the owner gains *run*: two changes, separated by a comma

By numbers. Add the numbers for each set: `rw-` is 6, `r-x` 5, `---` 0. So `600` is "the owner reads and writes, nobody else anything" — what the shell demands of `secrets.sh` before it reads it — and `755` is "everyone may run it, only the owner may change it":

```
chmod 600 ~/.config/fedokawaii/secrets.sh   # private
chmod +x ~/.local/bin/myscript              # add x for everyone
stat -c '%a %U:%G %n' /usr/local/bin/dwl    # 755 root:root /usr/local/bin/dwl
```

> **On Windows.** NTFS gives each file a list of entries — this user or group may read, write, modify — inherited from its folder. Unix gives every file the same three fixed classes: owner, group, everyone else. ACLs (below) add NTFS-style extra entries where needed.

umask. Why did `note.txt` start as `rw-rw-r--`? New files start from 666 (folders 777) with the bits in your *umask* taken away.

> **Word — umask.** The permission bits taken away from every file and folder you create. Like a default on a shared drive that says *others may not write here*, applied before you set anything yourself.

```
$ umask
0002
```

(for example) `0002` takes write away from others only: new files come out `664`, folders `775`. That is Fedora's mask for a user whose primary group has the user's own name — the usual case, and that group holds only you, so "the group may write" gives nobody else anything. It is set at every login, the desktop's and a console's alike: `/etc/login.defs` says `UMASK 022` and `USERGROUPS_ENAB yes`, and `pam_umask` turns that into 002 for such a user. A mask of `0022` takes write away from the group as well: files come out `644`. Whichever one `umask` prints on your machine, the arithmetic is the same: 666 minus the mask.

Three special bits sit in front of the usual nine:

Bit	Octal	Shows as	Does	Example
set-user-ID	4000	`s` in the owner's `x`	the program runs as its owner, not as you	`/usr/bin/sudo` is `---s--x--x`, owned by root: that is how it can become root
set-group-ID	2000	`s` in the group's `x`	on a folder: new files inherit the folder's group	shared project folders
sticky	1000	`t` in the others' `x`	on a folder: only a file's owner may delete or rename it	`/tmp` is `drwxrwxrwt`: everyone writes there, nobody deletes another's file

ACLs (access control lists) add rules for further users or groups to one file; `ls -l` shows a `+` after the permissions. logind, systemd's login manager (§29.8), uses them to give the person at the keyboard the GPU and the sound devices: `command ls -l /dev/dri/card1` shows `crw-rw----+`.

24.3 sudo

`sudo COMMAND` runs one command as root after asking for *your* password. For example:

```
$ sudo whoami
[sudo] password for you:
root
```

Nothing shows while you type the password; for the next five minutes this terminal's `sudo` does not ask again. Fedora's `/etc/sudoers` allows it to the `wheel` group with one line:

```
%wheel  ALL=(ALL)       ALL
```

(*members of wheel, on any host, as any user, may run any command.*) Change sudo's rules only with `sudo visudo`, which checks the file before it saves it, so a slip cannot lock you out; for any other system file, `sudoedit FILE` opens a copy in neovim as you and puts it in place as root. Fedora still ships the original sudo; a proposal to switch to sudo-rs, a Rust rewrite, was drafted for Fedora 44 but has not been accepted.

> **On Windows.** The UAC prompt that dims the screen is the same idea: an administrator works as an ordinary user and confirms each time more power is needed. Here the confirmation is always your password, and it covers one command, not a whole program.

24.4 polkit

`sudo` is for commands. Services need a different gatekeeper.

> **Word — polkit.** The system service that decides whether a user may ask another service to do something privileged: restart a unit, mount a disk, change a network. Its rules answer *yes*, *no* or *only after a password*.

When a program asks a system service over D-Bus (§32.1) to do something privileged, the service asks polkit whether this user may. Fedora's rules treat members of `wheel` as administrators. For the password polkit needs an **authentication agent** in your session: here `lxqt-policykit-agent`, started by the autostart. Try it:

```
systemctl restart bluetooth.service   # no sudo: polkit's dialog asks for your password
```

Bluetooth stops for a moment, so a connected device drops and may need connecting again.

On a text console there is no graphical agent, and `systemctl` asks on the terminal itself. Removable disks need no password at all: polkit allows mounting them to whoever sits at the machine, which is how `udiskie` mounts a USB stick for you (§32.6).

24.5 Why the desktop never runs as root

Root is borrowed for one command, never kept for a session:

Run as root, and	Because
every program it starts is root: a bad web page in the browser could change the system	privilege is inherited by every child process
files you save in your home belong to root, and your own programs cannot change them	new files take their creator's UID; `verify` checks for root-owned files in your home
the session's sockets, settings and services are in the wrong place	the session lives in `/run/user/1000` and your user manager, not root's
a typing slip in a terminal can delete the system	nothing stands between root and `rm`

Use `sudo` for the one command that needs it, and nothing else.

Read more: sudoers(5) · polkit(8).

24.6 Practice: permissions by hand

Five tasks, in `~/practice`. Only the practice files change.

1. Ask who you are.

```
$ id
uid=1000(you) gid=1000(you) groups=1000(you),10(wheel),39(video),… context=unconfined_u:…
```

(for example, shortened) Your UID, your primary group, every group you are in, and the SELinux type your shell runs as (§34.2).

If you see `uid=0(root)`*:* this terminal is a root shell; type `exit` to come back to yourself.

2. Give one file three sets of permissions in turn.

```
$ cd ~/practice
$ echo hello > perm.txt
$ chmod 644 perm.txt; command ls -l perm.txt
-rw-r--r--. 1 you you 6 … perm.txt
$ chmod 600 perm.txt; command ls -l perm.txt
-rw-------. 1 you you 6 … perm.txt
$ chmod 755 perm.txt; command ls -l perm.txt
-rwxr-xr-x. 1 you you 6 … perm.txt
```

You should see `644` as everyone reads, `600` as only you, `755` as everyone may run it.

If you see `chmod: cannot access 'perm.txt': No such file or directory`*:* you are not in `~/practice`; `cd ~/practice` first.

3. Make a script that will not run until it may.

```
$ echo 'echo it runs' > hello.sh
$ ./hello.sh
bash: ./hello.sh: Permission denied
$ chmod +x hello.sh
$ ./hello.sh
it runs
```

A text file runs as a program only with `x`. (Ch. 25 adds the first line every script should have.)

If you see `bash: hello.sh: command not found`*:* the `./` is missing. Without a path the shell looks only in the folders of `PATH` (§25.3), never in the folder you are in.

4. Ask what you may do as root.

```
$ sudo -l
[sudo] password for you:
Matching Defaults entries for you on fedora:
    !visiblepw, always_set_home, match_group_by_gid, always_query_group_plugin, env_reset, …

User you may run the following commands on fedora:
    (ALL) ALL
```

(shortened) The last line is the `%wheel` rule: as any user, any command.

If you see `Sorry, user you may not run sudo on fedora.`: you are not in `wheel` — the installer's box *Add administrative privileges to this user account* was left empty (§4.2).

5. See why `sudo` reads what you cannot.

```
$ command ls -l /etc/shadow
----------. 1 root root 1043 … /etc/shadow
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ sudo wc -l /etc/shadow
31 /etc/shadow
```

(for example) The file allows nobody anything, yet root reads it: root passes every permission check. `wc -l` counts its lines instead of printing them, so no password hash reaches the screen.

If you see `wc: /etc/shadow: Permission denied`: the `sudo` in front is missing.

What you learnt

- The kernel knows you as UID 1000 and your groups as numbers; `/etc/passwd` maps the names.
- `rwx` for owner, group and others decide what each may do; `chmod` sets them by letters (`g-w`) or numbers (`600`).
- Your umask decides what a new file starts with: `664` here.
- `sudo` runs one command as root after your password; polkit answers for services, through the agent's dialog.
- The desktop never runs as root: root is borrowed for one command, never kept.

Check yourself

1. What does `chmod 750 FILE` allow? → *the owner everything, the group read and run, others nothing*
2. Why does `./hello.sh` fail before `chmod +x`? → *a file runs as a program only with the `x` permission*
3. Who asks for your password when you restart a service without `sudo`? → *polkit, through the authentication agent `lxqt-policykit-agent`*

25. The shell as a language

> **In this chapter**
> - steer a command's output and errors into files and into other commands
> - say where a command is found, and what a program inherits from its shell
> - quote names and patterns safely
> - write, check and improve a script of your own
>
> **You need:** ch. 24 · ch. 18 (the shell's helpers).

In ch. 11 the shell was a place to type commands. It is also a small programming language: each command line is a sentence in it, and a script is a page of such sentences. This chapter teaches its grammar, by example, and ends with a script of your own.

25.1 The shell's ideas, by example

Ch. 11 named the first words: *variable* and *exit status* (§11.2). These are the rest, all at work in the examples below:

Term	Plain meaning	Here
readline	bash's line editor: what Tab, Up and the word keys do while you type	`~/.inputrc`
alias · function	a word typed in place of a longer command · a named piece of shell code that takes arguments and can decide	`ll` · `prowlarr`, `prowlarr stop`
environment variable	a variable marked with `export`, so every program started from this shell gets a copy	`EDITOR=nvim`
`PATH`	the folders searched, in order, for a command typed by its name	`~/.local/bin` first
pipe · redirection	one command's output into the next command · output into a file, or input from one	the vertical bar · `>`
glob	a pattern that bash turns into the matching file names before the command runs	`*.pdf`
quoting	`'…'` keeps text exactly as typed; `"…"` keeps it one word but still expands `$NAME`	`"$file"`

Small, safe commands for each idea; try them in any folder. Anything after `#` is a comment, which bash ignores.

Alias, function or program. `type -t` names what a word is:

```
$ type -t ll prowlarr nvim cd
alias
function
file
function
```

`ll` is an alias, `prowlarr` a function (it looks at its argument: `prowlarr stop`), `nvim` a program found through `PATH` (`command -v nvim` prints `/usr/bin/nvim`), and `cd` is zoxide's function. `type ll` shows what the alias stands for. Your own go in `~/.bashrc.d/local.sh`, which every new shell reads after this desktop's file, and which Theme never touches:

```
# ~/.bashrc.d/local.sh — yours
alias gs='git status'                          # an alias: typing gs runs git status
mkcd() { mkdir -p -- "$1" && cd -- "$1"; }     # a function: mkcd photos makes the folder, then goes in
```

An alias only swaps one word for others. Anything that needs an argument in the middle, or a decision, is a function.

Environment variables. Only an exported variable reaches the programs a shell starts:

```
NAME=hello; bash -c 'echo "[$NAME]"'    # prints []       — a plain variable stays in this shell
export NAME; bash -c 'echo "[$NAME]"'   # prints [hello]  — exported: the new bash has a copy
printenv EDITOR                         # prints nvim     — how git and sudoedit know your editor
```

Exit status, `&&` and `||`.

```
grep -q autocd ~/.bashrc.d/fedokawaii.sh; echo $?   # 0 — found
grep -q zsh ~/.bashrc.d/fedokawaii.sh; echo $?      # 1 — not found
mkdir -p ~/practice/tmp && cd ~/practice/tmp        # && runs the cd only if mkdir succeeded
grep -q zsh ~/.bashrc.d/fedokawaii.sh || echo none  # || runs the echo only after a failure
```

Pipes. The vertical bar `|` hands one command's output to the next command as its input. Your five most-used commands:

```
history | awk '{print $4}' | sort | uniq -c | sort -rn | head -5
```

Step	Passes on
`history`	every saved line: `812 2026-10-14 21:03:11 git status`
`awk '{print $4}'`	the fourth field of each line, the command's name: `git`
`sort`	the names in order, so the same ones stand together
`uniq -c`	each name once, with how many times it came: `251 git`
`sort -rn`	by that number (`-n`), largest first (`-r`)
`head -5`	the first five lines only

> **In SQL terms.** That pipeline is a query: `history` is the `FROM`, `awk` picks one column, `sort | uniq -c` is `GROUP BY` with `COUNT(*)`, `sort -rn` is `ORDER BY count DESC`, and `head -5` is `LIMIT 5`. Each bar hands its rows to the next clause.

Redirection.

```
ls ~/Downloads > ~/practice/list.txt   # > writes the output into a file, replacing what it held
date >> ~/practice/list.txt            # >> adds to its end instead
ls /nope 2> /dev/null                  # 2> sends the error messages away; /dev/null discards them
```

`>` empties its file before the command starts, so `sort list.txt > list.txt` leaves an empty file: write to a new name, then rename it.

Globs. Bash turns the pattern into file names before the command sees it:

Pattern	Matches
`*.pdf`	every name ending in `.pdf`, in this folder
`IMG_?.jpg`	`?` is exactly one character: `IMG_1.jpg`, not `IMG_12.jpg`
`[ab]*`	names that start with `a` or `b`
`**/*.pdf`	PDFs here and in every folder below (`globstar`)
`!(*.jpg)`	everything except JPEGs (`extglob`, on here)

A pattern matches a hidden name (`.bashrc`) only when its own first character is a dot. When nothing matches, bash passes the pattern on unchanged, and `ls` reports that `*.pdf` does not exist.

Quoting.

```
f='old notes.txt'
ls $f           # wrong: bash splits at the space, so eza looks for "old" and "notes.txt"
ls "$f"         # right: one name, space and all
echo '$HOME'    # prints $HOME    — single quotes keep every character as typed
echo "$HOME"    # prints /home/…  — double quotes keep the words together but expand $NAME
```

Put double quotes round every `"$variable"` : ShellCheck's most frequent warning, SC2086, is exactly this (§17.15), and your first script meets it (§25.6).

Command substitution.

> **Word — command substitution.** `$( … )` inside a command line: bash runs the command inside the brackets first and puts what it prints in its place. Like a cell formula that refers to another cell's result.

```
$ echo "Today is $(date +%A)"
Today is Friday
```

(for example) `date +%A` ran first and printed `Friday` ; `echo` received the finished sentence. The same shape is at work in `nvim "$(fzf)"` (§18.2) — edit whichever file you pick — and in `cd "$(mktemp -d)"` (§23.6), which goes into a new temporary folder. Keep the double quotes round it, for the same reason as round a variable.

Loops.

> **Word — loop.** A piece of a command line or script that runs again for each item of a list. `for f in *.jpg; do …; done` means *for every JPEG here, in turn, call it* `f` *and do this.*

```
$ for n in 1 2 3; do echo "line $n"; done
line 1
line 2
line 3
```

Part	Means
`for n in 1 2 3;`	take each word of the list in turn, and call it `n`
`do echo "line $n";`	the command to run each time; `$n` is the current word
`done`	the end of the loop

The list is usually a glob: `for f in *.docx; do abiword --to=pdf "$f"; done` turns every Word file in a folder into a PDF, one by one (§19.11); the picture recipes in §19.7 shrink every photo the same way.

25.2 Three streams, redirection and pipes

Every program reads from one place and writes to two: one for its results, one for its complaints. That is why `ls /nope 2> /dev/null` above printed nothing — the complaint went one way, and there were no results.

> **Word — file descriptor.** A small number a process uses for each file or channel it has open. Every process starts with three: 0 to read from, 1 for its results, 2 for its error messages. Like a desk's numbered trays: one in-tray, two out-trays.

Number	Name	Normally
0	standard input	the keyboard (or a pipe)
1	standard output	the terminal (or a pipe, or a file)
2	standard error	the terminal, even when output is redirected

```
ls ~/Books > books.txt          # 1 into a file
ls /nope 2> /dev/null           # 2 thrown away
journalctl -b 2>&1 | wc -l      # 2 joined to 1, then counted
```

The autostart uses the same tools: `waybar < /dev/null > ~/.cache/waybar.log 2>&1` gives the bar an empty input (`< /dev/null`) and sends both its outputs to its log. §25.1 has the full tour.

> **On Windows.** PowerShell pipes pass *objects* with named properties. Bash pipes pass plain lines of text, and each program splits them into columns itself (`awk '{print $4}'`). Less tidy, but any program, from any decade, can join any pipe.

25.3 Environment and PATH

Each process carries **environment variables**, a copy of its parent's; `export` marks a shell variable to be copied.

Try this. Ask for one variable of your environment.

```
$ printenv EDITOR
nvim
```

Every program you start from this shell gets that copy: it is how git and `sudoedit` know your editor.

The session's variables come from `start-dwl` (§29.8), so every program you start inherits `XDG_CURRENT_DESKTOP=wlroots` and the rest. **PATH** lists the folders searched for a command typed by its name:

```
$ echo "$PATH" | tr : '\n'
/home/you/.local/bin
/home/you/bin
…
/usr/local/bin
/usr/bin
$ type -a vi
vi is /usr/local/bin/vi
```

(for example) Services started by your user manager do not get your shell's PATH. That is why the screen-sharing portal's config names the helpers by their full paths (`/home/you/.local/bin/kawa-share`, written `@HOME@/…` in the toolkit): it runs with the manager's environment, not yours.

> **On Windows.** Windows has a `PATH` too (*System → Environment Variables*), with `;` between folders instead of `:`. `cmd.exe` also looks in the current folder first; bash never does — like PowerShell, which wants `.\script.ps1`, bash wants `./script`.

25.4 Quoting, globs and regular expressions

Two kinds of pattern look alike and are not: a glob matches file names, a regular expression matches text.

> **Word — regular expression.** A pattern for matching text, with its own small language: `.` any character, `*` repeated, `[a-z]` one of a range, `^` the start of a line. Like SQL's `LIKE`, but richer: `LIKE 'kawa-%'` is roughly `^kawa-`.

	A glob	A regular expression
used by	bash, before the command runs; `fd -g`	`rg`, `grep`, `sed`, `awk`, `journalctl -g`
matches	file names	text
any text	`*`	`.*`
one character	`?`	`.`
example	`*.pdf` — every PDF here	`kawa-[a-z]+` — `kawa-` followed by letters

Quote a regular expression (`rg 'kawa-[a-z]+'`), or bash may try to read it as a glob first. Quote every `"$variable"` that holds a file name (§25.1).

25.5 Scripts and the shebang

A script is a text file of commands with `x` permission (§24.2).

> **Word — shebang.** The first line of a script, starting `#!`, naming the program that runs it: `#!/usr/bin/env bash` means *run me with bash*. The name comes from *hash* (`#`) and *bang* (`!`).

First line	Runs with	Here
`#!/bin/sh`	the POSIX shell (bash in its plain mode, on Fedora)	`start-dwl`, `autostart.sh`
`#!/usr/bin/env bash`	the first `bash` on PATH	`kawa-theme`, `kawa-nowplaying`, `setup.sh`

Two checks before running one, both safe — neither runs the script:

```
bash -n ~/.local/bin/myscript     # check the syntax without running it
shellcheck ~/.local/bin/myscript  # explain likely mistakes
```

ShellCheck is the same checker neovim's bash language server uses (§17.15).

> **On Windows.** There the ending decides: `.bat`, `.ps1`, `.exe`. Here the first line and the `x` permission decide, and a script needs no ending at all: `kawa-theme` and `start-dwl` are scripts.

25.6 Your first script, step by step

This is the one place in Part IV where you write a file to keep: `space`, a command that tells you the free space on `/` and in your home. Every version is shown in full. Write it with neovim (ch. 17); `~/.local/bin` is first on your `PATH`, and the desktop uses no command called `space`.

1. Write the first version: `nvim ~/.local/bin/space`, type these three lines, then `:wq`.

```
#!/usr/bin/env bash
# space — how much room is left on the disk, for / and for your home
df -h --output=avail,target / ~
```

You should see nothing: neovim closes and the prompt comes back.

If you see `:wq` typed into the file: neovim was still in Insert mode; press **Esc**, remove the stray `:wq` with **x** (one character for each press), then type `:wq` again.

2. Run it — it refuses — then allow it to run.

```
$ space
bash: /home/you/.local/bin/space: Permission denied
$ chmod +x ~/.local/bin/space
$ space
Avail Mounted on
 412G /
 412G /home
```

(for example) Bash found `space` through `PATH`, but would not run it without `x`. The same number twice is no mistake: `/` and `/home` are two subvolumes of one Btrfs file system (§23.4), sharing its free space.

If you see `bash: space: command not found:` the file is not in `~/.local/bin`; check with `ls ~/.local/bin/space`.

3. Write the second version: an argument, an `if` and an exit status. Open the file again and replace everything in it with these lines:

```
#!/usr/bin/env bash
# space — how much room is left on the disk
#   space          for / and for your home
#   space FOLDER   for the disk that holds FOLDER
if [ $# -eq 0 ]; then
  df -h --output=avail,target / ~ | tail -n +2
  exit 0
fi
if [ ! -d $1 ]; then
  echo "space: $1 is not a folder" >&2
  exit 1
fi
df -h --output=avail,target $1 | tail -n +2
```

New	Means
`$#` · `$1`	how many arguments the script was given · the first of them
`if [ $# -eq 0 ]; then … fi`	*if the number of arguments equals 0*; `[ ]` is the test, `fi` ends the `if`
`[ ! -d $1 ]`	*if the first argument is not a folder*
`>&2`	this message goes to standard error, stream 2 (§25.2)
`exit 0` · `exit 1`	end here, with success · with a failure
the bar, then `tail -n +2`	a pipe into `tail`, which prints from the second line on: the heading line is left out

4. Check it, then try it three ways.

```
$ bash -n ~/.local/bin/space
$ space
 412G /
 412G /home
$ space ~/Documents
 412G /home
$ space ~/nowhere
space: /home/you/nowhere is not a folder
$ echo $?
1
```

(for example) `bash -n` prints nothing: the syntax is sound. A folder in your home is on the disk mounted at `/home`; a folder that does not exist ends the script with status 1.

If `bash -n` prints `syntax error`: a word is missing — usually the `fi`, or the `;` before `then`; compare your file line by line with the one above.

5. Find the hidden bug. Make a practice folder with a space in its name, then ask about it, then ask ShellCheck.

```
$ mkdir ~/practice/'old notes'
$ space ~/practice/'old notes'
/home/you/.local/bin/space: line 9: [: /home/you/practice/old: binary operator expected
df: /home/you/practice/old: No such file or directory
df: notes: No such file or directory
$ shellcheck ~/.local/bin/space

In /home/you/.local/bin/space line 9:
if [ ! -d $1 ]; then
          ^-- SC2086 (info): Double quote to prevent globbing and word splitting.
…
```

(shortened: the same finding follows for line 13) Without quotes, bash split the name at the space, so `[` and `df` each received two words. The script even ended with status 0: a pipe's status is its last command's, and `tail` succeeded.

If `mkdir` says `File exists`: the folder is there from before; go on with `space`.

6. Write the third version: the two `$1` quoted, as ShellCheck suggests.

```
#!/usr/bin/env bash
# space — how much room is left on the disk
#   space          for / and for your home
#   space FOLDER   for the disk that holds FOLDER
if [ $# -eq 0 ]; then
  df -h --output=avail,target / ~ | tail -n +2
  exit 0
fi
if [ ! -d "$1" ]; then
  echo "space: $1 is not a folder" >&2
  exit 1
fi
df -h --output=avail,target "$1" | tail -n +2
```

```
$ shellcheck ~/.local/bin/space
$ space ~/practice/'old notes'
 412G /home
```

(for example) ShellCheck is silent, and the name with a space arrives as one word.

If ShellCheck still reports SC2086: one `$1` is still bare; both lines need the quotes.

The script is yours, in your home: Theme and `update` never touch it. It is step 2 of the ladder of first projects (§43.6).

25.7 History rules

The shell saves what you type, so Up and Ctrl+R can find it again. A few things are kept out.

A command typed with a **leading space** is not saved: use it for anything with a password or token in it. Bare `ls`, `ll`, `lt`, `cd`, `cd -`, `pwd`, `exit`, `clear` and `history` are not saved either (`ls -la` is: a pattern must match the whole line), and a repeated command keeps only its newest copy.

```
 export TOKEN=…        # starts with a space: never written to the history
```

25.8 Secrets: keys and passwords

A secret is anything that lets someone act as you: an API key, a token, a password.

API keys go in `~/.config/fedokawaii/secrets.sh` (mode 600), which every new shell reads — and only while the file is yours alone: a copy others could read is not loaded, and the shell says so. Write them as `export NAME=value` — without `export` the programs you start never see them. Setup keeps Prowlarr's key there too (§20.1), deliberately without `export`: only the torrent search reads it, so no other program carries it. Nothing in the repository holds a key.

```
# ~/.config/fedokawaii/secrets.sh — yours alone (chmod 600)
export ANTHROPIC_API_KEY=…
```

Should its permissions ever widen, every new terminal says so instead of loading it:

```
secrets.sh not loaded: it must be yours and private — chmod 600 ~/.config/fedokawaii/secrets.sh
```

25.9 Why a new terminal opens at once

Every helper's shell code is written once to `~/.cache/fedokawaii/` and read from there: a new terminal starts no extra program, and the prompt itself runs none outside a git repository. zoxide learns a folder only when you arrive in it — one quick call then, nothing on any other prompt. (`time bash -ic exit` shows the cost: a few hundredths of a second.) After an `update` changes one of the tools, its cached code is written again by itself.

Read more: the GNU Bash manual — *Quoting*, *Pattern Matching* (globs), *Pipelines*, *Redirections*, *The Shopt Builtin*; its *Command Line Editing* chapter is readline (`man bash` holds the same text). fzf's keys and search syntax: github.com/junegunn/fzf; zoxide: github.com/ajeetdsouza/zoxide.

What you learnt

- Every process has three streams: input (0), output (1) and errors (2); `>`, `2>` and the pipe steer them.
- Only an exported variable reaches the programs you start; `PATH` decides where a command is found.
- Globs match file names before the command runs; regular expressions match text; quote both, and every `"$variable"`.
- A script is a text file with a shebang and `x` permission; `bash -n` and `shellcheck` check it without running it.
- A leading space keeps a command out of the history; keys belong in `secrets.sh`, mode 600.

Check yourself

1. What does `2> /dev/null` do? → *it throws away the error messages, stream 2*
2. Why did `space ~/practice/'old notes'` fail before the fix? → *the bare `$1` was split at the space into two words*
3. Why does a variable set without `export` not reach a program you start? → *only exported variables are copied into a new process's environment*

26. Processes and signals

> **In this chapter**
> - say how a program starts, and read the family tree of running programs
> - pause, resume and stop a program with signals and the shell's jobs
> - give a long job a lower priority
> - find and end what uses the CPU or memory, in btop
>
> **You need:** ch. 25 (the shell as a language).

A program on disk is a file; a program running is a *process*. This chapter follows a process from its start to its end: how it is made, how you talk to it with signals, how the shell keeps several at once, and how to watch them all.

26.1 How a program starts: fork and exec

> **Word — PID.** *Process ID*: the number the kernel gives a program when it starts, unique while it runs. Like a ticket at a service counter: you are called by number, and the number is handed out again only after you have gone.

Try this. Your shell is a process too; ask for its PID.

```
$ echo $$
2310
```

(for example) `$$` is the shell's own PID, different in every terminal window. A new process is always made by an existing one, in two steps:

1. **fork** — the process makes an exact copy of itself, the *child*, with a new number, the **PID**. The child records who made it: its parent's number, the **PPID**.
2. **exec** — the child replaces its own program with another one, keeping its PID, its open files and its environment.

When you type `ls` in bash, bash forks, the child execs `eza`, and bash waits for it to end. `start-dwl` does the same to run dwl: it stays behind as dwl's parent, so it can show the rescue screen if dwl ends at once. `kawa-greeter` instead ends with `exec agreety …`: no fork, the shell *becomes* agreety. Every process descends this way from process 1:

```
$ ps -o pid,ppid,user,args -p 1,$$,$PPID
    PID    PPID USER     COMMAND
      1       0 root     /usr/lib/systemd/systemd rhgb --switched-root --system --deserialize=…
   1352    1349 you      foot --server
   2310    1352 you      bash
```

(for example: `$$` is this shell's PID, `$PPID` its parent's; the order is by PID)

> **In SQL terms.** `ps -o pid,ppid,user,args` is a `SELECT` of four columns from a table of processes; `-p 1,$$,$PPID` is the `WHERE pid IN (…)`. PPID is a foreign key into the same table, so the process tree is a self-join: each row points at its parent's row.

Exit status. A process ends with a number for its parent: 0 is success, anything else a failure (§11.2). A process killed by a signal reports 128 plus the signal's number:

```
$ sleep 60          # press Ctrl+C
^C
$ echo $?
130
```

130 = 128 + 2, and 2 is SIGINT.

26.2 Signals

> **Word — signal.** A short numbered message the kernel delivers to a process: stop, pause, carry on, read your settings again. Like a tap on the shoulder: the program may answer in its own way, except to the two it cannot ignore, SIGKILL and SIGSTOP.

You have sent one already: **Ctrl+C** above was SIGINT. A program may *catch* most signals and decide what to do; if it does not, each signal has a default action, which for most is to end the process.

No.	Name	Default	Sent by, or used for
1	`SIGHUP`	end	the terminal closed; many services treat it as *re-read your settings*
2	`SIGINT`	end	**Ctrl+C** in a terminal
9	`SIGKILL`	end, at once	the last resort: it cannot be caught, so the program cannot tidy up
10	`SIGUSR1`	end	free for programs: waybar shows or hides the bar; foot's server switches to the dark palette
12	`SIGUSR2`	end	waybar and btop reload their settings; foot's server switches to the light palette
15	`SIGTERM`	end	the polite stop: `kill` sends it by default, and so does systemd to stop a service
18	`SIGCONT`	continue	`fg` and `bg`
19	`SIGSTOP`	pause	pauses; cannot be caught
20	`SIGTSTP`	pause	**Ctrl+Z** in a terminal
34+n	`SIGRTMIN+n`	end	real-time signals, free for programs: waybar's custom modules (`"signal": 8` means `SIGRTMIN+8`)

(the numbers are those of x86-64; `kill -l` lists them all)

This desktop uses signals in plain sight. Switching the theme (§9.1) amounts to these, among other things (the script sends each one only after the check below):

```
pkill -USR2 waybar      # the bar reads its files again
pkill -USR2 btop        # so does btop
pkill -USR1 foot        # foot: every window to the dark palette (USR2: light)
```

and `kawa-dwlstatus` updates one part of the bar by sending `SIGRTMIN+8` (workspaces), `+9` (layout), `+10` (title) or `+11` (keyboard layout) to waybar, which re-runs only the module with that `"signal"`; others poke through it — quick settings and the screen-share portal `+12` (the mode seal), `kawa-drives` `+13` (the drive mark). Because the default action of these signals is to *end* the process, both scripts first check that the program has installed its handler. The kernel publishes that in a bit mask:

```
grep SigCgt /proc/"$(pgrep -nx waybar)"/status    # SigCgt: the signals this process catches
```

Signal *n* is bit *n* - *1* of that hexadecimal number: `kawa-theme` tests bit 11 for `SIGUSR2` (12), and `kawa-dwlstatus` bit 33 + *n* for `SIGRTMIN+n`.

Read more: signal(7).

26.3 Jobs: the shell's own processes

> **Word — job.** A command the shell started and still keeps track of, numbered from 1 in each terminal: `%1` is the first. It can run in front, where it has the keyboard, or behind, in the background.

`command &` starts a program in the background, and the prompt comes back at once. **Ctrl+Z** pauses the one in the foreground (`SIGTSTP`); `bg` lets it carry on in the background, `fg` brings it back to the front, `jobs` lists them, and **Ctrl+C** ends the one in front:

```
$ sleep 300
^Z
[1]+  Stopped                 sleep 300
$ bg
[1]+ sleep 300 &
$ jobs
[1]+  Running                 sleep 300 &
```

Background jobs end with the terminal. Something that must outlive it belongs in a systemd unit (ch. 28), or at least in `systemd-run --user COMMAND`, which starts it as a unit of your user manager.

26.4 Priority: nice

> **Word — nice value.** A process's place in the queue for the CPU, from −20 (served first) to 19 (served last). The name says it: a *nicer* program lets the others go first.

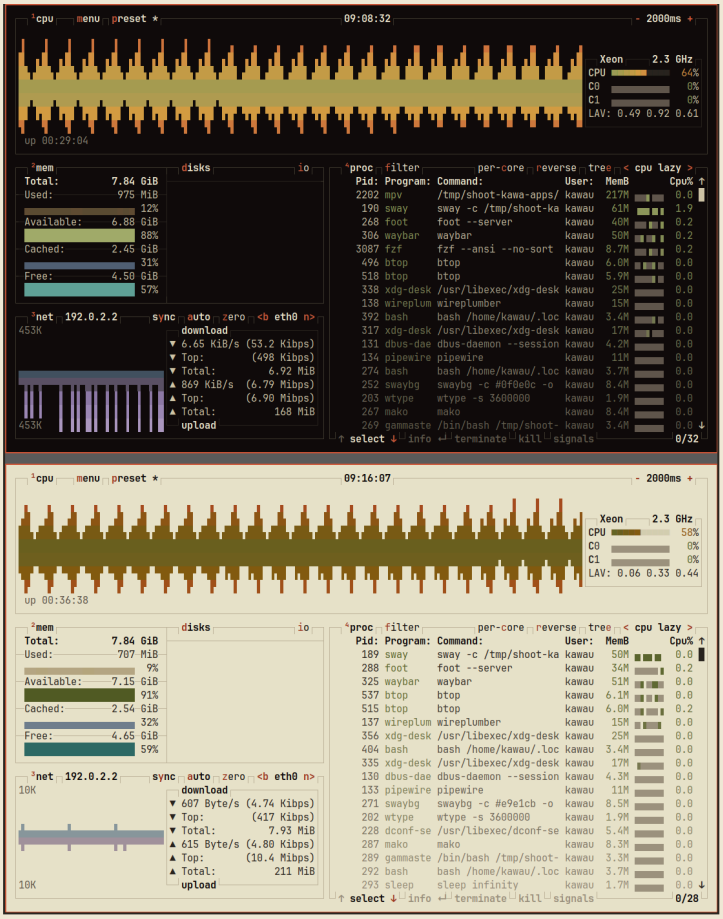
You may raise your own programs' niceness; lowering it needs root:

```
nice -n 19 tar -cf big.tar ~/Videos      # start at the lowest priority
renice -n 10 -p 2310                     # lower the priority of process 2310
ps -o pid,ni,args -C btop                # the NI column
```

The monthly scrub (§38.2) is an example: its unit runs `btrfs scrub start -B -c 3 /` with `Nice=19`. Nice orders the CPU; `-c 3` puts its disk reads in the *idle* class, served only when nothing else wants the disk.

26.5 Watching processes: btop

`ps` prints one picture of the processes; btop keeps the picture moving, with graphs of the CPU, memory and network above the process list.



btop: CPU, memory, network and the process list, in the palette.

Click the **CPU** part of the bar: btop opens in a floating window. Or type `btop`.

Keys	Does
↑ ↓ · Enter	choose a process · details of it
f or /	filter the process list (start with `!` for a regular expression); Delete clears it
← → · r	sort by the previous / next column · reverse
e	process tree
t · k · s	end the chosen process (SIGTERM: a request to stop, so it can save and tidy up) · kill it (SIGKILL: stopped at once, nothing saved) · send another signal
1 2 3 4 · 5	show / hide CPU, memory, network, processes · the GPU box
d	disks in the memory box
p · m · o	cycle layout presets · menu · options
q	quit

> **On Windows.** btop is this desktop's Task Manager: the graphs, the process list, and *End task* — here **t**, the polite SIGTERM. For a program that does not answer, **k** sends SIGKILL, as Task Manager does when it ends a process by force.

Pitfalls. Changes made in btop's own options menu last until you close it: the config says `save_config_on_exit = False`, so `~/.config/btop/btop.conf` stays exactly as the repository has it. Make lasting changes in your working copy (`configs/home/.config/btop/btop.conf`) and apply (chapter 42). On AMD, btop reads the card through the ROCm SMI library, which is not installed: it finds no GPU, so **5** shows no GPU box at all. LACT (§21.2) shows the card instead.

Read more: btop's README; **F1** (or **?**) inside btop lists every key.

26.6 Practice: start, pause and stop a process

Five tasks with `sleep`, a program that only waits. Do them in one terminal, in order.

1. Start a job in the background, and find it two ways.

```
$ sleep 300 &
[1] 4211
$ jobs
[1]+  Running                 sleep 300 &
$ pgrep -a sleep
4211 sleep 300
```

(for example) `[1]` is the job number, 4211 the PID; the prompt's header shows *jobs 1* too.

If `pgrep` lists more lines: another program is sleeping as well; yours is `sleep 300`.

2. Bring it to the front, pause it, send it back.

```
$ fg
sleep 300
^Z
[1]+  Stopped                 sleep 300
$ bg
[1]+ sleep 300 &
```

After `fg` the prompt does not come back until you press **Ctrl+Z** (SIGTSTP); `bg` sends SIGCONT.

If you see `bash: fg: current: no such job`: five minutes have passed and the sleep has ended; start it again.

3. Stop it politely, by job number.

```
$ kill %1
$ jobs
[1]+  Terminated              sleep 300
```

`kill` sends SIGTERM unless told otherwise; `%1` means *job 1 of this shell.*

If you see `bash: kill: %1: no such job`: the job had already ended.

4. Stop another by its PID.

```
$ sleep 200 &
[1] 4230
$ kill -TERM 4230
$ jobs
[1]+  Terminated              sleep 200
```

(for example) A PID works from any terminal, not only the one that started the program.

If you see `bash: kill: (4230) - No such process`: use the number bash printed, not the one in this book.

5. End one with Ctrl+C, and read the exit status.

```
$ sleep 60
^C
$ echo $?
130
```

130 is 128 + 2: ended by signal 2, SIGINT.

If you see `0`: another command ran between; `$?` holds only the last one's status.

What you learnt

- A process is made by fork (a copy) and exec (a new program); every process has a PID and a parent, back to process 1.
- A signal is a numbered message: Ctrl+C is SIGINT, Ctrl+Z SIGTSTP, `kill` sends SIGTERM, and SIGKILL cannot be refused.
- `&`, `jobs`, `fg` and `bg` juggle the shell's own processes; they end with the terminal.
- `nice` and `renice` set a process's place in the CPU queue; btop watches everything and ends what misbehaves.

Check yourself

1. What does `exec` do to the process that calls it? → *it replaces its program, keeping its PID, open files and environment*
2. Why try SIGTERM before SIGKILL? → *SIGTERM lets the program save and tidy up; SIGKILL stops it at once*
3. A command ended with status 130. What happened? → *it was ended by signal 2, SIGINT — Ctrl+C*

27. Packages: rpm and dnf

> **In this chapter**
> - read a package's name, version and contents, and find who owns any file
> - follow dnf's five steps, from fetching the lists to the finished install
> - know where every package comes from, and how it is proved genuine
> - follow one program down through its packages to the hardware
>
> **You need:** ch. 24 (users and permissions).

Every program on this machine, the kernel included, arrived as a package, and one tool installs and updates them all. This chapter opens a package, follows dnf through an install, and then traces how the pieces of this desktop depend on each other.

> **On Windows.** An RPM is like an `.msi` installer that also lists what it needs. dnf is the Microsoft Store, winget and Windows Update in one: it installs and updates everything, including the system itself, from a few signed sources — never from a download on a web page.

Its keeping-lean side, and how to add a package the manifest way, are in §38.1 and §42.11.

27.1 What an RPM holds

A package is one file, `NAME-VERSION-RELEASE.ARCH.rpm`.

Try this. Ask which mpv is installed.

```
$ rpm -q mpv
mpv-0.41.0-5.fc44.x86_64
```

The name, version, release and architecture together (the **NEVRA**, with an *epoch* in front when one is set) identify it exactly: `mpv-0.41.0-5.fc44.x86_64` is mpv 0.41.0, Fedora's fifth build of it, for Fedora 44, for 64-bit PCs.

Part	Holds	See it with
header (metadata)	name, version, summary, licence, what it **provides** and what it **requires**	`rpm -qi mpv` · `rpm -q --provides mpv` · `rpm -q --requires mpv`
payload	the files, in a compressed archive	`rpm -ql mpv`
scriptlets	small shell scripts run before or after installing or removing it: enabling a service, adding a user	`rpm -q --scripts greetd`
signatures	the packager's OpenPGP signature over the header and payload	`rpmkeys --checksig FILE.rpm`

`rpm` itself works on single packages and the database of what is installed (`/usr/lib/sysimage/rpm`); it does not download, and it does not solve dependencies. That is dnf's job. Asking rpm is always safe:

Question	Command	Answer, for example
which package owns this file?	`rpm -qf /usr/bin/foot`	`foot-…fc44.x86_64`
which files are in this package?	`rpm -ql greetd`	`/usr/bin/agreety`, `/usr/bin/greetd`, `/usr/lib/systemd/system/greetd.service`, …
what is it, and from whom?	`rpm -qi ffmpeg-libs`	`Name : ffmpeg-libs` … `Vendor : RPM Fusion` …
have its files changed since?	`rpm -V greetd`	nothing when they are as packaged; a line per changed file otherwise
which keys does rpm trust?	`rpmkeys --list`	one line per key: its fingerprint and its name — Fedora's, RPM Fusion's, Brave's, Claude Code's, each COPR's

Fedora 43 brought **RPM 6.0**, which identifies keys by their full fingerprint and makes `rpmkeys` the tool for them. From Fedora 45 rpm also refuses, by default, any package whose signature it cannot check (ch. 34).

27.2 dnf5

dnf (DNF 5 since Fedora 41) works with *repositories* (§5.3): servers holding packages and a **metadata** index of them — every package's name, version, provides and requires. Like a shop's catalogue with the stockroom behind it: dnf reads the catalogue first, then fetches only what it needs.

dnf fetches the index over HTTPS — for Fedora's own repositories it also checks it against checksums from Fedora's mirror list — and checks every package against its signature.

> **Word — dependency.** Something a package *requires* to work: another package, a library, a command. dnf installs dependencies by itself, the way a recipe's ingredients come with it.

Every install or update goes the same way:

1. **refresh** — download each enabled repository's metadata (`--refresh` forces it; this desktop fetches it only when you update, §36.4).
2. **solve** — work out a set of packages that satisfies every *requires*, with nothing in conflict. This is why `dnf install mpv` also brings mpv's libraries.
3. **show and ask** — the list of what will be installed, upgraded and removed.
4. **download and check** — each package's signature against the keys of its repository.
5. **transaction** — install everything as one unit, run the scriptlets, record it.

Around step 5 this desktop adds a snapper *pre* and *post* snapshot (§37.2), so a transaction can be undone from the disk as well. dnf keeps its own record:

```
$ dnf history list | head -n 3
ID Command line            Date and time       Action(s) Altered
58 dnf upgrade --refresh   2026-10-15 08:52:11               95
57 dnf install hyperfine   2026-10-02 19:40:03                1
```

(for example)

```
dnf history info 57          # what transaction 57 changed, package by package
sudo dnf history undo 57     # the reverse transaction: remove what it installed, and so on
```

27.3 DNF's settings on this desktop

`/etc/dnf/libdnf5.conf.d/90-fedokawaii.conf` holds only what differs from Fedora's defaults — its six settings, each with what it does and DNF's own default:

```
[main]
# ten downloads at once, from DNF_PARALLEL (default 3; 20 at most)
max_parallel_downloads=10
# use the mirror that connects fastest (default off)
fastestmirror=True
# a mirror sending 50 kB/s or less …
minrate=50k
# … for 20 seconds is dropped (defaults: 1000 bytes/s, 30 s)
timeout=20
# keep two kernels (default 3)
installonly_limit=2
# Enter means yes at dnf's question (default no)
defaultyes=True
```

Setup and `verify` check all six with `dnf --dump-main-config`, which prints every setting as DNF sees it once all its files are read:

```
dnf --dump-main-config |
  grep -E '^(defaultyes|fastestmirror|installonly_limit|max_parallel_downloads|minrate|timeout) '
```

```
defaultyes = 1
fastestmirror = 1
installonly_limit = 2
max_parallel_downloads = 10
minrate = 51200
timeout = 20
```

(`minrate` is shown in bytes: 50k = 51 200.) *Read more:* dnf5.conf(5).

27.4 Weak dependencies, install reasons and groups

Not every dependency is a must, and dnf remembers why each package is there:

Idea	Means	See it
Requires	must be there, or the package is not installed	`dnf repoquery --requires mpv`
Recommends (a weak dependency)	installed by default too, but may be left out or removed	`dnf repoquery --recommends tree-sitter-cli` → `nodejs` — the reason setup installs it with `--exclude='nodejs22*'` : this desktop has Node 24 under its own name, and keeps one Node.js
install reason	why a package is here: *User* (asked for by name), *Dependency*, *Weak Dependency*, *Group*	`dnf repoquery --installed --qf '%{name} %{reason}\n' mpv ffmpeg-libs` → `mpv User` and `ffmpeg-libs Dependency`
group (comps)	a named set of packages Fedora defines; the installer's *Minimal Install* is the `core` group	`dnf group list --installed`

The install reason is how `dnf autoremove` knows what may go: dependencies that nothing user-installed needs any more. Setup marks every manifest package *User*, so the clean-up can never take one (§38.1). Packages a group brought in carry the reason *Group*: a new release may add some (kmscon arrives with Fedora 45's `core` group), and `verify` counts them as Fedora's.

27.5 Keys and gpgcheck

> **Word — signature.** A short block of data made from a file with the packager's secret key. Anyone with the matching public key can check that the file is exactly what the packager signed. Like a wax seal: quick to check, impossible to copy.

Every repository file in `/etc/yum.repos.d/` names the key its packages must be signed with, and turns the check on:

```
# /etc/yum.repos.d/claude-code.repo, as setup writes it
[claude-code]
name=Claude Code
baseurl=https://downloads.claude.ai/claude-code/rpm/stable
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-claude-code
includepkgs=claude-code
```

`gpgcheck` (DNF 5's preferred name is `pkg_gpgcheck`) makes dnf refuse a package whose signature does not match. `includepkgs` limits what this repository may install to the one package. Setup checks the key's fingerprint against the one in `lib/manifest.sh` before trusting it; Fedora's own keys come with Fedora, in `/etc/pki/rpm-gpg/`.

Read more: Enforcing signature checking by default.

27.6 The repositories this desktop uses

```
dnf repolist          # the enabled ones, by id and name
```

Repository id	Holds	Why it is here
`fedora` · `updates`	the release as it shipped · every fix since	nearly everything
`fedora-cisco-openh264`	Cisco's H.264 codec, built and shipped by Cisco for licensing reasons	enabled by Fedora by default
`rpmfusion-free` (+ `-updates`)	free software Fedora leaves out, mostly for patent reasons	the full **ffmpeg**, Mesa's **freeworld** VA-API and Vulkan drivers (H.264 and HEVC), `libheif-freeworld`
`rpmfusion-nonfree` (+ `-updates`)	software that is free to use but not open source	**Steam**; Intel's full VA-API driver
`brave-browser`	Brave's own, signed	`brave-origin`
`claude-code`	Anthropic's own, signed, fingerprint checked	`claude-code`
`copr:copr.fedorainfracloud.org:ilyaz:LACT`	a COPR (§5.4)	LACT, the AMD GPU control
`copr:copr.fedorainfracloud.org:dejan:lazygit`	a COPR	lazygit

27.7 Pinned tools, and why

The few tools Fedora does not package come from their official releases, each **pinned**: an exact version and the SHA-256 of its file. The desktop's configs are written for exactly these — yazi's layout code, the fonts' pixel metrics every alignment is measured in — so an update never moves them, and a download that differs by one byte is refused.

Tool	Version	Held because
yazi	26.9.1	its theme, and the file dialog's layout code, use its internals
satty · presenterm · spotify_player	0.22.0 · 0.16.1 · 0.25.1	their configs are written for these
JetBrains Mono and Departure Mono (Nerd Fonts)	3.5.1	every alignment is measured in their pixels
PixelMplus 12	2013	its only release
torrra	2.5.1	its key wrapper replaces three of its methods; libtorrent under it moves to the newest
GE-Proton	the newest; two kept	games are what it is for, and it changes nothing else

dwl is built from a pinned commit (`DWL_COMMIT`), and so is the file dialog's portal (`TFC_COMMIT`).

A *SHA-256* is the kind of checksum you compared on Windows before writing the installer stick (§3.3): a 64-character fingerprint of a file's exact bytes — change one byte and it changes completely. The pins live in `lib/upstream.sh`; the version in place is recorded in your home:

```
cat ~/.local/share/fedokawaii/upstream/yazi     # → v26.9.1
```

and a download of that release must print exactly this:

```
$ sha256sum yazi-x86_64-unknown-linux-musl.zip
9b9c39decccf8cb0ff53a7d637d38f8a79d93bbd0099f4ea9c619ef6bb392f5d  yazi-x86_64-unknown-linux-musl.zip
```

27.8 A worked chain: a video, from mpv down to the GPU

What happens, in packages, when mpv plays a video with hardware decoding (`hwdec=vaapi` in `mpv.conf`)? Every link below can be checked on this machine. AMD is shown; §31.2 has the Intel equivalents.

```
mpv                                 the player                          Fedora
 └─ libavcodec.so.62 (ffmpeg-libs)  decoding, the full build            RPM Fusion free
     └─ libva.so.2 (libva)          the VA-API library                  Fedora
         └─ radeonsi_drv_video.so   the VA-API driver, freeworld        RPM Fusion free
             └─ /dev/dri/renderD128 the kernel's DRM interface
                 └─ amdgpu.ko       the kernel driver (kernel-modules)  Fedora
                     └─ amdgpu/*.bin.xz  the GPU's firmware (amd-gpu-firmware)  Fedora
```

1. The program, and its package.

```
$ command -v mpv
/usr/bin/mpv
$ rpm -qf /usr/bin/mpv
mpv-0.41.0-5.fc44.x86_64
```

2. What it needs. mpv needs ffmpeg's decoding library, among many others. A library dependency is written as the library's *soname*, not a package name:

> **Word — soname.** The name a shared library is known by, with its major version: `libva.so.2`. Programs ask for this name, not for a package, so any package that provides it will do. The nearest thing on Windows is a DLL's name.

```
$ rpm -q --requires mpv | grep -E '^lib(avcodec|va)\.'
libavcodec.so.62()(64bit)
libva.so.2()(64bit)
```

(for example, shortened)

3. Who provides it. Fedora's own build of that library is `libavcodec-free`, without the patent-encumbered codecs. Setup swapped it for RPM Fusion's full one (`dnf swap ffmpeg-free ffmpeg`), which provides the same soname, so mpv uses it without being rebuilt:

```
$ rpm -q --whatprovides 'libavcodec.so.62()(64bit)'
ffmpeg-libs-8.1.1-1.fc44.x86_64
$ dnf repoquery --installed --qf '%{name}  %{from_repo}\n' mpv ffmpeg-libs
ffmpeg-libs  rpmfusion-free-updates
mpv  updates
```

(for example)

4. Down to VA-API. ffmpeg hands decoding to the GPU through **libva**:

```
rpm -q --requires ffmpeg-libs | grep '^libva'    # libva.so.2, libva-drm.so.2, …
rpm -qf /usr/lib64/libva.so.2                     # libva-…fc44.x86_64
```

5. The driver: a runtime link, not a dependency. libva loads its driver *when it runs*, by name, from the kernel driver in use: amdgpu means `radeonsi`. No package requires the driver, so `--requires` cannot show this link; `vainfo` can:

```
$ vainfo --display drm 2>&1 | head -n 6
libva info: VA-API version 1.23.0
libva info: Trying to open /usr/lib64/dri-freeworld/radeonsi_drv_video.so
libva info: Found init function __vaDriverInit_1_23
libva info: va_openDriver() returns 0
vainfo: VA-API version: 1.23 (libva 2.23.0)
vainfo: Driver version: Mesa Gallium driver 26.0.6 for AMD Radeon RX 6700 XT (radeonsi, navi22, …)
$ rpm -qf /usr/lib64/dri-freeworld/radeonsi_drv_video.so
mesa-va-drivers-freeworld-26.0.6-1.fc44.x86_64
```

(for example) Since Fedora 44, Fedora's own VA-API drivers sit inside `mesa-dri-drivers`, in `/usr/lib64/dri/`, with H.264 and HEVC switched off. RPM Fusion's freeworld build installs beside them into `/usr/lib64/dri-freeworld/`, a folder Fedora's libva provides for exactly this, and libva opens it first. Further down `vainfo` lists `VAProfileH264High`, `VAProfileHEVCMain` and, on newer cards, `VAProfileAV1Profile0`: what `verify` checks.

6. The kernel's side. The driver talks to the kernel through a *render node* — a GPU device file any program may open (§31.1) — and the kernel's `amdgpu` driver does the rest:

```
$ lspci -k -d ::0300
03:00.0 VGA compatible controller: Advanced Micro Devices, Inc. [AMD/ATI] Navi 22 [Radeon RX 67…
    Subsystem: …
    Kernel driver in use: amdgpu
    Kernel modules: amdgpu
$ rpm -q --whatprovides 'kmod(amdgpu.ko)'
kernel-modules-7.2.7-200.fc44.x86_64
kernel-modules-7.2.8-200.fc44.x86_64
```

(for example, shortened: the full name is Navi 22 [Radeon RX 6700/6700 XT/6750 XT / 6800M/6850M XT] (rev c1)*; `-d ::0300` picks VGA controllers; one line per kernel kept)*

7. The firmware. The GPU runs small programs of its own, which the driver loads from files at start-up:

```
$ rpm -qf /usr/lib/firmware/amdgpu/aldebaran_mec.bin.xz
amd-gpu-firmware-20260916-1.fc44.noarch
```

(`aldebaran_*` is the first file in the folder; each GPU loads its own, named after its chip.) The package is part of Fedora's `linux-firmware` and is updated with it.

The lesson of the chain: three kinds of link join the pieces — *package dependencies* (`--requires`, solved by dnf), *runtime loading by name* (libva and its drivers, which no dependency records), and *the kernel's interfaces* (device files and firmware files). Only the first is visible to dnf, which is why setup installs the drivers explicitly and `verify` tests the result with `vainfo`.

27.9 A second chain: the music in the bar

The bar's middle shows what plays. Here the links are a config line, a script and a D-Bus name — no package requires another:

```
waybar                          the bar
 └─ "custom/nowplaying"         a module in ~/.config/waybar/config.jsonc: "exec": "kawa-nowplaying"
     └─ kawa-nowplaying         a bash script: prints one line of JSON per change
         └─ playerctl --follow  watches every MPRIS player
             └─ D-Bus, your session bus     the names org.mpris.MediaPlayer2.*
                 └─ mpv-mpris   a plugin inside mpv that registers org.mpris.MediaPlayer2.mpv
```

Link	Joined by	Check it
waybar → script	`"exec": "kawa-nowplaying", "return-type": "json"`: waybar runs it and reads each line	`pgrep -af kawa-nowplaying`
script → playerctl	it runs `playerctl --follow metadata --format …` and reads its lines	`playerctl metadata --format '{{status}} {{artist}} — {{title}}'`
playerctl → players	D-Bus: every name starting with `org.mpris.MediaPlayer2.` (§32.1)	`playerctl -l` → `mpv`
mpv → D-Bus	`mpv-mpris`, a plugin mpv loads from `/etc/mpv/scripts/` at start	`rpm -qf /etc/mpv/scripts/mpris.so` → `mpv-mpris-…`
a click → the player	`"on-click": "kawa-nowplaying play-pause"`: a D-Bus call to the player the bar shows	the button in the bar

The script prints lines such as `{"text":"… artist — title …","class":"playing"}`; waybar puts the text in the bar and the class in its stylesheet. Spotify's player and Brave join the same way, because they register MPRIS names too.

27.10 The manifest, setup and verify

The whole desktop is one list, applied, then proved:

```
 lib/manifest.sh            every package, by job; what is forbidden; which services
      │
      ▼
 setup.sh · update · repair   dnf installs the lists, marks them User, enables the services
      │
      ▼
 verify.sh                  every listed package present · nothing forbidden · nothing
                            user-installed beyond the list · services running · the chains
                            above working (vainfo, mpv-mpris, the portals)
```

A package you want to keep goes into the manifest (§42.12); one installed by hand shows in `verify` as *installed outside the manifest* until it is removed or listed.

Why no Flatpak or Snap. Both are second package systems, with their own runtimes, updates and sandbox. Here they would mean a second place for software to live, out of dnf's view: `verify` could no longer prove the machine matches the list, and an update would be two updates. Everything this desktop needs is packaged for Fedora or pinned from its upstream release (§27.7), so both are forbidden in the manifest, as are PackageKit and GNOME Software. The desktop portals (§32.2), first made for Flatpak's sandbox, serve the native programs here as well.

Read more: rpm.org documentation · rpmkeys(8) · dnf5 repoquery · dnf5 history · dnf5.conf(5) · RPM 6.0 in Fedora.

27.11 Practice: questions to ask rpm and dnf

Five questions, each answered by one command that only reads. dnf may first fetch the package lists if its copy is old; that too only reads.

1. Whose file is this?

```
$ rpm -qf /usr/bin/foot
foot-…fc44.x86_64
$ rpm -qf ~/.bashrc
file /home/you/.bashrc is not owned by any package
```

Files under `/usr` come from packages; your home's files never do.

If you see `error: file /usr/bin/fot: No such file or directory`*:* a typing slip in the path; Tab completes it.

2. What is this package?

```
$ dnf info mpv
Updating and loading repositories:
Repositories loaded.
Installed packages
Name            : mpv
…
Version         : 0.41.0
Release         : 5.fc44
Architecture    : x86_64
…
From repository : updates
Summary         : Movie player playing most video formats and DVDs
…
```

(for example, shortened) `From repository` names where it came from: Fedora's `updates`.

If you see `Available packages` *below as well:* a newer mpv is waiting for the next update.

3. What does it need?

```
$ dnf repoquery --requires mpv
…
libavcodec.so.62()(64bit)
…
libva.so.2()(64bit)
…
```

(for example, shortened) One requirement per line, many of them: mostly sonames, the libraries mpv links to (§27.8), and a few package names.

> **In SQL terms.** `dnf repoquery` is a `SELECT` over the package database. `--installed` is a `WHERE`, `--qf '%{name} %{reason}\n'` the column list, and `| wc -l` a `COUNT(*)`: `dnf repoquery --installed --qf '%{name} %{reason}\n'` reads like `SELECT name, reason FROM packages WHERE installed`.

If you see `No match for argument`: the package name is misspelt; `dnf search mpv` finds it.

4. What changed, and when?

```
$ dnf history list | head
ID Command line             Date and time       Action(s) Altered
58 dnf upgrade --refresh    2026-10-15 08:52:11               95
57 dnf install hyperfine    2026-10-02 19:40:03                1
…
```

(for example) The newest first; `dnf history info 58` lists what transaction 58 changed.

If the list scrolls past: `head` keeps the first ten lines; `| less` shows them all.

5. How many packages did you, or setup, ask for by name?

```
$ dnf repoquery --userinstalled | wc -l
171
```

(for example) A little more than the manifest's list: the hardware's packages and the installer's own choices count too. Everything else came as a dependency or with a group.

If a long list scrolls by instead of one number: the `| wc -l` is missing; it turns the list into its count.

What you learnt

- A package is one file with a header (what it provides and requires), a payload, scriptlets and a signature; `rpm` asks about installed ones.
- dnf refreshes, solves, asks, downloads and checks, then installs in one transaction — here between two snapshots.
- Every repository is signed and checked; this desktop adds RPM Fusion, Brave, Claude Code and two COPRs to Fedora's own.
- Programs need sonames, not packages; drivers are loaded by name at run time, which dnf cannot see — `verify` tests them.
- `rpm -qf`, `dnf info`, `dnf repoquery` and `dnf history` answer *whose, what, needs what, when.*

Check yourself

1. What does `rpm -qf FILE` tell you? → *which package owns the file, or that none does*
2. Why can ffmpeg-libs replace libavcodec-free under mpv without a rebuild? → *it provides the same soname, `libavcodec.so.62`*
3. Why is `radeonsi_drv_video.so` installed explicitly, not as a dependency? → *libva loads it by name at run time; no package requires it*

28. Services, timers and logs: systemd

> **In this chapter**
> - name the kinds of unit, and where their files come from
> - read any service's state, file and log — the system's and your own
> - change a packaged service with a drop-in that survives updates
> - read a timer, and filter the journal down to what matters
>
> **You need:** ch. 26 (processes and signals).

Much of what runs on this machine was started by nobody at the keyboard: the network, the login, the sound server, the monthly disk check. One program starts, watches and stops all of them, and keeps their logs: systemd, process 1.

28.1 systemd and its units

systemd starts, watches, restarts and stops everything that runs without you starting it by hand.

> **Word — unit.** Anything systemd manages, described by one small text file whose ending names its kind: `.service`, `.timer`, `.mount`, `.target` … Like a line in Windows' Services list, but also for timers, disks and whole stages of the start-up.

Try this. Ask about one unit.

```
$ systemctl is-active bluetooth.service
active
```

One word answers: `active`, `inactive` or `failed`.

A **service** is a program to start and watch; the other kinds, with their examples here:

Type	Is	Examples on this machine
`.service`	a program to start and watch	`greetd.service`, `NetworkManager.service`, `tuned.service`; yours: `pipewire.service`, `prowlarr.service`
`.socket`	a socket that starts its service when the first client connects	yours: `pipewire.socket` — PipeWire starts when something first plays a sound
`.timer`	a time at which to start a service	`fedokawaii-scrub.timer`, `snapper-timeline.timer`, `fstrim.timer`; yours: `fedokawaii-remind.timer`, `grub-boot-success.timer`
`.target`	a named group of units, a point in the start-up	`graphical.target`, `multi-user.target`; yours: `dwl-session.target`, `graphical-session.target`
`.mount` · `.swap`	a mounted file system · a swap device	`home.mount`, `boot.mount`, `tmp.mount` · `dev-zram0.swap`
`.path`	a path to watch; a change starts its service	`systemd-ask-password-console.path`
`.slice` · `.scope`	groups of processes for resources (cgroups, below)	`user.slice` · `session-2.scope`

> **Word — target.** A unit that only groups other units and marks a point in the start-up, such as *the network is up* or *the graphical login is ready*. Like a milestone in a project plan: reached when everything it needs is done.

Units come from three places, the later overriding the earlier: packages (`/usr/lib/systemd/system/`), generated at boot (`/run/systemd/generator/`), and the administrator (`/etc/systemd/system/`, with drop-in folders such as greetd's, §28.3). `systemctl status greetd` shows a unit's state, processes and last log lines; `systemctl cat greetd` the file and its drop-ins (the rest is in appendix D).

Enable and *start* are different: `start` runs a unit now, `enable` makes a link so it starts at every boot, from the `[Install]` section of its file (`enable --now` does both). A unit without one (Prowlarr's) runs only when asked.

> **On Windows.** `systemctl` is `services.msc` from a terminal. *Startup type: Automatic* is `enable`; the *Start* and *Stop* buttons are `start` and `stop`; *Status: Running* is `active (running)`. Task Scheduler's job is done by timers, below.

28.2 The system manager and yours

There are two kinds of systemd: the **system manager** (process 1, root's) and one **user manager** per user (`user@1000.service`), which runs your own units as you. Add `--user` to talk to yours — no `sudo`:

```
systemctl --user status pipewire.service wireplumber.service
systemctl --user list-timers              # fedokawaii-remind.timer, grub-boot-success.timer
journalctl --user -u prowlarr -n 20       # your units' logs
```

Your units live in `~/.config/systemd/user/` (this desktop's: `dwl-session.target` and the reminder's timer and service) and in `/usr/lib/systemd/user/` (the packages'). Normally a user manager starts with your first login and stops after your last logout. Setup turns on **linger** for you (`loginctl enable-linger`): your manager then starts at boot and stays, so `update`, `repair` and `verify` can reach Prowlarr and the portals even from a text console. `ls /var/lib/systemd/linger/` lists the users who have it.

28.3 Drop-in files: changing a unit without editing it

> **Word — drop-in.** A small file in a `NAME.d/` folder beside a config file or unit. It is read after the main file and overrides only the lines it sets — like a sticky note on a printed form, which leaves the form itself untouched.

This desktop puts everything system-wide into a drop-in folder (`*.d/`) or under `/usr/local` (appendix B lists each place, §B.3), never into a file Fedora's packages own — so a Fedora update never fights its settings, and removing one of them removes it cleanly. A drop-in folder sits beside the file it changes; each `.conf` file in it is read after the main file and overrides only what it sets. greetd's is a small example (shortened: the real file also keeps tty1 for greetd alone):

```
# /etc/systemd/system/greetd.service.d/fedokawaii.conf
[Service]
# an empty ExecStart= first clears the command in Fedora's greetd.service
ExecStart=
# then the one used instead: greetd with Fedokawaii's own config file
ExecStart=/usr/bin/greetd --config /etc/greetd/fedokawaii.toml
```

Command	Shows
`systemctl cat greetd.service`	Fedora's unit, then each drop-in, each under a `# /path` line
`rpm -qf /etc/greetd/fedokawaii.toml`	`file /etc/greetd/fedokawaii.toml is not owned by any package`: Fedokawaii's own
`rpm -qf /etc/greetd/config.toml`	`greetd-…`: Fedora's file, left as the package made it

Had a script edited a packaged config file instead, an update that brings a new version of that file would leave it beside yours as `.rpmnew`, to merge by hand; after a move to a new Fedora the menu lists any there are (the record `rpmnew` in `/var/lib/fedokawaii/`, §B.3).

Read more: systemd.unit(5), drop-in directories.

28.4 Timers, not cron

A timer is a unit that starts a service on a schedule. The monthly scrub, as setup writes it (its comment lines and `[Unit]` left out):

```
# /etc/systemd/system/fedokawaii-scrub.timer
[Timer]
OnCalendar=monthly
RandomizedDelaySec=1d
Persistent=true

[Install]
WantedBy=timers.target
```

Line	Means
`OnCalendar=monthly`	at midnight on the first of each month
`RandomizedDelaySec=1d`	plus a random delay of up to a day
`Persistent=true`	if the machine was off then, run at the next start

and it starts the service of the same name, `fedokawaii-scrub.service`. The reminder's timer counts from events instead: `OnActiveSec=2min` (two minutes after the session starts it) and `OnUnitActiveSec=6h` (six hours after each run). The older **cron** does the same job from a table of times. Fedora's minimal install brings no cron daemon and this desktop adds none: timers do the job better here, because each run is logged in the journal, a missed run catches up, and the job gets the same limits and security settings as any service. `systemctl list-timers fedokawaii-scrub.timer` shows when it runs next and when it last ran.

28.5 The journal: reading the logs

> **Word — journal.** The one log of the whole machine: every line any service, program or the kernel reports, each stamped with its time, its source and how serious it is. You read it, filtered, with `journalctl`.

systemd's **journald** collects the log lines of the kernel, every unit and every program that logs, with the time, the unit, the process and a priority, into binary files in `/var/log/journal`. `journalctl` reads them. Members of `wheel` may read the whole system journal without `sudo`. For example, the last line one of your own services wrote:

```
$ journalctl --user -u fedokawaii-remind -n 1
Oct 16 08:42:31 fedora systemd[1012]: Finished fedokawaii-remind.service - Fedokawaii — is any…
```

(for example, shortened) One line: the time, the machine, the program and its PID — here your user manager — and the message: the update reminder ran, two minutes after the session began. Filters narrow it down:

Filter	Shows	Example
`-b` · `-b -1`	this boot · the one before	`journalctl -b -1 -e` (the end of the last boot)
`-u UNIT`	one unit	`journalctl -u NetworkManager -b`
`--user -u UNIT`	one of your units	`journalctl --user -u wireplumber`

Filter	Shows	Example
`-k`	the kernel's messages	`journalctl -k -b -g amdgpu`
`-p PRIORITY`	that priority and worse: `err`, `warning`, `info`	`journalctl -b -p err`
`-t TAG`	one program's tag	`journalctl -t kawa-snap` (the DNF snapshots)
`--since` · `--until`	a time window	`journalctl --since "1 hour ago"`
`-g PATTERN`	lines matching a regular expression	`journalctl -b -g 'firmware'`
`-f`	follow new lines as they come; Ctrl+C stops	`journalctl -f -u greetd`
`-n N` · `-e`	the last N lines · jump to the end	`journalctl -n 50`

> **In SQL terms.** Each filter is one condition of a `WHERE`, joined by `AND`: `journalctl -b -u NetworkManager -p err --since today` reads like `SELECT * FROM journal WHERE boot = current AND unit = 'NetworkManager.service' AND priority <= 'err' AND time >= today`.

Several filters at once narrow further: `journalctl -b -p warning -u greetd` is this boot's warnings and errors from the login alone.

> **On Windows.** The journal is Event Viewer's *Windows Logs*, in a terminal: one list for the system and every program, each entry with its time, source and level. `-p err` is the *Error* level filter; `-u` picks the source.

28.6 cgroups

> **Word — cgroup.** *Control group*: a set of processes the kernel counts and limits together — CPU time, memory, disk and how many processes. Like a department's budget: every expense is booked to it, whoever in the department made it.

systemd puts every unit in one, which is how it knows exactly which processes belong to a service — a service that forks a hundred helpers is still stopped cleanly. Fedora uses **cgroups version 2**, one tree for everything (since Fedora 31). Any process can tell you where it sits:

```
$ cat /proc/self/cgroup
0::/user.slice/user-1000.slice/session-2.scope
```

The single `0::` line is the mark of version 2. This shell was started by foot's server, which the autostart started, which dwl started: all in session 2's scope. `systemd-cgls` draws the whole tree:

```
$ systemd-cgls --no-pager
…
├─user.slice
│ └─user-1000.slice
│   ├─user@1000.service
│   │ ├─session.slice
│   │ │ ├─pipewire.service
│   │ │ │ └─1402 /usr/bin/pipewire
│   │ │ └─wireplumber.service
│   │ │   └─1404 /usr/bin/wireplumber
│   │ └─…
│   └─session-2.scope
│     ├─1301 /bin/sh /usr/local/bin/start-dwl
│     ├─1310 /usr/local/bin/dwl -s /home/you/.config/dwl/autostart.sh
│     ├─1340 waybar
│     └─…
└─system.slice
  ├─greetd.service
  │ └─1102 /usr/bin/greetd --config /etc/greetd/fedokawaii.toml
  └─…
```

(for example, shortened) Your session's programs sit in `session-2.scope`; your user services under `user@1000.service`; system services in `system.slice`. `systemd-cgtop` shows the groups using the most CPU and memory, live (`q` quits).

Read more: systemd.unit(5) · systemd.timer(5) · journalctl(1) · loginctl(1).

28.7 Practice: look at a service

Five looks at services, all read-only. When an output fills the screen, **q** leaves it.

1. Read one service's state.

```
$ systemctl status bluetooth
● bluetooth.service - Bluetooth service
     Loaded: loaded (/usr/lib/systemd/system/bluetooth.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-16 08:40:13 +04; 41min ago
       Docs: man:bluetoothd(8)
   Main PID: 1088 (bluetoothd)
     Status: "Running"
…
     CGroup: /system.slice/bluetooth.service
             └─1088 /usr/libexec/bluetooth/bluetoothd
```

(for example, shortened; the lines vary a little between systemd versions) Loaded is the file and whether it is enabled; *Active* is what it is doing now; *CGroup* lists its processes.

If you see `Active: inactive (dead)` *and* `start condition unmet`*:* this computer has no Bluetooth adapter; the unit runs only when `/sys/class/bluetooth` exists.

2. Read a unit's file, drop-ins included.

```
$ systemctl cat greetd
# /usr/lib/systemd/system/greetd.service
[Unit]
…
# /etc/systemd/system/greetd.service.d/fedokawaii.conf
# installed by setup.sh: greetd reads fedokawaii.toml, and owns tty1 alone (Fedora 45 runs
# kmscon there, earlier releases agetty)
[Unit]
Conflicts=getty@tty1.service kmsconvt@tty1.service
After=getty@tty1.service kmsconvt@tty1.service

[Service]
ExecStart=
ExecStart=/usr/bin/greetd --config /etc/greetd/fedokawaii.toml
```

(shortened) Fedora's file first, then the drop-in, each under its own `# /path` line.

If you see `No files found for greet.service.`*:* a typing slip in the name; Tab completes unit names too.

3. Read the last lines a service wrote this boot.

```
$ journalctl -u NetworkManager -b -n 20
…
Oct 16 08:40:14 fedora NetworkManager[1050]: <info>  […] device (enp5s0): carrier: link connected
…
Oct 16 08:40:15 fedora NetworkManager[1050]: <info>  […] manager: NetworkManager state is now CON…
```

(for example, shortened) Time, machine, program and PID, then NetworkManager's own level (`<info>`) and timestamp, then the message.

If you see `-- No entries --`*:* the unit name is wrong; `systemctl list-units --type=service` lists the right ones.

4. See when every timer runs next.

```
$ systemctl list-timers
NEXT                         LEFT … UNIT                     ACTIVATES
Fri 2026-10-16 10:00:00 +04 38min … snapper-timeline.timer  snapper-timeline.service
Mon 2026-10-19 00:21:37 +04 2 days … fstrim.timer           fstrim.service
Sun 2026-11-01 13:42:07 +04 2 weeks … fedokawaii-scrub.timer fedokawaii-scrub.service
…
```

(for example, shortened: the LAST and PASSED columns are left out) The scrub's odd time is its random delay of up to a day.

If your own timers are missing: they belong to your user manager; `systemctl --user list-timers` shows them.

5. Read one of your own services.

```
$ systemctl --user status pipewire
● pipewire.service - PipeWire Multimedia Service
     Loaded: loaded (/usr/lib/systemd/user/pipewire.service; …)
TriggeredBy: ● pipewire.socket
     Active: active (running) since Fri 2026-10-16 08:40:31 +04; 41min ago
   Main PID: 1402 (pipewire)
…
```

(for example, shortened) TriggeredBy names the socket that started it, the first time a program played a sound.

If you see `Unit pipewire.service could not be found.`*:* the `--user` is missing; the system manager has no such unit.

What you learnt

- systemd manages units: services, sockets, timers, targets, mounts; files in `/usr/lib` come from packages, files in `/etc` from the administrator.
- `start` runs a unit now, `enable` at every boot; `--user` talks to your own systemd, which linger keeps running.
- A drop-in changes a packaged unit or config without editing it, so updates never fight it.
- Timers replace cron; the journal holds every log, and `journalctl`'s filters narrow it like a `WHERE`.
- Every unit lives in a cgroup, so systemd always knows which processes are whose.

Check yourself

1. What is the difference between `systemctl start` and `systemctl enable`? → *`start` runs it now; `enable` makes it start at every boot*
2. Why does this desktop change greetd with a drop-in instead of editing its unit? → *an update may replace the packaged file; the drop-in, its own file, stays*
3. Which command shows this boot's errors only? → *`journalctl -b -p err`*

29. How the machine starts

> **In this chapter**
> - follow the start-up from the power button to the desktop, stage by stage
> - read a boot entry and the kernel's arguments, and start the older kernel
> - know what the initramfs and early microcode are for
> - follow the login into your session and its autostart
>
> **You need:** ch. 28 (systemd) · ch. 23 (files and mounts).

Starting a computer is a relay: each stage finds, checks and starts the next, then steps back. This chapter runs the relay from the firmware to the bar, and shows where each stage leaves its trace, so that a stage that fails can be found.

#	Stage	What it does here	Where it lives
1	UEFI firmware	tests the hardware, reads its boot entry, loads shim from the EFI system partition	the motherboard; `/boot/efi`
2	shim	checks GRUB's signature (Secure Boot), then starts it	`/boot/efi/EFI/fedora/shimx64.efi`
3	GRUB2	shows the boot menu, loads the kernel and the initramfs, hands over the kernel arguments	`/boot/grub2/` , `/boot/loader/entries/`
4	the kernel	starts the CPU's cores, applies early microcode, starts its drivers, unpacks the initramfs	`/boot/vmlinuz-VERSION`
5	the initramfs	a small system in memory, made by dracut: finds the disk, mounts the Btrfs subvolume `root` , switches to it	`/boot/initramfs-VERSION.img`
6	systemd	process 1: mounts the rest, starts every service, reaches `graphical.target`	`/usr/lib/systemd/systemd`
7	greetd	the login manager on tty1, configured by `/etc/greetd/fedokawaii.toml`	`greetd.service`
8	agreety	greetd's plain greeter, started through `kawa-greeter` : user name, password	`/usr/local/bin/kawa-greeter`
9	your session	PAM checks the password; logind registers a session; `start-dwl` runs as you	`/usr/local/bin/start-dwl`
10	dwl	takes the screens and input, opens the Wayland socket, runs your autostart	`/usr/local/bin/dwl`
11	autostart	starts the bar, notifications, the terminal server, idle and lock, the polkit agent, automount, the clipboard history and the bar's status reader	`~/.config/dwl/autostart.sh`

Try this. Ask systemd how long the relay took, and where it ended.

```
$ systemd-analyze
Startup finished in 7.912s (firmware) + 2.017s (loader) + 1.104s (kernel) + 2.876s (initrd) + …
graphical.target reached after 3.389s in userspace.
$ systemctl get-default
graphical.target
```

(for example, shortened: the first line ends `+ 3.402s (userspace) = 17.312s`)

You typed	It means
`systemd-analyze`	the whole relay, timed: *firmware* is stage 1, *loader* stages 2–3, then the kernel, the initramfs (*initrd*) and everything systemd did
`systemctl get-default`	the target systemd starts towards: the graphical login

`systemd-analyze blame | head` lists the slowest units, and `systemd-analyze critical-chain graphical.target` the chain of units the target waited on.

29.1 Stages 1–2: UEFI, shim and Secure Boot

The firmware keeps a list of boot entries in its own memory (NVRAM); Fedora's points at **shim** on the *EFI system partition*, a small FAT partition mounted at `/boot/efi`:

```
$ sudo ls /boot/efi/EFI/fedora
BOOTX64.CSV  grub.cfg  grubx64.efi  mmx64.efi  shim.efi  shimx64.efi
```

(for example)

> **Word — Secure Boot.** A firmware setting that starts only boot programs signed with a key the firmware trusts. Like a guest list at a door: a changed boot loader or kernel is not on it, so it is not let in.

Firmware trusts Microsoft's key, so every distribution has a small first-stage loader, shim, signed by Microsoft. Shim carries Fedora's own certificate and uses it to check GRUB; GRUB and shim check the kernel the same way. A kernel started under Secure Boot also *locks itself down*: it refuses unsigned kernel modules, among other things. `check.sh` reads the firmware's own variable to say whether Secure Boot is on — a file, like nearly everything here (§23.1):

```
$ od -An -tu1 -j4 -N1 /sys/firmware/efi/efivars/SecureBoot-*
   1
```

`1` is on, `0` off. (The first four bytes of the file are the variable's attributes; `-j4` skips them and `-N1` reads the one byte of value.) Both work here: the drivers this desktop uses are part of Fedora's signed kernel.

> **On Windows.** Windows' own boot manager passes the same check, signed with a Microsoft key. That is why Fedora starts with Secure Boot left on, on a PC that came with Windows — the setting §3.6 said may stay as it is.

29.2 Stage 3: GRUB and the boot entries

GRUB reads one small file per installed kernel, in the **Boot Loader Specification** format, from `/boot/loader/entries/`. With two kernels kept (§29.5) there are two:

```
$ sudo ls /boot/loader/entries
3f9c…e1-7.2.7-200.fc44.x86_64.conf
3f9c…e1-7.2.8-200.fc44.x86_64.conf
```

The long prefix is the machine's ID (`/etc/machine-id`). One entry, shortened:

```
title Fedora Linux (7.2.8-200.fc44.x86_64) 44 (…)
version 7.2.8-200.fc44.x86_64
linux /vmlinuz-7.2.8-200.fc44.x86_64
initrd /initramfs-7.2.8-200.fc44.x86_64.img
options root=UUID=… ro rootflags=subvol=root rhgb quiet amdgpu.ppfeaturemask=0xfff7ffff …
```

`linux` and `initrd` are the kernel and the initramfs, as paths on the `/boot` partition; `options` holds the kernel's arguments (below; the line goes on with `vt.default_red=…`, the console's colours). Nobody edits these files by hand: `grubby` changes every entry at once, and also `/etc/default/grub`, so kernels installed later inherit the change — setup adds the AMD argument that way, and the theme the console colours:

```
sudo grubby --info=ALL | grep -E '^(kernel|args)='     # every entry's kernel and arguments
sudo grubby --update-kernel=ALL --args=quiet           # how an argument is added (for example)
```

The menu itself is hidden while the last boot succeeded (`menu_auto_hide` in GRUB's environment, `grub2-editenv - list`); a user timer, `grub-boot-success.timer`, marks a boot as successful two minutes after you log in. With the theme, the menu shows for `BOOT_MENU_SECONDS` instead (§29.3). On a machine that also starts another system — Windows beside Fedora — setup leaves the menu shown, as Fedora's installer does.

29.3 The boot menu



The boot menu at 1080p: sumi ink, a hem of seigaiha waves fading upwards in a dither, Departure Mono at twice its grid.

The **boot menu** is a GRUB theme in the same language, at your monitor's native resolution: plain sumi ink with a hem of seigaiha waves along the bottom edge, fading upwards in an ordered dither; the torii above a quiet box of entries, the chosen one marked by a vermilion bar, a thin vermilion line counting down. Its text is Departure Mono at two to four times its grid (22, 33 or 44 px, by the screen's height). It shows for `BOOT_MENU_SECONDS`; with `0` Fedora's hidden menu returns (it then appears only after a failed boot, or when you hold **Shift**, or press **Esc** or **F8**, as the machine starts). After changing monitors, apply the theme again (**Super+U** → *theme, applied again*) so it matches the new resolution.

29.4 Stage 4: the kernel and its arguments

A *kernel argument* is an option handed to the kernel as it starts (§5.3); the ones it was given are a file:

```
$ cat /proc/cmdline
BOOT_IMAGE=(hd0,gpt2)/vmlinuz-7.2.8-200.fc44.x86_64 root=UUID=… ro rootflags=subvol=root rhgb …
```

(for example, shortened: the line goes on with `quiet amdgpu.ppfeaturemask=0xfff7ffff vt.default_red=15,… vt.default_grn=… vt.default_blu=…`)

Argument	Means	Set by
`root=UUID=…`	the file system to start from, by its unique ID	the installer
`ro`	mount it read-only at first; systemd remounts it writable once it is checked	the installer
`rootflags=subvol=root`	on Btrfs, the subvolume called `root`	the installer
`rhgb quiet`	a graphical boot screen if one is installed; fewer kernel messages	the installer
`amdgpu.ppfeaturemask=0xfff7ffff`	amdgpu's own default plus OverDrive: the clock, voltage and fan controls LACT uses (all bits set would also switch on GFX DCS, which amdgpu keeps off as unstable)	setup (AMD only)
`vt.default_red=…` and its two partners	the text console's 16 colours	Theme (§6.1)

29.5 Two kernels, and the way back

DNF keeps two kernels: the newest and the one before it (`installonly_limit=2`, the DNF 5 minimum; DNF's default keeps three). Should a new one fail to boot or to start the desktop, choose the older one in the boot menu (if it is hidden, hold **Shift**, or press **Esc** or **F8**, as the machine starts); the next update replaces it.

```
rpm -q kernel-core    # the kernels installed: at most two lines
uname -r              # the one running now
```

29.6 Stage 5: the initramfs and early microcode

The kernel cannot mount the root file system before it has the drivers for the disk and for Btrfs, and those live *on* that file system.

> **Word — initramfs.** A small archive, loaded by GRUB beside the kernel, holding only enough of a system to find and mount the real one. Like the key to the house kept outside the house: the kernel needs it to get in.

dracut builds the initramfs for each kernel when the kernel is installed, from what is on this machine.

> **Word — microcode.** The processor maker's fixes to the processor itself, loaded into the CPU at every start. They are lost at power-off, so the system hands them over again each time.

At the front of the same file sits a tiny uncompressed archive with the CPU's microcode, so the kernel can apply it before anything else runs (`early_microcode`, on by default). Setup installs the microcode package for your CPU (`amd-ucode-firmware` or `microcode_ctl`, `lib/hardware.sh`) and has dracut rebuild the images.

```
$ sudo lsinitrd | grep microcode
drwxr-xr-x   2 root root    0 … kernel/x86/microcode
-rw-r--r--   1 root root  … … kernel/x86/microcode/AuthenticAMD.bin
$ journalctl -k -b -g microcode
… kernel: microcode: Current revision: 0x…
```

(for example; `GenuineIntel.bin` on an Intel CPU)

29.7 Stage 6: systemd

The kernel starts one program, process 1, and that is **systemd**. It reads its *units* — small text files describing services, mounts, timers and more (§28.1) — and starts what the *default target* needs, as much in parallel as the dependencies allow. A **target** is a named point in the start-up, a group of units:

Target	Means	Here
`multi-user.target`	the full system with text logins and network	Fedora Server's default
`graphical.target`	the same, plus a graphical login	this desktop's default: setup sets it

```
$ systemctl get-default
graphical.target
```

29.8 Stages 7–9: greetd, the login, the session

`greetd.service` is Fedora's unit with a drop-in of this desktop's (§28.3) that points it at its own file and gives it tty1 alone:

```
# /etc/greetd/fedokawaii.toml
[terminal]
vt = 1

[default_session]
command = "/usr/local/bin/kawa-greeter"
user = "greetd"
```

greetd runs that *greeter* as the unprivileged system user `greetd`. `kawa-greeter` clears the screen and replaces itself (`exec`, §26.1) with `agreety --cmd /usr/local/bin/start-dwl`. agreety asks for the name and password and passes them to greetd, which checks them through PAM and starts the command as you.

> **Word — PAM.** *Pluggable Authentication Modules*: the library every login uses to check a password and set up a session. greetd, `sudo` and the lock screen all ask it, so a password is checked the same way everywhere.

PAM also tells **logind**, systemd's login manager, that a session has begun:

```
$ loginctl
SESSION  UID USER SEAT  LEADER CLASS   TTY  IDLE SINCE
      2 1000 you  seat0 1290   user    tty1 no   -
      3 1000 you  -     1012   manager -    no   -

2 sessions listed.
```

(for example; the columns vary a little between systemd versions) Session 2 is your desktop on tty1. Session 3, class *manager*, is your own systemd — the *user manager*, `user@1000.service` — which runs your user services (PipeWire, the portals, Prowlarr). It keeps running with nobody logged in, because setup turned **linger** on for you (§28.2).

`start-dwl` sets the environment every program in the session inherits, then starts dwl:

Variable	Value	For
`XDG_SESSION_TYPE`, `XDG_CURRENT_DESKTOP`	`wayland`, `wlroots`	programs and portals: which kind of desktop this is (§32.2)
`XKB_DEFAULT_LAYOUT`, `XKB_DEFAULT_OPTIONS`	`us,ara`, `grp:alt_shift_toggle`	the keyboard, from `fedokawaii.conf`
`QT_QPA_PLATFORM`, `GDK_BACKEND`	`wayland;xcb`, `wayland,x11`	Qt and GTK: Wayland first, X11 as the fallback (§31.6)
`GTK_USE_PORTAL`	`1`	GTK 3 programs' file dialogs through the portal (yazi)
`EDITOR`, `TERMINAL`, `BROWSER`	`nvim`, `foot`, `brave-origin`	programs that open one of these
`XCURSOR_THEME`	`Fedokawaii`	the pixel pointers
`PATH`	`~/.local/bin` first	the `kawa-*` helpers

Should dwl be missing, unable to start because a library is gone, or stop again within seconds, `start-dwl` tries the build before it (`dwl.prev`, and the session says so in a notice) and otherwise shows a rescue screen with a shell, instead of a login that loops (§36.7). It runs one desktop at a time: started again while the desktop runs (typed in a terminal, or on another console), it says so and stops. Each login keeps the log of the session before as `~/.cache/dwl.log.old` — what dwl said before a crash.

Read more: loginctl(1).

29.9 Stages 10–11: dwl and the autostart

dwl opens the GPU through the kernel (§31.1), the keyboard and mouse through libinput, and a Wayland socket in `$XDG_RUNTIME_DIR` (`/run/user/1000/wayland-0`) for programs to connect to. Then it runs `~/.config/dwl/autostart.sh`, which first hands the session's variables to your user manager and D-Bus (so services they start later can open windows here), starts `dwl-session.target` (which binds systemd's `graphical-session.target`, the signal the portals wait for), and then starts the helpers:

Started	Job	Restarted if it crashes
`kawa-display`, `kawa-theme apply`	resolution and refresh rate; the theme's settings and wallpaper	— (run once)
`waybar`	the bar	yes (`keep`)
`mako`	notifications	D-Bus starts it again with the next notification
`foot --server`	every Super+Return terminal	yes
`lxqt-policykit-agent`	the password dialog for administrative actions (§24.4)	yes
`udiskie`	mounts USB drives as you plug them in, and hands each change to `kawa-drives` (ch. 13, §32.6)	yes
`kawa-clipkeep` and `wl-paste --watch cliphist store`	copies that outlive their program; the clipboard history	yes
`gammastep`	night light	—
`swayidle`	the notice, the lock and the screens off when you are away	yes
`kawa-dwlstatus`	reads dwl's status for the bar's workspaces, layout, title and keyboard	yes, in the foreground

`keep` is a small shell function in the file: it runs the program again when it exits with an error, gives up with a notice after five crashes in a minute, and says once in a notice when the program is not installed (`fedokawaii repair` puts it back). The autostart runs only as dwl starts it: run by hand, it says so and starts nothing (log out and in instead). When dwl ends — you log out, or it crashes — the autostart ends everything it started, so nothing runs twice after the next login.

The result, as the process tree shows it — reduced to the lines that matter (the full view is `ps -ef --forest`):

```
greetd --config /etc/greetd/fedokawaii.toml
 └─ greetd                                  (its worker for your session)
    └─ /bin/sh /usr/local/bin/start-dwl
       └─ /usr/local/bin/dwl -s /home/you/.config/dwl/autostart.sh
          ├─ /bin/sh /home/you/.config/dwl/autostart.sh
          │  ├─ sh ── waybar                (sh: a keep loop)
          │  ├─ mako
          │  ├─ sh ── foot --server ── bash ── …   (your terminals)
          │  ├─ sh ── lxqt-policykit-agent
          │  ├─ sh ── udiskie
          │  ├─ sh ── swayidle -w …
          │  └─ kawa-dwlstatus
          └─ Xwayland …                     (only once an X11 program asks for it)
```

systemd sees the same programs grouped differently, by *control group*: everything dwl started sits in your session's scope, beside your user services (§28.6).

Reading the whole story afterwards. Every stage from the kernel on writes to the **journal** (§28.5): `journalctl -b` is this boot from the start, `journalctl -b -1 -p err` the errors of the boot before, `journalctl -b -u greetd` the login. dwl and what it starts write to `~/.cache/dwl.log` instead.

Read more: Boot Loader Specification · systemd-analyze(1) · bootup(7) · the kernel's microcode loader · Secure Boot on Fedora.

The relay, one line per stage

#	In one line	If it fails, look at
1–2	the firmware loads shim, which checks GRUB's signature	the firmware's own screen; the Secure Boot setting
3	GRUB shows the menu and loads a kernel and its initramfs	the boot menu: choose the older kernel
4–5	the kernel applies microcode, the initramfs mounts `/`	`journalctl -b -1 -k`
6	systemd starts every unit towards `graphical.target`	`journalctl -b -p err` · `systemctl --failed`
7–9	greetd and agreety log you in; PAM and logind open your session	`journalctl -b -u greetd`
10–11	dwl takes the screens and the autostart starts the desktop	`~/.cache/dwl.log` · the rescue screen

What you learnt

- The start-up is a relay of eleven stages; each checks and starts the next.
- Secure Boot can stay on: shim is signed by Microsoft, and checks GRUB and the kernel with Fedora's key.
- Boot entries live in `/boot/loader/entries/`; `grubby` changes the kernel's arguments in all of them; two kernels are kept.
- The initramfs mounts `/` for the kernel; early microcode fixes the CPU before anything runs.
- systemd starts towards `graphical.target`; greetd, PAM and logind open your session; dwl and the autostart build the desktop.

Check yourself

1. Why can Secure Boot stay on? → *shim is signed with Microsoft's key, and the drivers used here are part of Fedora's signed kernel*
2. Where do you see the arguments the running kernel was given? → *`cat /proc/cmdline`*
3. The new kernel will not start the desktop. What do you do? → *choose the older kernel in the boot menu (if it is hidden: hold Shift, or press Esc or F8)*

30. Hardware: drivers, firmware and devices

> **In this chapter**
> - tell a driver from firmware and microcode, and know how this desktop keeps each
> - see how a device appears in `/dev`, and which driver holds it
> - talk to the Bluetooth stack from a terminal
> - say what the performance profile changes, and why
>
> **You need:** ch. 29 (how the machine starts).

Every device needs two pieces of software: a driver in the kernel, and often firmware inside the device itself. This chapter shows where both come from on this machine, how a device gets its file in `/dev`, and how to list everything inside the computer.

30.1 Drivers and firmware

A **driver** (§2.1) is the code in the kernel that runs one kind of device: `amdgpu` for AMD graphics, `r8169` for a common Realtek network card. Most are **kernel modules**, separate files the kernel loads only when it finds the device.

The kernel carries the drivers. Many devices also need **firmware**, and the CPU needs microcode (§29.6), loaded at every boot.

> **Word — firmware.** Software that runs inside a device rather than on the CPU: the motherboard's UEFI, a Wi-Fi card's, a GPU's, a sound DSP's. Some stays in the device's own chip; some the driver loads into the device each time it starts.

Try this. Ask which driver holds the graphics card.

```
$ lspci -k -d ::0300 | grep driver
    Kernel driver in use: amdgpu
```

(for example) One line answers: the kernel's `amdgpu` holds it. Fedora ships firmware in separate packages, and a minimal install does not bring them all: Intel's Wi-Fi and Bluetooth firmware, for one, comes only when asked for. So setup, every update and every repair look at the hardware itself and install what it needs:

Looks at	Finds	Installs
the kernel's own log, this boot	firmware a driver asked for and did not find: `Direct firmware load for iwlwifi-so-a0-gf-a0-89.ucode failed with error -2`	the package that provides that file (one of Fedora's `iwlwifi-…-firmware` packages, here)
`/proc/cpuinfo`	the CPU's maker: `GenuineIntel` or `AuthenticAMD`	its microcode (`microcode_ctl` or `amd-ucode-firmware`), then the boot images again, so the next start loads it early
`/sys/class/ieee80211`	a Wi-Fi radio	NetworkManager's Wi-Fi part (without it the card is not managed at all) and the regulatory database
the sound card's driver	Intel's audio DSP (`sof-audio-pci-intel-…`)	its firmware, `alsa-sof-firmware`
the GPUs	an Intel GPU beside the AMD card the desktop drives	Intel's video decoding too (`intel-media-driver`)

What it installs is recorded in `/var/lib/fedokawaii/hardware-pkgs` and belongs to the manifest from then on: `verify` counts it as the desktop's, and dnf's clean-up never takes it. A file the kernel asked for that no package

has is not a fault — a driver tries the newest firmware first, then loads one it has. A device no driver holds at all is reported (`verify`, *drivers and firmware*): that is a kernel matter, and that device will not work here.

Firmware inside the devices — the SSD's, the motherboard's, a dock's — comes from the device makers through the LVFS, with **fwupd**. The update fetches the list and says what is new; installing it is yours to decide, because some updates restart the machine:

```
fwupdmgr get-devices      # every device fwupd can update, with its firmware version
fwupdmgr get-updates      # what is new for them (the update has fetched the list)
fwupdmgr update           # install them; it says first which ones need a restart
```

To see what holds a device:

```
lspci -nnk | rg -A3 -i 'vga|audio|network'   # PCI devices: name, IDs, "Kernel driver in use"
lsusb                                       # USB devices: the dongle, the keyboard…
journalctl -k -b -g 'firmware'              # what the kernel said about firmware this boot
```

> **On Windows.** Device Manager's list is `lspci -k` and `lsusb`; its *Driver* tab is the line *Kernel driver in use*. The difference: here drivers come with the kernel, not from each maker's website, and firmware arrives through `update` and fwupd.

Read more: Fedora's firmware packages · fwupd · LVFS.

30.2 Devices: udev and /dev

When a device appears — at boot, or a USB stick plugged in — the kernel's driver announces it, and **udev** (`systemd-udevd`) reacts: it creates the file in `/dev`, gives it its owner, permissions and stable names, and tells anyone listening. An example is a name udev made, which stays the same whichever port a disk is in:

```
$ command ls /dev/disk/by-label/
FEDOKAWAII
```

(for example, with the project's USB stick from §3.5 plugged in) Its rules live in `/usr/lib/udev/rules.d/` (the packages') and `/etc/udev/rules.d/` (yours).

```
command ls -l /dev/disk/by-label/                 # stable names, by label (also by-uuid, by-id)
udevadm info /dev/dri/card1 | head                # what udev knows about one device
udevadm monitor --udev                            # watch devices come and go; Ctrl+C stops
```

Kernel modules that no device asks for are loaded from `/etc/modules-load.d/`: setup puts `i2c-dev` there, for monitor brightness over DDC/CI (§8.1).

libinput reads keyboards, mice and touchpads for the compositor: wlroots uses it, so dwl gets pointer acceleration, tapping and scrolling the same way every Wayland desktop does. The keyboard layout itself comes from **XKB** (`XKB_DEFAULT_LAYOUT`, §29.8).

Read more: udev(7).

30.3 Bluetooth: BlueZ

BlueZ is the Bluetooth stack: the kernel part, and the daemon `bluetoothd` (`bluetooth.service`). Quick settings and Blueman use it; so can you:

```
bluetoothctl show          # the adapter: powered, discoverable
bluetoothctl devices       # paired and known devices
```

When quick settings cannot switch Bluetooth on, a notice says why: *it is blocked* — by an airplane-mode key or the machine's own switch; `rfkill unblock bluetooth` lifts it — or *it did not switch on*, when the service is not answering (`systemctl status bluetooth`).

Bluetooth headphones appear in PipeWire as one more sound output — a *sink* (§31.8) — once connected.

30.4 The performance profile: tuned

tuned is a service that applies a named set of kernel and hardware settings, a *profile*. For example, ask which one is active:

```
$ tuned-adm active
Current active profile: fedokawaii
```

The machine runs the **tuned** profile `fedokawaii` (`system/tuned.conf`). It takes Fedora's *throughput-performance* (the CPU at full speed — this is a desktop that is always plugged in) and changes the memory settings for a desktop whose swap is *zram*, compressed memory rather than a disk, so a large copy never stalls the desktop:

Setting	Kernel default	throughput-performance	here	Why
`vm.swappiness`	60	10	100	swapping to zram is cheap: use it rather than drop file cache
`vm.dirty_ratio`	20 %	40 %	20 %	a big copy is written out long before it fills 40 % of memory
`vm.dirty_background_ratio`	10 %	10 %	10 %	writing starts early, in the background
`vm.page-cluster`	3 (8 pages)	—	0 (1 page)	reading back from zram is a memory copy: no read-ahead
`vm.watermark_boost_factor`	15000	—	0	no extra bursts of memory reclaim, so frames stay even under load

```
tuned-adm active                      # → Current active profile: fedokawaii
sysctl vm.swappiness vm.dirty_ratio   # → vm.swappiness = 100, vm.dirty_ratio = 20
```

Wi-Fi power saving is off as well (`wifi.powersave = 2`, meaning *disable*, in `/etc/NetworkManager/conf.d/90-fedokawaii.conf`), so the connection never naps.

Read more: tuned's throughput-performance profile · the kernel's vm settings.

30.5 Practice: what is inside this computer?

Five lists of the hardware, each read for one line. None changes anything.

1. The PCI devices, each with the driver that holds it.

```
$ lspci -k
…
03:00.0 VGA compatible controller: Advanced Micro Devices, Inc. [AMD/ATI] Navi 22 [Radeon RX 67…
    Subsystem: …
    Kernel driver in use: amdgpu
    Kernel modules: amdgpu
…
05:00.0 Ethernet controller: Realtek Semiconductor Co., Ltd. RTL8111/8168/8211/8411 PCI Expre…
    Kernel driver in use: r8169
…
```

(for example, shortened) Read the GPU's *Kernel driver in use*: `amdgpu`. The network card at `05:00.0` is why it is called `enp5s0` (ch. 33).

If a device has no `Kernel driver in use` line: no driver holds it, and it will not work; `verify` reports it.

2. The USB devices.

```
$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 003: ID 046d:c52b Logitech, Inc. Unifying Receiver
…
```

(for example, shortened) Each line: where it is plugged in, its maker's and model's numbers, and its name. Solaar (§21.2) manages that Logitech receiver.

If you see only `root hub` lines: nothing is plugged in by USB; plug in a stick and look again.

3. The disks, with their file systems.

```
$ lsblk -f
NAME        FSTYPE FSVER LABEL UUID FSAVAIL FSUSE% MOUNTPOINTS
zram0       swap   1           …                  [SWAP]
nvme0n1
├─nvme0n1p1 vfat   FAT32       …     580.1M     3% /boot/efi
├─nvme0n1p2 ext4   1.0         …       723M    19% /boot
└─nvme0n1p3 btrfs              …     412.3G    11% /home
                                                 /
```

(for example, shortened) Read the last partition: one Btrfs file system, mounted twice — as `/home` and as `/` (§23.4). `zram0` is the swap in memory.

If you see a `sda` too: that is a USB stick or a second disk (ch. 13).

4. What the kernel said about firmware this boot.

```
$ journalctl -k -b -g firmware
… kernel: iwlwifi 0000:04:00.0: Direct firmware load for iwlwifi-so-a0-gf-a0-89.ucode failed …
… kernel: iwlwifi 0000:04:00.0: loaded firmware version 86.… so-a0-gf-a0-86.ucode op_mode iwlmvm
```

(for example, on a machine with an Intel Wi-Fi card; shortened) Error −2 means *no such file*: the driver asked for the newest firmware, then loaded the one it has. Not a fault.

If you see `-- No entries --`: nothing on this machine asked for firmware by name this boot — also fine.

5. The devices whose firmware fwupd can update.

```
$ fwupdmgr get-devices
…
├─Samsung SSD 980 1TB:
│     Device ID:          …
│     Current version:    3B4QFXO7
…
```

(for example, shortened) The SSD's own firmware and its version. `fwupdmgr get-updates` says whether its maker offers a newer one.

If you see `No detected devices`: nothing in this computer offers its firmware through the LVFS — common, and fine.

What you learnt

- A driver runs a device from inside the kernel; firmware runs inside the device; microcode fixes the CPU.
- Setup, `update` and `repair` read the hardware and install the firmware it needs; fwupd updates the firmware stored in devices, when you choose.
- udev makes each device's file in `/dev`, with stable names under `/dev/disk/by-*`.
- `lspci -k`, `lsusb`, `lsblk -f` and `journalctl -k` list the hardware and what holds it.
- The tuned profile keeps the CPU at full speed and tunes memory for zram.

Check yourself

1. Which line of `lspci -k` names the driver holding a device? → *`Kernel driver in use`*
2. Is `Direct firmware load … failed with error -2` a fault? → *no: the driver tried the newest firmware, then loaded one it has*
3. Why is `vm.swappiness` 100 here? → *swap is zram, compressed memory, so swapping is cheap*

31. Graphics and sound

> **In this chapter**
> - follow a window's picture from the program to the monitor
> - tell DRM, KMS, Wayland, wlroots and a toolkit apart
> - know how text becomes pixels, in any script
> - follow a sound from a program to the speakers, and steer it
>
> **You need:** ch. 30 (drivers and firmware).

Two streams leave every program you use: pictures for the screen and sound for the speakers. This chapter follows both, layer by layer, from the program down to the kernel's driver.

> **Word — buffer.** A block of memory that holds one finished picture — of a window, or of the whole screen. Programs draw into buffers and hand them on; nobody draws onto the screen directly.

From a program's drawing to light on the screen:

```
a program (Brave, foot, mpv)   draws its window into a buffer, through its toolkit (GTK, Qt):
  │                            on the GPU with Mesa (radeonsi · RADV · ANV) and the kernel's DRM,
  │                            or on the CPU
  ▼
the Wayland protocol           the program hands the finished buffer to the compositor
  ▼
dwl (on wlroots)               places the buffers and composes the screen, also on the GPU
  │  KMS
  ▼
the kernel: amdgpu, i915 or xe sends the finished picture to the monitor
```

31.1 DRM and KMS

DRM (Direct Rendering Manager) is the kernel's GPU subsystem; **KMS** (kernel mode setting) is its part that sets resolutions and refresh rates and puts pictures on the screens. Both are reached through files in `/dev/dri`.

Try this. List the GPU's device files.

```
$ command ls /dev/dri
by-path  card1  renderD128
```

(for example)

Node	Opened by	For
`card1`	the compositor alone, through logind	KMS: screens and modes, plus drawing
`renderD128`	any program	drawing and video decoding only: no control of the screens

> **Word — render node.** A GPU device file, such as `/dev/dri/renderD128`, that any program may open to draw or decode video, but not to change what the screens show. Like a shared kitchen: everyone may cook; only one person, the compositor, decides what is served.

31.2 The drivers

The kernel's driver talks to the GPU; Mesa's drivers, in userspace, turn OpenGL and Vulkan calls into the GPU's own commands. Each GPU family has one of each:

GPU	Kernel driver	OpenGL (Mesa)	Vulkan (Mesa)	Video decoding (VA-API)
AMD Radeon	`amdgpu`	radeonsi	**RADV**	radeonsi, RPM Fusion's freeworld build
Intel, up to Meteor Lake and Arc Alchemist	`i915`	iris	**ANV**	iHD (`intel-media-driver`)
Intel Lunar Lake, Arc Battlemage, Panther Lake	`xe`	iris	ANV	iHD

Facts worth knowing for 2025–2026:

- **RADV** is the Vulkan driver for AMD. AMD ended its own open driver, AMDVLK, in September 2025 and now backs RADV.
- Intel's new kernel driver **xe** is the default for Lunar Lake and Battlemage since Linux 6.12, and for Panther Lake since 6.17. Meteor Lake was the last generation to use `i915` by default; older chips stay on it.
- The kernel driver also needs the GPU's **firmware** files (`amd-gpu-firmware` on AMD); setup installs whatever the kernel asked for and did not find (`lib/hardware.sh`).

31.3 Wayland, wlroots and dwl

Under X11, a separate *X server* owned the screen, and the window manager was one more client of it. Under **Wayland** the compositor *is* the display server: dwl owns the screens (KMS) and the input devices, and each program talks to it directly over a Unix socket using the Wayland protocol. Programs draw their own windows into buffers and hand them over; the compositor only arranges and composes them, and sends input to the window it belongs to.

> **On Windows.** Windows has worked this way since Vista: every program draws into its own buffer, and the Desktop Window Manager (`dwm.exe`) composes the screen from them. dwl does that job here — and, unlike `dwm.exe`, also decides where each window goes.

wlroots is a library that does the heavy parts of a compositor — DRM and KMS, input through libinput, the protocols, XWayland. **dwl** is about three thousand lines of C on top of it: the layouts, the keys and the bar's status. Setup builds it against its own copy of wlroots 0.20 (§36.7).

31.4 Layer shell: the bar, notifications and menus

Ordinary windows are tiled by dwl. The desktop's own pieces use the **layer-shell** protocol (`wlr-layer-shell-unstable-v1`) instead: a surface placed on one of four layers, anchored to edges, outside the tiling.

Layer	Holds here
background	the wallpaper (swaybg)
bottom	—
top	the bar (waybar, `"layer": "top"`), which also reserves its strip of the screen (an *exclusive zone*) so windows tile below it
overlay	notifications and the level display (mako, `layer=overlay`)

fuzzel's menus are layer surfaces too. The lock screen uses its own protocol (`ext-session-lock-v1`), which guarantees that nothing else shows while it is up.

31.5 XWayland

Programs written only for X11 — Steam, many games — run through **XWayland**, an X server that is itself a Wayland client. dwl is built with it switched on (`XWAYLAND=-DXWAYLAND`) and starts it only when an X11 program first connects; copy and paste work in both directions.

31.6 Toolkits

Most programs do not speak Wayland directly; their **toolkit** does — the library that draws their buttons, menus and text: GTK for most programs here, Qt for a few. `start-dwl` tells both to prefer Wayland and fall back to X11 through XWayland (`GDK_BACKEND=wayland,x11`, `QT_QPA_PLATFORM="wayland;xcb"`, §29.8), and Qt to take its colours and fonts from qt6ct (§40.2).

31.7 Fonts

Three libraries put text on the screen, in this order:

Library	Job	Ask it
fontconfig	which font file to use for a name, a language and a missing character	`fc-match monospace` · `fc-list :lang=ar family`
HarfBuzz	*shaping*: which glyphs, in which order and position — essential for Arabic, which joins its letters	(inside every toolkit)
FreeType	turns each glyph's outline into pixels	(inside every toolkit)

```
$ fc-match monospace
JetBrainsMonoNerdFont-Regular.ttf: "JetBrainsMono Nerd Font" "Regular"
```

(for example) The rules are in `~/.config/fontconfig/fonts.conf` (§40.3); after changing them, `fc-cache -f` rebuilds the index.

Read more: Wayland's architecture · wlr-layer-shell · the kernel's DRM documentation · Intel's xe GPUs · Mesa.

31.8 Sound: ALSA, PipeWire, WirePlumber

Sound takes the opposite road: from many programs at once down to one card, mixed on the way.

> **Word — sink.** Where sound goes: a sound output such as the speakers, the HDMI port or Bluetooth headphones. Its opposite, where sound comes from, is a **source**: a microphone.

For example, the volume of the default sink:

```
$ wpctl get-volume @DEFAULT_AUDIO_SINK@
Volume: 0.40
```

(for example) The bar shows that as 40. The layers that carry the sound:

Layer	Is	Here
ALSA	the kernel's sound drivers, and the oldest way for a program to reach a sound card	`/proc/asound/cards`; `/dev/snd/`
PipeWire	one sound server for every program: it mixes streams, moves them between devices and passes sound between programs (and video, for screen sharing)	`pipewire.service`, started by `pipewire.socket`
pipewire-pulse	PipeWire speaking PulseAudio's protocol, so programs written for PulseAudio (Brave, Steam) work unchanged	`pipewire-pulse.service` (package `pipewire-pulseaudio`)

Layer	Is	Here
pipewire-alsa	sends programs that open ALSA directly (spotify_player) through PipeWire	a plugin, no process of its own
WirePlumber	PipeWire's *session manager*: the policy — which device is the default, where a new stream goes, what is remembered	`wireplumber.service`

PipeWire replaced PulseAudio (desktop sound) and JACK (low-latency studio sound) as Fedora's default in Fedora 34; WirePlumber became its session manager in Fedora 35. All of it runs as your own user services, not as root:

```
$ wpctl status
…
Audio
 ├─ Devices:
 │      45. Navi 21/23 HDMI/DP Audio Controller [alsa]
 │      46. Family 17h/19h HD Audio Controller  [alsa]
 ├─ Sinks:
 │  *   58. Family 17h/19h HD Audio Controller Analog Stereo [vol: 0.40]
 ├─ Sources:
 │  *   59. Family 17h/19h HD Audio Controller Analog Stereo [vol: 1.00]
…
```

(for example, shortened) Under *Sinks* are the outputs, under *Sources* the inputs; `*` marks the default. The list also holds *Share* and *Share (as microphone)*, this desktop's own pair for calls (`~/.config/pipewire/pipewire.conf.d/60-share.conf`, §14.4).

```
wpctl get-volume @DEFAULT_AUDIO_SINK@        # Volume: 0.40 — what the bar shows as 40
wpctl set-mute @DEFAULT_AUDIO_SINK@ toggle   # what right-clicking the volume does
pw-top                                       # every stream live, with its load (q quits)
```

Read more: PipeWire · pipewire-pulse(1) · WirePlumber.

What you learnt

- A program draws its window into a buffer; dwl composes the buffers and sends the screen through KMS; the kernel's driver drives the GPU.
- `card1` is the compositor's; `renderD128`, the render node, is open to any program for drawing and decoding.
- Under Wayland the compositor is the display server; X11 programs run through XWayland; toolkits speak Wayland for most programs.
- fontconfig picks the font, HarfBuzz shapes the text, FreeType draws the pixels.
- Sound goes from programs through PipeWire, steered by WirePlumber, to ALSA's drivers; a sink is an output, a source an input.

Check yourself

1. Which device file may any program open, and which only the compositor? → *`renderD128` · `card1`*
2. Why does Steam need XWayland? → *it is written for X11 only; XWayland is an X server that runs as a Wayland client*
3. Which program decides where a new sound stream goes? → *WirePlumber, PipeWire's session manager*

32. How desktop programs talk: D-Bus and portals

> **In this chapter**
> - read the names, objects and interfaces on the two buses
> - follow a program's request for a file dialog through the portal to yazi
> - know how a notice reaches mako, and the music reaches the bar
> - watch messages go by while you use the desktop
>
> **You need:** ch. 31 (graphics and sound) · ch. 28 (systemd).

The bar shows what mpv plays, a notice pops up when a download ends, an upload button opens yazi: in each case one program asked another for something. This chapter is about the message bus they ask over, and the portals that sit on it.

32.1 D-Bus

Programs on a desktop need to ask each other things: *show this notice*, *pause*, *mount this disk*, *open a file dialog*. **D-Bus** (§15.5) is the message bus they use. Two buses run:

Bus	Run by	Carries	Examples
the **system bus**	the system's `dbus-broker.service`	messages to system services	NetworkManager, logind, systemd, polkit, udisks2, BlueZ, resolved
your **session bus**	your user manager's `dbus-broker.service`, socket `/run/user/1000/bus`	messages between your programs	mako, the portals, MPRIS players

Fedora uses **dbus-broker**, a newer implementation of the same protocol, since Fedora 30.

> **Word — D-Bus name.** The address a program takes on a bus so that others can find it, written like a web address back to front: `org.mpris.MediaPlayer2.mpv`. Like a shop's sign: messages for that name reach whoever holds it now.

Try this. List the players that hold a media-player name; start a video in mpv first.

```
$ busctl --user list | grep mpris
org.mpris.MediaPlayer2.mpv      2650 mpv            you :1.58  session-2.scope  2 -
```

(for example, while mpv plays; the columns are the name, its process, its user, its unique connection name, its unit and session) Every service on a bus has a **name**, publishes **objects** at paths, and each object offers **interfaces** with methods, properties and signals:

```
name       org.mpris.MediaPlayer2.mpv
object     /org/mpris/MediaPlayer2
interface  org.mpris.MediaPlayer2.Player
  method     PlayPause()
  property   PlaybackStatus  "Playing"
```

`busctl` reads all of it:

```
busctl list | head                    # the system bus: every name
busctl --user list | grep -E 'mpris|Notifications|portal'
busctl --user tree org.mpris.MediaPlayer2.mpv
busctl --user get-property org.mpris.MediaPlayer2.mpv /org/mpris/MediaPlayer2 \
  org.mpris.MediaPlayer2.Player PlaybackStatus      # s "Playing"
busctl --user monitor org.freedesktop.Notifications # watch messages go by; Ctrl+C stops
```

> **Word — activation.** Starting a service the moment the first message for its name arrives, instead of at login. Like a shop that opens its door when a customer knocks.

A service need not be running to be reached: D-Bus can **activate** it. That is how mako comes back after it was closed, and how the portals start the first time a program asks for a file dialog.

32.2 The desktop portals

> **Word — portal.** A D-Bus service through which a program asks the desktop for something it should not, or cannot, do alone: a file dialog, a screen to share, the dark or light setting. Like a hotel's front desk: guests ask, the desk arranges it.

The front end, `xdg-desktop-portal`, takes every request and hands it to a **backend**, chosen by a file named after `XDG_CURRENT_DESKTOP` — here `wlroots`, so `~/.config/xdg-desktop-portal/wlroots-portals.conf` (§10.4):

Interface	Backend	Does
`FileChooser`	`termfilechooser`, else `gtk`	yazi in a floating terminal, through `kawa-pick`
`ScreenCast` · `Screenshot`	`wlr`	screen sharing and screenshots for Brave and others
`Settings`	`gtk`	the colour scheme GTK 4 and Brave follow
everything else	`gtk`	

An upload in Brave, link by link:

1. Brave calls the portal's `OpenFile` method on your session bus.
2. `xdg-desktop-portal` reads `wlroots-portals.conf` and passes the request to the termfilechooser backend (D-Bus starts it if it is not yet running).
3. The backend runs `~/.local/bin/kawa-pick`, which opens yazi in a floating foot window.
4. You choose a file; the path travels back the same way, and Brave reads that file.

The portals start only inside a graphical session, which is why the autostart starts `dwl-session.target` (it binds systemd's `graphical-session.target`) and `start-dwl` stops it at logout.

32.3 How the file dialog is wired

How it is wired, should you look. A *portal* is a D-Bus service through which a program asks the desktop for something it should not do alone — a file dialog, a screen share, the colour scheme; *xdg-desktop-portal* takes each request and hands it to a *backend* named in a small config file. Here the file-dialog backend `xdg-desktop-portal-termfilechooser` (built by setup from a pinned release, `/usr/local/libexec/`) hands every dialog to `~/.local/bin/kawa-pick`, which opens yazi; `~/.config/xdg-desktop-portal/wlroots-portals.conf` routes dialogs there, with GTK's own dialog as the fallback — should the portal ever be missing, programs show that one, in the desktop's colours, instead. The portal's settings are in `~/.config/xdg-desktop-portal-termfilechooser/config`.

The routing file's settings, one comment each:

```
[preferred]
# anything not named below: the GTK backend
default=gtk
# screen sharing and screenshots: the wlroots backend
org.freedesktop.impl.portal.ScreenCast=wlr
org.freedesktop.impl.portal.Screenshot=wlr
# the dark / light setting that GTK 4 and Brave follow
org.freedesktop.impl.portal.Settings=gtk
# file dialogs: yazi first, GTK's own if it is missing
org.freedesktop.impl.portal.FileChooser=termfilechooser;gtk
```

Its name comes from `XDG_CURRENT_DESKTOP=wlroots`, which the session sets: the portal looks for `wlroots-portals.conf`. *Read more:* portals.conf(5).

32.4 Notifications

The **Desktop Notifications** specification defines one name on the session bus, `org.freedesktop.Notifications`, with a `Notify` method. Whatever program owns the name shows the notices; here that is **mako**. Any program can call it — `notify-send` from a shell:

```
notify-send "Backup" "Copied 1 240 files"   # a notice, as any program would send it
makoctl dismiss --all                       # what Super+Shift+N does
makoctl mode                                # the current modes: do-not-disturb, sharing
```

32.5 MPRIS

MPRIS is the specification for media players on D-Bus: each player takes a name starting with `org.mpris.MediaPlayer2.`, at the object `/org/mpris/MediaPlayer2`, with the interfaces `org.mpris.MediaPlayer2` and `org.mpris.MediaPlayer2.Player` (play, pause, next, what is playing). The bar, the media keys and `playerctl` all speak it, so any player that does works with all three (§27.9).

32.6 polkit and udisks

polkit (§24.4) is itself a D-Bus service on the system bus, `org.freedesktop.PolicyKit1`; services ask it, and it asks your agent for a password when the rules want one. **udisks2** (`org.freedesktop.UDisks2`) manages disks for ordinary users: listing, mounting, unmounting, powering off. **udiskie**, started by the autostart, listens to udisks2 and mounts each new removable drive under `/run/media/you/LABEL` (the machine's own disks it leaves closed: udisks marks them *HintSystem*, and udiskie's config says not to open those by itself); every change it sees, it hands to `kawa-drives`, which shows the notice, lights the bar's drive mark and ejects (§13.2). The same calls, by hand:

```
udisksctl status                        # the drives udisks knows: model, serial, device
busctl --system tree org.freedesktop.UDisks2 | head   # its objects: one per drive and block device
kawa-drives eject /dev/sdb1             # write out, close everything on that drive, switch it off
```

Read more: D-Bus specification · busctl(1) · portals.conf · Desktop Notifications · MPRIS · udisks.

32.7 Practice: listen to the bus

Four tasks, best done with two terminals side by side (**Super**+**Return** twice). Listening changes nothing.

1. In the first terminal, start listening to the notification service.

```
$ busctl --user monitor org.freedesktop.Notifications
Monitoring bus message stream.
```

You should see that one line, and the cursor waiting.

If nothing more happens: that is right — the monitor waits for messages; leave it running and go on in the second terminal.

2. In the second terminal, send a notice; then look at the first.

```
$ notify-send hello
```

```
…
‣ Type=method_call  Endian=l  Flags=0  Version=1 Cookie=9  Timestamp="Thu 2026-10-15 20:41:07.…"
  Sender=:1.87  Destination=:1.20  Path=/org/freedesktop/Notifications  …  Member=Notify
  UniqueName=:1.87
  MESSAGE "susssasa{sv}i" {
          STRING "notify-send";
          UINT32 0;
          STRING "";
          STRING "hello";
          …
```

(for example, shortened) A *method call* to `Notify`, with its arguments in order: the program's name, no notice to replace, no icon, the summary *hello*, and more below. mako's *method return* follows, with the new notice's number. Before it, `notify-send` called `GetServerInformation`: it asked who was listening. **Ctrl+C** stops the monitor.

If nothing appears: the monitor runs in the other window; look at the first terminal.

3. Ask which media players are on the bus. Start a video in mpv first.

```
$ playerctl -l
mpv
```

If you see *`No players found`*: nothing is playing, or the player does not speak MPRIS.

4. Find the portals.

```
$ busctl --user list | grep portal
org.freedesktop.impl.portal.PermissionStore         1838 xdg-permission- you :1.30 xdg-pe…
org.freedesktop.impl.portal.desktop.gtk             1902 xdg-desktop-por you :1.41 xdg-de…
org.freedesktop.impl.portal.desktop.termfilechooser    - -               -   (activatable) -
org.freedesktop.impl.portal.desktop.wlr             1897 xdg-desktop-por you :1.40 xdg-de…
org.freedesktop.portal.Desktop                      1850 xdg-desktop-por you :1.35 xdg-de…
…
```

(for example, shortened) `org.freedesktop.portal.Desktop` is the front end; the `impl.portal.desktop.*` names are its backends. *(activatable)* means not running yet: D-Bus starts the file dialog's backend at the first upload.

If you see no lines: grep is case-sensitive; type `portal` in small letters.

What you learnt

- Programs ask each other things over D-Bus: the system bus for system services, your session bus for your programs.
- A service is found by its D-Bus name, offers objects and interfaces, and can be started by activation at the first message.
- Portals let programs ask the desktop for a file dialog, a screen share or the colour scheme; here the file dialog is yazi.
- Notifications go to whoever holds `org.freedesktop.Notifications` — mako; players speak MPRIS to the bar.

Check yourself

1. Which bus carries a message from Brave to the portal? → *your session bus*
2. Why does mako come back after it was closed? → *D-Bus activates it when the next notification arrives*
3. Which file decides that file dialogs open yazi? → *`wlroots-portals.conf`, in `~/.config/xdg-desktop-portal/`*

33. The network

> **In this chapter**
> - see and change the machine's connections
> - follow a name such as `fedoraproject.org` to its address
> - know what may connect to this machine from outside, and what listens here
> - find which step fails when "the internet is down"
>
> **You need:** ch. 28 (systemd) · §8.2 (getting online).

Getting online takes three services, one after the other: one brings up the connection, one turns names into addresses, one decides what may come in. This chapter takes them in that order, then turns them into a checklist for the day the internet seems to be down.

33.1 NetworkManager

NetworkManager brings up the network cards, keeps Wi-Fi and wired connections, and hands out DNS settings. `nmcli` is its command-line face (and `nmtui`, in quick settings, a menu).

> **Word — IP address.** The number a machine is reached at on a network, like a street address for letters: `192.168.1.23` at home, `127.0.0.1` for the machine itself. The `/24` after it says how much of it names the local network.

Try this. List each network card, with its addresses.

```
$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp5s0           UP             192.168.1.23/24 fe80::…/64
```

(for example) `lo` is the machine talking to itself; `enp5s0` is the wired card, named after where it sits (PCI bus 5, slot 0). NetworkManager's own view:

```
$ nmcli device
DEVICE  TYPE      STATE                   CONNECTION
enp5s0  ethernet  connected               Wired connection 1
lo      loopback  connected (externally)  lo
```

(for example) Its settings are in `/etc/NetworkManager/`; this desktop adds one drop-in (`conf.d/90-fedokawaii.conf`) that turns Wi-Fi power saving off.

> **On Windows.** `ip -br addr` is `ipconfig`, shorter; `nmcli` and `nmtui` are the network settings page. The adapter called *Ethernet* on Windows is `enp5s0` here: Linux names a card by where it sits, so the name never changes.

33.2 Names: systemd-resolved

> **Word — DNS.** The *Domain Name System*: the internet's phone book, which turns a name such as `fedoraproject.org` into an IP address. Your router, or your provider, runs the servers that answer.

Turning `fedoraproject.org` into an address is the job of **systemd-resolved**, Fedora's default since Fedora 33. Programs ask a local *stub* at `127.0.0.53`; resolved asks the DNS servers NetworkManager gave it, and caches the answers:

```
$ grep -v '^#' /etc/resolv.conf
nameserver 127.0.0.53
options edns0 trust-ad
search .
$ resolvectl status
Global
  resolv.conf mode: stub
…
Link 2 (enp5s0)
…
       DNS Servers: 192.168.1.1
…
```

(for example) `resolvectl query NAME` looks a name up through it, and `resolvectl flush-caches` empties the cache.

33.3 The firewall: firewalld

firewalld holds the kernel's packet filter: the rules for what may connect to this machine from outside. Every connection arrives at a *port*.

> **Word — port.** A numbered door on one machine, from 1 to 65535: a program *listens* at a port, and others connect to it. Web servers use 443; Prowlarr here uses 9696.

firewalld sorts networks into zones, and a network card belongs to one zone.

> **Word — zone.** In firewalld, a named list of what may come in, applied to the network cards assigned to it. Like Windows' *Public* and *Private* network profiles: the same machine can be open at home and closed in a café.

On Fedora Server the default zone is normally `FedoraServer`. Setup takes `ssh` and `cockpit` out of it (with `SSH_SERVER="no"`) and opens torrra's port:

```
$ sudo firewall-cmd --get-default-zone
FedoraServer
$ sudo firewall-cmd --list-services
dhcpv6-client
$ sudo firewall-cmd --list-ports
6881/tcp 6881/udp
```

(for example) Port 6881 is open on purpose, so other peers can reach torrra while it downloads. Prowlarr listens on this machine only (`127.0.0.1:9696`), out of the network's reach whatever the firewall says. `ss -tln` shows everything that listens, and where.

Read more: nmcli(1) · systemd-resolved(8) · firewalld.

33.4 Practice: is the network up? A checklist

When "the internet is down", one step of five has failed. Ask them in this order; the first that fails is the one to fix. Each check only reads.

1. An address? The card must be up and have one.

```
$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp5s0           UP             192.168.1.23/24 fe80::…/64
```

(for example) Healthy: `UP`, with an address that is not `127.…`.

If you see `DOWN`, or `UP` with only `fe80::…`: the cable is out, or no network gave the card an address — check the cable or the Wi-Fi in quick settings (§8.2).

2. A route? Reach an address on the internet, without any names.

```
$ ping -c 3 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
64 bytes from 1.1.1.1: icmp_seq=1 ttl=57 time=12.4 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=57 time=12.1 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=57 time=12.3 ms

--- 1.1.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 12.1/12.3/12.4/0.1 ms
```

(for example) Healthy: three replies, *0% packet loss*.

If you see `ping: connect: Network is unreachable`, or `100% packet loss`: there is no way out — the router, or the line behind it.

3. Names? Turn a name into an address.

```
$ resolvectl query fedoraproject.org
fedoraproject.org: 38.145.60.20                           -- link: enp5s0
                   38.145.60.21                           -- link: enp5s0

-- Information acquired via protocol DNS in 23.1ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: no
-- Data from: network
```

(for example: the addresses differ by time and place) Healthy: at least one address.

If you see `resolve call failed: All attempts to contact name servers or networks failed`: step 2 works but names do not — the DNS server is not answering; `resolvectl status` shows which one is in use.

4. The web? Fetch the first lines of a real page.

```
$ curl -I https://fedoraproject.org
HTTP/2 200
…
```

(for example, shortened) `-I` asks only for the headers. `200` means *OK*.

If you see `curl: (6) Could not resolve host: fedoraproject.org`: go back to step 3. If you see `curl: (7) Failed to connect …`: names work, but the site, or the way to it, does not.

5. What listens here? The other direction: which doors are open on this machine.

```
$ ss -tln
State  Recv-Q Send-Q Local Address:Port  Peer Address:PortProcess
LISTEN 0      4096   127.0.0.53%lo:53         0.0.0.0:*
LISTEN 0      4096      127.0.0.54:53         0.0.0.0:*
LISTEN 0      4096       127.0.0.1:9696       0.0.0.0:*
…
```

(for example, shortened) `-t` TCP, `-l` listening, `-n` numbers instead of names. The two `:53` lines are systemd-resolved's stubs; `127.0.0.1:9696` is Prowlarr, while it runs. `127.…` means *this machine only*: nothing outside can reach them.

If you see a line with `0.0.0.0:22`: an SSH server listens for the whole network: `SSH_SERVER` in `fedokawaii.conf` is `yes` (§5.2).

What you learnt

- NetworkManager brings the cards up and hands out DNS servers; `ip -br addr` and `nmcli` show the result.
- systemd-resolved turns names into addresses through its stub at `127.0.0.53`, and caches them.
- firewalld lets in only what the zone allows; here, torrra's port 6881 and no SSH.
- When the internet is down, check in order: an address, a route, names, the web.

Check yourself

1. `ping -c 3 1.1.1.1` works but `curl -I https://fedoraproject.org` cannot resolve the host. Which step fails? → *names: DNS (step 3)*
2. Why is Prowlarr unreachable from other machines whatever the firewall says? → *it listens on `127.0.0.1`, this machine only*
3. What is a firewalld zone? → *a named list of what may come in, applied to the network cards in it*

34. Security and containers

> **In this chapter**
> - name each layer that protects this machine, and what it stops
> - read an SELinux denial as a sentence, and restore a file's label
> - understand the rootless container Prowlarr runs in, and look after it
> - keep the machine safe by six habits
>
> **You need:** ch. 24 (users and permissions) · ch. 33 (the network).

No single lock keeps a computer safe; several do, each against a different mistake or attack. This last chapter of Part IV puts them side by side, looks closely at SELinux and at the one container this desktop runs, and ends with the habits that keep it all working.

Each layer limits a different kind of mistake or attack:

Layer	Stops	Here
permissions (§24.2)	one user's programs touching another's files	your home is yours alone; `secrets.sh` is 600
users and privilege (§24.1)	ordinary programs changing the system	root only through `sudo` and polkit, after your password
SELinux	a program doing what its type may not, even as root	`enforcing`, the *targeted* policy
firewalld	connections from outside that nothing asked for	`FedoraServer` zone, no SSH unless chosen
Secure Boot	a changed boot loader or kernel	shim, GRUB and the kernel checked at every start (§29.1)
signed packages	a package changed on its way, or from someone else	every repository with `gpgcheck`; keys checked by fingerprint (§27.5)
rootless containers	a containerised program reaching the system	Prowlarr as you, in its own user namespace

34.1 SELinux

Permissions ask *who are you?* SELinux asks a second question: *what kind of program are you, and is this kind allowed to touch that kind of file?* It asks even of root.

> **Word — SELinux type.** The part of a security label that says what kind of thing a process or file is: `user_home_t` for files in a home, `bin_t` for programs. The policy lists which process types may do what to which file types.

Try this. Ask whether SELinux is on.

```
$ getenforce
Enforcing
```

Enforcing means a refused access is refused, not only logged. (`sestatus` adds the policy's name, `targeted`, and more.) Fedora's SELinux policy is **targeted**: the services most exposed — network daemons, the login, containers — run in confined types with exactly the access their policy grants, while your own programs run `unconfined_t`.

The next section reads the labels that carry these types; the one after it, a refusal.

34.2 SELinux labels on files

On Fedora every file and process also carries an **SELinux label**, and the kernel's policy says which labels of process may touch which labels of file — on top of the permissions (§24.2), even for root (§34.1). `-Z` shows them:

```
$ command ls -Z ~/.bashrc /usr/local/bin/dwl
unconfined_u:object_r:user_home_t:s0 /home/you/.bashrc
unconfined_u:object_r:bin_t:s0 /usr/local/bin/dwl
$ id -Z
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

(for example) The part that matters is the third, the **type**: `user_home_t` for files in a home folder, `bin_t` for programs. Your own shell runs as `unconfined_t`, which the policy lets do what the permissions allow; services run in confined types. Files moved into place keep the label of where they came from, which is why setup runs `restorecon` after installing a file: it sets the label the policy expects for that path. Prowlarr's folder is mounted into its container with `:Z`, which relabels it `container_file_t`, the one type a container may use.

34.3 Reading a denial

When the policy refuses an access, the refusal is logged as an *AVC* message (*access vector cache*, the part of SELinux that decides):

```
$ sudo ausearch -m avc -ts recent
----
time->Thu Oct 15 20:41:07 2026
type=AVC msg=audit(1792082467.512:812): avc:  denied  { read } for  pid=4120 comm="myservice" …
```

(an invented service, for example; shortened — the one line goes on: `name="notes.md" dev="nvme0n1p3" ino=93321 scontext=system_u:system_r:myservice_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=0`)

Field	Reads
`denied { read }`	the access that was refused
`comm="myservice"`, `pid=4120`	the program that asked
`name="notes.md"`, `ino=93321`	the file it wanted
`scontext=…:myservice_t:…`	the asker's type
`tcontext=…:user_home_t:…`	the file's type
`tclass=file`	the kind of object

Read it as a sentence: *a process of type `myservice_t` was refused `read` on a `file` of type `user_home_t`*. The usual cause on a desktop is a file with the wrong label — made elsewhere and moved into place — and the usual fix is `restorecon -v FILE`, which gives it the label its path should have. Never switch SELinux off to make a problem go away (§39.2).

Read more: SELinux on Fedora · Troubleshooting SELinux.

34.4 Rootless containers

A *container* is a program fenced off on this kernel (§34.5 defines the words). Prowlarr runs in podman *without* root.

> **Word — user namespace.** A private numbering of users for one container: inside, it sees user 0 and up; outside, those are borrowed numbers that belong to nobody real. Like a hotel's room numbers: room 1 inside is not house number 1 on the street.

Its container gets its own user namespace: inside, it sees users from 0 upwards; outside, those numbers are a range of IDs set aside for you in `/etc/subuid` and `/etc/subgid`, none of which is a real user on the machine. The quadlet maps your own UID through as 1000 (`UserNS=keep-id`), so the files it writes stay yours:

```
$ grep "^$USER:" /etc/subuid
you:524288:65536
$ podman unshare cat /proc/self/uid_map
         0       1000          1
         1     524288      65536
```

(for example) `podman unshare` runs a command in podman's own user namespace, the one your containers start from: there 0 is you, and 1 and up are the borrowed IDs. Prowlarr's quadlet then maps you once more, to 1000 inside its container. Add `NoNewPrivileges=true`, a port bound to `127.0.0.1` only, the SELinux type `container_t` for its processes and `container_file_t` for its folder, and Prowlarr can reach very little even if it misbehaves.

Read more: podman(1).

34.5 podman and the Prowlarr container

Term	In one line
image	a read-only bundle of one program and everything it needs, downloaded once
container	a running copy of an image, walled off from the rest of the system
container or VM	a virtual machine boots a whole second operating system on pretend hardware; a container is only a fenced-off process on this kernel, so it starts in seconds and costs only the program's own memory
rootless	the container runs as you, not as root: a break-out would have only your rights
quadlet	a small file that tells systemd how to run a container as a service
health check	a command run inside the container from time to time, asking whether it still answers

podman runs one container here: **Prowlarr**, the indexer service torrra searches through (ch. 20). It runs rootless, as you — you are its user inside too, so every file in `~/.config/prowlarr` is yours — holds no capabilities and can gain none, runs **only when wanted** (a torrra search starts it and stops it again; the command `prowlarr` starts it and opens its page — idle it would hold some 300 MB of memory), and listens only on this machine, at http://localhost:9696. systemd manages it from a *quadlet* file, `~/.config/containers/systemd/prowlarr.container`, which `update` keeps current. A health check asks it every five minutes whether it answers; three misses in a row restart it. The quadlet's main lines, and what each does:

Line in `prowlarr.container`	Does
`Image=lscr.io/linuxserver/prowlarr:latest`	the image to run
`UserNS=keep-id:uid=1000,gid=1000`	you are the same user inside as outside, so its files stay yours
`NoNewPrivileges=true`	it can never gain more rights than it starts with
`PublishPort=127.0.0.1:9696:9696`	its port 9696, reachable from this machine only
`Volume=%h/.config/prowlarr:/config:Z`	its settings live in `~/.config/prowlarr` (`%h` is your home; `:Z` labels the folder for SELinux)
`HealthInterval=5m` · `HealthRetries=3`	the health check: every five minutes, three misses allowed

Command	Does
`systemctl --user status prowlarr`	is it running, and its last lines of output
`journalctl --user -u prowlarr -e`	its full log
`systemctl --user restart prowlarr`	restart it
`prowlarr stop`	stop it when you are done with its page (`prowlarr` leaves it running; it is stopped anyway at the next restart of the computer)
`podman ps` · `podman logs -f prowlarr`	running containers (with their health) · follow the log
`podman image ls lscr.io/linuxserver/prowlarr`	the Prowlarr image you have, and when it was made (`update` fetches the newest)
`podman image prune`	remove image layers left behind by updates

Pitfalls.

- Never `sudo podman`: root has its own, separate — and here empty — list of containers.
- Do not `podman rm` or `podman run` it by hand: systemd recreates the container from the quadlet at every start. To change it, edit `system/prowlarr.container` in your working copy and apply (§42.4): a change under `system/` makes apply run repair, which puts the new file in place (so does `update`).

Read more: podman-systemd.unit(5), every quadlet key, and Podman's documentation.

34.6 A container as a systemd unit: the quadlet

Prowlarr runs in a rootless podman container (§34.5, §34.4), and systemd runs the container. The file `~/.config/containers/systemd/prowlarr.container` is a **quadlet**: a description in systemd's own format, which podman's *generator* — a small program systemd runs at boot and at every `daemon-reload` — turns into an ordinary service:

```
$ systemctl --user cat prowlarr.service | head -n 3
# /run/user/1000/systemd/generator/prowlarr.service
# Automatically generated by /usr/lib/systemd/user-generators/podman-user-generator
#
```

(for example) The quadlet names the image, the port (`127.0.0.1:9696`, this machine only), the folder (`%h/.config/prowlarr`), the key (a podman secret) and a health check; systemd adds what it adds to any service: restart on failure, logs in the journal, start and stop on demand. The file has no `[Install]` section, so Prowlarr runs only when a torrra search or the `prowlarr` shell function starts it.

34.7 Good habits

The layers above work by themselves, as long as nobody switches them off or hands over the keys. Six habits keep it so:

Habit	Why	Taught in
Never switch SELinux off; read the denial and `restorecon` the file	switching it off removes a whole layer, for every program, to fix one	§34.3
`sudo` for one command only; never a root shell to work in	everything a root shell starts is root too	§24.5
Read what you paste: download a script, look at it, then run it	a script piped from `curl` straight into `bash` runs whatever that server sends, as you — with `sudo`, as root	§25.5
Keys and tokens only in `secrets.sh`, mode 600; a leading space for a command with a password	a key in a config file or the history is a key anyone can read later	§25.8

Habit	Why	Taught in
Update each season, when the reminder comes	fixes for holes found since arrive with updates	§36.2
Read Claude Code's permission prompts; do not wave them through	each prompt is a command or an edit you are about to allow	§44.2

> **On Windows.** Switching SELinux off to make something work is like switching off Microsoft Defender to let an installer run: the problem goes away, and so does the protection, for everything else too.

The one-line pipe `curl -fsSL URL | bash` deserves its own word. Done in two steps, it costs a minute and shows you what you are about to run:

```
curl -fsSLo install.sh https://example.org/install.sh   # download only
less install.sh                                         # read it; q quits
```

Part IV in one table — each chapter's idea, and one command that shows it:

Chapter	The idea	See it with
ch. 22	the machine is layers; five questions explore any of it	`type` · `rpm -qf` · `journalctl`
ch. 23	one tree from `/`; an inode is the file, a name points at it	`findmnt` · `stat`
ch. 24	owner, group, others; root is borrowed	`id` · `ls -l` · `sudo -l`
ch. 25	streams, pipes, quoting, scripts	`space` · `shellcheck`
ch. 26	fork, exec, signals, jobs	`ps` · `kill` · btop
ch. 27	packages, repositories, signatures, sonames	`dnf repoquery`
ch. 28	units, drop-ins, timers, the journal, cgroups	`systemctl status` · `journalctl -u`
ch. 29	a relay of eleven stages	`systemd-analyze`
ch. 30	drivers, firmware, udev	`lspci -k` · `lsblk -f`
ch. 31	buffers, KMS, Wayland; PipeWire and WirePlumber	`wpctl status`
ch. 32	D-Bus names, portals, activation	`busctl --user monitor`
ch. 33	an address, a route, names, the web	`ip -br addr` · `resolvectl query`
ch. 34	layers of protection; SELinux types; rootless containers	`getenforce` · `podman ps`

What you learnt

- Seven layers protect this machine: permissions, privilege, SELinux, the firewall, Secure Boot, signed packages and rootless containers.
- An SELinux denial reads as a sentence — this type was refused this access to that type — and `restorecon` usually mends it.
- Prowlarr runs rootless, in its own user namespace, on `127.0.0.1` only; systemd runs it from a quadlet.
- Six habits keep the layers working: SELinux on, `sudo` once, read before running, keys in `secrets.sh`, update each season, read Claude Code's prompts.

Check yourself

1. A service is refused reading a file you moved into place. What is the usual fix? → *`restorecon -v FILE`, which gives the file the label its path should have*
2. Why does `sudo podman ps` show no Prowlarr? → *root has its own, separate list of containers; Prowlarr is yours*
3. What is wrong with `curl URL | bash`? → *it runs whatever the server sends without your reading it first*

Part V — Looking after it

The machine is built to be left alone. You know now how it works underneath (Part IV); this Part is what you do with that knowledge: once a season, update it; once a year, move it to the next Fedora; when a change goes wrong, go back in time; now and then, tidy it; and when something breaks, find the cause calmly. Start with ch. 35, the one menu that does most of it.

Chapter	What it gives you
35. The system's control: Super+U	the menu, entry by entry, and what verify proves
36. Updates and new Fedora releases	the season's update, and the yearly move
37. Snapshots: going back in time	undo a change, a file or a whole update
38. Keeping it clean	packages, disk space, logs, and a seasonal routine
39. When something goes wrong	a method, a table of symptoms, and the console when nothing else starts

35. The system's control: Super+U

> **In this chapter**
> - open the system's control and run any of its twelve entries
> - read the status page and the marks beside each entry
> - read verify's report: passed, failed, worth knowing
>
> **You need:** ch. 27 (packages) and ch. 28 (services); you met the control in §8.5.

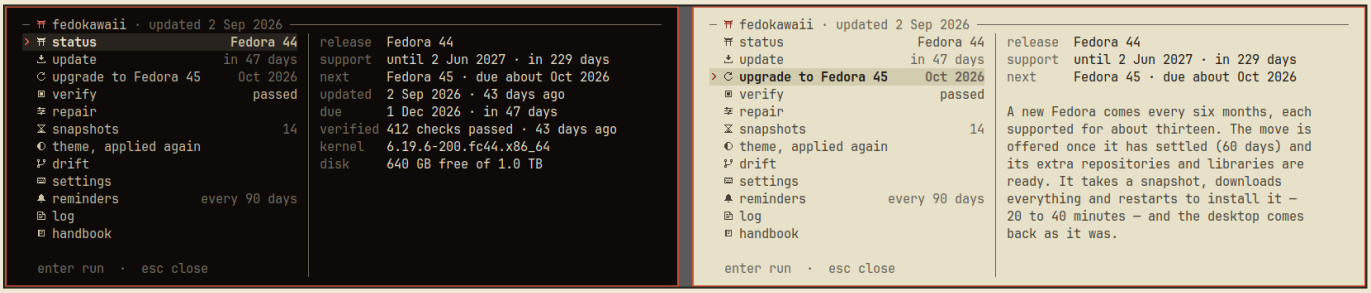
The machine is built to be left alone for months and to run for years on a toolkit that is never edited again: once a season you **update** it, about once a year you **move** it to the next Fedora, and both are one entry in one menu.

These words come up throughout this Part. The ones Part IV taught are there for reminding:

Term	Plain meaning
update	newer versions of the packages of the release you have: Fedora 44 stays 44
upgrade, the move	going to the next release, 44 → 45: nearly every package is replaced in one go
kernel	the core of Linux, which drives the hardware (§2.1); each new one is installed beside the one before
transaction	one `dnf` run that installs, updates or removes a set of packages together (ch. 27)
SELinux	Fedora's mandatory access control (ch. 34): rules in the kernel that limit what each program may touch, even when it runs as root. *Enforcing* means the rules are applied, not only logged; `getenforce` says which
linger	systemd keeps your own services (Prowlarr, the portals) available with nobody logged in, so a run as root can reach them

Read more: SELinux on Fedora.

35.1 The menu, entry by entry



The system's control (Super+U): what is due at the right of each entry, how things stand in the pane — the status on the dark theme, the move to the next Fedora on the light.

Super+U opens it (also quick settings → *system*, or `fedokawaii` in a terminal): a small window with nothing to search, only twelve entries — **↑ ↓** (or **j k**) to choose, the mark at the right of each saying what is due, and beside them a pane with how things stand for the entry under the cursor. **Enter** runs it in the same window — what changes the system asks for your password — and a key afterwards brings the menu back with its marks brought up to date; **Esc** (or **q**) closes it.

Entry	Does	Its mark
status	how things stand, on one page	the release
update	the season's update (§36.2)	when the next is due; **due** in red once it is

Entry	Does	Its mark
upgrade to Fedora N	the move to the next release (§36.5)	when it is expected or offered; **ready** in red once it has settled
verify	every check (§35.2); changes nothing	**passed**, or how many to fix
repair	verify, put right what it found — packages, this machine's drivers and firmware (§30.1), anything doubled or fighting (a package twice, a second login manager, an unneeded dependency…), services, the login, dwl (rebuilt if it will not start; work of yours in its folder is set aside as a branch or a stash, never lost), the session, the tools, neovim's plugins, the configs where the theme was chosen — and verify again; offline too, as far as it can	
snapshots	the snapshots of `/` (§37.2)	how many
theme, applied again	every config back as the toolkit has it (yours kept in the backup)	
drift	the configs that differ from the toolkit's; changes nothing	
settings	a copy of `fedokawaii.conf` in neovim; when you close it every value is checked, and only a valid file replaces the toolkit's (a wrong one is shown, and **y** opens your edit again to put it right; **n** leaves everything as it was); the next update applies it	
reminders	Enter changes the season: every 60 · 90 · 120 days · off	the season
log	the last run's log, at its end	
handbook	this book (on a text console: its text, in less)	

Every entry is a command too — `fedokawaii status`, `update`, `upgrade` (`--now`: without waiting), `verify`, `repair`, `theme`, `drift`, `snapshots`, `log`, `settings`, `guide`, `remind [check | every DAYS | wait DAYS | on | off]` — and `update`, `verify` and `repair` work on their own in a terminal. On a text console (Ctrl+Alt+F3), or should fzf ever fail to start, `fedokawaii` shows the same menu as a numbered list: the system can be looked after even when the desktop cannot start.

A word more than a command takes is refused (`fedokawaii upgrade now` is not `--now`). While an update or a repair runs, a second update, move or repair says so at once instead of asking for your password. The exit status, for scripts: 0 done, 1 it could not, 2 misuse, 3 it ran and something still needs you, 4 (drift) in sync but for an optional part, 130 stopped with Ctrl+C.

> **On Windows.** Windows Update downloads and installs by itself, and restarts when it decides. Here nothing updates in the background: the menu tells you when the season has come, and the update runs when you press **Enter** on *update* — with a snapshot first.

Try this. In a terminal, ask for the status page — the same as the *status* entry:

```
$ fedokawaii status
release  Fedora 44
support  until 2 Jun 2027 · in 230 days
next     Fedora 45 · due about Oct 2026
updated  17 Jul 2026 · 90 days ago
due      now · every 90 days
verified 212 checks passed · 90 days ago
…        (then the running kernel and the free disk space)
```

(for example: Fedora 44 in mid-October 2026, ninety days after an update)

An update that stopped before its end (a power cut, no network) is not counted: `updated` keeps the last one that finished, and a line under it says `tried 16 Oct, stopped: see the log`.

Line	It means
`release · support`	the Fedora you run, and until when Fedora fixes it (its `SUPPORT_END`)
`next`	the next release, and when it is expected or offered (§36.5)
`updated · due`	your last update, and when the next is due: here, now
`verified`	the last verify's outcome (§35.2)

On a text console the same page uses `-` for `·`, as the console's font may lack it.

Reminders. A quiet check tells you, in a notice, when the update season has come round, when a new Fedora has settled, and a month before yours ends — from what the machine already knows, never from the network. The season, and how to change it, is in §36.1.

The toolkit. What root runs — every update, repair and move — is the copy setup installed in `/usr/local/lib/fedokawaii`, owned by root, so nothing running as you can change it; the `fedokawaii` command is installed from it. The folder you installed from can go once setup is done. *settings* edits the toolkit's own `fedokawaii.conf`. Every other change is made in your working copy and applied from there (chapter 42): the copy becomes the toolkit.

Its `fedokawaii.conf` is merged key by key — a value you saved through *settings* stays, unless the copy changed that key since the toolkit was last made from a copy; then the copy's is meant. For example:

Key	Saved through *settings*	In the copy	The toolkit gets
`SSH_SERVER`	`yes`	`no`, as it always was	`yes` — yours stays
`MONITOR_HZ`	`144`	`165`, changed in the copy	`165` — the copy's change
`DWL_COMMIT`	(a pin)	a newer pin	the copy's

The log names every value taken from the copy, and the toolkit's file before is kept as `/var/lib/fedokawaii/fedokawaii.conf.before`. The toolkit holds plain files and folders only: a copy with a symbolic link in it is refused — the toolkit stays as it was, and the run stops there, before anything is applied.

35.2 What verify proves

verify compares the machine with the toolkit's lists — every package, service, file and setting — and changes nothing. Every check fails if its probe fails — a broken tool can never produce a pass. (A *probe* is the small command a check runs: `getenforce` for SELinux, `findmnt` for the file system.) The output is one line per check under each area, and a verdict at the end:

```
$ fedokawaii verify
…
system
  ✔ Fedora 44 (44 or newer)
  ✔ SELinux enforcing
  ✔ / is Btrfs
  …
  ! Prowlarr has no indexers yet — open http://localhost:9696 → Indexers → Add Indexer (torrra…
  …
✔ verified — 212 checks passed, 1 warning(s)
```

(shortened)

> **In SQL terms.** verify is a suite of data-quality tests: each check is a query that must come back clean — like `SELECT name FROM installed EXCEPT SELECT name FROM manifest` returning no rows. A ✘ line names the rows that broke the rule.

✔ passed · ✘ failed (`fedokawaii repair` fixes most) · ! worth knowing, never a failure; a line after a dim dot only reports (which driver opens NTFS, say). The outcome is kept for the menu as one line — when, passed, failed, warnings:

```
cat /var/lib/fedokawaii/last-verify    # → 1792039927 212 0 1
```

Area	Checks
system	Fedora ≥ 44 · SELinux enforcing · Btrfs root · no root-owned files in your home · groups · neovim as the editor · the tuned profile · Wi-Fi power saving off · `secrets.sh` private · linger · ntsync and i2c-dev · the SSH policy and firewall ports (6881 for torrents)
packages and redundancy	every manifest package installed · none of the forbidden ones · nothing user-installed beyond the pre-setup record and the manifest · no Flatpak or Snap · no package installed twice · `dnf check` clean · no service that fights the desktop's own (a second login manager, power-profiles-daemon, a second notification daemon…) · the login greetd's alone · dependencies nothing needs (a warning) · configuration files an update could not merge (a warning) · nothing left behind by a stopped run
drivers and firmware	every package this hardware needs (microcode, Wi-Fi, sound DSP, missing firmware, Intel's video decoding beside an AMD card; one missing is a warning: update and repair install it) · microcode loaded at boot · every network, display, sound and USB device held by a driver · Wi-Fi managed by NetworkManager · device firmware updates, when fwupd knows of any (shown, never a fault) · udisks answers, so drives open · which driver opens NTFS
repositories and DNF	RPM Fusion, Brave, Claude Code, COPRs · all six DNF values
upkeep	the toolkit root's alone · the `fedokawaii` command the toolkit's · the manifest marked as yours · the snapshots' bounds
boot	the boot-menu setting · at most two kernels, the newest running · no rescue image · the GPU kernel argument
GPU and media	hardware video decoding · Vulkan · ffmpeg encoders
compositor and login	dwl version and pinned commit, XWayland, its own wlroots and every library, the patch (keys, gaps, frames, centring, layout), built from `config.h` · the session launcher and its rescue screen · the greetd SELinux module, agreety · the boot-success timer
services	greetd, Bluetooth, snapper timers, fstrim, PipeWire, Prowlarr · the monthly Btrfs scrub, and what the last one found
torrents	the key in `secrets.sh` · Prowlarr's container file current, running, answering, taking the key · its files yours · indexers (a warning when there are none) · podman's database · `~/Downloads` outside the snapshots
release binaries	present, every library resolves, `--version` runs · fonts
applications	Brave Origin actually starts · Claude Code · LazyVim · torrra loads and runs through its wrapper, with no key in its config · mpv in the bar · yt-dlp's JavaScript runtime · lazygit · qalc
theme	every file in sync · foot and fuzzel accept their configs · waybar config parses · GTK and the portal follow the current theme · no window buttons · default programs · Brave in the desktop's colours · Qt palette · fonts resolve (Arabic, Japanese, Chinese, Korean) · mpv's colours · pixel art drawn for both themes · the two C helpers built · the clipboard window · the wallpaper pack · the pixel pointers · the yazi file dialog · cava · the boot menu theme
live session	bar, notifications, terminal server, wallpaper, idle, automount, status readers, polkit agent, clipboard history and keeper, the update reminder · the graphical session known to systemd · the file-dialog and screen-sharing portals · the "Share" sound devices · the refresh rate is the best available · a monitor answers DDC/CI · night light (shown, never a fault: it is yours to switch off)

The *live session* area appears only while your desktop session runs (verify looks for dwl): with nobody logged in, verify checks everything else.

What you learnt

- **Super+U** (or `fedokawaii`) opens the control: twelve entries, each with what is due at its right.
- Every entry is also a command: `fedokawaii status`, `update`, `verify`, `repair`…
- On a text console the same menu appears as a numbered list.
- verify changes nothing: ✔ passed, ✘ failed (repair fixes most), ! worth knowing.
- Its outcome is kept in `/var/lib/fedokawaii/last-verify` for the menu.

Check yourself

1. Which entry puts right what verify found? → *repair*
2. Is a `!` line in verify's report a failure? → *no: worth knowing, never a failure*
3. How do you reach the control on a text console? → *log in and type fedokawaii: a numbered menu*

36. Updates and new Fedora releases

> **In this chapter**
> - know when the update season comes, and change its length
> - run the season's update, and read what it did
> - update one program between seasons
> - move to the next Fedora once a year
>
> **You need:** ch. 35.

Updating is the one chore this machine asks of you, four times a year. This chapter shows what one update does, step by step, so that its report reads like a list you recognise.

36.1 The update season

> **Word — season.** The time between two updates, chosen by you: **90 days** unless you change it. When it has passed, a notice says *the update season is here*, and *update* in **Super+U** turns red.

Two minutes after you log in, and every six hours while you stay, a quiet check reads what is known — your last update, your release's end of support, whether the next release has settled — and shows at most one notice a day, only when something is due. It never goes on the network.

Try this. Ask the check yourself, in words:

```
$ fedokawaii remind check
nothing due — the next update in 53 days (8 Dec 2026)
```

(for example) Once the season has come round, the same command says `the update season is here: 90 days since the last update (Super+U)`.

The season is in `~/.config/fedokawaii/upkeep.conf`: `EVERY` (days between updates, 90), `WAIT` (days a new release settles before the move is offered, 60), `REMIND` (`yes` or `no`). The file is written the first time you change the season; until then the defaults hold:

```
# ~/.config/fedokawaii/upkeep.conf — the update season (fedokawaii remind; Super+U → reminders)
EVERY=90     # days between updates: a reminder once they have passed (7–365)
WAIT=60       # days a new Fedora release settles before the move to it is offered (0–180)
REMIND=yes   # yes | no
```

The easiest way to change it is `fedokawaii remind`:

```
fedokawaii remind every 120   # an update season of 120 days
fedokawaii remind wait 30     # offer a new release 30 days after it is out
fedokawaii remind off         # no notices (on: back again)
systemctl --user list-timers fedokawaii-remind.timer    # when the quiet check runs next
```

In the menu, *reminders* does the same with **Enter**: each press moves the season on — every 60, 90 or 120 days, then off.

> **In SQL terms.** Think of a Power BI dataset with no scheduled refresh: it stays exactly as of its last refresh until you press *Refresh*. The season is the note on the report that says "refreshed 90 days ago — time to refresh".

36.2 The season's update

update brings everything up to date in one run, in this order:

Step	What happens	If it fails
your working copy	only when the update is run from it (chapter 42): the copy becomes the toolkit, your saved settings merged in	a copy with a link in it is refused: it stops before anything has changed
checks	10 GB free on `/` and 300 MB on `/boot`, no earlier update waiting for a restart, dnf's own state (packages left twice by an interrupted run are tidied); a clock that is not synchronised is only reported	it stops before anything has changed, and says what to do
snapshot	a numbered snapshot of `/`, *before update DATE*, checked to exist	it stops: nothing is updated without one
packages	one `dnf upgrade --refresh` — Fedora, RPM Fusion, Brave, Claude Code, the COPRs — run as a system service of its own, its output followed in the window and kept in the log: closing the window, or the session ending, cannot cut it off halfway	tried again; if a third-party repository is what fails, Fedora's own updates go ahead without it and the report says so; otherwise it stops with nothing changed
`fedokawaii.conf`	its system values applied again: keyboard, time zone, DNF, SSH, the performance profile	
the manifest	anything missing installed; a package a newer Fedora no longer has is listed, never fatal	
this machine's drivers	the firmware the kernel asked for and did not find, the CPU's microcode, Wi-Fi's and the sound card's parts — whatever this hardware needs (§30.1); device firmware updates are listed, never applied by themselves	each on its own
nothing doubled or fighting	packages left twice removed; what `dnf check` finds brought in line; services that fight the desktop's own switched off; the login made greetd's again; dependencies nothing needs removed; what a stopped run left cleared. Configuration files an update could not merge are listed in `/var/lib/fedokawaii/rpmnew`, never touched	each on its own
the desktop's own parts	dwl's wlroots, the pinned tools and fonts (§27.7), GE-Proton, the file dialog	each on its own: one that fails — GitHub down, say — is reported, the rest goes on, and the next update tries it again
torrra · Prowlarr	libtorrent to its newest under torrra; Prowlarr's newest image, then started and checked	
neovim's plugins	your current versions kept first (`~/.local/state/fedokawaii/lazy-lock/`, the last six), then `Lazy! sync`	
verify	every check, the outcome recorded for the menu	

The dnf transaction itself also gets its pre/post pair of snapshots. At the end the update says whether a new kernel waits for a restart, and notes whether the next Fedora is out. The outcome — complete, *partial* (a part to retry next time), *checks* (verify found something: run *repair*) or *stopped* — shows in the menu, and the run's last line says it too (`— update done (ok) — the next one in 90 days`); the log keeps every line (`fedokawaii log`). Not included: language servers you installed through Mason in neovim (`:Mason`, then `U`).

36.3 Recipe: update everything

1. **Super+U** → *update* (its mark reads **due** once the season has come round), or type `update`. It snapshots, upgrades every package, installs what the manifest gained, checks the pinned tools and fonts, brings GE-Proton, torrra's libtorrent and Prowlarr's image up to date, syncs neovim's plugins, and finishes with `verify` (§36.2).
2. If it reports a new kernel, restart; the one before stays installed as the way back.

3. Not included: Mason's tools in neovim (`:Mason`, then `U`). The toolkit itself stays the version you installed (§35.1).

Tip: the update clears dependencies nothing needs any more by itself (`dnf autoremove`, the list in the log); `dnf repoquery --unneeded` shows any that are left (§38.1).

36.4 Between seasons

In between, nothing changes underneath you — nothing updates, or even checks for updates, in the background:

Would otherwise	Here
DNF fetches package lists every three hours or so (its *makecache* timer: ten minutes after boot, then three hours after each run, plus up to an hour at random)	off: lists are fetched when you update (`dnf upgrade --refresh`); `systemctl is-enabled dnf5-makecache.timer` says `disabled`
neovim's plugin manager looks for new plugin versions	off (`checker = { enabled = false }` in `lua/config/lazy.lua`); plugins move only with an update, which keeps the old versions first
Prowlarr's image is replaced as soon as a new one appears	fetched by an update only; its next start uses it
tldr refreshes its pages	at most once a quarter, and only when you run it
the release tools and fonts	pinned (§27.7)

Two programs live on the internet's schedule rather than yours, and are worth updating on their own when they ask — each command touches that one program only:

When	Command
Brave reports a security fix you want now	`sudo dnf upgrade --refresh brave-origin`
YouTube stops playing in mpv, or yt-dlp says to update	`sudo dnf upgrade --refresh yt-dlp`
security fixes only, between seasons (optional)	`sudo dnf upgrade --refresh --security`

If an update breaks neovim

An update keeps the plugin versions you had in `~/.local/state/fedokawaii/lazy-lock/` (the last six), as *lockfiles*: lists of every plugin and the exact version it was at. To go back:

1. Find the newest — the versions from just before the last update: `ls -t ~/.local/state/fedokawaii/lazy-lock/ | head -n 1`
2. Copy it over neovim's own lockfile, for example `cp ~/.local/state/fedokawaii/lazy-lock/lazy-lock.20261015-0852.json ~/.config/nvim/lazy-lock.json`
3. Open neovim and run `:Lazy restore`: every plugin goes back to the version in the file.

Anything else an update changed, snapper undoes (§37.2): every `dnf` run is wrapped in a pre/post pair, and the update itself starts from a snapshot.

Read more: lazy.nvim, Lockfile.

36.5 A new Fedora — upgrade

A new Fedora comes about every six months, in April and October (Fedora 43 on 28 October 2025, Fedora 44 on 28 April 2026); each is supported for about thirteen, until four weeks after the release two later — Fedora 44 until 2 June 2027. The menu shows your release's date from Fedora's own record of it, the line `SUPPORT_END=YYYY-MM-DD` in `/etc/os-release` (`grep SUPPORT_END /etc/os-release` shows it), and offers the move when it is sensible:

1. **Out.** A release is out once Fedora publishes its release tree. An update finds that out, and so does opening the menu — one request to Fedora's own servers, at most once a day.

2. **Settled.** The move is offered `WAIT` days (60) after the release day, once its first fixes are in — sooner when your release ends within 60 days. `fedokawaii upgrade --now` does not wait.
3. **Ready.** When you choose it, the move checks first, and stops with nothing changed unless RPM Fusion already has the new release (ffmpeg and the drivers come from there), there are 20 GB free, and every library dwl needs is in it (one that a tool of the desktop's own needs is reported). A COPR that has not built for the new release yet is left out, its packages kept as they are, until it has.
4. **The move.** Your release fully updated, a snapshot of `/`, everything downloaded — should dnf report conflicts, it lists what it would remove and asks — then a restart into Fedora's own installation: 20–40 minutes with nothing else running. Leave the computer on.
5. **After.** Once, by itself, after the restart: the manifest again, packages left twice tidied, torrra for the new Python, the theme (dwl rebuilt if needed), a new baseline snapshot for the new release, the checks — and the outcome, told once in a notice when you next log in (nobody is logged in while it runs). Configuration files the new packages could not merge (an `.rpmnew` beside yours) and packages no longer in any repository are listed in the status, for you to look at — never changed by themselves.

If the installation fails, the computer restarts into your old release as it was, and the notice at your next login says so. If the new release starts but the desktop does not, the boot menu still has your previous kernel, and the snapshot taken before the move holds the old system (§37.2).

Underneath, the move is Fedora's own *offline upgrade*: everything is downloaded first, then installed in a special start before anything else runs. `fedokawaii upgrade` puts the checks above around these two commands of dnf's, so you never type them yourself:

```
sudo dnf system-upgrade download --releasever=45   # all of Fedora 45; nothing installed yet
sudo dnf system-upgrade reboot                      # restart into the installation
```

Read more: Upgrading Fedora Linux using DNF.

36.6 Recipe: move to a new Fedora

1. Wait for the menu: *upgrade to Fedora N* reads **ready** once the new release has settled (and a reminder says so). Before then it shows when it will be offered.
2. Have an hour, and 20 GB free (the status shows the space). Close what you are working on.
3. **Super+U** → *upgrade to Fedora N*, Enter, your password. It checks everything first and stops — nothing changed — if something is not ready yet, saying what.
4. If dnf reports conflicts it lists the packages it would remove and asks; *y* goes on (any the desktop needs come back afterwards), anything else stops with nothing changed.
5. The computer restarts into the installation: 20–40 minutes, then the login. A minute or two later a notice says the move is finished; the status lists anything to look at.

36.7 Built to be left alone

The toolkit is never edited after it is installed, so whatever changes around it over the years is handled one way — checked for, worked around, or reported, never a silent break:

In time	What happens
Fedora moves to a newer wlroots and drops the one dwl is written for	dwl uses its own copy, kept from Fedora's build at the first install (`/usr/local/lib/fedokawaii-wlroots-0.20`); a move checks the new release has every library under it
dwl cannot start — a library missing, or it stops within seconds	the login starts the build before it (`dwl.prev` , a notice says so); with neither, a rescue screen on the console says what to do, instead of a login that loops back
a new kernel misbehaves	the one before it stays installed, in the boot menu
a package is renamed or retired	listed, never fatal: the rest goes on without it

In time	What happens
a third-party repository is down, changes its key, or has nothing yet for a new release	skipped with a warning; Fedora's own updates never wait on it
a release tool's download changes	refused by its checksum; the version in place stays
a run is interrupted — the window closed, the power cut	dnf runs apart from the window; the next run tidies a half-finished transaction; every record is written whole or not at all
the disk fills	every run checks for room first; snapshots stay within bounds (a fifth of the disk stays free)
dnf's clean-up (`autoremove`) would take a package the desktop needs	never: every package in the manifest is marked as yours. Nothing is *protected* by name, so a later Fedora can still replace one (say, sudo by sudo-rs) without holding back every update
a tool's output changes its language or format	setup, verify and the `fedokawaii` command read it in one fixed language (`LC_ALL=C.UTF-8`), whatever yours is
two runs at once	one at a time; the menu shows when one is going on

What you learnt

- The season is 90 days unless you change it (`fedokawaii remind every DAYS`); a notice says when it has come.
- *update* takes a snapshot, upgrades every package, brings the desktop's own parts up to date and ends with verify.
- Between seasons nothing updates by itself; one program at a time with `sudo dnf upgrade --refresh NAME` .
- A new Fedora is offered 60 days after its release; the move restarts into an installation of 20–40 minutes, then the login.
- What changes over the years is checked for, worked around or reported — never a silent break.

Check yourself

1. Which command says, in words, whether an update is due? → *fedokawaii remind check*
2. What does an update take before it changes anything? → *a snapshot of /, checked to exist*
3. YouTube stops playing in mpv between seasons. What do you update? → *yt-dlp:* `sudo dnf upgrade --refresh yt-dlp`

37. Snapshots: going back in time

> **In this chapter**
> - picture a snapshot, and say what it is not
> - list the snapshots and see what changed between two of them
> - undo one transaction, one file, or a whole bad update
> - read how many snapshots are kept, and change it
>
> **You need:** §23.5 (how Btrfs makes them) and §36.2 (the update takes one first).

This is the safety net under everything else in Part V: whatever an update or an experiment changes, a snapshot from just before it still has the old files.

37.1 What a snapshot is, with one example

A **snapshot** (§4.2) is a subvolume frozen at one moment. Part IV showed why it is instant and nearly free: Btrfs never writes over old blocks, so the snapshot keeps pointing at them while the live files move on (§23.5). One file, three steps:

Time	What happens	`~/Documents/expenses.ods` now	in the 10:00 snapshot
10:00	the hourly snapshot of `/home` is taken	40 rows	40 rows — the same blocks, no extra space
10:20	you delete a month by mistake, and save	30 rows	still 40 rows
10:25	you copy the file back out of the 10:00 snapshot (§37.2)	40 rows again	40 rows

The rule: a snapshot holds every file of its subvolume as it was, and costs space only for what has changed since.

> **On Windows.** A snapshot is a System Restore point that covers your files too, and *Previous versions* in a file's properties, in one: made in an instant, every hour and before every change of software, on the same disk.

What a snapshot is not: a backup. It lives on the same disk as the files it protects. If that disk dies, is stolen or is wiped, the snapshots go with it.

> **Word — backup.** A copy of your files on **another** disk, or another place altogether — an external drive, a cloud drive. Snapshots undo mistakes; only a backup survives the loss of the disk.

So both: snapshots for "undo", a backup for "the disk is gone" (the seasonal routine in §38.4 ends with one).

37.2 Snapshots on this desktop

snapper does here what System Restore and File History do on Windows: it keeps earlier states of the system so a change can be undone. Setup made two configurations, both Btrfs snapshots:

Config	Covers	Snapshots kept in
`root`	`/` — the system, `/etc`, `/usr`, `/var`	`/.snapshots/`
`home`	`/home` (only when it is its own Btrfs subvolume, as the installer makes it) — **except** `~/Downloads`	`/home/.snapshots/`

Try this. List the snapshots of `/` — `snaps` is the desktop's short name for `sudo snapper -c root list`:

```
$ snaps
 # | Type   | Pre # | Cleanup  | Description
---+--------+-------+----------+-------------------------
 0 | single |       |          | current
 1 | single |       |          | fedokawaii baseline
40 | single |       | number   | before update 2026-10-15
41 | pre    |       | number   | dnf install hyperfine
42 | post   |    41 | number   | dnf install hyperfine
57 | single |       | timeline | timeline
```

(for example; shortened to the columns that matter here)

Column	It means
`#`	the snapshot's number — what every other command names it by; `0` is the system as it is now
`Type`	`single` on its own; `pre` and `post` a pair around one dnf run (`Pre #` links them)
`Cleanup`	the rule that will delete it one day; empty: kept until you delete it
`Description`	why it was taken: the dnf command, the update's date, the hour

What makes a snapshot

Trigger	Config	Cleaned up by
every DNF transaction: a *pre* before and a *post* after, described by the command that ran (`dnf install htop`; the date when the command cannot be read). dnf5's actions plugin runs `/usr/local/sbin/kawa-snap` from `/etc/dnf/libdnf5-plugins/actions.d/fedokawaii-snapper.actions`	`root`	the *number* rule
every hour (`snapper-timeline.timer`)	`root` and `home`	the *timeline* rule
before every update: a single snapshot, *before update YYYY-MM-DD*, marked important; `fedokawaii update` checks that it exists before it changes anything	`root`	the *number* rule, counted among the important ones — so a busy month of dnf runs never pushes it out
before every move to a new Fedora: *before Fedora N*, marked important	`root`	the *number* rule, counted among the important ones
the *fedokawaii baseline*, taken once after the verified build — and again for each new Fedora	`root`	none — kept until you delete it, or until the next release's baseline replaces it (then the *number* rule)
`sudo snapper -c CONFIG create -d "…"`	either	none — kept until you delete it

`snapper-cleanup.timer` applies the rules once a day; how many of each kind it keeps is in §37.3.

The baseline has no cleanup rule, so the automatic cleanup never removes it: it stays as the known-good state of the freshly built system until you delete it yourself, or until a move to the next Fedora takes a new one (*fedokawaii baseline (Fedora N)*) and hands the old one to the numbered cleanup. To keep any other snapshot for good the same way: `sudo snapper -c root modify --cleanup-algorithm "" NUMBER` (the number from `snaps`).

Looking and comparing

Command	Does
`snaps`	the list for `/` (an alias for `sudo snapper -c root list`)
`sudo snapper -c root list --type pre-post`	only the DNF pairs, one line each
`sudo snapper -c home list`	the list for `/home`
`sudo snapper -c root status 41..42`	every file changed between two snapshots
`sudo snapper -c root diff 41..42 /etc/some.conf`	the change inside one file
`sudo snapper -c root status 41..0`	everything changed since 41 — `0` is the system as it is now

In `status`, each line starts with six characters: `+` created, `-` deleted, `c` content or `t` type changed, then `p`, `u`, `g`, `x` and `a` for permissions, owner, group, extended attributes and ACL; `.` means unchanged. For a pair around a transaction that installed `htop` (say 45 and 46), shortened:

```
$ sudo snapper -c root status 45..46
+..... /usr/bin/htop
c..... /usr/lib/sysimage/rpm/rpmdb.sqlite
```

> **In SQL terms.** A snapshot is time travel: Trino reads an Iceberg table as it was with `SELECT … FROM t FOR TIMESTAMP AS OF …`. `status 41..0` is the diff between that old state and now — the rows (here, files) added, removed and changed.

Undoing

You want	Do
undo one DNF transaction	`sudo snapper -c root undochange 41..42`, reboot, `verify` (§37.5)
one system file as it was	`sudo snapper -c root undochange 41..42 /etc/some.conf`
one of your files as it was	copy it out of the snapshot: `sudo ls /home/.snapshots/57/snapshot/$USER/` to find it, then `sudo cp -a /home/.snapshots/57/snapshot/$USER/Documents/letter.odt ~/Documents/` (`-a` keeps you as its owner)
a file deleted by mistake	the same, from the newest hourly snapshot that still has it
the whole system back to an earlier state	`sudo snapper -c root undochange 41..0` — every file changed since 41 goes back. There is no one-step rollback on this layout (§37.4): the boot entries name the `root` subvolume, so `snapper rollback` would boot the same system again

A whole-system undo is the same tool as undoing one transaction, only wider: it puts files back and runs no package scripts, and `/boot` (the kernels) is not in the snapshots. Reboot and run `verify` afterwards.

> **Careful.** `undochange N..0` puts back **every** file changed since N — including the ones you changed on purpose. Read `status N..0` first, and undo a single file where one is enough.

Before a risky change — a new repository, a driver, a big edit in `/etc` — take your own:

```
sudo snapper -c root create -p -d "before new repo"   # -p prints its number, say 58
sudo snapper -c root status 58..0                    # later: everything changed since
sudo snapper -c root undochange 58..0                # if it went wrong: all of it back, then reboot
```

A snapshot made this way has no cleanup rule: delete it yourself once the change has proved itself (§38.4).

Deleting

`sudo snapper -c root delete 30-40` (a range) or `delete 31 35`; the same with `-c home`. Delete a *pre* together with its *post*. Btrfs frees the space in the background; `delete --sync` waits until it is free. Snapshots hold every file that was there when they were taken, so deleting a large file frees nothing until the last snapshot holding it is gone:

Step	Space used
you save a 4 GB video in `~/Videos`	4 GB
the hourly snapshot of `/home` is taken	still 4 GB: the snapshot shares the same blocks
you delete the video	still 4 GB: the snapshot holds it
the last snapshot holding it is cleaned up — up to two months later if a monthly one caught it, sooner when space is short	4 GB free again

Not in any snapshot:

- `~/Downloads` — its own subvolume (setup made it while the folder was empty; `verify` warns if it is not). A Btrfs snapshot never includes a subvolume nested inside it.
- `/boot` and `/boot/efi` — separate partitions; the previous kernel is the safety net (§29.5).
- everything else in your home **is** in the `home` snapshots: `~/.cache`, Steam's games, podman's images. Deleting them frees their space only as the snapshots holding them expire.

To keep another big folder out the way `~/Downloads` is, make it a subvolume while it is still empty — as yourself, with no `sudo`, the way setup made `~/Downloads`:

```
btrfs subvolume create ~/Games   # a new, empty subvolume, owned by you
restorecon ~/Games               # its SELinux label, as for any folder in your home
stat -c %i ~/Games               # prints 256: the top folder of every subvolume has inode 256
```

To remove such a subvolume later, empty it and `rmdir` it, as with a plain folder.

Read more: snapper(8) · snapper-configs(5) · Btrfs subvolumes · Working with Btrfs – Snapshots (Fedora Magazine).

37.3 How many are kept

snapper keeps timeline snapshots of `/` (and `/home` when it is its own subvolume), a *baseline* taken right after the verified build, a pre/post pair around every DNF transaction, and one before every update and every move. For a machine left alone for months they stay within bounds — far fewer than snapper's own defaults. Setup gives both configs the same limits:

Kind	Setting	snapper's default	Here
numbered: the DNF pairs, old baselines	`NUMBER_LIMIT`	50	`4-20`: the newest 20; down to 4 when space is short
numbered and *important*: *before update*, *before Fedora N* — counted apart	`NUMBER_LIMIT_IMPORTANT`	10	`4-10`: the newest 10; down to 4
hourly: the first snapshot of each hour	`TIMELINE_LIMIT_HOURLY`	10	`2-8`: the last 8; down to 2
daily: the first of each day	`TIMELINE_LIMIT_DAILY`	10	`2-7`: a week; down to 2
weekly	`TIMELINE_LIMIT_WEEKLY`	0	`0`: none
monthly: the first of each month	`TIMELINE_LIMIT_MONTHLY`	10	`0-2`: two; down to none
yearly	`TIMELINE_LIMIT_YEARLY`	10	`0`: none

A range such as `4-20` means: keep twenty, and when space runs short go down as far as four. Space is short when less than a fifth of the disk is free (`FREE_LIMIT=0.2`), or when the snapshots hold more than half of it (snapper's default `SPACE_LIMIT`). The cleanup runs once a day and never removes a snapshot younger than an hour (snapper's `NUMBER_MIN_AGE` and `TIMELINE_MIN_AGE`, 3600 seconds). The baseline stays until you delete it — or until the next Fedora: a move takes a new baseline for the new release, and the old one then ages out with the numbered ones. `~/Downloads` is a subvolume of its own (when it was still empty at setup; `verify` warns otherwise), so downloads stay out of `/home`'s snapshots: the space of a deleted download comes back at once.

Read the limits with `sudo snapper -c root get-config`. Change one with, for example:

```
sudo snapper -c root set-config NUMBER_LIMIT=4-30     # a range, or low space no longer trims it
sudo snapper -c home set-config TIMELINE_LIMIT_DAILY=2-5 TIMELINE_LIMIT_MONTHLY=0-1
sudo snapper -c home set-config TIMELINE_CREATE=no    # no hourly snapshots of /home at all
```

`fedokawaii repair` sets the limits back to setup's (`snapper_limits` in `setup.sh`); to keep yours, change them there, in your working copy, and apply. It leaves `TIMELINE_CREATE` as you set it.

Read more: snapper-configs(5).

37.4 snapper at a glance

The chapter on one page. snapper keeps snapshots of `/` and `/home`: one every hour, thinned out over time; one before each update and each move; the *fedokawaii baseline*; and a **pre / post pair around every dnf transaction**. A snapshot costs almost nothing until files change after it. `~/Downloads`, `/boot` and `/boot/efi` are not in them (§37.2).

Command	Does
`snaps`	list the snapshots of `/` (`sudo snapper -c root list`)
`sudo snapper -c root list --type pre-post`	only the dnf pairs, one line each
`sudo snapper -c root status 41..42`	which files the transaction changed: `+` created, `-` deleted, `c` content, `p` permissions
`sudo snapper -c root diff 41..42 /etc/some.conf`	what changed inside one file
`sudo snapper -c root undochange 41..42`	**put back every file the transaction changed**
`sudo snapper -c root undochange 41..42 /etc/some.conf`	put back one file only
`sudo snapper -c root create -d "before trying X"`	a snapshot of your own. It has no clean-up rule, so it stays until you delete it; add `-c number` to let it age out with the dnf pairs
`sudo snapper -c root delete 30-40`	delete a range
`sudo snapper -c home list`	the snapshots of `/home`

Recommended. Before an experiment — a new repository, a driver, a big config change — take a snapshot of your own with a description. If it goes wrong, `status` from that number to the newest one shows everything that changed, and `undochange` puts it back. §37.5 walks through undoing a bad update.

Pitfalls.

- **`undochange`, not `rollback`.** `snapper rollback` works by changing Btrfs's *default* subvolume, but Fedora's boot entries name the subvolume outright (`rootflags=subvol=root` — check with `cat /proc/cmdline`), so after a rollback the same old `root` would boot again. Undo with `undochange`.
- `/boot` is a separate partition and is **not** in the snapshots. For a kernel that misbehaves, pick the previous one in the boot menu (if it is hidden, hold **Shift**, or press **Esc** or **F8**, as the machine starts) — it stays installed beside the new one (§29.5).

- snapper's own manual warns that consistency after `undochange` is not guaranteed: it puts files back; it does not re-run package scripts. Reboot afterwards, then run `verify`.
- Deleting files does not free space while snapshots still hold them (§38.3).

Tip: `sudo snapper -c root status N..0` shows everything changed since snapshot N — the quickest way to see what an experiment really touched (§37.2).

Read more: `man snapper` and `man snapper-configs`, also at http://snapper.io/manpages/snapper.html.

37.5 Recipe: undo a bad update

1. `sudo snapper -c root list --type pre-post` — find the pair for the update, by its date: a *pre* snapshot taken just before the dnf run and a *post* just after. Say they are 41 and 42.
2. `sudo snapper -c root status 41..42 | less` — read what it changed.
3. `sudo snapper -c root undochange 41..42` — every one of those files goes back.
4. Reboot, then `verify`.
5. The next `update` would bring the same packages back, because it upgrades everything. Until a fixed version arrives, upgrade with `sudo dnf upgrade --exclude=PACKAGE` instead.

If the problem is the new kernel, use the boot menu instead (§29.5). If the desktop does not start at all, log in on a text console (**Ctrl**+**Alt**+**F3**) and run the same commands there (§39.4).

What you learnt

- A snapshot is a subvolume frozen at one moment: instant, and costing only what changes after.
- A snapshot is not a backup: it lives on the same disk.
- `snaps` lists them; `status A..B` shows what changed; `undochange A..B` puts it back.
- One of your own files comes back with a plain copy out of `/home/.snapshots/N/snapshot/`.
- The cleanup keeps 4–20 numbered, 4–10 important, a few hourly, daily and monthly ones; the baseline and your own stay until you delete them.

Check yourself

1. Your disk dies. Do the snapshots help? → *no: they are on the same disk — only a backup does*
2. Which command shows every file changed since snapshot 58? → *`sudo snapper -c root status 58..0`*
3. Why `undochange`, not `snapper rollback`? → *Fedora's boot entries name the root subvolume, so a rollback would boot the same system again*

38. Keeping it clean

> **In this chapter**
> - see what is installed and why, and let unneeded packages go
> - find what fills the disk, and free it safely
> - keep the journal small, and know which caches may go
> - follow a seasonal routine of a quarter of an hour
>
> **You need:** ch. 37 (snapshots hold deleted files) and ch. 27 (packages).

Chapter 35 keeps the machine working. This one keeps it tidy: how to keep the package list short, how to find and free disk space, and what to do once a season. Nothing here runs by itself unless it says so.

Words used in this chapter. Part IV taught them; here they are again in one line each:

Term	In plain words
Btrfs · subvolume · snapshot · copy-on-write	the file system here; its separately snapshotted trees; their frozen copies; why those copies are free until files change (§23.5)
dependency	a package installed because another package needs it, not because anyone asked for it (ch. 27)
user-installed	a package asked for by name: by you, or by setup from the manifest
autoremove	`dnf autoremove`: removes the dependencies that no user-installed package needs any more
journal	systemd's log of the system and of your own services, read with `journalctl` (§28.5)
cache	files a program can make again: deleting them costs time, not work (a few programs keep more there: §38.2)
XDG base directories	the freedesktop.org rule for where programs keep settings, data, state and cache (§23.3)
drop-in folder	a `NAME.d/` folder beside a config file: each file in it adds or overrides settings, and the main file stays untouched (§28.3)

> **On Windows.** *Storage Sense* can empty the Recycle Bin and temporary files by itself. Here little is cleared by itself — old snapshots, and unneeded packages at each update; the rest is a short list you run once a season (§38.4).

38.1 Keeping packages lean

Every package you want lives in `lib/manifest.sh`, and `verify` fails on anything user-installed beyond the manifest and what was there before setup. That rule keeps the list honest; these keep the disk lean.

Command	Does
`dnf repoquery --userinstalled`	packages asked for by name, or brought by a group such as the installer's; the rest came in as dependencies. `verify` compares this list with the manifest and its record from before setup
`dnf repoquery --unneeded`	what `sudo dnf autoremove` would remove: a preview that changes nothing
`sudo dnf autoremove`	removes dependencies nothing needs any more. `update` and `repair` run it for you (in a snapshotted dnf run, the list in the log); by hand, read the list before saying yes. Manifest packages are user-installed, so it never touches them
`dnf repoquery --installed --whatrequires NAME`	which installed packages require NAME
`rpm -qf FILE`	which package a file belongs to, for example `rpm -qf /usr/bin/foot`

Command	Does
`sudo dnf clean all`	empties DNF's download and metadata caches in `/var/cache/libdnf5` (they come back at the next upgrade). `dnf` run without `sudo` keeps its own copy in `~/.cache/libdnf5`; `dnf clean all` without `sudo` empties that
`rpm -q kernel-core`	two kernels at most: the newest, and the one before it; `uname -r` names the one running

Kernels

Two at most: the newest and the one before it, kept as the way back should a new one misbehave (§29.5). DNF removes the oldest by itself when a third arrives — never the one running — and `dnf autoremove` never removes a kernel; there is nothing to clean by hand:

```
rpm -q kernel-core   # two lines, one per kernel
uname -r             # the one running: normally the newest
```

The rest of what this desktop installs

What	Where	Clean with
torrra (pipx)	`~/.local/share/pipx/venvs/torrra`	nothing to do: torrra stays at its version, and `update` moves libtorrent under it to the newest. `pipx list` shows it; do not `pipx uninstall` it — `verify` checks it
Prowlarr's image	podman's storage in `~/.local/share/containers`	`podman system df` shows the size; `podman image prune` removes *dangling* images — older versions left without a name when a newer one was pulled (`update` already runs it). Not `podman image prune -a`: Prowlarr's container exists only while it runs, so that would delete its current image too
GE-Proton	`~/.local/share/Steam/compatibilitytools.d`	nothing: `update` keeps the newest two
games	`~/.local/share/Steam/steamapps`	Steam's *Settings → Storage*: uninstall games there. Each game's Proton prefix (`steamapps/compatdata`) and shader cache (`steamapps/shadercache`) can grow to gigabytes; a folder there for a game you no longer have can go
neovim's tools	Mason, in `~/.local/share/nvim/mason`	`:Mason`, `X` on a tool you no longer use

38.2 Disk space and logs

How full is it? On Btrfs, `df` and `du` cannot tell what the snapshots hold: `du` counts a file in full even when a snapshot shares its blocks, so neither says what deleting something would free. Ask Btrfs:

Command	Shows
`sudo btrfs filesystem usage /`	the real picture: *Free (estimated)*, and *Device unallocated*
`sudo btrfs filesystem du -s ~/Downloads ~/.local/share/Steam`	how much of a folder is its own and how much is shared with snapshots
`du -xh --max-depth=1 ~ | sort -h | tail`	the biggest folders in your home (`-x` stays on one subvolume, so it skips `~/Downloads`; run it there separately)
`fd -HI -S +1g . ~`	single files of 1 GB or more (`-H`: hidden ones too; `-I`: also those a `.gitignore` hides, such as build folders in `~/repos`)

Try this. Ask Btrfs how full the disk is, then how much of two folders is their own:

```
$ sudo btrfs filesystem usage /
Overall:
    Device size:                 931.51GiB
    Device allocated:            212.02GiB
    Device unallocated:          719.49GiB
    Free (estimated):            741.22GiB      (min: 381.47GiB)
$ sudo btrfs filesystem du -s ~/Downloads ~/.local/share/Steam
     Total   Exclusive  Set shared  Filename
  38.40GiB    38.40GiB       0.00B  /home/you/Downloads
  61.20GiB     2.10GiB    59.10GiB  /home/you/.local/share/Steam
```

(for example; shortened)

Free (estimated) is roughly what you can still write. *Device unallocated* is space Btrfs has not yet handed to data or metadata (see *balance* below). *Exclusive* is what deleting the folder would free at once: all of `~/Downloads`, which no snapshot holds, but little of Steam's — the rest is shared with the `home` snapshots and comes back only as they expire.

`compsize`, `gdu` and `ncdu` are not in the manifest; `du`, `btrfs filesystem du` and `fd` do the job.

The journal

The journal is kept on disk in `/var/log/journal`. It stays within systemd's default limits — at most 10 % of the file system, while leaving 15 % of it free, each capped at 4 GB; nothing here changes them.

Command	Does
`journalctl --disk-usage`	how much the journal uses: `Archived and active journals take up 1.1G in the file system.`
`sudo journalctl --rotate --vacuum-time=4weeks`	close the files in use, then delete entries older than four weeks
`sudo journalctl --rotate --vacuum-size=500M`	the same, down to 500 MB

Vacuuming deletes only archived (closed) journal files; `--rotate` closes the active ones first, so their old entries can go too.

For a lasting limit, write a drop-in of your own:

```
sudo mkdir -p /etc/systemd/journald.conf.d        # the folder may not exist yet
sudoedit /etc/systemd/journald.conf.d/50-size.conf
```

```
[Journal]
# the most the journal may use on disk
SystemMaxUse=500M
```

`sudo systemctl restart systemd-journald` applies it; `journalctl --disk-usage` shows the result.

Read more: journald.conf(5) · journalctl(1).

Caches you can clear

What	Clear with	Comes back
scaled wallpapers and picker previews	`rm -r ~/.cache/kawa-wallpaper`	at the next wallpaper change, or `kawa-wallpaper --restore`
thumbnails other programs share (the freedesktop.org folder; yazi keeps its own previews under `/tmp`, emptied at every restart)	`rm -r ~/.cache/thumbnails`	when a program needs them again
the clipboard history	Super+V, then Ctrl+X (or `cliphist wipe`)	from your next copy
the launcher's counts (a single file)	`rm ~/.cache/fuzzel`	the order of programs starts afresh
book covers and page counts	`rm -r ~/.cache/kawa-books`	as the books window (Super+R) draws them again
old config backups	`rm -r ~/.local/state/fedokawaii/backup/<time>`	— keep the newest until you are sure
the trash	**g t** in yazi → *Empty trash bin*, or `gio trash --empty` (`gio trash --list` shows what is there first)	—
DNF's caches	`sudo dnf clean all`	at the next upgrade

Clear single folders while you are logged in; the logs (`dwl.log`, `waybar.log`) are open by running programs and are best left alone.

Btrfs care

TRIM tells the SSD which blocks are free, so it can keep writing at full speed. Btrfs does it as it goes (`discard=async`, its default on SSDs since Linux 6.2), and `fstrim.timer` catches up once a week. Two more tools exist; neither is needed often:

- **scrub** reads every block and checks it against its checksum, to find a failing disk early. The Btrfs documentation suggests once a month, and a timer does exactly that (`fedokawaii-scrub.timer`, at idle disk priority); `verify` reports what the last one found. By hand: `sudo btrfs scrub start /`, then `sudo btrfs scrub status /`. On one disk it can repair only what Btrfs keeps two copies of — normally its metadata (`sudo btrfs filesystem df /` shows *DUP* there). Damaged file data it can only report; the copy to restore is your backup.
- **balance** rewrites block groups — the chunks Btrfs hands out to data and metadata — so the free space inside half-empty ones becomes *unallocated* again. It is almost never needed: only when `btrfs filesystem usage /` shows *Device unallocated* close to zero while data is far from full. Then a gentle one: `sudo btrfs balance start -dusage=50 /` (only data block groups less than half full). Never run a full balance as routine: it rewrites everything.

Read more: btrfs filesystem · btrfs scrub · btrfs balance · mount options.

38.3 Recipe: free disk space

1. Look first: `sudo btrfs filesystem usage /` for the real picture (`df` is not reliable with snapshots), and `du -xh -d1 ~ | sort -h | tail` for the biggest folders in your home.
2. Snapshots hold deleted files: `snaps`, then delete old ranges you do not need, `sudo snapper -c root delete 30-60`. Keep the baseline (it is never cleaned up by itself).
3. The trash: **g t** in yazi, or `gio trash --empty`.
4. Big files: `fd -u -S +1g . ~` (`-u` also looks in hidden folders and in what a `.gitignore` leaves out, such as build output in a code folder; `-H` alone skips that).

5. Caches and logs: `podman image prune`, `sudo journalctl --vacuum-size=200M`, `sudo dnf clean all`, and Steam's own *Settings* → *Storage* for games.

Tip: chapter 38 explains each of these, and which caches are safe to clear (§38.2).

38.4 A seasonal routine

Once a season, with the update (§36.2):

Step	Command
update everything	**Super+U** → update (or `update` in a terminal) — it ends with `verify` (§36.2)
check the result	the status in the same menu: every check passed, nothing listed to look at
once a year	**Super+U** → upgrade, once the menu offers the next Fedora (§36.5)
drop unneeded dependencies	nothing to do: the update does it (`dnf repoquery --unneeded` shows what is left)
look at the space	`sudo btrfs filesystem usage /`
sort the downloads	move keepers out of `~/Downloads`, delete the rest; same for `~/Pictures/Screenshots`
empty the trash	**g t** in yazi → *Empty trash bin*, or `gio trash --empty`
prune snapshots you made by hand	`snaps`, then `sudo snapper -c root delete N`
trim the journal	`sudo journalctl --rotate --vacuum-time=4weeks`
check the disk	nothing to do: a scrub runs every month; `sudo btrfs scrub status /` shows the last one
back up	copy `~/repos`, `~/Documents` and your other folders to another disk

Most of these take a minute; the update itself takes longest, and runs on its own once started.

What you learnt

- Every wanted package is in the manifest; `dnf repoquery --unneeded` previews what `autoremove` would drop, and the update drops it for you.
- On Btrfs ask Btrfs how full the disk is: `sudo btrfs filesystem usage /`; *Exclusive* is what deleting a folder would free at once.
- The journal stays within systemd's limits; `journalctl --vacuum-…` trims it further.
- Caches can go; a deleted file's space returns only when the last snapshot holding it does.
- Once a season: update, look at the space, sort the downloads, back up.

Check yourself

1. Why can `du` mislead on this disk? → *it counts a file in full even when a snapshot shares its blocks*
2. You deleted a 4 GB video and nothing was freed. Why? → *a snapshot still holds it; the space returns when that snapshot is cleaned up*
3. Which step of the seasonal routine protects against a dead disk? → *the backup to another disk*

39. When something goes wrong

> **In this chapter**
> - follow a calm method from a symptom to its cause and its fix
> - find a symptom in a table, with the log to read
> - run the desktop's own self-tests
> - recover from a text console when the desktop will not start
>
> **You need:** ch. 38, and the journal (§28.5).

Something will go wrong one day: a monitor that ignores the brightness keys, an update that stops, a desktop that will not start. None of it is a disaster if you go step by step.

39.1 A calm method

Six steps, in this order. Most problems end at step 2 or 3.

1. **What changed?** An update, a new program, a setting, a new monitor? The cause is usually the last change.
2. **Run `fedokawaii verify`.** It checks everything and names the exact piece that is wrong.
3. **Read the log.** Which one: the table at the end of §39.2. Read the lines just before the first error.
4. **Search for the exact message**, in quotes, on the web: someone has usually seen it.
5. **Undo.** A snapshot from before the change (§37.5), or the older kernel in the boot menu (§29.5).
6. **Ask, with the log.** Paste the error into a question — or hand it to Claude Code (§44.1): `journalctl -b -p err | claude -p "what stands out here?"`.

> **On Windows.** The same habits as with *Event Viewer* and *Reliability Monitor*: find the red entry, read its message, search for it. Here the logs are text, read with `journalctl` or `less`, and one command, verify, checks the whole machine.

A worked example. The symptom: the brightness keys do nothing on a new monitor.

1. What changed? A new monitor, plugged in yesterday. The old one's brightness keys worked.

2. verify. Run it from the desktop, so it checks the live session too:

```
$ fedokawaii verify
…
live session
  ✔ PipeWire + WirePlumber active
  …
  ! no monitor answered DDC/CI — switch DDC/CI on in the monitor's on-screen menu, then try Super+]
…
✔ verified — 212 checks passed, 1 warning(s)
```

(for example; shortened)

The `!` line names the cause and the fix: the brightness keys talk to the monitor over DDC/CI, a channel in the display cable, and this monitor has it switched off.

3. Fix. In the monitor's own menu (its buttons, under the screen or at the back), find *DDC/CI* — often under *Settings* or *System* — and switch it **on**.

4. Check. Press **Super+]**: the level display appears at the bottom of the screen, and the screen grows brighter. Run verify again: the `!` line has become `✔ a monitor answers DDC/CI (brightness control works)`.

Four steps, and no log needed: verify did the looking. When verify finds nothing, step 3 takes over — the next section says which log belongs to which symptom.

39.2 Symptoms, and where to look

Symptom	Look at
setup stopped	the red line on screen and `/var/log/fedokawaii.log`; fix; run it again
something feels off	`verify` — it names the exact piece
the login loops back	`~/.cache/dwl.log` (the session before: `dwl.log.old`); `sudo journalctl -b -u greetd`
a rescue screen instead of the desktop	dwl could not start (it says why): `fedokawaii repair` there, then log out (Ctrl+D) and in again
an update stopped, or did only part	the menu's status line (Super+U) and `fedokawaii log`; most causes go with `fedokawaii repair`, then update again
the move to a new Fedora was refused	it says why (not settled, RPM Fusion not ready, a library missing, too little room) — nothing changed; the release you have stays supported until the date in the status
the bar shows no workspaces	`~/.cache/waybar.log`; `pgrep -f kawa-dwlstatus`
the bar, the terminal server or the clipboard keeper vanished	they start again by themselves within a second; if one keeps failing, a notice says so once — `~/.cache/dwl.log`. mako is started again by the next notification
a theme or wallpaper change has no transition	`ls ~/.local/bin/kawa-veil`; **Super+U** → *theme, applied again* (it builds it; the build log is in the theme log)
a copy is gone after closing its program	`pgrep kawa-clipkeep`; **Super+U** → *theme, applied again*; Super+V still has it
an upload button does nothing	`verify` (the portal checks); then restart Brave — it looks for the file-dialog portal when it starts
a USB drive does not open	`lsblk -f` (is a file system there?); `pgrep -a udiskie`; open it by hand: `udisksctl mount -b /dev/sdb1` says why not; an NTFS disk from Windows: §13.7
"is in use" when ejecting	the notice names the program: close it (a terminal standing in the drive: `cd` out), then eject again
no music meter	`cava -v`; the meter appears only while an MPRIS player plays
brightness does nothing	switch **DDC/CI on** in the monitor's own menu, then try Super+]; on a laptop, `ls /sys/class/backlight` should list its screen
Brave not in the desktop's colours	it was open while the theme was applied: close Brave, then **Super+U** → *theme, applied again* (or Settings → Appearance → Theme → GTK)
torrra says Prowlarr has no indexers	type `prowlarr` in a terminal (it starts Prowlarr and opens its page) → Indexers → Add Indexer (§20.1)
the boot menu has the wrong size	after changing monitors: **Super+U** → *theme, applied again*
a wallpaper-pack picture is missing	`kawa-wallpaper --pack` when online
the theme key does nothing	run `kawa-theme` in a terminal: it says why. "current is a folder, not a link" (a backup or sync tool copied it): **Super+U** → *theme, applied again*
Print does nothing, or "Screenshot not saved"	`ls -ld ~/Pictures/Screenshots` — it must be a folder you can write to
Super+Print saves a plain shot: *not annotated: satty is missing*	satty is fetched from GitHub, so an install made offline may lack it: once online, **Super+U** → *repair*
the screen stays locked with no padlock (the lock program crashed)	Ctrl+Alt+F2, log in, then `WAYLAND_DISPLAY=wayland-0 swaylock` and type your password at the desktop again; Ctrl+Alt+F1 goes back to it
SELinux denial	`sudo ausearch -m avc -ts recent`; never disable SELinux

Symptom	Look at
a config change didn't apply	`fedokawaii drift`; foot fonts and dwl changes apply at the next login
want an old config back	`~/.local/state/fedokawaii/backup/<time>/`

Known issue	Status
foot does not reorder right-to-left text	Arabic is complete in GTK/Qt programs and Brave; in the terminal the letters show but are not reordered — a foot limitation
Steam's interface	not themeable
Brave's own pages	settings pages, the Shields panel, about:blank and error pages keep Brave's own greys; private windows keep its private colours
sharing a screen with system sound	carries the call's own sound too (an echo for the others); use a tab's sound or the Share output instead (§14.4)

Reading the logs. Most rows in the first table end in a log; these commands read them:

Log	Read it with
setup, update, repair, moves	`fedokawaii log` — `/var/log/fedokawaii.log`, opened at its end
the session: dwl and what it starts	`tail -n 40 ~/.cache/dwl.log` (the session before a crash: `dwl.log.old`)
one system service, since the last start	`sudo journalctl -b -u greetd`
one of your own services	`journalctl --user -u prowlarr -n 50` (the last 50 lines)
every error since the last start	`journalctl -b -p err`

`ausearch -ts recent` in the SELinux row means the last ten minutes.

Read more: Troubleshooting SELinux (Fedora Quick Docs) · journalctl(1).

39.3 The desktop's own self-tests

verify checks the **machine**. The project also carries tests of its own **scripts**: every helper, the engine behind the control and the installer, tried edge case by edge case. They run in a throw-away home, against stand-ins, so nothing on the machine is touched.

> **Word — stub.** A stand-in for a real program or device in a test: a fake USB drive, a fake monitor, a fake `dnf`, which answers the way the real one would. The test then checks what the script did with the answer.

Try this. Run the tests of one area, as yourself, without `sudo`. They run in any copy of the project: here, the toolkit itself, which everyone may read and only root may change (once you have a working copy, ch. 42, run them there):

```
$ cd /usr/local/lib/fedokawaii
$ bash tests/run.sh drives
test_drives: 95 passed, 0 failed

✔ all 1 test files passed
```

(for example)

You typed	It means
`bash tests/run.sh drives`	run only the tests in `tests/drives/` — the USB-drive helper's
`95 passed, 0 failed`	each check of that file, one per edge case
`✔ all 1 test files passed`	the verdict; `bash tests/run.sh` alone runs every area

Each test file builds a fresh home in a temporary folder, with stubs for the hardware and services a helper talks to — USB drives, monitors, sound, the network, dnf — and removes it afterwards. No root, no display and no network are needed; bash, jq and python3 are. One area takes about a minute or less; all of them take a good while longer.

A failed check prints one line, the case it checks and what went wrong, then the counts:

```
  ✘ CASE — WHAT WENT WRONG
test_NAME: 94 passed, 1 failed

✘ 1 of 1 test files failed: AREA/test_NAME.sh
```

(the form: the capitals stand for the details. For a comparison, WHAT WENT WRONG reads `expected [THIS], got [THAT]`: what was expected, and what came instead)

When to run them.

- **After changing a script** in your working copy (ch. 42), before you apply it: a test that fails now is a problem you did not ship.
- **Before reporting a problem**: if every test passes, the scripts behave as written, and the cause is in this machine — verify's field. If one fails, its line is the best report you can give.

`tests/drives/test_drives.sh` is the worked example to copy for a test of your own; the harness every file uses is `tests/lib/harness.sh`.

39.4 When the desktop will not start

Two things can stand between you and the desktop: the compositor cannot start, or the whole system starts badly. Both are mended from a text console — the one place that needs nothing but the kernel and a login.

The rescue screen. If dwl cannot start — a library missing, or it stops within seconds, and no earlier build of it (`dwl.prev`) starts instead — you do not get a login that loops back. After your password, the first console shows this instead:

```
  The desktop could not start: dwl is missing a library (libwlroots-0.20.so).

  This console works as usual. To put things right:

      fedokawaii repair      finds what broke and fixes it (or: fedokawaii, the menu)

  What dwl said: /home/you/.cache/dwl.log · Ctrl+D logs out.
```

(for example: the reason after the colon varies)

Under it is an ordinary prompt. Type `fedokawaii repair`, give your password, and let it finish. Then **Ctrl+D** logs out, and the login comes back: log in again.

From any text console. When there is no rescue screen — the login loops back, or the screen stays black — switch to a console of your own: **Ctrl+Alt+F3**, then log in as yourself. `fedokawaii` there shows the control as a numbered list:

```
$ fedokawaii

  FEDOKAWAII

  release  Fedora 44
  support  until 2 Jun 2027 - in 230 days
  …

   1  status
   2  update
   3  upgrade to Fedora 45
   4  verify
   5  repair
   6  snapshots
   7  theme, applied again
   8  drift
   9  settings
  10  reminders
  11  log
  12  handbook

  number (q quits):
```

(for example; shortened)

Type **5** and **Enter** for *repair*. Afterwards a key brings the list back; **q** leaves it. **Ctrl+Alt+F1** returns to the first console, where the login is.

Try this — now, while everything works, so the console is familiar on a bad day:

You do	You see	It means
Ctrl+Alt+F3	a black screen and a `login:` line	a console of its own, beside the desktop
log in, type `fedokawaii`, **Enter**	the status and twelve numbered entries	the same control as **Super+U**
1, **Enter**, then a key	the status page, then the list again	an entry runs and comes back
q, then `exit`	the login line again	you are logged out of that console
Ctrl+Alt+F1	your desktop, as you left it	the desktop runs on the first console

The older kernel. If the trouble began with a new kernel — after an update that said a new kernel waits for a restart — start the one before it. Restart; the boot menu shows for two seconds (if it is hidden, hold **Shift**, or press **Esc** or **F8**, as the machine starts). Choose the entry below the first: it carries the older kernel's number. If the desktop comes back, the new kernel was the cause; the one before stays installed as the way back (§29.5).

Undo with snapper. If a change to the system caused it, the same commands as on the desktop work at the console: find the pair with `sudo snapper -c root list --type pre-post`, read it with `status`, put it back with `undochange`, reboot, and run `fedokawaii verify` (§37.5).

Part V, looking back.

Chapter	You can now
35	run the control's twelve entries, and read verify's report
36	update once a season, and move to a new Fedora once a year
37	find, compare and undo changes with snapshots
38	keep packages lean, free disk space, and follow the seasonal routine
39	go from a symptom to a fix, and recover from a text console

What you learnt

- The method: what changed → verify → the log → the exact message → undo → ask, with the log.
- The symptoms table gives the first place to look; the logs table, the command to read it.
- `bash tests/run.sh` tests the project's own scripts against stubs, touching nothing.
- A rescue screen or a console (**Ctrl+Alt+F3**) gives you `fedokawaii repair` and the menu.
- The older kernel is one choice away in the boot menu; snapper undoes from the console too.

Check yourself

1. What is the first question of the calm method? → *what changed?*
2. Do the self-tests need root or a network? → *no: they run in a throw-away home against stubs*
3. The desktop will not start. Where do you run repair? → *on a text console (Ctrl+Alt+F3), or at the rescue screen's prompt*

Part VI — Making it yours

You use this desktop every day, and Part IV showed what runs underneath it. Part VI is about making it yours: how its look is made, how git keeps a history of every change, how to change any part so that every update keeps it, and how to learn to program here — alone and with an AI assistant. Read the chapters in order: the working copy of ch. 42 is a git repository, so ch. 41 comes first.

40. How the look is built

> **In this chapter**
> - read the two palettes and their contrast
> - follow a theme switch from one link to every program
> - change an icon, a pointer or a colour at its source
> - say why the bar costs almost nothing
>
> **You need:** ch. 9 (the theme) · §26.2 (signals) · §23.6 (links).

Chapter 9 showed the switch between the dark and the light theme. This chapter opens it up: where every colour, font, icon, pointer and transition comes from, and which file to change for each. All of it is drawn by this project from text files you can read — the palette is a table in a Python script, and the icons and pointers are grids of `#` and `.`. You change those files in your working copy (ch. 42); this chapter tells you which ones.

40.1 The two palettes

Each theme is one palette: a ground, text in three strengths, and a set of accents named after traditional Japanese colours. Both are written at the top of `tools/build-themes.py`:

	dark — *sumi* (ink)	light — *washi* (paper)
ground	`#0f0e0c` ink	`#e9e1cb` paper
text / strongest	`#cfc3a6` / `#ece3cc`	`#2b2721` / `#0f0e0c`
quiet text	`#877f6f`	`#5f594d`
bar / raised key / hairline	`#161411` / `#1f1c18` / `#2b2721`	`#dcd2b8` / `#efe8d5` / `#c2b79b`
vermilion (accent: seals, frames, fills)	`#d0583a`	`#c0462a`
vermilion for words	`#d15b3d`	`#9c3922`
beni (red) · matcha (green) · yamabuki (yellow)	`#d64b4b` · `#8e9a5b` · `#d29f45`	`#a8342a` · `#5c682d` · `#845b0f`
ai (blue) · fuji (purple) · asagi (teal)	`#5f7fa0` · `#9079a9` · `#7fb8b0`	`#3f6185` · `#6f5a8a` · `#2f6e67`

The accents keep their character on both grounds; on paper they are deepened so they keep their contrast, and on paper the terminal's "bright" colours are the deeper ones.

> **Word — contrast ratio.** How much lighter one colour is than another: from 1 : 1 (the same colour) to 21 : 1 (black on white). Body text reads comfortably from 4.5 : 1 up.

Every colour used for reading — quiet grey and vermilion included — reaches 4.5 : 1 or better against the surface it sits on (the WCAG AA level for body text). This is why vermilion comes in two shades: the accent is for seals, frames and fills, and a shade of its own is used for words. Measured on the ground:

	dark	light
text	11.0 : 1	11.4 : 1
quiet text	4.9 : 1	5.3 : 1
vermilion for words	4.8 : 1	5.3 : 1
vermilion accent	4.7 : 1	3.9 : 1 — never used for words on paper

Read more: WCAG, Contrast (Minimum).

Decoration that is meant to recede — an empty workspace key, a hairline, the terminal's deliberately faint grey for comments — sits lower on purpose. Terminal programs see the usual 16 colours, arranged the way gruvbox does it, so a program that uses "black" for faint lines and "white" for text reads correctly on paper too.

Terminal colour	usually means	dark	light
0, "black"	faint lines and backgrounds	`#2b2721`	`#d6cdb3` — a shade of paper
7, "white"	ordinary text	`#cfc3a6`	`#4a443a` — ink
15, "bright white"	the strongest text	`#ece3cc`	`#1f1c18`

gruvbox is a well-known retro colour scheme; its light version swaps these roles in the same way.

40.2 Where every app gets its colours

Try this. Ask which theme is on, three ways. Then press **Super+Shift+W** and ask again.

```
$ kawa-theme get
dark
$ readlink ~/.config/fedokawaii/current
themes/dark
$ gsettings get org.gnome.desktop.interface color-scheme
'prefer-dark'
```

You typed	It means
`kawa-theme get`	the theme switch's own answer: `dark` or `light`
`readlink ~/.config/fedokawaii/current`	where the link points: every app reads its colours through it
`gsettings get … color-scheme`	what GTK programs and web pages are told

After **Super+Shift+W** the three answers are `light`, `themes/light` and `'prefer-light'`.

All theme files live in `~/.config/fedokawaii/themes/dark/` and `…/light/`, written from one palette by `tools/build-themes.py`. **`~/.config/fedokawaii/current`** is a symbolic link (§23.6) to one of them; every app reads its colours through that link, and `kawa-theme` switches it, then tells whatever is running to look again. Replacing a link in one step switches every reader at once. `ls ~/.config/fedokawaii/current/` lists the files the apps below read.

App	Reads the theme by	On a switch
foot (and everything in it: neovim, btop, yazi, lazygit, torrra, spotify_player, bat, nmtui, the clipboard window)	both palettes in `foot.ini`; `include=` picks the one to start in	SIGUSR1 / SIGUSR2 to the foot server switches every window; neovim notices by itself
neovim	gruvbox.nvim with the palette remapped; `background` is never forced	follows the terminal (DEC mode 2031)
yazi, lazygit, bat, torrra, the clipboard window, nmtui	the terminal's 16 colours by name (yazi's `theme.toml`, lazygit's config, bat's `ansi` theme, torrra's `ansi-dark` (right on both: the palettes keep "black" a shade of the ground), `NEWT_COLORS` for nmtui)	with foot, at once
waybar	`@import` of `current/waybar.css`	SIGUSR2 (reload)
fuzzel	`include=current/fuzzel.ini`	read fresh at every menu
mako	`include=current/mako` (outside any section, as mako requires); the file carries its own sections for low-urgency, critical and level notices	`makoctl reload`

App	Reads the theme by	On a switch
GTK 4 / libadwaita	one `gtk.css` with both palettes, chosen by GTK's color-scheme media query	gsettings `color-scheme`
GTK 3	two GTK themes of its own, `Fedokawaii` and `Fedokawaii-dark` (adw-gtk3 + the palette, in `~/.local/share/themes`)	gsettings `gtk-theme` — every open window, the file dialog included
Brave	GTK mode (set by Theme): its tab strip, toolbar and menus come from the GTK theme, a few rules in it reach Brave alone (the address bar keeps Brave's own colours); the new-tab page is the ground colour (`light-dark()`); web pages follow the portal's dark / light	live
Qt	`qt6ct.conf` is a link into the theme	the link is replaced; qt6ct watches its folder
mpv	`include` of `current/mpv.conf`: on-screen text, the controller, menus	every open player loads the new file over its socket
btop	`color_theme = "fedokawaii"`: `~/.config/btop/themes/fedokawaii.theme` is a link through `current` (btop takes themes only by name)	SIGUSR2
zathura	`include ../fedokawaii/current/zathurarc`	`SourceConfig` over D-Bus
imv, presenterm	their config file is a link into the theme	next start
swaylock	`kawa-lock` passes `-C current/swaylock`	next lock
the screenshot region	`kawa-screenshot` reads `current/colors.sh`	next screenshot
wallpaper	each theme's own choice	`kawa-wallpaper --restore`
MangoHud (games)	`MangoHud.conf` in the dark palette and Departure Mono	—
the console, boot menu	always the dark theme's colours	—

The news travels as signals (§26.2). `SIGUSR1` and `SIGUSR2` are left for each program to use as it likes — foot switches its palette on them, waybar reloads its style on `SIGUSR2`. `kawa-theme` sends them only to programs that already catch them, since an uncaught `SIGUSR1` would end the program.

To change a colour: edit the palette at the top of `tools/build-themes.py` in your working copy, run `python3 tools/build-themes.py` there, then apply (ch. 42). **Super+Shift+U → colours** does the middle step for you: it runs the generator when you save, then offers to apply. A few files cannot read a generated one — dwl's `config.h` (compiled in), mpv's subtitle colours, satty, MangoHud, `fetch` and the scripts' fallbacks — so they hold the colours themselves, and every run of the generator checks that each one is still a palette colour (it stops if not).

40.3 Fonts

Font	Where	Why
Departure Mono (Nerd Font build)	bar, menus, notifications, boot menu, the game overlay	a pixel font on an 11 px grid — set at 11 px (or 22, 33), never in points, with smoothing off
Sumi Icons	every icon in the bar, menus and notifications — and, as *Sumi Icons Term*, in the prompt, fetch and the clipboard and wallpaper windows	the desktop's own icon font, built from `art/icons.txt`
PixelMplus 12	Japanese in the pixel UI (song titles in the bar, menus, notices) and in the terminal	a 12 px Japanese pixel font, so a Japanese title in the bar stays sharp
JetBrains Mono (Nerd Font)	the terminal and neovim	readable code at 11.5 pt

Font	Where	Why
Noto Sans CJK	Japanese, Chinese and Korean in programs, web pages and subtitles	a full, smooth face where text is read at length
Terminus	the text console	sharp bitmap font for the login
Noto Sans, Noto Naskh Arabic, Amiri, Liberation, Noto Color Emoji	GTK and Qt programs, documents, Arabic, emoji	

A *point* is a length (1/72 inch) that turns into a different number of pixels with each screen's scaling; a pixel size is exact, which is what a pixel font needs. Fontconfig, the library every program asks which font to use and how to draw it (§31.7), has rules in `~/.config/fontconfig/fonts.conf` switch three things **off** for the three pixel fonts (that rule comes after the general one, because in fontconfig the last matching edit wins):

Setting	What it does	Other fonts	Pixel fonts
`antialias`	smooths edges with in-between shades	on	off
`hinting`	nudges outlines onto the pixel grid	slight	off — they sit on it already
`rgba`	colour fringes (subpixel smoothing)	`rgb`	`none`

PixelMplus is drawn on a 12-pixel grid, so it stays unsmoothed only at 12 and 24 px; at any other size (Japanese inside the 11 px bar, say) it keeps the normal smoothing, which reads better than broken pixels.

Try this. Ask fontconfig which font a program gets for code, for Arabic and for Japanese.

```
$ fc-match monospace
JetBrainsMonoNerdFont-Regular.ttf: "JetBrainsMono Nerd Font" "Regular"
$ fc-match 'sans-serif:lang=ar'
…: "Noto Naskh Arabic" "Regular"
$ fc-match 'sans-serif:lang=ja'
…: "Noto Sans CJK JP" "Regular"
```

(shortened: the last two file names are cut.) Each answer is the font file, then the family and the style in quotes. `monospace` and `sans-serif` are the generic names programs ask for; `:lang=ar` adds "it must have Arabic letters".

Read more: the Arch Wiki's Font configuration.

40.4 Icons



The icons of Sumi Icons, eight times their size: each square is one pixel on screen. Here each is centred in its tile; in the font it sits on the text's middle line.

Every icon is drawn as text in `art/icons.txt` — `#` for a pixel, `.` for nothing — on the same 11 px grid as Departure Mono, ten pixels tall and centred on the middle of the text line. `tools/build-icons.py` merges each drawing into a few rectangles and writes `SumiIcons.ttf` (code points U+FE000 onward, clear of every Nerd Font), its terminal cut `SumiIconsTerm.ttf` ("Sumi Icons Term": the same drawings two pixels higher, centred on the terminal font's taller capitals) and `~/.config/fedokawaii/icons.sh`, which gives the scripts names like `$ICON_SPEAKER`. Each font carries one harmless hinting instruction, so FreeType never auto-hints it: every pixel lands exactly where it is drawn.

A drawing is a block that starts with `@name`; this is the first in the file, the torii:

```
@torii
#........#
##########
.########.
..#....#..
##########
..#....#..
..#....#..
..#....#..
..#....#..
..#....#..
```

To change an icon:

1. Edit its drawing in `art/icons.txt`, in your working copy (chapter 42). Add new icons at the end of the file — inserting one in the middle renumbers the rest.

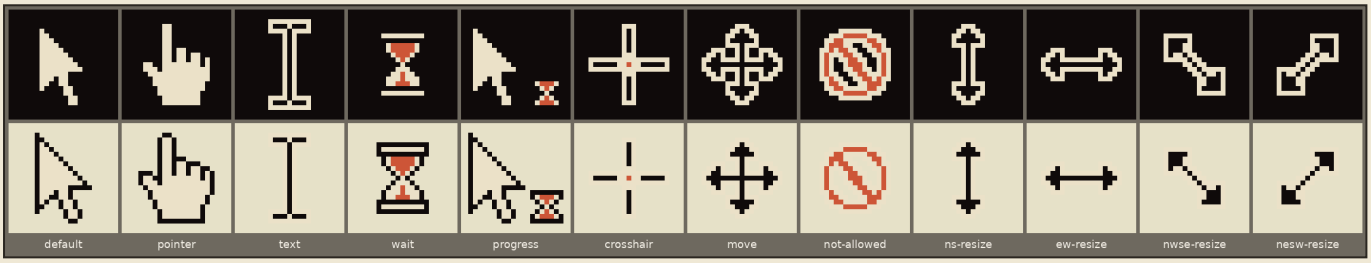
2. Build the fonts in a throwaway Python environment with `fonttools` (and `Pillow` for `--preview out.png`, a picture of every icon) — the commands below.
3. Apply it: `fedokawaii apply`, or *apply* in **Super+Shift+U** (chapter 42).

```
python3 -m venv /tmp/v                    # a private Python, thrown away later
/tmp/v/bin/pip install fonttools pillow   # the two libraries the script uses
/tmp/v/bin/python tools/build-icons.py    # writes both fonts and icons.sh
```

Super+Shift+U → icons does all three for you: it opens `art/icons.txt` in the working copy, draws the fonts when you save (fetching fontTools once, into a folder of its own), and offers to apply.

The meters are made of icons too: `eq-0` … `eq-5` are 3-pixel columns with up to five lit segments and a 4-pixel advance, and `eq-cap` has no width at all — printed after a full column in vermilion, it repaints that column's top segment. The music meter and the CPU history are built from them, and so is the level display's wedge (at twice the size); the brightness menu and the volume menu's title use the framed-bar icons `lv-*` (cells 8 px tall).

40.5 Pointers



The mouse pointers at 8× their size, on ink and on paper.

The mouse pointer is drawn for the desktop too: the theme **Fedokawaii**, in `~/.local/share/icons/Fedokawaii/`. The arrow and the hand are the classic shapes in ink with a paper body, so they read on both themes; the thin ones — the text cursor, the cross for screenshots, the arrows for moving and resizing a window — are ink lines with a one-pixel paper rim; the hourglass's sand, the cross's centre and the "not here" sign are vermilion. Each is a drawing in `art/cursors.txt` at the desktop's own pixel scale: 24 px, every pixel one screen pixel, like the fonts and icons, and 48 px with every pixel doubled for a 2× screen — so a pointer is never smoothed or blurred.

Where	What
`~/.local/share/icons/Fedokawaii/cursors/`	the twelve pointers, and links for the older names programs ask for (`left_ptr`, `hand2`, `xterm`…)
`~/.local/share/icons/default/index.theme`	makes Fedokawaii the default for dwl itself and older (XWayland) programs
`XCURSOR_THEME`, `XCURSOR_SIZE` · `cursor-theme`	the same for everything else: the session's environment (`start-dwl`) and GTK's setting (`/etc/dconf`)

Shapes not drawn here — the rarer ones, like a "copy" or "zoom" pointer — come from Adwaita underneath, through one line of the theme's `index.theme` (the comment is added here):

```
[Icon Theme]
Name=Fedokawaii
Comment=Pixel pointers in ink and paper (tools/build-cursors.py)
# any shape this theme does not draw is taken from Adwaita
Inherits=Adwaita
```

To change one: edit its drawing in your working copy, run `python3 tools/build-cursors.py` there (standard Python only; `--preview out.png` draws them all), then apply (chapter 42); the change shows at the next login.

40.6 Transitions

`kawa-veil` (C, about 750 lines in `src/kawa-veil/`, built by **Theme** with the tools that build dwl) makes the theme and wallpaper changes smooth. It takes a still of every screen (the compositor's screencopy), shows it on the topmost layer with every click passing through, runs the change underneath, waits a moment for the apps to redraw, then lets the still go block by block over about two thirds of a second, with an ease in and out. Each block is one pixel of the wallpaper pack's art on that screen (3 px at 1080p, 4 at 1440p, 6 at 4K), and a block is only ever cleared, never blended, so every frame stays crisp. The order the blocks go in is the design: an 8×8 ordered dither for a wallpaper, and for a theme a front of seigaiha crests — arcs 96 px wide at 1080p (128 at 1440p, 192 at 4K) with a dithered fringe — rising or falling across the screen. It works on rotated screens too. If anything is missing (no compositor, no screencopy) the change happens without it, and a watchdog removes the veil after 20 seconds whatever happens.

Change	Style	Takes
to the light theme	crests rising from the bottom edge (`--rise`), like dawn	about 0.65 s
to the dark theme	crests falling from the top edge (`--fall`), like nightfall	about 0.65 s
a new wallpaper	an 8×8 ordered dither (`--dissolve`)	about 0.5 s

> **Word — ordered dither.** A fixed 8×8 pattern of thresholds that decides which blocks go first, so a change spreads as an even, fine grain rather than as random noise.

40.7 The terminal's invisible marks

> **Word — OSC sequence.** *Operating System Command*: a short message a program prints to the terminal. It never shows on screen; it tells the terminal something — here, where the prompt and each command's output begin and end.

The prompt sends four, and foot acts on them:

The prompt prints	When	foot uses it for
`OSC 7 ; file://host/folder`	before each prompt	**Ctrl+Shift+N** opens the new terminal in that folder
`OSC 133 ; A`	where each prompt starts	**Ctrl+Shift+Z** / **X** jump between prompts
`OSC 133 ; C`	when a command starts	the start of its output
`OSC 133 ; D`	when it has finished	the end of its output: **Ctrl+Shift+G** sends what lies between to neovim

Read more: foot's README, *Shell integration* (codeberg.org/dnkl/foot).

40.8 fetch and the sixteen colours

`fetch` prints one of ten pictures drawn for this desktop — a beckoning cat, a daruma, a torii at sunset over the sea, a beige computer showing Mt Fuji, Fuji at dawn, an onigiri, a paper lantern, a cassette, a fox mask and a bowl of ramen — beside the system facts. `fetch NAME` picks one (`neko` , `daruma` , `torii` , `pc` , `fuji` , `onigiri` , `chochin` , `cassette` , `kitsune` , `ramen`). It is pure bash and prints in a few milliseconds.



The ten fetch pictures, on ink and on paper: two pixels per character cell, drawn with half blocks.



The terminal's sixteen colours on both themes, with bold, dim, italic, underline, reverse and strike.

40.9 Why the bar costs almost nothing

> **Word — poll.** To ask again and again, on a timer, whether something has changed. The opposite is to be told: a program sleeps until another one wakes it with the news.

The bar costs almost nothing at rest, because nothing in it polls:

Part	Wakes
workspaces, layout, title, keyboard layout	only when dwl reports a change; only that part is redrawn
clock	once a second (waybar's own clock)
CPU	every two seconds, in pure bash reading `/proc/stat` — no program is started
drive mark	only when a drive comes, opens, closes or goes: `kawa-drives` pokes it (`SIGRTMIN+13`)
music meter	its analyser (cava) runs only while a player is playing: paused, it is not running at all; silent, it sleeps

What you learnt

- Both themes come from one palette in `tools/build-themes.py`, and every colour used for reading reaches 4.5 : 1 against its ground.
- `~/.config/fedokawaii/current` is a link to one theme's folder; `kawa-theme` re-points it and tells the running programs to look again.
- The pixel fonts are set in pixels and never smoothed; `fc-match` shows which font a program gets.
- Icons and pointers are text grids (`art/icons.txt` , `art/cursors.txt`) that small Python tools turn into fonts and pointer files.
- The prompt marks itself for foot with OSC sequences, and nothing in the bar polls.

Check yourself

1. Why does vermilion come in two shades? → *the accent is too faint for words on paper (3.9 : 1), so words get a deeper shade of their own*
2. What does `readlink ~/.config/fedokawaii/current` print on the light theme? → *`themes/light`*
3. Where do you change an icon? → *its drawing in `art/icons.txt`, in the working copy; then the fonts are drawn again and applied*

41. Keeping history: git and lazygit

> **In this chapter**
>
> - make a repository and a first commit, in a terminal and in lazygit
> - stage part of a file, undo a step, put work aside
> - branch, merge, pull and push, and settle a conflict
> - find the way back from a mistake
>
> **You need:** ch. 17 (neovim) · ch. 25 (the shell).

Every change you make to this desktop deserves a history: what changed, when, why — and a way back. git keeps that history for any folder, and lazygit shows it on one screen, a key for each action. Learn it here, before ch. 42: the working copy you change the desktop in is a git repository.



lazygit in a repository with changes: files, the diff, branches and commits.

41.1 What git is

git keeps the history of a folder. The folder becomes a *repository*: you change its files as usual, and whenever a set of changes is worth remembering, you record a *commit* — a snapshot of those changes with a one-line message saying why, your name and the time. Commits line up into a history you can read, compare and return to; nothing committed is lost to later edits. Before committing you choose what goes in: *staging* a change puts it on the tray for the next commit, so one commit can hold the alias you added and not the half-finished experiment in the same file. git keeps all of this in a hidden `.git` folder inside the repository.

A *branch* is a named line of commits. The usual one is called `main`; a second branch lets you try something — a new bar layout, a rewrite of a chapter — without disturbing it, then *merge* it in if it works or delete it if not. A *remote* is another copy of the same repository somewhere else: a private repository on GitHub or Codeberg, or

another computer. *Push* sends your new commits there; *pull* brings in the commits made elsewhere — on your other machine, say. For dotfiles and notes, that means every change is recorded with its reason, any old version is a few keys away, and the same files live on every machine you use.

> **In SQL terms.** The history is an audit log: one row per commit — who, when, why — and rows are added, not edited. A branch is a copy of a table to try changes on, like `SELECT * INTO main_test FROM main`; a merge writes them back into `main`.

A small history, as `git log --oneline` prints it in a terminal — newest first, one commit per line:

```
a0044fc notes: fonts to try
cd74973 notes: a clock with seconds
7398591 notes: first ideas for the bar
```

The seven characters in front are the start of each commit's *hash*, its unique name; `git show a0044fc` prints that commit in full. lazygit's *Commits* panel is the same list.

Term	In one line
working tree	the files as they are on disk now, with edits not yet committed
staging area (index)	the tray: exactly what the next commit will hold
hash	a commit's unique name, a long hexadecimal number; its first seven characters are enough
`HEAD`	"where you are now": normally the newest commit of the branch you are on
upstream	the remote branch yours follows, such as `origin/main`: push and pull go there
fetch	download the remote's new commits without touching your files or your branch
rebase	replay your commits on top of another branch, as new commits (*Branches*, below)
reflog	git's private record of every place `HEAD` has been, which makes undo possible (*Recipes*)

Read more: chapters 1–3 of the free book Pro Git.

41.2 Once, before the first commit

```
git config --global user.name "Your Name"        # written into every commit you make
git config --global user.email you@example.com   # likewise: the address your Git host knows
git config --global init.defaultBranch main      # new repositories start on a branch called main
git config --global pull.rebase false            # pull joins two diverged histories with a merge
```

Try this. Type the four lines, with your own name and address, then read them back. They are kept in `~/.gitconfig`; git writes the names in lower case:

```
$ git config --global --list
user.name=Your Name
user.email=you@example.com
init.defaultbranch=main
pull.rebase=false
```

Without the fourth line, a `pull` on a branch that has new commits on both sides stops with `fatal: Need to specify how to reconcile divergent branches.` With it, git joins the two lines with a merge commit, as drawn under *Branches* below.

41.3 Opening it

Type `lazygit` inside any git repository, press **Space g g** in neovim (it opens the repository of the current file), or use `:` then `lazygit` in yazi. Started in a folder that is not a repository, it asks *Create a new git repository?* — `y`, then Enter for git's default branch name: the way to start keeping a folder under git (`mkdir -p ~/repos/notes && cd ~/repos/notes && lazygit`). A repository that already exists on a website comes with `git clone URL ~/repos/NAME`. Everything lazygit does is an ordinary git command, listed in its command log, so `git`, the prompt (§11.3) and neovim see the same thing.

41.4 The screen

Five panels on the left; on the right the *main view*, showing whatever the selected line stands for — a file's changes, a commit, a branch's log.

Panel	Key	Lists	More tabs, with `[ ]`
Status	1	the repository and branch; ↑ commits not pushed yet, ↓ commits waiting to be pulled	—
Files	2	every changed file, as a folder tree	Worktrees, Submodules
Branches	3	the local branches; the one you are on is marked `*`	Remotes, Tags
Commits	4	the history of the current branch, newest at the top	Reflog
Stash	5	changes put aside	—

A file's status letters are git's own, the same `git status --short` prints. Each file has two columns: the left one for the staged part, the right one for the part not yet staged.

Letters	Means
`??`	new and not yet tracked: git has never stored it
`M` · `A` · `D` · `R`	modified · added · deleted · renamed
`UU`	in conflict (*Resolving a conflict*, below)

Four files, as `git status --short` shows them:

```
MM local.sh     left M: a change staged · right M: another change, not staged yet
 M bar.md       a change, not staged (the left column is blank)
A  ideas.md     a new file, staged whole
?? todo.md      new, untracked
```

Keys	Does
1 … 5 · 0	jump to a panel · to the main view
← → or h l (Tab, Shift+Tab)	previous · next panel
↑ ↓ or k j · `<` `>`	move in the panel · to its top, bottom
Enter · Esc	go into the selected item (a file's lines, a commit's files) · back out
/ · n N	search or filter the panel · next, previous match
?	every key of the panel you are in, with what it does
`+` · `_`	a bigger main view (half, then the whole screen) · back
:	run any command — `git log --stat`, say
q	quit

Read more: lazygit's key reference, the same list **?** shows, panel by panel.

41.5 The colours here

The repository's `config.yml` names the terminal's colours instead of fixing its own, so lazygit switches with the theme along with foot:

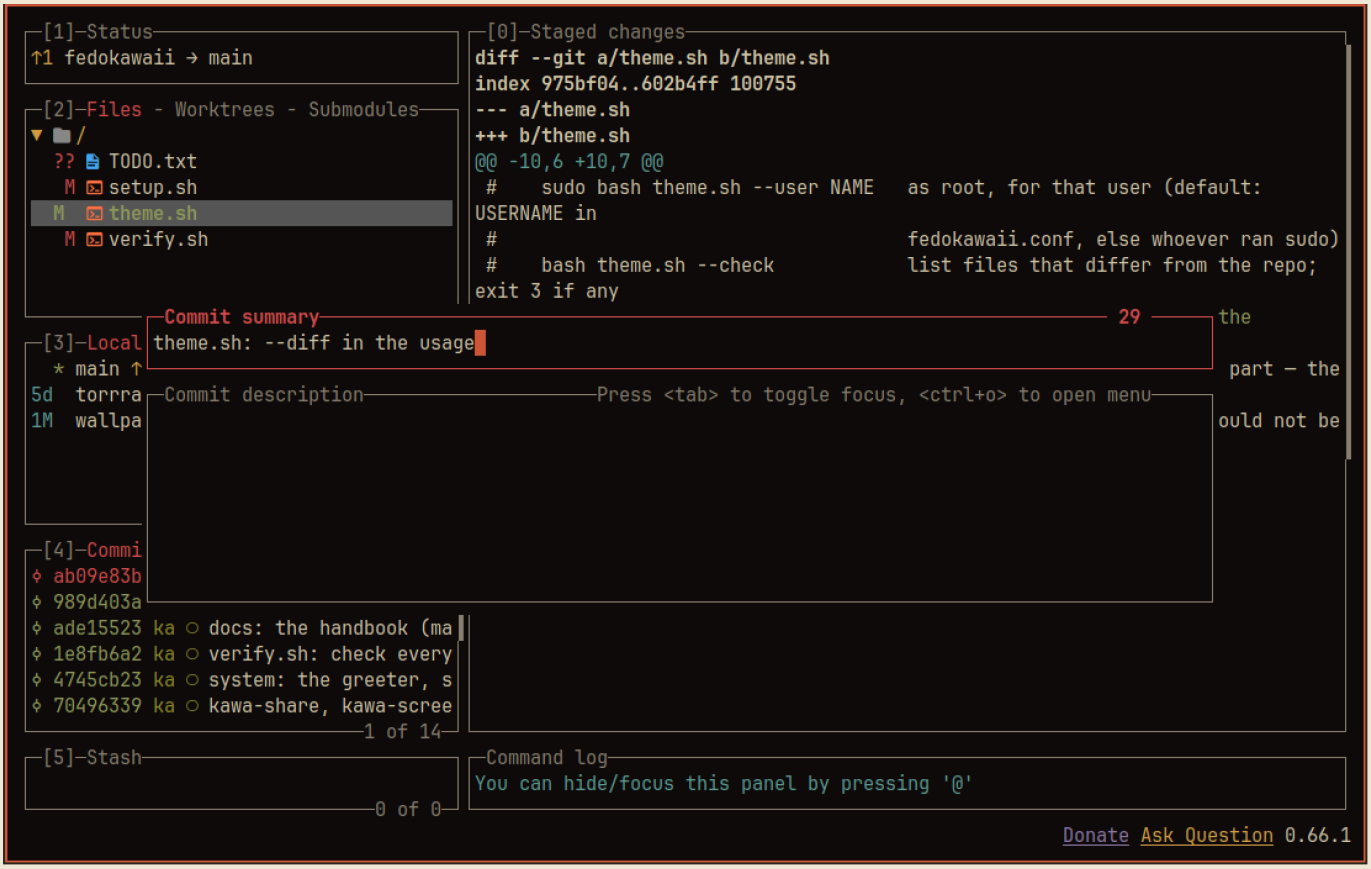
What	Colour
the frame of the panel you are in	vermilion, bold — like the frame of the focused window
the other frames and their titles	a quiet warm grey (■ `#877f6f`, one value that reads on ink and on paper)
the frame while you search with `/`	yamabuki (yellow)
the key hints along the bottom	ai (blue)
the selected line	the terminal's "black": a shade off the ground, on either theme
unstaged changes	vermilion
a commit's hash	vermilion: only here, not pushed yet · yamabuki (yellow): pushed · matcha (green): already in `main`

Frames have square corners (`border: single`), files carry Nerd Font icons, and `overrideGpg: true` keeps lazygit on screen while a signed commit is made, instead of handing the terminal to git — which suits a key whose passphrase gpg-agent already holds. Nothing here turns commit signing on; until you do, the setting has no effect. Inside neovim (Space g g), Snacks lays a colour file made from the editor's colours over this one, so the frames and the selected line follow the editor there.

41.6 A first commit, step by step

Say you keep notes in `~/repos/notes` and have just written `ideas.md`.

1. Start `lazygit` in the folder (or Space g g in neovim). *Files* lists `ideas.md` with `??`: git does not know the file yet.
2. With the file selected, **Space** stages it: its letters turn to `A`, in green. The main view shows what will go into the commit.
3. **c** opens the commit box. Type a short summary — what changed and why, in one line: `notes: first ideas for the bar` — and press **Enter**. For more, **Tab** moves to the description box below (Enter there starts a new line); Tab again goes back, and Enter in the summary commits both.
4. The file leaves *Files* — there is nothing left to commit — and *Commits* shows the new commit at the top, its hash in vermilion: so far it exists only on this machine.



The commit box: a one-line summary above, the description below, the staged file behind them.

If nothing is staged when you press **c**, lazygit asks whether to commit every change.

The same commit typed in a terminal — this is what lazygit runs for you:

```
git status --short                              # ?? ideas.md
git add ideas.md                                # stage it (Space)
git commit -m "notes: first ideas for the bar"  # commit what is staged (c)
git log --oneline -1                            # the new commit, at the top
```

Tip: before trying anything unfamiliar, commit or stash what you have. With nothing uncommitted, a step that goes wrong can be taken back with **z**, or from the reflog (*Recipes*, below).

41.7 Practice: a first commit of your own

Ten minutes in a repository you can throw away, `~/repos/practice`, typing git's own commands — the ones lazygit runs for you. Set your name first (§41.2). The hashes you see will differ from these: each commit's hash includes your name and the time.

1. Make a folder and turn it into a repository.

```
$ mkdir -p ~/repos/practice
$ cd ~/repos/practice
$ git init
Initialized empty Git repository in /home/you/repos/practice/.git/
```

If you see `Reinitialized existing Git repository`*:* the folder was a repository already. No harm is done; go on.

2. Write a file, and ask git what it sees.

```
$ echo 'rice, eggs, dates' > shopping.txt
$ git status --short
?? shopping.txt
```

`??`: a new file, never stored. *If you see `fatal: not a git repository`:* you are in another folder — `cd ~/repos/practice` first.

3. Stage the file, then commit it.

```
$ git add shopping.txt
$ git status --short
A  shopping.txt
$ git commit -m "shopping: first list"
[main (root-commit) c33e59b] shopping: first list
 1 file changed, 1 insertion(+)
 create mode 100644 shopping.txt
```

`A` in the left column: staged, ready for the commit. `root-commit` marks the first commit of a repository; `c33e59b` is the start of its hash *(for example). If you see `Author identity unknown`:* git does not know your name yet — run the lines in §41.2, then commit again.

4. Add a line, and look at the change before you commit it.

```
$ echo 'mangoes' >> shopping.txt
$ git diff
diff --git a/shopping.txt b/shopping.txt
index 8b7b6b9..9770f23 100644
--- a/shopping.txt
+++ b/shopping.txt
@@ -1 +1,2 @@
 rice, eggs, dates
+mangoes
```

The `+` line is the one you added; a line that went would start with `-`. *If you see a `-rice, eggs, dates` line:* you typed `>` instead of `>>` and replaced the list. `git restore shopping.txt` brings back the committed version; add the line again.

5. Commit the change.

```
$ git commit -am "shopping: add mangoes"
[main 196a1e6] shopping: add mangoes
 1 file changed, 1 insertion(+)
```

`-a` stages every changed file git already knows, so one command stages and commits. *If you see `nothing to commit`:* the line was not added — run step 4 again.

6. Read the history, then see the same in lazygit.

```
$ git log --oneline
196a1e6 shopping: add mangoes
c33e59b shopping: first list
$ lazygit
```

In lazygit, press **4**: *Commits* lists the same two, newest at the top. Select one, and the main view shows its change. **q** quits.

You typed	It means
`git init`	make this folder a repository: git's records go into a hidden `.git` inside it
`git status --short`	what changed since the last commit, two letters per file
`git add FILE`	stage it: put it on the tray for the next commit
`git commit -m "…"`	record what is staged, with a message that says why
`git diff`	the changes not staged yet, line by line
`git log --oneline`	the history, newest first, one commit per line

Keep the folder: branches, the stash and undo (below) are safe to try here first.

41.8 Staging part of a file

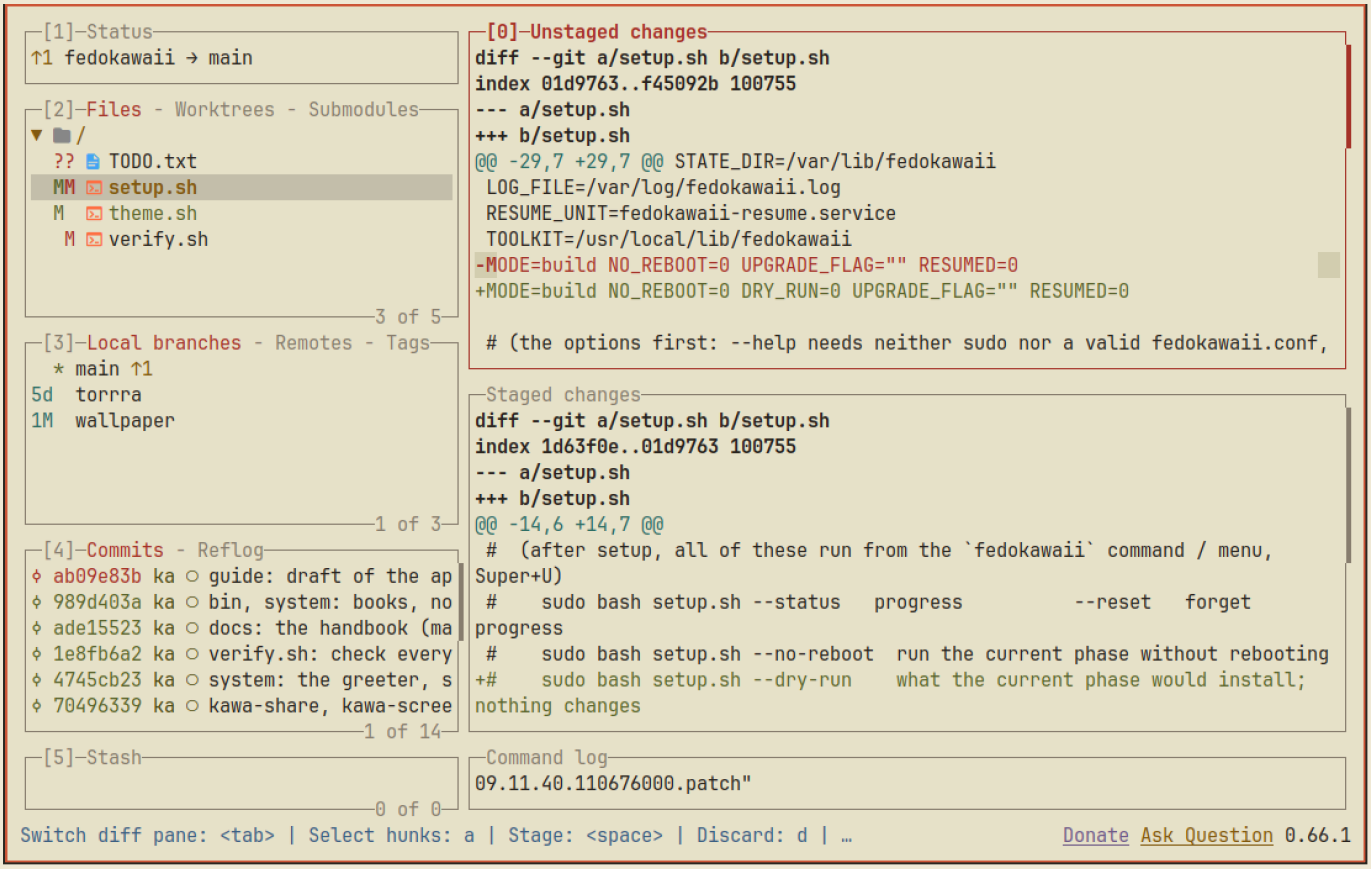
A commit should hold one change you can describe in one line. When a file holds two — a fixed alias and a half-written function — stage them apart:

1. In *Files*, select the file and press **Enter**. The main view takes over, showing the changes not yet staged, with one *hunk* (a block of changed lines) selected.
2. ↑ ↓ move from hunk to hunk; **a** switches to single lines and back; **v** starts a range.
3. **Space** stages the selection, which moves to the *Staged changes* pane beside it. **Tab** switches between the two panes; Space in the staged one unstages.
4. **Esc** returns to the files, and **c** commits only what is staged. The rest stays as it is, for the next commit.

In git's own notation that file — `local.sh`, say — has two hunks. Each starts at an `@@` line that says where it sits (`-3 +3`: line 3, before and after); `-` lines go, `+` lines come:

```
@@ -3 +3 @@
-alias ll='eza -la'
+alias ll='eza -la --git'
@@ -23,0 +24,3 @@
+backup() {
+  # half-written
+}
```

Stage the first hunk only, and *Files* shows `MM local.sh`: one change staged, one not.



Staging by hunk: the unstaged changes on one side, the staged ones on the other, the selected hunk highlighted.

In this view **d** removes the selected change **from the file** (in the staged pane, it unstages it); **E** opens the hunk in neovim to edit it before staging. Neovim can stage too, from inside the file: the coloured bars in the margin mark changed lines, `]h` · `[h` jump between hunks, Space g h s stages the hunk under the cursor, Space g h r resets it, and Space g h p shows what changed.

41.9 Commit, amend, undo

Keys (in Files)	Does
c	commit what is staged
C	commit, writing the message in neovim — save and quit (`:wq`) to finish
A	amend: add what is staged to the last commit (it asks first)
w	commit without running the repository's hooks — scripts a repository can run before each commit, a code formatter say; a new repository has none switched on
z · Z	undo · redo the last git action

z reads git's own record of actions (the *reflog*) and asks before acting. Undoing a commit takes the commit back and leaves its changes staged; a checkout, a reset or a rebase is undone too. It does not bring back changes discarded with **d** that were never committed, nor a stash entry, nor a branch you created; it cannot take back a push; and in the middle of a rebase it does nothing — **m** → abort is the way out there.

Tip: a typo in the last commit's message: select it in *Commits* and press **r**. A file forgotten from it: stage the file and press **A** in *Files*.

41.10 Stash

Stashing puts uncommitted changes aside and leaves the files as they were at the last commit — for when you need to switch branches or pull in the middle of something.

Keys	Does
s (Files)	stash the changes to tracked files; it asks for a name — Enter alone will do
S (Files)	stash options: *including untracked files*, only the staged changes, only the unstaged …
Space (Stash, 5)	apply the entry: the changes come back, the entry stays
g (Stash)	pop: the changes come back, the entry goes
d (Stash)	drop the entry

A typical use, in three steps — halfway through an edit, you want the commits from your other machine first:

1. *Files*: **s**, Enter. The files go back to the last commit; the edit waits in *Stash* (5).
2. **p** pulls.
3. *Stash*: select the entry, **g**. Your edit is back, on top of the pulled commits. If both changed the same lines it stops with a conflict (below), and the entry stays in the list until you drop it with **d**.

Read more: Pro Git, Stashing and Cleaning.

41.11 Branches

Keys (in Branches, 3)	Does
n	a new branch from the selected one, and switch to it
Space · `-`	switch to the selected branch · back to the previous one
M	merge the selected branch into the one you are on (then *Regular merge*)
r	rebase the branch you are on onto the selected one (then *Simple rebase*)
R · d	rename · delete (it asks: the local branch, the remote one, or both)
f	fast-forward the selected branch to its remote
]	the *Remotes* tab: the branches of other copies

Merge or rebase. Both bring the work of one branch into another; they differ in the history they leave. Say `main` gained one commit while the branch `bar-layout` gained two (`git log --oneline --graph --all`; lazygit draws the same graph):

```
* fb0d748 (bar-layout) bar: smaller clock
* 10519bf bar: try two rows
| * 985820e (HEAD -> main) notes: ideas for the bar
|/
* 6c84175 notes: start
```

	Merge (M)	Rebase (r)
what it does	joins the two lines with a new *merge commit*	replays your branch's commits, one by one, on top of the other branch
the history after	both lines kept, with a fork and a join	one straight line
your commits	unchanged, same hashes	rewritten, with new hashes
on a branch others share	safe	only for commits nobody else has

After merging `bar-layout` into `main`, the join shows:

```
*   a2b2184 (HEAD -> main) Merge branch 'bar-layout'
|\
| * fb0d748 (bar-layout) bar: smaller clock
| * 10519bf bar: try two rows
* | 985820e notes: ideas for the bar
|/
* 6c84175 notes: start
```

Rebasing `bar-layout` onto `main` instead gives one line — and its two commits have new hashes, because they now sit on a different parent:

```
* 124d160 (HEAD -> bar-layout) bar: smaller clock
* 46ba52e bar: try two rows
* 985820e (main) notes: ideas for the bar
* 6c84175 notes: start
```

Merging that into `main` then needs no merge commit: `main` moves up to `124d160`, a *fast-forward*.

For an experiment that worked on its own branch: switch to `main` (Space), select the experiment's branch, **M**. To bring `main`'s new commits under a branch you are still working on: stay on that branch, select `main`, **r**.

Read more: Pro Git on branching and merging and on rebasing.

41.12 Pull and push

A repository needs a remote before it can push. Create an empty private repository on GitHub or Codeberg, then in *Branches* press **]** for *Remotes*, **n**, name it `origin` and paste the address the site shows (an SSH address needs your SSH key added to the site; an HTTPS one asks for a token instead of a password).

Keys	Does
P	push the branch. The first time, lazygit asks which remote branch to track: `origin main`
p	pull: fetch the remote's new commits and merge them
f (Files)	fetch only: see what is new (↓ in *Status*) without merging it yet

lazygit also fetches by itself every minute while it is open (`git.autoFetch`), so the counts in *Status* are current. Reading them, in a repository called `notes`:

Status shows	Means	Do
`✓ notes → main`	the same commits here and on the remote	nothing
`↑2 notes → main`	two commits here the remote does not have	**P**
`↓1 notes → main`	one commit on the remote, not here yet	**p**
`↓1↑1 notes → main`	both sides moved on: the branch has *diverged*	**p** (it merges), then **P**

After amending or rebasing commits that were already pushed, **P** says the branch has diverged and offers a *force push*: Enter replaces the remote's history with yours. lazygit pushes with `--force-with-lease`, which still refuses if the remote holds commits you have not fetched. Do it only on a branch nobody else uses — on a one-person dotfiles repository, that is every branch.

41.13 Resolving a conflict

A merge, rebase or pull stops with a *conflict* when both sides changed the same lines — the same alias edited on two machines, say. Nothing is lost; git waits for you to choose.

In the file itself a conflict looks like this — here `main` and the branch `laptop` both changed one alias:

```
<<<<<<< HEAD
alias ll='eza -lah'
=======
alias ll='eza -la --git'
>>>>>>> laptop
```

Above `=======` is your side (`HEAD`, the branch you are on), below it the incoming one. The fix is the file as it should be, with the three marker lines gone — here perhaps `alias ll='eza -lah --git'`.

1. lazygit reports the conflicts; *Files* lists the files concerned, marked `UU`.
2. Select one and press **Enter**. The main view shows each conflict: your side (`HEAD`), then the incoming side, between git's `<<<<<<<`, `=======` and `>>>>>>>` lines.
3. ↑ ↓ choose a side and **Space** keeps it; **b** keeps both; ← → go to the next or previous conflict; **z** takes a choice back. Or **e** edits the file in neovim: write the lines as they should be, delete the three marker lines, `:wq`.
4. When a file has no conflict left, lazygit stages it. When none remain, it asks *All merge conflicts resolved. Continue the merge?* (or the rebase) — Enter. A merge then commits; a rebase moves on to its next commit, which may stop again.

m offers *continue* and *abort* at any time; abort puts everything back as it was before the merge or rebase began.

41.14 Rewriting history: interactive rebase

Commits not pushed yet — vermilion hashes — can be tidied before anyone sees them: joined, reworded, reordered, dropped. Select a commit in *Commits*:

Keys	Does
s	squash it into the commit below; both messages are kept, joined
f	fixup: into the commit below, its own message dropped
r · R	reword its message · reword it in neovim
d	drop the commit, and its changes with it
Ctrl+J · Ctrl+K	move it down · up the list
A	amend it — any commit, not only the last — with what is staged
e	stop the history at this commit to change it: edit the files, stage, **A**, then **m** → continue
i	start a rebase of the branch's commits: mark several with s, f, d (or p, to keep), then **m** → continue runs them all

Each single key runs at once, after asking. While a rebase is stopped, **m** → abort puts everything back; once it has finished, **z** undoes it.

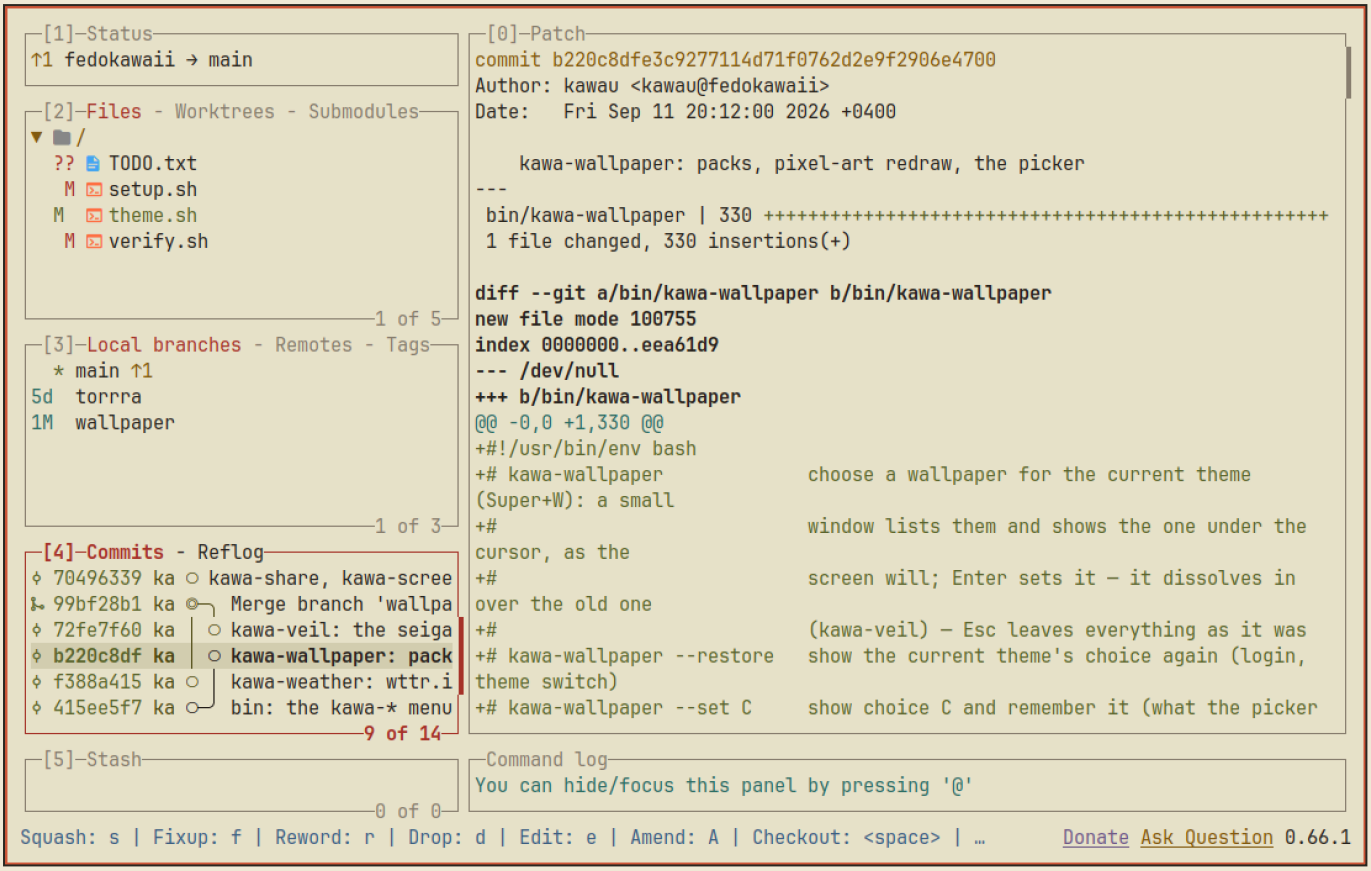
An example: four commits, newest at the top, where `typo` only fixed a slip in the commit below it. Select `typo` and press **f**:

Before	After
`606c48b bar: seconds in grey`	`17912ba bar: seconds in grey`
`7d19c63 typo`	—
`c40a45e bar: show seconds`	`4bf80c7 bar: show seconds`, now with the slip fixed
`bb03eba bar: smaller clock`	`bb03eba bar: smaller clock`

The commit below takes in the change and keeps its own message. Every commit from there up gets a new hash, because what it builds on changed; the oldest one is untouched.

Read more: Pro Git, Rewriting History.

41.15 History and diffs



The commit history: the branch graph beside the hashes, one commit's changes on the right.

Keys	Does
4, then ↑ ↓	the commits; the main view shows each one's message and changes
Enter · Enter · Esc	the files a commit changed · one file's diff · back
/	search the messages
Ctrl+S	filter the history: only the commits touching a path (a file's history), or by one author
W	compare: **W** → *Diff* the selected commit, move to another, and the main view shows the difference; **Esc** (or **W** → *Exit diff mode*) ends it
a (in Status, 1)	every branch, as a graph
Ctrl+L	log options: order, graph
] (in Commits)	the *Reflog* tab: every position the branch has been in, even ones no longer in the history

41.16 Cherry-pick

To copy a commit from one branch onto another without merging the whole branch: in *Branches*, press Enter on the branch that has it, to see its commits; select the commit and press **C** (for several: **C** on each, or a range with **v**); switch to the branch where it should land; in *Commits*, **V** pastes it there as a new commit. Esc or Ctrl+R forgets the copied set.

41.17 Recipes

I made a mess — go back.

The mess	The way back
one file's uncommitted changes are wrong	*Files*: select it, **d** → *Discard all changes*. Gone for good
everything uncommitted is wrong, but might be useful later	**s**: it waits in panel 5 until you **g** it back or **d** it
the last commit is wrong, not pushed	**z** (its changes come back as staged), or fix the files and **A**
an older commit is wrong, already pushed	*Commits*: select it, **t** (revert) — a new commit that undoes it, safe to push
a merge or rebase is going badly	**m** → abort
a rebase or reset has finished, and the result is wrong	**z**; or *Commits* → **]** *Reflog*, select the line from before, **g** → *Hard reset*
everything since the last commit must go	**D** → *Nuke working tree* — think twice: untracked files go too

A hard reset also throws away whatever is uncommitted, so stash first. git keeps commits that dropped out of the history in the reflog for about a month (`gc.reflogExpireUnreachable`, 30 days), so a commit you can still remember is almost never really lost.

What the reflog shows after the rebase drawn under *Branches* — newest first, one line for each move of `HEAD` (`git reflog` in a terminal, the *Reflog* tab here):

```
124d160 HEAD@{0}: rebase (finish): returning to refs/heads/bar-layout
124d160 HEAD@{1}: rebase (pick): bar: smaller clock
46ba52e HEAD@{2}: rebase (pick): bar: try two rows
985820e HEAD@{3}: rebase (start): checkout main
fb0d748 HEAD@{4}: checkout: moving from main to bar-layout
```

The line below `rebase (start)` is where the branch stood before: select `fb0d748`, **g** → *Hard reset*, and `bar-layout` is back as it was.

Commit only part of a file. Files, Enter on the file, select the hunk (**a** for single lines), **Space**, **Esc**, **c** — see *Staging part of a file*, above.

Move a commit to a new branch. You committed on `main` and meant to start a branch:

1. In *Branches*, with `main` selected, **n** and a name: the new branch starts here, with the commits, and you are on it.
2. Select `main` and press **Space** to go back to it.
3. In *Commits*, select the last commit that belongs on `main`, **g** → *Hard reset*. `main` is back where it was; the commits live on in the new branch.

N (*Move commits to new branch*) does the same in one step for commits not pushed yet, and asks where the new branch should start. For a single commit from the middle: cherry-pick it to the other branch (above), then **d** drops it here.

Recommended. From inside neovim (Space g g), lazygit is set up by LazyVim itself: it takes the editor's colours, and **e** opens the file in that same neovim instead of starting a second one. Started on its own, `e` runs neovim inside lazygit's window and returns when you quit it. Commit small and often, with a message that says why: `bar: show seconds` reads better in a year than `update`.

Pitfalls.

- **z** undoes git actions, not edits you have not committed — discarded changes (**d**) are gone.
- Commits are made with your git identity: set it once (above) if git complains.
- **Space** on a *commit* checks that commit out: the files show the old state, and lazygit reports a *detached HEAD*. Space on a branch in *Branches* (or **-**) brings you back.

- Rewriting commits that are already pushed — amend, squash, reword, drop — needs a force push and breaks other copies that have them. On a one-person repository that is harmless; anywhere shared, revert (**t**) instead.

Cheat sheet.

To	Keys
move between panels · jump to one	← → · 1 … 5
every key here	?
stage a file · all files · single lines	Space · a · Enter, then Space
commit · in neovim · amend the last	c · C · A
discard a file's changes · everything	d · D
stash · pop it back	s · g (in Stash)
undo · redo	z · Z
pull · push · fetch	p · P · f
new branch · switch · back to the previous	n · Space · `-`
merge into this branch · rebase onto	M · r
squash · fixup · reword · drop · move	s · f · r · d · Ctrl+J, Ctrl+K
copy commits · paste them here	C · V
revert a commit · reset to it	t · g
look inside a commit · back out	Enter · Esc
a file's history	Ctrl+S, by path
continue or abort a merge or rebase	m
quit	q

Read more: lazygit's own pages on configuration and undo; git's reference manual at https://git-scm.com/docs.

What you learnt

- A repository is a folder with a history; a commit records what is staged, with a message that says why.
- `git status --short`, `git diff` and `git log --oneline` answer *what changed*, *how* and *when*; lazygit shows all three on one screen.
- In lazygit, **Space** stages, **c** commits, **z** undoes the last git action and **s** puts work aside.
- A branch is a line of work for trying something; a merge brings it back; pull and push keep a remote copy in step.
- The reflog remembers every place `HEAD` has been, so a commit is almost never lost.

Check yourself

1. What do `??` and `A` mean in `git status --short`? → *new and never stored · added, staged for the next commit*
2. Which lazygit key takes back the last git action? → ***z**** **— and** *Z* redoes it*
3. A commit you already pushed is wrong: amend it, or revert it? → *revert it (t): a new commit that undoes it, safe to push*

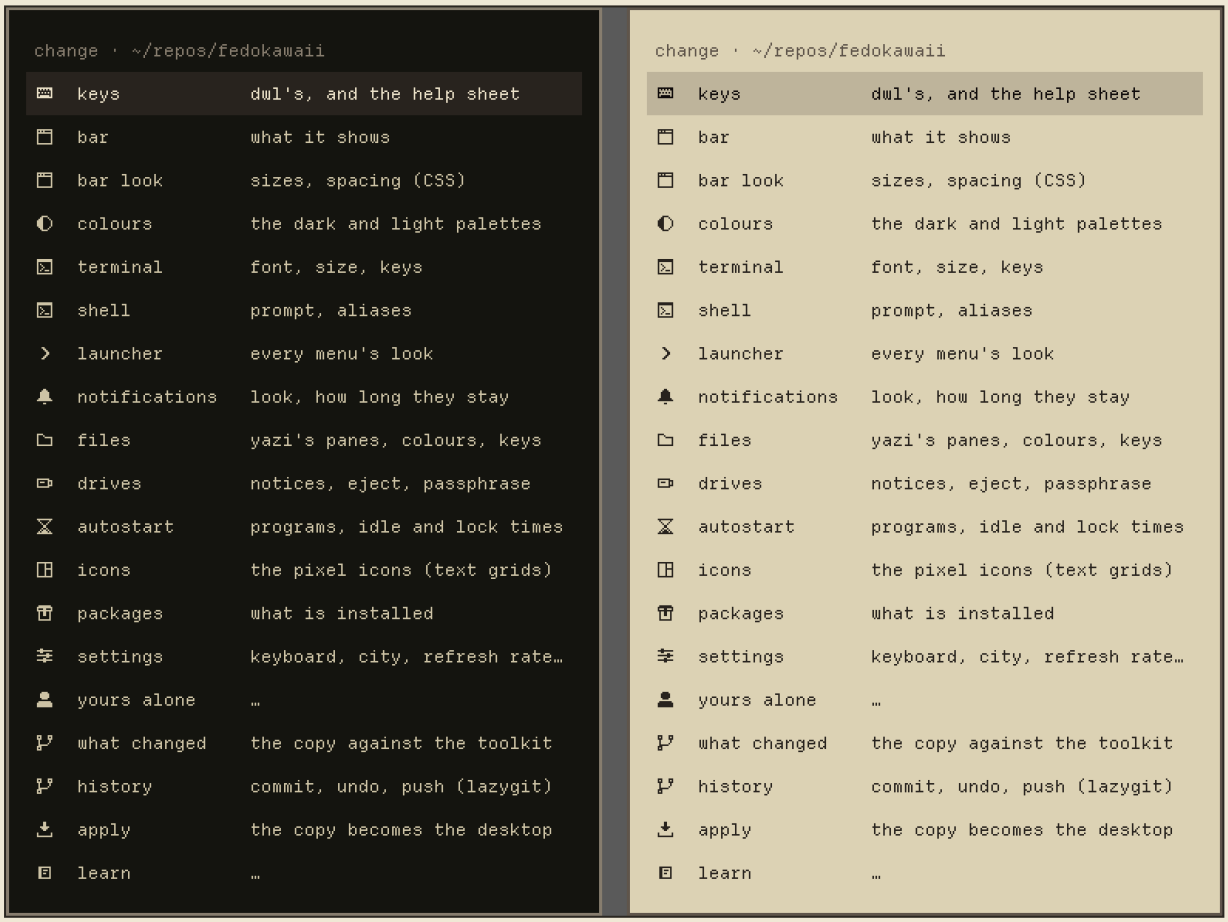
42. Changing it yourself

> **In this chapter**
> - tell a file that is yours alone from one the toolkit owns
> - change any part of the desktop in your working copy, test it, apply it
> - keep each change in git, and undo one
> - add or take out a package
>
> **You need:** ch. 41 (git) · ch. 40 (how the look is built).

Everything this desktop is — every key, colour, menu, config and package — is a text file in one folder, and any of it can be changed. This chapter shows how to do that so it lasts. Every change takes the same five steps, and the sections below follow them in order:

1. **Which kind?** A file is yours alone, or the toolkit's (§42.1).
2. **Which file?** Who owns each config, and where each part of the desktop lives (§42.2, §42.3).
3. **Change it** in your working copy — or try it in place first, then take it into the copy (§42.4, §42.5).
4. **Check and apply**: the self-tests, then `fedokawaii apply` (§42.6, §42.7).
5. **Keep it** in git, so that it can be undone and outlives a reinstall (§42.8).

The rules the scripts count on, two recipes and packages close the chapter. Chapter 43 is about learning the languages these files are written in.



The change menu (Super+Shift+U) on both themes: every part of the desktop by name, then what is yours alone, what changed, history, apply and learn.

42.1 Two kinds of change

Every file you might change is one of two kinds: yours alone, or the toolkit's.

> **On Windows.** Settings live in many places — each program's dialogs, the Registry — and a big update may reset some of them. Here every setting is a line in a text file, and this section says which files an update leaves alone.

Yours alone. The theme never writes these files. Change them where they are, and nothing will undo it.

File	For	Read
`~/.bashrc.d/local.sh` (any name but `fedokawaii.sh`)	your aliases, functions and `export`s. Fedora's `~/.bashrc` reads every file in `~/.bashrc.d` by name, so `local.sh` comes after `fedokawaii.sh`, and what you set there wins	by every new terminal
`~/.config/dwl/autostart.local.sh`	programs to start at every login. End each one that keeps running with `&`	at the next login (its output goes to `~/.cache/dwl.log`)
`~/.config/nvim/lua/plugins/*.lua`, all but `colorscheme.lua`	LazyVim plugins, and changes to the ones it has	at neovim's next start
`~/.local/bin/` — names the desktop does not use there (`kawa-…`, `fetch`, `brave-origin`, the release tools such as `yazi`, `torrra`)	your own scripts, on your `PATH`	at once
the right-hand column of the table in §42.2	each program's own settings: `yazi.toml`, `mpv/input.conf`…	by that program

Super+Shift+U → yours alone opens the first three in neovim — the first time, it starts each file with a few lines saying what it is for — and `~/.local/bin` in yazi. Two small examples:

```
# ~/.bashrc.d/local.sh
alias gs='git status --short'           # gs: what changed in this repository
mkcd() { mkdir -p -- "$1" && cd -- "$1"; } # mkcd notes/2026: make a folder and go in
export LESS='-R --mouse -i -S'          # the desktop's less, and -S: long lines cut, not wrapped
```

```
# ~/.config/dwl/autostart.local.sh
syncthing serve --no-browser &          # keeps running: ends with &
sleep 5 && notify-send "logged in" &    # a one-off, also in the background
```

The desktop itself. Every file the toolkit ships belongs to the toolkit. The theme puts the toolkit's version back each time it runs (yours goes to `~/.local/state/fedokawaii/backup/`), and every update and repair runs from the toolkit. So the desktop is changed where the toolkit comes from: your **working copy** (§42.4). First, find the file.

42.2 Who owns which config

The programs' settings in `~/.config` are of both kinds, side by side. Every file under `configs/home/` in the toolkit belongs to **Theme**: change it in your working copy and apply (§42.4; **Super+Shift+U** opens each by name). An edit made only in `~/.config` is listed by `fedokawaii drift` and replaced at the next theme run (the old copy goes to `~/.local/state/fedokawaii/backup/`). Files the toolkit does not ship are yours and are never touched.

```
fedokawaii drift      # the managed configs that differ from the toolkit (changes nothing)
```

Managed by Theme (edit in the working copy)	Yours (edit in place)
`foot/foot.ini`, `fuzzel/fuzzel.ini`, `mako/config`, `waybar/`, `btop/btop.conf`, `lazygit/config.yml`, `bat/config`, `mpv/mpv.conf` and `mpv/scripts/kawa-*.lua`, `yt-dlp/config`, `zathura/zathurarc`, `satty/config.toml`, `spotify-player/`, `yazi/theme.toml`, `yazi/init.lua`, `yazi/keymap.toml` and `yazi/plugins/` (drives, smart-enter), `udiskie/config.yml`, `nvim/lua/config/lazy.lua`, `nvim/lua/plugins/colorscheme.lua`, `nvim/after/ftplugin/sh.lua`, `MangoHud/MangoHud.conf`, `mimeapps.list`, `xdg-terminals.list`, the portal files (`xdg-desktop-portal/`, `xdg-desktop-portal-wlr/`, `xdg-desktop-portal-termfilechooser/`), the pointers in `~/.local/share/icons/`, `pipewire/pipewire.conf.d/60-share.conf`, `~/.bashrc.d/fedokawaii.sh`, `~/.inputrc`, `dwl/autostart.sh`, the launcher entries in `~/.local/share/applications`	every other file in `~/.config/nvim`, `yazi/yazi.toml`, `mpv/input.conf`, `cava/config`, `~/.config/fedokawaii/secrets.sh`, `~/.config/torrra/config.toml`, the browser and Steam profiles (except the two Brave settings Theme makes — its colours and its launcher entry, §5.3)

42.3 Where each thing lives

For the desktop itself, look the part up here. Every path is inside your working copy (§42.4); "When" is when the change shows after apply; "In the menu" is the entry of **Super+Shift+U** that opens the file.

To change	Edit	Written in	When	In the menu
keys	`src/dwl/config.h` (`keys[]`), and a line in `configs/home/.config/fedokawaii/keybinds.txt`	C: one `{ … }` row per key	next login	keys
gaps, borders, which windows float	`src/dwl/config.h`: `gappx`, `borderpx`, `rules[]`	C	next login	keys
frame and border colours	`src/dwl/config.h`: `bordercolor`, `focuscolor`, from the names `SUMI`, `SHU` … defined above them	C	next login	keys
the bar: modules, their order	`configs/home/.config/waybar/config.jsonc`	JSON with comments	at once	bar
the bar: sizes, spacing	`configs/home/.config/waybar/style.css` (its colours come from the palette)	CSS, as GTK reads it	at once	bar look
what a bar module shows	`bin/kawa-cpu`, `kawa-nowplaying`, `kawa-dwlstatus`, `kawa-drives` … (the weather above quick settings: `kawa-weather`)	bash	at once	—
the menus	`bin/kawa-quick`, `kawa-power`, `kawa-audio`, `kawa-config` …; their look: `bin/kawa-menu` and `configs/home/.config/fuzzel/fuzzel.ini`	bash · INI	the next menu that opens	launcher
every colour, both themes	`tools/build-themes.py` (the palettes near its top) — the menu runs it after you save	Python: a table of hex colours	at once; the terminal's at the next login	colours
the terminal	`configs/home/.config/foot/foot.ini`	INI	next login (the terminal server)	terminal
the shell: prompt, aliases, the tools' settings	`configs/home/.bashrc.d/fedokawaii.sh`	bash	every new terminal	shell
notifications	`configs/home/.config/mako/config`	INI-like	at once	notifications
files (yazi)	`configs/home/.config/yazi/init.lua` (the panes), `theme.toml`, `keymap.toml` (keys); the file dialog's keys: `configs/home/.config/fedokawaii/yazi-pick/keymap.toml`	Lua · TOML	the next time yazi starts	files

To change	Edit	Written in	When	In the menu
drives	`configs/home/.config/udiskie/config.yml` (what opens by itself, the hook); `bin/kawa-drives` (the notices, the mark, the menu, eject); `configs/home/.config/yazi/plugins/drives.yazi/main.lua` (M in yazi)	YAML · bash · Lua	`kawa-drives` at once; udiskie's file at the next login; M the next time yazi starts	drives
autostart, idle and lock times	`configs/home/.config/dwl/autostart.sh` (the `swayidle` line: seconds)	POSIX sh	next login	autostart
the lock screen	`bin/kawa-lock`; its picture: `configs/home/.local/share/fedokawaii/{dark,light}/lock.xpm`	bash · XPM	the next lock	—
pixel art	`configs/home/.local/share/fedokawaii/{dark,light}/*.xpm`	XPM: a text grid, one letter per pixel	at once	—
icons	`art/icons.txt` — `#` is a pixel. The menu draws them into the font with `tools/build-icons.py`, fetching fontTools once	text grids	next login	icons
pointers	`art/cursors.txt`, then `python3 tools/build-cursors.py` in the copy	text grids	next login	—
packages	`lib/manifest.sh`, and a row in `PACKAGES.md`	bash lists	apply installs them (repair)	packages
settings	`fedokawaii.conf` (§5.2)	`KEY="value"` lines	apply puts each in place (repair)	settings
the system's own steps	`setup.sh` (one function per step: `s_…`, `r_…` for repair); `verify.sh` (its checks)	bash	the next update or repair	—
what this hardware gets	`lib/hardware.sh` (what it looks for: firmware, microcode, Wi-Fi, sound)	bash	the next update or repair	—
what counts as doubled or fighting	`lib/manifest.sh` (`FORBIDDEN_PKGS`), `lib/hygiene.sh` (services that fight)	bash lists	the next update or repair	—
the control (Super+U)	`system/fedokawaii`	bash	after apply	—
this handbook	`docs/fedokawaii-guide.md`; the PDF is built by `docs/build/`	Markdown	—	—

An example of each language, as it appears in these files:

```
/* C — src/dwl/config.h: one row per key: modifiers, key, what it does, its argument */
{ MODKEY,          XKB_KEY_b,   spawn,   {.v = browsercmd} },   /* Super+B → Brave */
```

```
// JSON — waybar/config.jsonc: which modules sit in the middle of the bar
"modules-center": ["custom/nowplaying"],
```

```
/* CSS — waybar/style.css: one rule for one module (the time), in the palette's colours */
#clock.time { color: @strong; padding: 0 10px 0 0; border-right: 1px solid @edge; }
```

```
# INI — foot/foot.ini: a [section], then key=value lines
[main]
font=JetBrainsMono Nerd Font:size=11.5,Sumi Icons Term:pixelsize=11
```

```
-- Lua — a table describes a plugin to LazyVim
return { { "folke/zen-mode.nvim", cmd = "ZenMode" } }
```

```
# bash — lib/manifest.sh: a list (an array) of package names
PKG_APPS=(
  hyperfine            # benchmarks a command
)
```

42.4 The working copy

The working copy is `~/repos/fedokawaii`: the toolkit as a folder of your own, kept as a git repository. You change it, and **apply** makes it the desktop's. (To keep the copy somewhere else, set `FEDOKAWAII_COPY=/that/folder` in `~/.bashrc.d/local.sh`. That reaches `fedokawaii` in a terminal only: **Super+Shift+U**, started by dwl, never reads `~/.bashrc.d` and keeps `~/repos/fedokawaii`.)

Try this. Make the copy, then ask for it again.

```
$ fedokawaii copy
  your working copy: /home/you/repos/fedokawaii — change it there (fedokawaii change), then: …
$ fedokawaii copy
  your working copy: /home/you/repos/fedokawaii
```

(shortened) The first run copies the toolkit and commits it unchanged; every later run only says where the copy is — and, once you have changed files, how many since the last commit (`— 2 file(s) changed since its last commit`). If git does not know your name yet, it says so and leaves that first commit to you (§41.2).

Step	Command, or Super+Shift+U	What it does
1. make it, once	`fedokawaii copy` (the menu makes it on first use)	copies the toolkit to `~/repos/fedokawaii` and commits it unchanged: a point you can always return to
2. change	a part in the menu, or `nvim ~/repos/fedokawaii/FILE`	the menu opens the part's file in neovim, in a window of its own, with the copy as the project (**Space f f** finds any file in it)
3. look	`fedokawaii diff` · *what changed*	every line the copy changes against the toolkit, in a pager; `q` closes
4. apply	`fedokawaii apply` · *apply*	the copy becomes the desktop's (§42.7)
5. keep	*history* (lazygit in the copy)	commit with a line that says why, and push (§42.8)

A whole change, start to finish — a wider gap between windows:

1. **Super+Shift+U → keys.** `src/dwl/config.h` and `keybinds.txt` open side by side.
2. Find `gappx` near the top (`/gappx` in neovim): its line ends in `= 8;`, before a comment. Change `8` to `12`, save with `:w` and close neovim with `:qa`.
3. The menu asks what next. Choose *apply now*, type your password, and apply rebuilds dwl.
4. Log out and in (**Super+Shift+E → log out**): the gaps are 12 pixels.
5. *history*: lazygit shows `src/dwl/config.h` changed. Press **Space** to stage it, **c** to commit (`dwl: gaps 12 px, wider for the 1440p screen`), **P** to push.

The copy and GitHub. The project lives on GitHub as the private repository `MoeDia/fedokawaii`. Clone it to `~/repos/fedokawaii` instead of running `fedokawaii copy`:

```
git clone https://github.com/MoeDia/fedokawaii ~/repos/fedokawaii
```

A private repository asks for your GitHub user name, and for a *personal access token* as the password. You make the token on github.com: *Settings → Developer settings → Personal access tokens*. The clone's `fedokawaii.conf` is GitHub's version. Apply merges your saved settings into it (§35.1): a setting you changed

through *settings* stays, unless GitHub's file changes that same setting itself since you last applied — then GitHub's value is taken, and apply names it.

42.5 The one rule for configuration

A file the theme manages is changed in your working copy and applied — never only where the program reads it. Trying a change in place first is fine, and often the quickest way to see it. To make it last:

1. Edit the file where the program reads it — say `~/.config/waybar/style.css` — and watch the result (§42.3 says when each part re-reads its file).
2. `fedokawaii keep ~/.config/waybar/style.css` takes it into your working copy.
3. `fedokawaii diff` shows the change as the toolkit will see it; `fedokawaii apply` makes it the desktop's (§42.7).

Try this. Ask `keep` about a file you have not changed.

```
$ fedokawaii keep ~/.config/waybar/style.css
  configs/home/.config/waybar/style.css: already the same in the copy
```

Nothing to do: the copy has that file as it is. After a change in place, the same command copies it, to the place it came from — `configs/home/…` for a config, `bin/…` for a script from `~/.local/bin`:

```
$ fedokawaii keep ~/.config/waybar/style.css
  kept: configs/home/.config/waybar/style.css — fedokawaii apply makes it the desktop's
```

`keep` does not copy three kinds of file:

- files a tool draws — the colours, the icon font, the pointers. It names the source to change instead;
- files the theme fills in from `fedokawaii.conf` — they contain `@HOME@`, `@WEATHER_CITY@` or a similar token. Make the same change in the copy by hand;
- files the toolkit does not ship. They are yours already: the theme never replaces them.

Until you keep and apply, `fedokawaii drift` lists the file, and the next theme run puts the toolkit's version back (yours is kept in `~/.local/state/fedokawaii/backup/<time>/`). Files the theme never writes are yours to edit where they are (§42.1).

42.6 Check before you apply: the self-tests

You met the desktop's self-tests in §39.3: each one runs a `kawa-…` script against stubs — a stick pulled out while open, a machine with no network — and checks that the script does what it should. Here they do the job they are for. Change a script in the working copy, run its area, and you know whether you broke something before apply makes the change the desktop's.

The tests check the scripts of the folder they sit in, so run them in the working copy, and run the area once **before** you change anything: a check that fails then was not you. `ls tests` lists the areas:

Area	Tests, for example
`drives`	`kawa-drives`: sticks, disks, SD cards, eject
`status`	the bar and its keys: `kawa-cpu`, `kawa-nowplaying`, `kawa-weather`, the volume keys
`menus`	quick settings, the power menu, the calendar, Super+/, Super+Shift+U
`look`	the theme switch, the wallpapers, the lock, screenshots
`engine`	`fedokawaii`: the menu, `status`, `copy`, `keep`, `diff`
`static`	every script parses; nothing a script calls is missing; drawn files match their source

After a change. Say the notice for a stick pulled out while open feels too loud, and you take `-u critical` out of its line in `bin/kawa-drives` (**Super+Shift+U → drives** opens the file; when it asks, choose *later*). Run the area again:

```
$ cd ~/repos/fedokawaii
$ bash tests/run.sh drives
  ✘ pulled out while open: a critical notice — no call matching [notify-send -a drives -u crit…
  ✘ an old eject mark does not hide a real pull-out — no call matching [critical] — calls: not…
test_drives: 93 passed, 2 failed

✘ 1 of 1 test files failed: drives/test_drives.sh
```

(shortened) Every failed check prints a line of the same form, `✘ <case> — <what>`:

Part	Here	Means
`✘`	—	one check failed
case	`pulled out while open: a critical notice`	the behaviour under test, in the test's own words
what	`no call matching [notify-send … -u critical …]`	what was expected; after `calls:`, what the stubs saw instead

The case is written in the test file, above its checks — here in `tests/drives/test_drives.sh`:

```
case_ "pulled out while open: a critical notice"
setup
kawa-drives event device_mounted /dev/sdb1 True KINGSTON ""
reset_calls
kawa-drives event device_unmounted /dev/sdb1 True KINGSTON ""
kawa-drives event device_removed /dev/sdb1 True KINGSTON ""
called 'notify-send -a drives -u critical -- .*KINGSTON was pulled out while open'
```

It plays what udiskie would report — a stick opened, closed by surprise, gone — then checks the stubs' log for a critical notice. Now you choose. A slip: undo the change (§42.8) and the area passes again. Meant: the test is out of date, so change what it expects (here, the `called` line), run the area again, and commit script and test together. A change under `tests/` makes apply run repair: slower, never harmful.

Apply itself only checks that each script parses (§42.7); whether it still behaves is the tests' job, and they run only when you run them.

42.7 Apply: the copy becomes the desktop

When the copy holds the change you want, apply makes it the desktop's: `fedokawaii apply` in a terminal, or *apply* in **Super+Shift+U**. After an edit, the menu offers *apply now*, *later* (then `fedokawaii apply` when you are ready) or *what changed*. Apply asks for your password once (sudo), lists the files that differ from the toolkit, then works in order:

1. ShellCheck reads the shell scripts you changed and prints its notes. These are advice: apply goes on.
2. Every shell script in the copy must parse (`bash -n`). If one does not, nothing is applied, and apply names the file.
3. The copy becomes the toolkit. It is copied into a folder only root can enter, refused if it holds a symbolic link, and its `fedokawaii.conf` is merged with what you saved through *settings* (§35.1).

4. What changed decides the rest:

You changed	Apply runs	Because
`configs/`, `bin/`, `src/` (dwl's `config.h`, the C helpers), `art/`, `theme.sh`	the theme	it installs the configs, rebuilds dwl when `config.h` changed, and reloads the running session
`lib/manifest.sh`, `fedokawaii.conf`, `setup.sh`, `lib/`, `system/` — and anything not named in the other rows, `tests/` included	repair	it installs what the manifest gained, puts the system's parts in place (login, session launcher, SSH, time zone, DNF), and runs the theme
`docs/`, `tools/`, `*.md`, `verify.sh`, `check.sh`, `fedokawaii.sh`	nothing more	they travel with the toolkit; `verify.sh` is used at the next check
nothing	nothing	it says so

Run from the copy with `sudo bash ~/repos/fedokawaii/fedokawaii.sh theme`, `update` or `repair`, the theme, update and repair also make the copy the toolkit first, in the same way. `fedokawaii theme` from the menu re-applies the toolkit as it is.

When something goes wrong:

What you see	Why	Do
apply: *these do not parse*	a shell script has a syntax error	`bash -n FILE` shows the line; often a missing `fi`, `done` or quote
apply: *dwl failed to build with src/dwl/config.h*	`config.h` has a C mistake	`fedokawaii log` shows the compiler's message and line; the dwl you have keeps running
the bar is gone after apply	`config.jsonc` is not valid JSON	`waybar -l debug` in a terminal names the line; fix it and apply
a menu looks wrong	`fuzzel.ini` has a key fuzzel does not know	`fuzzel --check-config`
the terminal will not start at login	`foot.ini` has a mistake	`foot --check-config`

Apply's last line is a reminder of the last step: keep the change in your copy's history.

42.8 Keep it: commit, push, undo

A change that is applied but not committed works, but it has no history: nothing says why it was made, no key takes it back, and a reinstall would not have it. Keeping it takes a minute:

1. **Super+Shift+U → history** opens lazygit in the working copy (ch. 41).
2. *Files* lists what you changed. Select each file and press **Space** to stage it.
3. **c**, then one line that says why — `dwl: gaps 12 px, wider for the 1440p screen` — and **Enter**.
4. **P** pushes it to your GitHub repository, if the copy has one (§41.12).

Try this. Read the copy's history from any folder.

```
$ git -C ~/repos/fedokawaii log --oneline
4f1c2d9 dwl: gaps 12 px, wider for the 1440p screen
9e07b31 fedokawaii as installed (2026-10-16)
```

(for example) `-C FOLDER` runs git as if you were in that folder. The oldest line is the commit `fedokawaii copy` made: the toolkit as it came.

Undoing.

- *In the copy*: lazygit → *Files* → **d** discards a change not yet committed. For a commit you regret, select it in *Commits* and press **t** to revert it (§41.17). Then run the self-tests and apply again.

- *A config, as it was before a theme run*: `~/.local/state/fedokawaii/backup/<time>/`.
- *The system*: a snapshot (§37.2).

42.9 Rules the scripts count on

1. **A file under `configs/home/` lands at the same place in your home.** `configs/home/.config/foot/foot.ini` becomes `~/.config/foot/foot.ini`, and a new file there is installed by the next apply. A file you *remove* from the copy is not deleted from your home; delete it there yourself.
2. **Drawn files are made, not edited**: `configs/home/.config/fedokawaii/themes/`, the three `gtk.css` files (GTK 4's and the two GTK 3 themes'), the colour block of `foot.ini`, `nvim/lua/plugins/colorscheme.lua`, `icons.sh`, the `SumiIcons` fonts and the pointers. Change their source — the palette, `art/icons.txt`, `art/cursors.txt` — and run the tool. The next run of the tool replaces any hand edit.
3. **Tokens are filled in at apply**: `@HOME@`, `@MONITOR_HZ@`, `@WEATHER_CITY@`, `@NIGHT_LIGHT@`, `@MPV_SCALING@`, `@ICON_CHEVRON_RIGHT@`. Leave them as they are. (`@DEFAULT_AUDIO_SINK@` is no token: it is wpctl's own name for the default output.)
4. **A script's first line decides its mode.** The theme makes a file executable when it starts with `#!`, so a new script in `bin/` begins with `#!/usr/bin/env bash`.
5. **Names**: the desktop's helpers are `kawa-…`, beside `fetch`, `brave-origin` and the release tools in `~/.local/bin`. Give your own scripts other names.
6. **A key in `config.h` gets its line in `keybinds.txt`**, so **Super+/** stays complete.
7. **A package goes into the manifest.** Never only `dnf install`: `verify` would list it (§42.11).
8. **Afterwards, `fedokawaii verify`** proves the rest still holds. A changed `kawa-…` script gets its area of the self-tests first: `bash tests/run.sh AREA` (§42.6).

What you see when one of these is broken, and what to do, is in §42.7.

42.10 Recipe: change a setting for good

1. Is the file managed? The table in §42.2 says; `fedokawaii drift` lists the managed files that differ from the toolkit.
2. Managed: **Super+Shift+U**, pick the part, change it, close neovim, *apply now* (§42.4). It reloads what it can; the terminal's settings and dwl's keys take effect at the next login.
3. Yours: edit it where it is. Nothing will overwrite it.
4. Keybindings live in `src/dwl/config.h`; the *keys* part opens it beside `keybinds.txt`, so Super+/ stays complete.
5. Commit the change in *history* (§42.8).

42.11 Adding a package properly

A package you keep goes into the manifest, in your working copy: its name in `PKG_APPS`, a row in `PACKAGES.md`, then *apply* — apply runs repair, which installs it (§42.12 has the steps). An `update` alone does not read your working copy. A one-off — something to try for an afternoon — can be `sudo dnf install NAME`; `verify` will list it as *installed outside the manifest* until you `sudo dnf remove NAME` again, which also removes the dependencies it brought (DNF's `clean_requirements_on_remove`, on by default). Never leave it there to be forgotten.

42.12 Recipe: install a program and keep verify happy

`verify` fails on anything user-installed that is not in the manifest, so the manifest comes first.

1. Check that it is not on the *never present* list in `PACKAGES.md`, and that it is not a second tool for a job one already does.
2. **Super+Shift+U → packages**: `lib/manifest.sh` and `PACKAGES.md` open side by side in your working copy. Add the name to `PKG_APPS` with a short comment, for example `hyperfine # benchmarks a command (runs it many times)`, and a row to `PACKAGES.md`, so the list stays true.

3. Close neovim and choose *apply now*: apply sees the manifest changed and runs repair, which installs every package the manifest has gained, then runs `verify`.

A package from Fedora or RPM Fusion needs nothing more. One from a new COPR also needs its name in `COPRS` and `sudo dnf copr enable owner/project` by hand — `update` does not add repositories. Flatpak and Snap are refused outright.

42.13 Taking a package out

In your working copy, move it from its list to `RETIRED_PKGS` in `lib/manifest.sh`, then *apply*. Repair (and every update after it) marks it as a dependency (`dnf mark dependency`) rather than removing it — unless it was already user-installed before setup:

	Before	After apply
listed by `dnf repoquery --userinstalled`	yes	no
counted by `verify`	as a manifest package	not at all
`sudo dnf autoremove` (the next update or repair runs it)	leaves it	removes it once nothing needs it

Read more: dnf5 autoremove · repoquery · mark · clean · dnf5.conf.

What you learnt

- A file is yours alone — the theme never writes it — or the toolkit's, changed in the working copy and applied.
- The working copy, `~/repos/fedokawaii`, is the toolkit as a git repository of your own; **Super+Shift+U** opens each part of it by name.
- A config tried in place goes into the copy with `fedokawaii keep`; `fedokawaii diff` shows what apply would change.
- `bash tests/run.sh AREA` checks a changed script against what it should do, before you apply.
- Apply checks every script, makes the copy the toolkit and runs the theme or repair; a commit in lazygit keeps the change.
- A package goes into `lib/manifest.sh` and `PACKAGES.md`, never only `dnf install`.

Check yourself

1. You changed `~/.config/foot/foot.ini` in place. What does the next theme run do, and how do you keep your version? → *it puts the toolkit's back (yours goes to the backup folder); `fedokawaii keep` the file, then `fedokawaii apply`*
2. A self-test prints `✘ pulled out while open: a critical notice — …`. What does the text before the dash name? → *the case: the behaviour the test expected*
3. Where does a program you want to keep go? → *its name in `PKG_APPS` in `lib/manifest.sh`, a row in `PACKAGES.md`, then apply*

43. Learning to program here

> **In this chapter**
> - choose a first language, and the order of the next
> - read a real script of this desktop, one idea at a time
> - name the ideas behind programming, each with an example
> - pick a first project from a ladder of real changes
>
> **You need:** ch. 42 (changing it) · ch. 25 (the shell).

This desktop is a good place to learn programming. Every part of it is a short, readable file you can change, apply and undo, and the files cover most of what working programmers write: shell scripts, a little C, Python, Lua, CSS, JSON and TOML. This chapter covers what to learn, in what order, how to read a real script, the ideas behind it all in plain words with an example each, and where the reliable documentation is. Appendix F says where Linux and the trade stand in October 2026.

43.1 How to learn here

Habit	Why it works	For example
change something real	a change you can see sticks better than an exercise	the gaps between windows (§42.4), a new bar module, a shell alias
one small step, then check	each step shows whether you understood the last	`bash -n script` · `fedokawaii diff` · `fedokawaii verify`
type examples, never paste them	your hands learn the shapes: brackets, quotes, `fi`	the examples in this chapter
read the manual first, then search	the manual is right for your version; a search result may be for another	`man 5 foot.ini` before a web search
write down what you learnt	you will need it again in a month	`~/Notes/notes.md` (**Super+N**): one line per thing learnt
ask an AI to *explain*, not only to *write*	you learn from the explanation; you only get used to the code	§44.2
one language at a time	two new syntaxes at once blur together	finish §43.2's step 1 before step 2

Three tools on this desktop answer most small questions:

```
tldr tar          # a few common examples of a command
man 1 tar         # its whole manual (/word searches, n the next match, q quits)
tar --help | less # the short list of its options
```

43.2 Languages worth learning now

The surveys agree on a short list. In the 2026 Stack Overflow Developer Survey, the most used languages were JavaScript (62 %), then SQL, HTML/CSS and Python (58 % each). Rust rose to 12th place. In the 2025 survey, Python grew by 7 percentage points in a year, "the go-to language for AI, data science, and back-end development", and Rust was again the most admired language (72 %). GitHub's Octoverse 2025 found TypeScript the most used language on GitHub for the first time (August 2025): typed languages make AI-assisted code more reliable, because the type checker catches the AI's mistakes before they run. Shell scripts were the fastest-growing language in AI-related projects (+324 %). IEEE Spectrum's 2025 ranking put Python first, overall and for jobs.

The order below suits this desktop: each step uses what you learnt in the one before.

Step	Language	What it is for	Why now	Start here, on this desktop
1	**bash** (the shell)	gluing programs together; most of this desktop	half of all developers use it (48.7 %, 2025 survey); every server and container speaks it	`~/.bashrc.d/local.sh`, then `bin/kawa-weather` in your working copy
2	**Python**	data, AI and machine learning, automation, web back ends	first in IEEE Spectrum, overall and for jobs; the language of AI libraries	`tools/build-themes.py` — the palettes are a Python table
3	**SQL**	asking questions of data kept in tables	second most used in 2026 (58 %); every company keeps its data in tables	Python's built-in `sqlite3`, below
4	**TypeScript** (and JavaScript)	web pages, web apps, tools that run in Node.js	the most used on GitHub (2025); types keep AI-written code honest	Node.js 24 is installed: it runs a `.ts` file directly
5	one of **Rust** or **Go**	fast, safe programs; command-line tools; servers	Rust: the most admired language for years. Go: the language of containers and cloud tools	add `rust cargo` or `golang` to the manifest when you start (chapter 42)
beside	**C**, only to read	dwl, the kernel, most of Linux	`config.h` is C; reading it is enough at first	`src/dwl/config.h`
beside	**Lua**	neovim, yazi, mpv scripts	small: learnt in an afternoon	`~/.config/nvim/lua/plugins/local.lua`
beside	**Markdown, JSON, TOML, YAML**	notes, configs, data files	every tool reads one of them	`~/Notes/notes.md`, `waybar/config.jsonc`

Each language's first program, runnable here:

```
# 1. bash — save as ~/hello.sh, then: bash ~/hello.sh Kyoto
name=${1:-world}                  # the first argument, or "world"
echo "hello, $name — it is $(date +%H:%M)"   # e.g. hello, Kyoto — it is 21:03
```

```
# 2. Python — save as ~/hello.py, then: python3 ~/hello.py
import json, urllib.request       # parts of Python itself: nothing to install
with urllib.request.urlopen("https://wttr.in/Sharjah?format=j1") as r:
    now = json.load(r)["current_condition"][0]
print(f"{now['temp_C']} °C, {now['weatherDesc'][0]['value']}")   # e.g. 31 °C, Sunny
```

```
# 3. SQL, inside Python — save as ~/books.py, then: python3 ~/books.py
import sqlite3
db = sqlite3.connect(":memory:")  # a database that lives only while this runs
db.execute("CREATE TABLE books (title TEXT, pages INTEGER)")
db.executemany("INSERT INTO books VALUES (?, ?)", [("Kokoro", 248), ("Snow Country", 175)])
for title, pages in db.execute("SELECT title, pages FROM books WHERE pages > 200"):
    print(title, pages)           # Kokoro 248
```

```
// 4. TypeScript — save as ~/hello.ts, then: node ~/hello.ts
// (Node.js removes the types and runs the rest; it may print a one-line "experimental" note)
function area(width: number, height: number): number {
  return width * height;
}
console.log(area(1920, 1080));    // 2073600
// area("1920", 1080) would be refused by a type checker: a string is not a number
```

What AI changes, and what it does not. AI assistants now write a large share of everyday code. In the 2026 survey, 73 % of developers who use them use them daily. That makes some things matter more than before:

- **Reading code** you did not write, quickly and critically.
- **Fundamentals**: data structures, algorithms, how a computer and a network work. IEEE Spectrum expects architecture and the choice of algorithm to stay vital skills.
- **Types and tests**, which catch an assistant's mistakes before they run.
- **Version control**, so that any change, yours or an assistant's, can be undone (§41).
- **Explaining a problem precisely**: to a person, to an assistant, or in a commit message.

43.3 Read a real script: the weather line

Step 1 of the table ends in `bin/kawa-weather`: the script that writes the weather line at the top of quick settings. It is 56 lines long, a quarter of them comments, and most of the rest use one of the five ideas below, one idea each. Run it, then open it beside this page with `bat ~/repos/fedokawaii/bin/kawa-weather`.

```
$ kawa-weather
☼  Sharjah  31°C  Clear
```

(for example: the sun is the desktop's own icon, and the weather is today's.) The very first time it prints only the city: the weather is fetched in the background and shows from the next run.

1. A variable.

```
CACHE="$HOME/.cache/kawa-weather"
```

`CACHE` now holds `/home/you/.cache/kawa-weather`: bash put your home folder in place of `$HOME` before storing it, and every later `$CACHE` stands for that path. *The rule:* `NAME=value`, with no spaces round `=`; `$NAME` reads it back. A few lines higher, `. "$HOME/.config/fedokawaii/env"` reads another file into the script — that is where `WEATHER_CITY="Sharjah"` comes from (your `fedokawaii.conf`, §5.2).

2. A question, and `&&`.

```
[[ -s $CACHE ]] && read -r cached <"$CACHE"
```

`[[ -s FILE ]]` is true when the file exists and is not empty; only then does `&&` run `read`, which puts the file's first line into `cached`. *The rule:* `[[ … ]]` answers with an exit status, and `&&` (or `||`) acts on it (§25.1). Ask the same question yourself: `[[ -s ~/.cache/kawa-weather ]] && echo cached || echo empty`.

3. A command's output as a value.

```
line=$(curl -fsS … "https://wttr.in/$city?format=%t++%C&m" 2>/dev/null) || return 0
```

(shortened) `$( … )` runs curl and keeps what it prints — `+31°C Clear` — in `line`. If curl fails, offline say, `|| return 0` leaves quietly and the old line stays. *The rule:* `$(command)` turns output into a value. See the raw answer:

```
$ curl -fsS "https://wttr.in/Sharjah?format=%t++%C&m"
+31°C  Clear
```

(for example)

4. Cutting text.

```
printf '%s  %s  %s\n' "$WEATHER_CITY" "${BASH_REMATCH[1]#+}" "${BASH_REMATCH[2]}" >"$CACHE.tmp" \
  && mv -f "$CACHE.tmp" "$CACHE"
```

The line just before it checked curl's answer against a regular expression (`[[ $line =~ … ]]`) and kept its two parts: `${BASH_REMATCH[1]}`, the temperature `+31°C`, and `${BASH_REMATCH[2]}`, the sky `Clear`. `${BASH_REMATCH[1]#+}` is the temperature with one `+` cut from its start: `31°C`. (The line goes to a new file that is then renamed over the old one, so the menu never reads half a line.) *The rule:* `${NAME#text}` cuts from the start, `${NAME%text}` from the end.

```
$ line='+31°C  Clear'
$ echo "${line#+}"
31°C  Clear
```

5. Choosing by pattern.

```
case ${cond,,} in                       # the sky only, never the city's name ("Nice")
  *thunder*|*storm*)                    icon=$ICON_STORM ;;
  *snow*|*sleet*|*blizzard*|*ice*)      icon=$ICON_SNOW ;;
  *rain*|*drizzle*|*shower*)            icon=$ICON_RAIN ;;
  *fog*|*mist*|*haze*|*dust*|*sand*)    icon=$ICON_FOG ;;
  *cloud*|*overcast*)                   icon=$ICON_CLOUD ;;
  *)                                    icon=$sky ;;
esac
```

`${cond,,}` is the sky in small letters — `clear`. `case` tries each pattern from the top and runs the first that matches; `clear` matches none until `*)`, which takes anything, so the icon is the sun (or the moon at night). The comment explains why only the sky is matched: *Nice* holds `ice`, and its weather would always show snow. *The rule:* `case` picks the first pattern that matches, and the patterns are globs (§25.4).

Now read the whole file from the top. Most lines are these five ideas again; the others — a function, `refresh() { … }`, a pattern test, `[[ … =~ … ]]`, an `if`, a sum inside `(( … ))` — are in the bash manual (§43.5), and their comments say what they are for. Its self-tests are `tests/status/test_weather.sh` (§42.6).

43.4 Ideas, defined, with an example each

Idea	In one line	See it here	Taught in
process	a program while it runs, with its own number (PID)	`pgrep -a waybar` → `1234 waybar`	§26.1
signal	a short message to a process: stop, reload, …	`pkill -USR2 waybar` makes the bar re-read its files	§26.2
standard input / output / error	the three streams every program has: what it reads, what it prints, where its errors go	`ls /nope 2>/dev/null` hides the error (stream 2)	§25.2
pipe `|`	one program's output becomes the next one's input	`ls ~/Books | wc -l` counts your books	§25.1
redirection `>` `>>`	output into a file, replacing it or adding to it	`date >> ~/log.txt` adds a line	§25.1
exit status	a number each program ends with: 0 means success	`grep -q x file; echo $?` → `0` or `1`	§25.1
variable · **environment variable**	a named value; an *exported* one is passed to every program started after it	`export EDITOR=nvim` → git opens neovim	§25.3
PATH	the folders the shell searches for a command, in order	`command -v kawa-menu` → `/home/you/.local/bin/kawa-menu`	§25.3
permissions	who may read (r), write (w) and run (x) a file	`ls -l secrets.sh` → `-rw-------`: only you, never run	§24.2
root · **sudo**	the user allowed everything · run one command as root	`sudo dnf install X` asks for your password	§24.3

Idea	In one line	See it here	Taught in
package · **repository** · **dependency**	software as one file · a server that offers packages · a package another needs	`dnf repoquery --requires mpv`	ch. 27
service (systemd unit)	a program systemd starts, watches and restarts	`systemctl status bluetooth`	§28.1
configuration file	settings a program reads when it starts or reloads	`~/.config/foot/foot.ini`	§23.3
regular expression	a pattern that matches text	`rg 'kawa-[a-z]+'` finds `kawa-menu`, `kawa-lock`…	§25.4
version control · **commit** · **branch**	a history of every change · one saved change · a line of work	`git log --oneline` in any repository	ch. 41
container	a program with its own files, sharing the computer's kernel	Prowlarr runs in one (§34.5)	§34.4
compositor	the program that draws every window on the screen and handles input	dwl, on Wayland	§31.3
API · **HTTP** · **JSON**	a program's way in for other programs · how the web asks and answers · text that holds data	`kawa-weather` asks wttr.in's API (§43.3)	below
type	what kind of value something is: a number, text, a list	`area(1920, 1080)`, the TypeScript example above	§43.2
compile · **interpret**	turn the whole program into machine code first · run it line by line	dwl is compiled (C); bash and Python are interpreted	—
LLM · **token** · **context** · **prompt**	a large language model · the pieces of text it reads and writes · everything it can see at once · what you ask it	`claude` in your working copy	ch. 44

Try this. Two of these ideas in one line — redirection and the exit status.

```
$ pgrep -x waybar >/dev/null && echo "the bar runs" || echo "the bar is down"
the bar runs
```

`pgrep` ends with 0 when it finds the bar; `>/dev/null` throws away what it prints, so only the exit status is left. `&&` runs only after a 0, `||` only after anything else.

JSON, and asking it questions. Many programs answer in JSON: text that holds named fields and lists. `jq` picks out what you need. `kawa-drives` asks `lsblk` for every block device as JSON, then keeps the names of the drives:

```
jq -r '.blockdevices[] | select(.type == "disk" or .type == "rom" or .type == "loop") | .name'
```

Try this. The same kind of question, about your own disks.

```
$ lsblk -J -o NAME,TYPE | jq -r '.blockdevices[] | select(.type == "disk") | .name'
nvme0n1
zram0
```

(for example) `zram0` is the swap kept compressed in memory (§30.4): to the kernel, a disk too.

jq part	Means
`.blockdevices[]`	every entry of the list called `blockdevices`, one at a time
`select(.type == "disk")`	keep only the entries whose `type` is `disk`
`.name`	of each one left, print the name
`-r`	raw text: the names without their quotes

> **In SQL terms.** A jq filter is a small query, read left to right: `.blockdevices[] | select(.type == "disk") | .name` is `SELECT name FROM blockdevices WHERE type = 'disk'`. The `[]` is the FROM, `select()` the WHERE, and the field at the end the column list.

43.5 Where to read

Reliable, free and kept up to date. Official documentation comes first.

Subject	Read	Where
the shell and Linux, from zero	*The Linux Command Line*, William Shotts (a free book)	linuxcommand.org/tlcl.php
the missing basics: shell, editors, git, data	*The Missing Semester of Your CS Education* (MIT)	missing.csail.mit.edu
programming from the start	*CS50* (Harvard, free)	cs50.harvard.edu/x
bash, the reference	the GNU Bash manual · `man bash`	gnu.org/software/bash/manual
shell mistakes, explained	the ShellCheck wiki: one page per finding	shellcheck.net/wiki
git	*Pro Git* (free) · `man git-commit`	git-scm.com/book
Python	the official tutorial	docs.python.org/3/tutorial
regular expressions	Python's *Regular Expression HOWTO*	docs.python.org/3/howto/regex.html
SQL	SQLite's language reference · PostgreSQL's tutorial	sqlite.org/lang.html · postgresql.org/docs/current/tutorial.html
TypeScript	the TypeScript Handbook	typescriptlang.org/docs/handbook
JavaScript and the web	MDN Web Docs	developer.mozilla.org
Rust	*The Rust Programming Language*	doc.rust-lang.org/book
Go	*A Tour of Go*	go.dev/tour
Lua	*Programming in Lua* · neovim's `:help lua-guide`	lua.org/pil
C	the dwl wiki and its `config.def.h` · `man 3 printf` and friends	codeberg.org/dwl/dwl/wiki
machine learning	Google's Machine Learning Crash Course · fast.ai's *Practical Deep Learning*	developers.google.com/machine-learning/crash-course · course.fast.ai
prompting an assistant	Anthropic's prompt engineering guide	docs.claude.com
Fedora	the Fedora documentation · `man dnf5`	docs.fedoraproject.org
any Linux program	its man page · the Arch Wiki (written for Arch, mostly true here)	`man NAME` · wiki.archlinux.org

43.6 A ladder of first projects

Each step is a real change to this desktop, made the chapter 42 way, and each teaches one more thing.

Step	Project	You learn
1	three aliases and a function in `~/.bashrc.d/local.sh` (§42.1)	variables, quoting, functions
2	a script in `~/.local/bin` that tells you the free space on `/` and `~` — §25.6 writes it with you	arguments, `if`, pipes, exit status
3	a new key in `config.h`: **Super+Shift+B** opens Brave in a private window — the same cycle as the gaps in §42.4	reading C, rebuild and apply
4	a bar module that shows how many updates wait (`dnf check-update -q | wc -l`, every hour)	waybar's JSON, signals, a helper script
5	a Python script that keeps each day's weather in an SQLite file and prints the week	Python, JSON from an API, SQL
6	a small TypeScript tool that turns your notes' headings into a table of contents	types, Node.js, reading files
7	the same disk-space tool from step 2, in Rust or Go	compiling, a type system, a package manager

Commit each step in the working copy's history, and run the self-tests of any `kawa-…` script you change on the way (§42.6).

Sources for the numbers in this chapter: the Stack Overflow Developer Survey 2025 and its 2026 results; GitHub Octoverse 2025; IEEE Spectrum, The Top Programming Languages 2025.

What you learnt

- Learn one language at a time: bash first, then Python, then SQL — each step uses the one before.
- Every part of this desktop is a short file you can read: `kawa-weather` is mostly five ideas, in 56 lines.
- `NAME=value` stores, `[[ … ]]` asks, `$( … )` captures output, `${NAME#…}` cuts, `case` picks by pattern.
- `jq` asks JSON a question the way `SELECT … WHERE` asks a table.
- The manual first (`tldr`, `man`, `--help`), then the sources in §43.5; a first project from the ladder above.

Check yourself

1. What does `${line#+}` make of `+31°C Clear`? → *`31°C Clear`: one `+` cut from the start*
2. Two patterns of a `case` match. Which one runs? → *the first, from the top*
3. In `jq '.blockdevices[] | select(.type == "disk") | .name'`, which part plays WHERE? → *`select(.type == "disk")`*

44. Learning with an AI assistant

> **In this chapter**
> - start Claude Code in a folder, steer it and stop it
> - ask it to explain, plan and test, rather than only to write
> - keep control: commit first, read every change, keep secrets out
>
> **You need:** ch. 43 (learning to program) · ch. 41 (git).

An AI assistant can read this desktop's scripts with you, explain any line and propose a change. Used well, it is a patient teacher who never tires of a question; used carelessly, it changes files you have not read. This chapter shows the first way.

44.1 Claude Code, an assistant in the terminal

Type `claude` in a terminal, inside the folder you want to work on. It reads and edits files there and runs commands, asking first. It came from Anthropic's signed dnf repository (stable channel), so it updates with `dnf upgrade` (and `update`) and does not update itself; `claude doctor` checks the installation.

Signing in

Either sign in with a Claude account (Pro, Max, Team or Enterprise): the first start opens the sign-in page in Brave. Or use an API key — and keep it only in `~/.config/fedokawaii/secrets.sh` as `export ANTHROPIC_API_KEY=…` (§25.8); Claude Code asks once whether to use the key it finds in the environment. Never put a key in the repository or in a project's settings files.

Keys or input	Does
Esc	stop Claude mid-answer, keeping what it has done
Esc Esc	with an empty prompt: go back to an earlier point (also `/rewind`)
Shift+Tab	switch mode: ask before edits, accept edits, **plan** (it only reads and proposes)
`\` then Enter · Ctrl+J	a new line in the prompt
Ctrl+V	paste a picture from the clipboard — a screenshot, say
Ctrl+G	write the prompt in neovim
`!command`	run a shell command and show Claude the output
`@path`	point at a file
`/init`	write a `CLAUDE.md` describing the project, read at every start
`/clear` · `/compact`	start over · summarise to free room
`/model` · `/usage` · `/resume` · `/theme`	model · cost and limits · an earlier session · colours

`claude -c` continues the last conversation in this folder, `claude -r` picks an older one, and `claude -p "question"` answers once and exits, which makes it usable in pipes: `journalctl -b -p err | claude -p "what stands out here?"`.

Recommended. Use plan mode for anything touching this repository or the system, read the plan, then let it work. Run `claude` in your working copy, `~/repos/fedokawaii`, to ask about the desktop itself — it can read the scripts and this book (`docs/`). For colours that follow the theme, `/theme` → the *ANSI colours only* variant of dark or light.

Pitfalls. It acts as you: it can read anything you can, `secrets.sh` included, and every command it runs has your rights. Read the permission prompts instead of approving them by habit. When it edits a managed config in `~/.config`, the next Theme run replaces the change — point it at the repository copy instead.

Read more: Claude Code's documentation; interactive mode lists every key, and commands every `/` command.

44.2 Learning with it, safely

Claude Code reads the files in the folder you start it in, so start it in your working copy.

Try this. Check that nothing is uncommitted, then start it.

```
$ cd ~/repos/fedokawaii
$ git status --short
$ claude
```

You typed	It means
`cd ~/repos/fedokawaii`	the folder it may read: the desktop's scripts, configs and this book
`git status --short`	no output: nothing uncommitted, so any change it makes is one step from undone
`claude`	starts the assistant here; the first time, you sign in (§44.1)

Press **Shift+Tab** until it is in plan mode, and ask your first question: *"Explain `bin/kawa-weather` line by line. Do not edit anything."* Compare its answer with §43.3: where the two differ, the script itself decides. **Esc** stops it mid-answer.

Ask	You learn
"Explain line 120 of `bin/kawa-quick`, word by word."	what each piece of the syntax does
"Why does ShellCheck say SC2086 on line 40? Show the fix and why it is safer."	the rule behind the warning
"I want the bar's CPU module to turn red above 90 %. Which file, and what would you change? Do not edit yet."	where things live, and the plan before the change
"Add a case for my change to `tests/drives/test_drives.sh`, in the style of the others, and explain each check."	how a behaviour is pinned down (§42.6)
"Here is the error from `fedokawaii log`. What does it mean, step by step?"	how to read an error

Rules that keep it safe:

1. **Commit first** (lazygit, **c**). Whatever it changes is then one key away from undone.
2. **Read every change before you apply it.** `fedokawaii diff` shows them all, and `bash tests/run.sh AREA` checks a changed script (§42.6).
3. **Never paste a secret** — `secrets.sh`, a token or a password — into a prompt.
4. **Ask for sources** when it states a fact about a program, and check them in the manual. In the 2026 survey, 93 % of developers said they need sources to trust an AI's answer.
5. **Type the fix yourself** now and then, instead of accepting it. It is slower, and it is how the syntax becomes yours.

Part VI, looking back.

Chapter	You can now
ch. 40	trace any colour, font, icon or pointer to the file it comes from
ch. 41	record, compare, branch and undo changes with git and lazygit
ch. 42	change any part of the desktop so that every update keeps it
ch. 43	choose what to learn next, and read the desktop's own scripts
ch. 44	learn with an assistant without losing control of your files

The appendices that follow are for looking things up: the software and why each piece, where everything lives, the keys, the commands by job, the terms, and the sources.

What you learnt

- `claude` in a folder starts an assistant that reads, edits and runs commands there, asking first; **Esc** stops it mid-answer.
- Plan mode (**Shift+Tab**) makes it read and propose only: read the plan before you let it work.
- Ask it to explain, to find the file, to plan and to write tests — the explanation is what you learn from.
- Commit first, read `fedokawaii diff`, run the self-tests, and never paste a secret.

Check yourself

1. Where does an API key for Claude Code belong? → *only in `~/.config/fedokawaii/secrets.sh`, as `export ANTHROPIC_API_KEY=…`*
2. Why commit before a session? → *every change it makes is then one key away from undone*
3. It edited `~/.config/waybar/style.css`. What does the next theme run do? → *it puts the toolkit's version back: point it at the working copy instead*

Appendices

A. Software, and why each piece

The full table is `PACKAGES.md`, in the project and in your working copy (`~/repos/fedokawaii/PACKAGES.md`); these are the choices worth knowing. Adding or taking out a package is §42.11.

Job	Tool	Why this one
compositor	dwl 0.9	small and fast; patched for gaps, retro frames, centred dialogs, Arabic-proof shortcuts, the layout in the bar, menus closed by a click outside, monitors set up when plugged in
bar	waybar	its own clock; custom modules fed by one status reader
launcher and every menu	fuzzel	one look for all menus; no second menu program
terminal	foot	fast, live dark/light palettes, prompt integration, pictures (Sixel)
login	greetd + agreety	a plain text login, no extra greeter
lock / idle	swaylock · swayidle · wlopm	the padlock is a picture, the ring sits in its keyhole; wlopm powers the screens off without moving windows
transitions	`kawa-veil` (this repository, C)	nothing packaged does animated changes on dwl; built locally
clipboard	wl-clipboard · cliphist · fzf · `kawa-clipkeep` (this repository, C)	history with a searchable window and picture previews (ImageMagick, as sixels); copies that outlive their program — the one clipboard keeper for wlroots is not packaged for Fedora, and a small one is quick to check
notifications	mako	
drives	udiskie (udisks) · `kawa-drives` (this repository)	udiskie only mounts; the notices, the bar's mark, the menu, yazi's **M** and an eject that says *safe to unplug* are one small script, so a GUI file manager and its daemons are not needed
music meter	cava	the only extra process for the bar, asleep in silence
brightness	ddcutil · logind (a laptop's own screen)	the monitor's real backlight
fetch	`fetch` (this repository)	replaces fastfetch: a few milliseconds, pictures drawn here
torrents	torrra + Prowlarr	a terminal client that searches through Prowlarr's hundreds of indexers
video	mpv (+ mpv-mpris, yt-dlp, Node.js)	plays nearly any file, and web links through yt-dlp; Node.js answers YouTube's JavaScript challenges for yt-dlp

Removed as redundant across versions: tuigreet (agreety is part of greetd), wlogout (a fuzzel menu), fastfetch (own `fetch`), a separate clock script (waybar's clock), starship (native prompt), GStreamer plugins (nothing uses them), and the Rust toolchain (release binaries instead).

B. Where everything lives

Every path this desktop uses, in three tables: your own files, each program's files, and what the scripts put on the system. Settings that Theme manages are replaced at its next run if edited in place (§42.5); data and caches belong to the programs. The columns of §B.2 follow the XDG folders (§23.3): settings in `~/.config`, data in `~/.local/share`, state in `~/.local/state`, cache in `~/.cache`.

B.1 Your files

Path	What	Notes
`~/Documents/` · `~/Pictures/` · `~/Videos/` · `~/Music/`	what you keep	the standard folders: file dialogs and yazi know them
`~/Downloads/`	arrivals: Brave, torrra and yt-dlp save here	its own Btrfs subvolume, outside every snapshot (§37.2)
`~/Books/`	your library — the books window lists it (Super+R, §16.2)	folders inside are fine
`~/Notes/notes.md`	your notes (Super+N, §16.3)	plain Markdown
`~/Pictures/Screenshots/`	Print, Shift+Print, satty's Ctrl+S, mpv's `s`	
`~/Pictures/Wallpapers/`	your own wallpapers, on both themes	
`~/repos/`	your git repositories	
`~/.config/fedokawaii/secrets.sh`	API keys, as `export NAME=value` (and Prowlarr's, deliberately without `export`)	yours, mode 600; nothing else holds a key

The standard folders were made at setup by `xdg-user-dirs-update`, which records them in `~/.config/user-dirs.dirs`; programs ask for them by name:

```
$ xdg-user-dir DOWNLOAD
/home/you/Downloads
```

B.2 Each program's files

Program	Settings	Data	Cache, state, logs
the desktop (dwl, bar, menus)	`~/.config/fedokawaii/` (`env`, `themes/`, `current`, `icons.sh`, `keybinds.txt`, `cava.conf`, `fzf.opts` — the look of every list) · `~/.config/{waybar,mako,fuzzel,dwl}/` · dwl's `~/.local/src/dwl/config.h`	`~/.local/share/fedokawaii/` (pixel art, `wallpapers.list`, the pack in `wallpapers/`, `upstream/`)	`~/.cache/dwl.log`, `waybar.log`, `fedokawaii-theme.log`, `kawa-wallpaper/` · `~/.local/state/fedokawaii/` (each theme's wallpaper, stamps, `backup/`, `lazy-lock/`)
upkeep (Super+U)	`~/.config/fedokawaii/upkeep.conf` (the season) · `~/.config/systemd/user/fedokawaii-remind.{timer,service}` (the reminder)	—	`~/.cache/fedokawaii/` (when the next Fedora came out, the day of the last reminder)
foot and bash	`~/.config/foot/foot.ini` · `~/.bashrc.d/fedokawaii.sh` · `~/.inputrc`	`~/.bash_history` · zoxide's folders: `~/.local/share/zoxide/`	`~/.cache/fedokawaii/` (fzf's and zoxide's shell code)
the clipboard	—	`~/.cache/cliphist/db` (the history)	

Program	Settings	Data	Cache, state, logs
books and notes	—	`~/Books/` · `~/Notes/`	`~/.cache/kawa-books/` (covers, page counts)
neovim	`~/.config/nvim/` (`lua/config/lazy.lua`, `lua/plugins/colorscheme.lua` from Theme; the rest is LazyVim's starter, yours)	`~/.local/share/nvim/` (plugins in `lazy/`, Mason's tools in `mason/`) · `~/.config/nvim/lazy-lock.json` (plugin versions)	`~/.local/state/nvim/` (undo history, swap files, logs)
lazygit	`~/.config/lazygit/config.yml`	—	`~/.local/state/lazygit/`
yazi (files and dialogs)	`~/.config/yazi/theme.toml`, `init.lua`, `keymap.toml`, `plugins/drives.yazi/`, `plugins/smart-enter.yazi/` · the dialog's keys: `~/.config/fedokawaii/yazi-pick/` · `~/.config/xdg-desktop-portal-termfilechooser/config`	the trash: `~/.local/share/Trash/`	`~/.local/state/yazi/`
drives (udiskie, kawa-drives)	`~/.config/udiskie/config.yml`	—	which drives are open: `$XDG_RUNTIME_DIR/kawa-drives/` (cleared at each login)
zathura	`~/.config/zathura/zathurarc`	`~/.local/share/zathura/bookmarks.sqlite` (each book's page, bookmarks)	
mpv and yt-dlp	`~/.config/mpv/` (`mpv.conf`, `scripts/`) · `~/.config/yt-dlp/config`	—	`~/.local/state/mpv/` (where each video stopped)
torrra (torrents)	`~/.config/torrra/config.toml` — Prowlarr's address, the download folder, colours; never the key	the program: `~/.local/share/pipx/venvs/torrra/` · downloads: `~/Downloads/`	
Prowlarr	`~/.config/containers/systemd/prowlarr.container` (how it runs) · its key: `secrets.sh`	`~/.config/prowlarr/` (its database: indexers, login, history) · the image: `~/.local/share/containers/`	`journalctl --user -u prowlarr`
Brave Origin	`~/.config/BraveSoftware/Brave-Origin/` (profile, settings, extensions)	the same folder	`~/.cache/BraveSoftware/`
spotify_player	`~/.config/spotify-player/` (`app.toml`, `theme.toml`)		`~/.cache/spotify-player/` (login, cover art)
btop	`~/.config/btop/btop.conf`		
tldr	its colours and refresh in `~/.bashrc.d/fedokawaii.sh`		`~/.cache/tldr/` (the pages)
Steam and games		`~/.local/share/Steam/` · GE-Proton in `compatibilitytools.d/` there	
GTK, Qt, pointers	`~/.config/gtk-4.0/` · `~/.config/qt6ct/` · `/etc/dconf`	`~/.local/share/themes/Fedokawaii*` · `~/.local/share/icons/Fedokawaii/` (pointers)	

B.3 On the system

Path	What
`/usr/local/lib/fedokawaii/`	the toolkit: this project, root's own copy, run by every update, repair and move (§5.5)
`/usr/local/bin/`	`fedokawaii` (the control, Super+U), `dwl` and `dwl.prev` (the build before it), `start-dwl`, `kawa-greeter`, `kawa-torrra`
`/usr/local/lib/fedokawaii-wlroots-0.20/`	dwl's own copy of wlroots, kept from Fedora's build (§36.7)
`/usr/local/sbin/`	`kawa-snap` (the DNF snapshots)
`/usr/local/libexec/xdg-desktop-portal-termfilechooser` (+ its service files under `/usr/local/lib` and `/usr/local/share`)	the yazi file dialog's portal, built by setup
`/etc/dnf/libdnf5.conf.d/90-fedokawaii.conf` · `/etc/dnf/libdnf5-plugins/actions.d/`	DNF's settings · the snapshot around every DNF run
`/etc/tuned/profiles/fedokawaii/` · `/etc/NetworkManager/conf.d/90-fedokawaii.conf`	the performance profile · Wi-Fi power saving off
`/etc/dconf/db/local.d/00-fedokawaii`	GTK's defaults: theme, icons, pointers, fonts
`/etc/greetd/fedokawaii.toml` · `/etc/systemd/system/greetd.service.d/fedokawaii.conf`	the login: agreety on tty1 · greetd pointed at that file
`/etc/snapper/configs/` (`root`, `home`) · `/.snapshots` and `/home/.snapshots`	the snapshot settings · the snapshots
`/boot/grub2/themes/sumi/`	the boot menu theme
`/var/lib/fedokawaii/` · `/var/log/fedokawaii.log` · `/var/log/fedokawaii-{check,verify}.log`	setup's progress and the records the menu reads (`last-update`, `last-verify`, `last-upgrade`, `release-N`, `snapshots`, and what to look at after a move: `unavailable`, `rpmnew`) · the log · the check and verify logs (root's alone: `fedokawaii log`)

C. Keys at a glance

The keys you use most, one table per program, to keep beside the keyboard. Each is a selection; the full lists are in §7.2 (and on screen with **Super**+/) for the desktop, §11.9 and §18.1 for the terminal, §10.3 for yazi, the end of §17.22 for neovim and §41.17 for lazygit. **Super** is the Windows key.

The desktop. The keys in **bold** are the dozen to learn first.

Keys	Does
Super+Return · Space · B · E	terminal · launcher · Brave Origin · files (yazi)
Super+X · V · /	quick settings · clipboard history · every key, searchable
Super+A · C · W	sound · calendar · wallpaper
Super+R · N	books · notes (again to put them away)
Super+Q	close the window
Super+J / K	focus the next · previous window
Super+H / L	shrink · grow the main area
Super+Shift+Return · F · Shift+Space	move to the main area · fullscreen · float or unfloat
Super+T / Y / M	tiled · floating · monocle
Super+1…9 · Shift+1…9	go to a workspace · send the window there
Super+Tab	back to the previous workspace
Super+Escape · Super+Shift+E	lock · power menu
Super+Shift+W	dark ⇄ light theme
Print · Shift+Print · Super+Print	screenshot a region · the whole screen · a region, then annotate it
Super+U · Super+Shift+U	the system's control · change the desktop

The terminal (foot and bash).

Keys	Does
Ctrl+Shift+C · Ctrl+Shift+V	copy · paste (selecting with the mouse copies too)
Ctrl+C	stop the program that is running
q · Esc, then `:q!`	leave a pager or a man page · leave neovim without saving
Ctrl+D	end of input; on an empty line, close the terminal
Tab	complete a name, ignoring case; one press lists the choices
Up / Down	earlier · later commands that start with what you typed
Ctrl+R	search the whole history (fzf)
Ctrl+T · Alt+C	insert a path · go into a folder (fzf)
Alt+.	the last argument of the previous command
`cd WORD`	jump to a folder you use whose name matches (zoxide)
Ctrl+Shift+N	a new terminal in this folder
Shift+PageUp / PageDown	scroll back (the mouse wheel too)
Ctrl+Shift+R	search the scrollback
Ctrl+Shift+Z / X	jump to the previous · next prompt
Ctrl+Shift+G	the last command's output, in neovim

yazi (Super+E, and every file dialog).

Keys	Does
h j k l (or the arrows)	parent folder · down · up · into the folder
Enter · O	open (a folder: go in) · choose the program
g h · g d · g c	home · Downloads · `~/.config`
Space · v · Ctrl+A	select · select a range · select all
y · x · p	copy · cut · paste
d · D	to the trash · **delete for good**
a · r	create (end the name with `/` for a folder) · rename
.	show · hide hidden files
s · S	search file names (fd) · file contents (ripgrep)
f · /	filter as you type · find
z · Z	jump below here (fzf) · to a folder you use (zoxide)
M · g m	drives: open or eject one · go where they open
c c	copy the file's full path
Tab	details of the file
~ · q	every key · quit

neovim.

To	Keys
get back to Normal mode	Esc
save · quit · both · quit without saving	Ctrl+S · `:q` · `:wq` · `:q!`
undo · redo · repeat the last change	`u` · Ctrl+R · `.`
move by character · word · to the line's start, end	`h` `j` `k` `l` · `w` `b` · `0` `$`
top · bottom of the file · line 42	`gg` · `G` · `42G`
type before · after the cursor · on a new line	`i` · `a` · `o`
delete a character · a line	`x` · `dd`
change a word · inside quotes	`ciw` · `ci"`
copy a line · paste	`yy` · `p`
select characters · lines	`v` · `V`
search · next · previous	`/text` · `n` · `N`
replace in the whole file	`:%s/old/new/g`
find a file · text	Space Space · Space /
the file tree · lazygit	Space e · Space g g
every key	Space, then wait

lazygit.

To	Keys
move between panels · jump to one	← → · 1 … 5
every key here	?
stage a file · all files	Space · a

To	Keys
commit · amend the last commit	c · A
discard a file's changes	d
stash · bring it back	s · g (in *Stash*)
undo · redo the last git action	z · Z
pull · push · fetch	p · P · f
new branch · switch to one	n · Space (in *Branches*)
merge into this branch · rebase onto	M · r
revert a commit	t
look inside a commit · back out	Enter · Esc
continue or abort a merge or rebase	m
quit	q

D. Commands by job

Every command here is installed on this desktop: Fedora's minimal set (coreutils, util-linux, procps-ng, iproute, iputils, findutils, grep, sed, gawk, tar, systemd, dnf5, rpm, and from the same `@core` group `less`, `man`, `curl`, firewalld, SELinux's tools and the audit tools) plus what the manifest adds. `tldr NAME` gives more examples; `man NAME` the full manual — §11.7 shows how to read both, and `--help`. Rows whose *Does* changes something — copy, move, delete, install — act at once: read the example before you run it.

D.1 Files and folders

Command	Does	Example
`ls` · `ll` · `lt`	list a folder (eza here): plain · everything with sizes and git state · as a tree	`ll ~/Documents`
`command ls -l`	coreutils' own long listing, with permissions and link counts	`command ls -l /usr/local/bin`
`cd` · `cdi`	change folder; zoxide jumps by a remembered word · pick one in fzf	`cd repos`
`pwd`	print the current folder	`pwd`
`mkdir -p`	make a folder and any parents	`mkdir -p ~/Documents/2026/tax`
`cp` · `cp -a`	copy · copy keeping owner, times and links	`cp -a ~/Notes /mnt/backup/`
`mv`	move or rename	`mv report.pdf ~/Documents/`
`rm` · `rmdir`	delete files · delete an empty folder (no undo; prefer the trash)	`rm -i old.txt`
`gio trash`	move to the trash instead	`gio trash old.txt` · `gio trash --empty`
`ln -s`	make a symbolic link	`ln -s ~/Books/manual.pdf ~/Desktop/`
`touch`	create an empty file, or update its time	`touch notes.md`
`stat`	every detail of one file: inode, links, permissions, times	`stat -c '%a %U %s %n' ~/.bashrc`
`realpath` · `readlink -f`	the full path, links resolved	`realpath ~/.config/fedokawaii/current`
`yazi`	the file manager (Super+E)	`yazi ~/Downloads`

D.2 Viewing

Command	Does	Example
`cat`	print a file	`cat /etc/fedora-release`
`bat`	print with line numbers and colour	`bat ~/.config/foot/foot.ini`
`less`	page through: Space, `b`, `/word`, `n`, `q`	`less ~/.cache/dwl.log`
`head` · `tail`	the first · last lines	`tail -n 40 ~/.cache/dwl.log`
`tail -f`	follow a growing file; Ctrl+C stops	`sudo tail -f /var/log/fedokawaii.log`
`file`	what kind of file it really is	`file ~/Downloads/*`
`wc`	count lines, words, bytes	`wc -l ~/Notes/notes.md`
`nvim -R`	open read-only in neovim	`nvim -R /etc/greetd/fedokawaii.toml`

D.3 Searching

Command	Does	Example
`fd`	find files by name (regex; `-g` for a glob), fast, skipping hidden and ignored ones	`fd -e pdf . ~/Documents`
`find`	find files by any property	`find ~ -xdev -size +1G`
`rg`	search inside files (ripgrep)	`rg -n 'kawa-[a-z]+' ~/.config/waybar`
`grep`	search lines in a file or a pipe	`grep -i error ~/.cache/waybar.log`
`fzf`	pick from any list by typing a few letters	`nvim "$(fd -e md | fzf)"`
`command -v` · `type -a`	where a command comes from: program, alias or function	`type -a cd`
`rpm -qf`	which package a file belongs to	`rpm -qf "$(command -v waybar)"`

D.4 Text processing

Command	Does	Example
`sort`	sort lines (`-n` numbers, `-h` sizes like 2G, `-r` reverse)	`du -sh ~/* | sort -h`
`uniq -c`	collapse repeated lines, counting them (sort first)	`journalctl -b -o cat -u NetworkManager | sort | uniq -c | sort -rn | head`
`cut`	take fields by a separator	`cut -d: -f1,7 /etc/passwd`
`awk`	fields and small programs	`ps -eo rss,comm --sort=-rss | awk 'NR>1 && NR<7 {printf "%6.0f MB %s\n", $1/1024, $2}'`
`sed`	replace or delete by pattern	`sed -n '/^\[preferred\]/,$p' ~/.config/xdg-desktop-portal/wlroots-portals.conf`
`tr`	translate or delete characters	`echo "$PATH" | tr : '\n'`
`jq`	query and reshape JSON	`pw-dump | jq -r '.[] | .info.props["node.description"] // empty' | sort -u`
`column -t`	line text up in columns	`mount | column -t | head`
`tee`	write to a file and the screen at once	`dnf repoquery --userinstalled | tee ~/pkgs.txt`
`xargs`	turn lines into arguments	`fd -0 -e tmp | xargs -0 -r rm` (deletes them, spaces in names and all)
`git diff --no-index`	compare two files or folders	`git diff --no-index old.conf new.conf`

D.5 Archives

Command	Does	Example
`tar -czf` · `tar -xzf`	pack into · unpack from a gzip-compressed tar	`tar -czf notes.tar.gz ~/Notes`
`tar -tf`	list what is inside	`tar -tf notes.tar.gz | head`
`7z a` · `7z x` · `7z l`	make · extract · list 7z, zip and most other formats	`7z x ~/Downloads/photos.zip -o"$HOME/Pictures/trip"`
`bsdtar -xf`	extract almost any archive, comic books included	`bsdtar -xf book.cbz -C pages/`
`sha256sum`	the fingerprint of a file, to compare with a published one	`sha256sum fedora.iso`

D.6 Processes

Command	Does	Example
`ps`	list processes (`-ef` all; `--forest` as a tree)	`ps -ef --forest | less`
`pgrep -a`	find processes by name	`pgrep -a waybar`
`kill`	send a signal (TERM by default) to a PID	`kill 4120` · `kill -USR2 4120`
`pkill`	the same, by name	`pkill -USR2 waybar` (reloads the bar)
`top` · `btop`	live view of CPU, memory and processes	`btop`
`nice` · `renice`	start at · change to a lower priority	`nice -n 19 7z a big.7z ~/Videos`
`jobs` · `fg` · `bg`	this shell's jobs	after **Ctrl+Z**: `bg`
`timeout`	stop a command after a time	`timeout 10 ping fedoraproject.org`
`watch`	run a command again every two seconds	`watch -n 5 free -h`
`cat /proc/PID/status`	everything the kernel knows of one process	`grep -E 'State|VmRSS' /proc/"$(pgrep -nx waybar)"/status`

D.7 Services and logs

Command	Does	Example
`systemctl status`	a unit's state, processes and last log lines	`systemctl status NetworkManager`
`systemctl --user status`	one of your units	`systemctl --user status wireplumber`
`systemctl start` · `stop` · `restart`	now	`sudo systemctl restart bluetooth`
`systemctl enable` · `disable`	at every boot	`sudo systemctl enable --now fstrim.timer`
`systemctl list-timers`	every timer, next and last run	`systemctl --user list-timers`
`systemctl cat`	a unit with its drop-ins	`systemctl cat greetd`
`systemctl list-units --failed`	what failed	`systemctl --user list-units --failed`
`journalctl`	read the journal (filters in §28.5)	`journalctl -b -p err`
`systemd-analyze`	boot time, the slow units, the chain	`systemd-analyze blame | head`
`systemd-cgls` · `systemd-cgtop`	the cgroup tree · live use per group	`systemd-cgtop`
`loginctl`	sessions, users, linger	`loginctl session-status`
`busctl`	D-Bus: names, objects, calls	`busctl --user list | grep mpris`
`systemd-run --user`	run a command as a unit of its own	`systemd-run --user --scope 7z a big.7z ~/Videos`

D.8 Packages

Command	Does	Example
`dnf search` · `dnf info`	find a package · describe one	`dnf info btop`
`dnf install` · `dnf remove`	add · take away (and what only it needed)	`sudo dnf install hyperfine`

Command	Does	Example
`dnf upgrade --refresh`	update everything (the season's update does more, §36.2)	`sudo dnf upgrade --refresh brave-origin`
`dnf provides`	which package has a file or command	`dnf provides '*/bin/7z'`
`dnf repoquery --requires` · `--recommends`	what a package needs · would like	`dnf repoquery --requires waybar`
`dnf repoquery --installed --whatrequires`	what installed packages need this one	`dnf repoquery --installed --whatrequires libva`
`dnf repoquery --userinstalled`	what was asked for by name	`dnf repoquery --userinstalled | wc -l`
`dnf repolist`	the enabled repositories	`dnf repolist`
`dnf history`	past transactions; undo one	`dnf history list | head`
`dnf group list`	the comps groups	`dnf group list --installed`
`rpm -qi` · `-ql` · `-qf`	one package's details · its files · a file's package	`rpm -ql mpv-mpris`
`rpm -q --changelog`	what changed in recent builds	`rpm -q --changelog greetd | head`
`rpm -V`	check a package's files against how it was installed	`rpm -V greetd`
`rpmkeys --list`	the trusted signing keys	`rpmkeys --list`

D.9 Users and permissions

Command	Does	Example
`id` · `id -nG`	your UID, groups and SELinux context · group names	`id -nG`
`whoami` · `groups`	your name · your groups	`whoami`
`getent passwd` · `getent group`	a user's · a group's entry	`getent group wheel`
`sudo`	run one command as root	`sudo dnf upgrade --refresh`
`sudoedit`	edit a system file safely, in neovim	`sudoedit /etc/systemd/journald.conf.d/50-size.conf`
`passwd`	change your password	`passwd`
`chmod`	change permissions	`chmod 600 ~/.config/fedokawaii/secrets.sh`
`chown`	change the owner (root only)	`sudo chown you:you ~/somefile`
`umask`	the mask for new files	`umask`
`getenforce` · `sestatus`	SELinux mode · full status	`getenforce`
`ls -Z` · `id -Z` · `ps -eZ`	SELinux labels of files · of you · of processes	`command ls -Z ~`
`restorecon -v`	put back a file's proper SELinux label	`restorecon -v ~/.local/bin/myscript`
`ausearch -m avc`	SELinux refusals	`sudo ausearch -m avc -ts recent`

D.10 Disks and file systems

Command	Does	Example
`lsblk`	disks and partitions as a tree	`lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINTS`
`findmnt`	what is mounted where, as a tree	`findmnt -no OPTIONS /`

Command	Does	Example
`df -h`	space per file system (on Btrfs, ask Btrfs: §38.2)	`df -h / /boot`
`du -sh`	space a folder's files use	`du -sh ~/Videos`
`btrfs filesystem usage`	the real space on Btrfs	`sudo btrfs filesystem usage /`
`btrfs subvolume list`	the subvolumes and snapshots	`sudo btrfs subvolume list /`
`snapper list` · `snaps`	the snapshots of `/`	`snaps`
`snapper status`	what changed between two snapshots	`sudo snapper -c root status 41..42`
`swapon --show` · `zramctl`	swap in use · the zram device	`swapon --show` → `/dev/zram0 partition 8G 0B 100`
`blkid`	file system types, labels and UUIDs	`sudo blkid /dev/nvme0n1p3`
`mount` · `umount`	attach · detach a file system	`sudo mount /dev/sdb1 /mnt`
`fstrim`	tell the SSD which blocks are free (a weekly timer does it)	`sudo fstrim -v /`

D.11 Network

Command	Does	Example
`ip -br addr` · `ip route`	addresses per card · the routes, with the default gateway	`ip route show default`
`nmcli`	NetworkManager: devices, connections, Wi-Fi	`nmcli device wifi list`
`nmtui`	the same, as a menu	`nmtui`
`ss -tlnp`	what listens on which port (your own processes named)	`ss -tlnp`
`ping -c 3`	is a host reachable	`ping -c 3 1.1.1.1`
`tracepath`	the route to a host, hop by hop	`tracepath fedoraproject.org`
`curl`	fetch a URL	`curl -sS -o /dev/null -w '%{http_code}\n' https://fedoraproject.org/`
`resolvectl`	DNS: status, look-ups, cache	`resolvectl query fedoraproject.org`
`firewall-cmd`	the firewall's zones, services and ports	`sudo firewall-cmd --list-all`
`hostnamectl`	the machine's name and identity	`hostnamectl`

D.12 Hardware

Command	Does	Example
`lspci -k`	PCI devices and the driver holding each	`lspci -k -d ::0300` (the GPU)
`lsusb`	USB devices	`lsusb`
`lscpu`	the processor: model, cores, caches	`lscpu | head`
`free -h`	memory and swap in use	`free -h`
`uptime`	how long since the start, and the load	`uptime`
`uname -r`	the running kernel	`uname -r`
`cat /etc/fedora-release`	the release	`cat /etc/fedora-release`
`vainfo`	the video-decoding driver and its codecs	`vainfo --display drm`

Command	Does	Example
`wlr-randr`	monitors, modes and refresh rates (in the session)	`wlr-randr`
`ddcutil`	the monitor's own settings over DDC/CI	`ddcutil getvcp 10`
`fwupdmgr`	firmware inside devices: list, check, update	`fwupdmgr get-updates`
`lsmod` · `modinfo`	loaded kernel modules · one module's details	`modinfo -F description amdgpu`
`journalctl -k`	the kernel's messages	`journalctl -k -b -g firmware`

E. Glossary, A–Z

Every term the book defines, one line each, with the place that teaches it. A word used in two fields carries the field in brackets: *buffer (neovim)* is not *buffer (graphics)*. The commands themselves are in appendix D.

Term	Meaning
ACL	extra permissions for further users or groups on one file; `ls -l` shows a `+` (§24.2)
activation	starting a service when the first message for its D-Bus name arrives, not at login (§32.1)
agreety	greetd's plain text greeter on tty1: it asks for your user name and password (§6.1)
alias · function	a short word typed in place of a longer command · a named piece of shell code that takes arguments (§25.1)
ALSA	the kernel's sound drivers and their interface; PipeWire sits on top of it (§31.8)
AMDVLK	AMD's own open Vulkan driver, ended in 2025 in favour of RADV (§31.2)
ANV	Mesa's Vulkan driver for Intel graphics (§31.2)
API key	a long password one program shows another; kept only in `secrets.sh` (ch. 20)
API · HTTP · JSON	a program's way in for other programs · how the web asks and answers · text that holds data (§43.4)
apply	make the working copy the toolkit and put it in place: `fedokawaii apply` (§42.7)
archive	one file holding a whole folder, often compressed: `.zip`, `.tar.gz` (§3.5)
argument	what a command acts on, usually a file or folder: `~/Downloads` in `ls ~/Downloads` (§11.6)
autoremove	`dnf autoremove`: removes the dependencies no user-installed package needs any more (ch. 38)
AVC	*access vector cache*, the part of SELinux that decides; a *denied* AVC in the audit log is a refusal (§34.3)
backend (portal)	the program that carries out a portal's request: wlr, gtk or termfilechooser (§32.2)
backup	a copy of your files on another disk or place; unlike a snapshot, it survives losing the disk (§37.1)
bitrate	how many bits one second of a stream takes: more is sharper and larger (§15.2)
BLS	Boot Loader Specification: one small file per boot entry, in `/boot/loader/entries` (§29.2)
BlueZ	the Bluetooth stack: the kernel's part and the `bluetoothd` service (§30.3)
boot menu	the firmware's one-time list of disks to start from; GRUB's list of kernels is also called so (§3.6)
branch (git)	a named line of commits, for trying something without disturbing `main` (§41.1)
Btrfs	the copy-on-write file system of `/` and `/home`, with subvolumes and instant snapshots (§4.2)
buffer (graphics)	a block of memory holding one finished picture of a window or a screen (ch. 31)
buffer (neovim)	a file's text held in memory while you edit it; the file changes only when you save (ch. 17)
bus (D-Bus)	the channel D-Bus messages travel on: the system bus, or your session bus (§32.1)
cache	files a program can make again: deleting them costs time, not work (ch. 38)
cgroup	a group of processes the kernel counts and limits together: CPU time, memory, disk (§28.6)
checksum	a short fingerprint computed from every byte of a file; one changed byte changes it completely (§3.3)
codec	how one stream is compressed: H.264, HEVC or AV1 for video, AAC or Opus for sound (§15.2)
command	a line you type at the prompt and send with **Enter**; its first word names the program (§11.4)
command substitution	`$( … )`: bash runs the command inside first and puts its output in its place (§25.1)
commit	one recorded set of staged changes, with a message that says why, your name and the time (§41.1)

Term	Meaning
compile · interpret	turn a whole program into machine code first, as for C · run it line by line, as bash does (§43.4)
compositor	the program that places every window on the screen and handles input; under Wayland, the display server (§2.3)
comps	Fedora's package groups, such as `core`, which a Minimal Install is (§27.4)
configuration file	settings a program reads when it starts or reloads: `~/.config/foot/foot.ini` (§43.4)
conflict (git)	both sides changed the same lines; git stops and marks them for you to choose (§41.13)
console · text console	a full screen of text on the machine itself, without windows: tty1–tty6, **Ctrl+Alt+F1…F6** (§4.3)
container	a program fenced off from the rest of the system, with its own files, sharing the kernel (ch. 20)
container format	the file type that holds a video's streams together: `.mkv`, `.mp4`, `.webm` (§15.2)
contrast ratio	how much lighter one colour is than another, 1 : 1 to 21 : 1; text needs 4.5 : 1 (§40.1)
COPR	Fedora's build service, where anyone may publish a small repository; lazygit and LACT come from two (§5.4)
copy-on-write	writing a changed block to a new place instead of over the old one, which snapshots keep (§23.5)
cron	the older way to run jobs on a schedule; systemd timers do it here (§28.4)
D-Bus	the desktop's message system: programs send each other short messages over it (§15.5)
D-Bus name	the address a program takes on a bus, such as `org.mpris.MediaPlayer2.mpv` (§32.1)
dbus-broker	the D-Bus implementation Fedora uses (§32.1)
dependency	something a package requires to work: another package, a library, a command (§27.2)
device file	a file in `/dev` that stands for hardware: `/dev/nvme0n1` is the whole SSD (§23.2)
distribution	a kernel and thousands of programs, tested and released together as one system: Fedora is one (§2.2)
DNS	the internet's phone book: it turns a name such as `fedoraproject.org` into an IP address (§33.2)
dracut	the tool that builds the initramfs (§29.6)
driver	the part of the system that operates one kind of hardware; on Linux most are in the kernel (§2.1)
DRM	the kernel's graphics subsystem, the *Direct Rendering Manager* (§31.1)
drop-in	a file in a `NAME.d/` folder that overrides lines of a config or unit without editing it (§28.3)
dwl	the tiling Wayland compositor of this desktop: a small C program built on wlroots (§2.3)
edition	one of Fedora's starting sets for a purpose: Workstation, Server… (§2.2)
EFI system partition	the small FAT partition at `/boot/efi` from which the firmware loads shim (§23.2)
eject	write out what is still on its way to a drive, close it, and say *safe to unplug* (§13.1)
encoding	how letters are stored as bytes: UTF-8 today, Windows-1256 in many older Arabic files (ch. 19)
environment variable	a variable marked with `export`, so every program started afterwards gets a copy (§11.2)
exec	a process replacing its program with another, keeping its PID (§26.1)
exit status	the number a command ends with: 0 is success, anything else a failure (§11.2)
fast-forward (git)	a merge that only moves the branch on to the other's newest commit, with no merge commit (§41.11)
fetch (git)	download a remote's new commits without touching your files or your branch (§41.1)
ffmpeg	the library and tool that decode and encode nearly every video and sound format (§19.8)
file descriptor	a numbered open file of a process: 0 its input, 1 its output, 2 its errors (§25.2)
file system	how a disk's space is organised into files and folders: NTFS, exFAT, Btrfs (§3.5)
fingerprint	the long, unique number of a signing key, checked before the key is trusted (§27.5)

Term	Meaning
firewalld	the service that holds the firewall's rules, by zones (§33.3)
firmware	software in a device's own chips: the motherboard's UEFI, a Wi-Fi card's, a GPU's (§3.6)
focus	the window that receives what you type; its frame is vermilion (§6.4)
fontconfig	the library that picks the font file for a name, a language and a missing character (§31.7)
fork	a process copying itself to make a child (§26.1)
formatter	a program that rewrites a file's layout without changing what it does (ch. 17)
free software	software anyone may read, change and pass on: free as in freedom, not as in price (§2.4)
FreeType	the library that turns font outlines into pixels (§31.7)
freeworld	RPM Fusion's builds of Mesa and libheif with the codecs Fedora leaves out (§27.6)
GE-Proton	a community build of Proton, with fixes and media patches ahead of Valve's (§21.1)
generator	a small program systemd runs at boot to write units, such as podman's for quadlets (§34.6)
git	the version-control program: it keeps the history of a folder as commits (§41.1)
glob	a pattern bash turns into matching file names before the command runs: `*.pdf` (§25.1)
greetd	the login manager: it holds tty1, asks for your name and password, then starts the desktop (§5.4)
group	a named set of users given access together; its number is the **GID** (§24.1)
GRUB2	the boot loader that shows the boot menu and loads the kernel (§29.2)
hard link	a second name for the same inode (§23.6)
hardware decoding	the graphics card unpacks video instead of the processor, through VA-API (§15.2)
HarfBuzz	the library that shapes text: which glyphs go where; it joins Arabic letters (§31.7)
hash (git)	a commit's unique name, a long hexadecimal number; its first seven characters are enough (§41.1)
`HEAD`	git's "where you are now": normally the newest commit of the branch you are on (§41.1)
health check	a command run inside a container from time to time, asking whether it still answers (§34.5)
home folder	your own folder, `/home/you`, written `~` (§10.1)
hunk (git)	one block of changed lines in a diff; lazygit stages them one by one (§41.8)
image (container)	a read-only bundle of one program and everything it needs, from which containers run (§34.5)
image (disk)	one file holding a whole disc or stick, byte for byte: an `.iso` (§3.3)
indexer	a website that lists torrents and answers searches; Prowlarr asks all of yours at once (ch. 20)
initramfs	a small system loaded beside the kernel, only to find and mount the real one (§29.6)
inode	a file's record: type, permissions, owner, size, times and where its data lies — all but its name (§23.6)
install reason	why dnf has a package: *User*, *Dependency*, *Weak Dependency* or *Group* (§27.4)
ioctl	a special call on a device file, beyond reading and writing: how Mesa drives the GPU (§23.1)
IP address	the number a machine is reached at on a network: `192.168.1.23`; `127.0.0.1` is itself (§33.1)
job	a command the shell started and still tracks, numbered `%1`, `%2` … in each terminal (§26.3)
journal	systemd's one log of the whole machine, read with `journalctl` (§28.5)
kdump	memory set aside at every boot to record a kernel crash for later study (§4.2)
kernel	the core program that drives the hardware and shares it out between programs: Linux (§2.1)
kernel argument	an option handed to the kernel as it starts; `cat /proc/cmdline` shows them (§5.3)
kernel module	a driver in a file of its own, loaded only when its device is found (§30.1)
KMS	the kernel's control of screens and their modes, *kernel mode setting* (§31.1)

Term	Meaning
kmscon	the program that draws the text consoles in Fedora 45, in place of the kernel (§6.1)
label	the name a file system carries, such as `KINGSTON` (§13.1)
language server · LSP	a separate program that understands one language and answers the editor's questions · the protocol it speaks (ch. 17)
launcher	the box that starts a program by its name: fuzzel (§6.4)
layer shell	the Wayland protocol for bars, notifications, menus and wallpapers (§31.4)
leader	the key that starts LazyVim's own commands: Space (ch. 17)
libinput	the library that reads keyboards, mice and touchpads for the compositor (§30.2)
libva	the VA-API library: programs' way to the GPU's video decoder (§27.8)
linger	your own systemd manager kept running with nobody logged in (§28.2)
LLM · token · context	a large language model · a piece of text it reads or writes · all it can see at once (§43.4)
logind	systemd's login manager: sessions, seats and access to devices (§29.8)
loop	part of a script that runs again for each item: `for f in *.jpg; do …; done` (§25.1)
lossless · lossy	keeps every pixel or sound exactly (PNG, zip) · drops detail for a much smaller file (JPEG, MP3) (ch. 19)
LUKS	Linux's standard disk encryption: an encrypted drive opens only with its passphrase (§13.1)
LVM · XFS	the volume manager and file system of Fedora Server's automatic layout; not used here (§4.2)
magnet link	a link that names a torrent by its fingerprint, the *info-hash* (ch. 20)
main area	the large left part of a tiled screen; the other windows share the column on the right (§6.4)
Markdown	plain text with a few marks for structure: `##` before a heading, `-` before a list item (§16.3)
merge (git)	bringing the commits of one branch into another (§41.1)
Mesa	the open OpenGL, Vulkan and VA-API drivers (§31.2)
metadata (file)	facts kept inside a file: author, dates, and in a photo the phone and the place (ch. 19)
metadata (repository)	a repository's index of its packages, their versions and what each requires (§27.2)
microcode	the processor maker's fixes, loaded into the CPU at every start (§29.6)
MIME type	the standard name for a kind of file: `application/pdf`, `image/png`, `text/plain` (§10.5)
mode (neovim)	what keys do at the moment: give commands (Normal), type text (Insert), select (Visual) (ch. 17)
mount · mount point	to attach a file system to a folder, so its files appear there · that folder (§5.1)
MPRIS	the D-Bus standard through which players say what they play and take play and pause (§15.2)
NetworkManager	the service that runs the network connections: `nmcli`, `nmtui` (§33.1)
NEVRA	name, epoch, version, release, architecture: a package's exact identity (§27.1)
nice value	a process's place in the queue for the CPU, from −20 (served first) to 19 (last) (§26.4)
OCR	*optical character recognition*: a program reads the letters in a picture as text (ch. 19)
OpenGL	the older GPU programming interface; Vulkan is the newer (§31.2)
operating system	the software between programs and hardware: it shares out the machine and keeps programs apart (§2.1)
operator · motion · text object	in neovim: a verb (`d`, delete) · a move (`w`, next word) · a named piece (`iw`, this word) (ch. 17)
option	a word starting with `-` or `--` that changes how a command works: `-l`, `--sort=size` (§11.6)
ordered dither	a fixed 8×8 pattern that decides which blocks go first, for an even, fine grain (§40.6)
OSC sequence	an invisible message a program prints to the terminal, such as where the prompt begins (§40.7)

Term	Meaning
package	one program or library in Fedora's format, an `.rpm` file, installed and removed as one (§2.3)
PAM	the library every login uses to check a password and open a session (§29.8)
partition	a fixed part of a disk used as if it were a disk of its own; Windows' *C:* is one (§4.2)
patch	a file of line changes applied to a program's source before it is built: `src/dwl/dwl.patch` (§5.4)
path	the address of a file or folder: `/home/you/Documents/tenancy.pdf` (§10.1)
`PATH`	the folders searched, in order, for a command typed by its name (§25.1)
peer · seeder · leecher	a computer sharing a torrent · one that has all of it · one still downloading (ch. 20)
permissions	who may read (r), write (w) and run (x) a file (§24.2)
personal access token	a long password GitHub makes for one purpose, which git takes in place of your password (§5.1)
picker	a search window in neovim: type, choose from the list, see a preview (ch. 17)
PID · PPID	a process's number · the number of its parent (§26.1)
pinned commit	one exact version of a program's source, named by its commit hash (§5.4)
pipe · redirection	one command's output into the next command · output into a file, or input from one (§25.1)
PipeWire	the sound server: it carries every program's sound, and a screen share's picture (§14.4)
pipewire-pulse	PipeWire's PulseAudio-compatible server, for programs that speak PulseAudio (§31.8)
plugin (neovim)	an add-on written in Lua; LazyVim is a collection of them (ch. 17)
polkit	the service that decides whether a user may ask another service for something privileged (§24.4)
polkit agent	the session program that asks for your password for polkit: `lxqt-policykit-agent` (§24.4)
poll	to ask again and again, on a timer, whether something has changed (§40.9)
port	a numbered door on a machine where a program listens: 443 for the web, 9696 for Prowlarr (§33.3)
portal	a D-Bus service through which a program asks the desktop for a file dialog, a screen to share… (§32.2)
process	a program while it runs, with its own number, the PID (§26.1)
prompt	the text a command line shows while it waits for you; to an AI assistant, what you ask it (§4.3)
Proton	Valve's compatibility layer, built on Wine: it runs Windows games on Linux (§21.1)
PTY	a pseudo-terminal, `/dev/pts/N`, made for a terminal program in a window (§11.1)
push · pull (git)	send your new commits to a remote · bring in the commits made elsewhere (§41.1)
quadlet	a small file that tells systemd how to run a container as a service (§34.5)
quickfix list	in neovim, a list of places — file and line — to step through, such as search results (ch. 17)
quoting	`'…'` keeps text exactly as typed; `"…"` keeps it one word but expands `$NAME` (§25.1)
radeonsi	Mesa's OpenGL and VA-API driver for AMD graphics (§31.2)
RADV	Mesa's Vulkan driver for AMD graphics (§31.2)
readline	bash's line editor: what Tab, Up and the word keys do while you type (§25.1)
rebase (git)	replay your commits on top of another branch, as new commits (§41.1)
reflog	git's record of every place `HEAD` has been, which makes undo possible (§41.1)
register (neovim)	one of neovim's clipboards, named by a character: `a` … `z`, and `+` for the desktop's (ch. 17)
regular expression	a pattern for matching text: `kawa-[a-z]+` matches `kawa-menu` (ch. 19)
remote (git)	another copy of a repository, on GitHub say: push and pull go there (§41.1)
render node	`/dev/dri/renderD128`: the GPU for drawing and decoding, without control of the screens (§31.1)

Term	Meaning
repository (git)	a folder with a history: git keeps it in a hidden `.git` folder inside (§41.1)
repository (packages)	a server that offers packages and an index of them; `dnf repolist` lists those in use (§5.3)
Requires	a dependency that must be there, or the package is not installed (§27.4)
resolution · dpi	how many dots (pixels) per inch of paper: a sharp A4 scan is 300 dpi (ch. 19)
resolved	systemd-resolved: name look-ups through the local stub at `127.0.0.53` (§33.2)
root	the administrator account, UID 0, allowed everything; here nobody logs in as it (§4.2)
rootless container	a container run by an ordinary user, in a user namespace (§34.4)
RPM	Fedora's package format, and the tool that installs single packages (§27.1)
RPM Fusion	the community repositories for what Fedora does not ship, such as the full ffmpeg (§5.4)
scope · slice	systemd's groups of processes: one session or service · a branch of the tree (§28.6)
scriptlet	a small script inside a package, run when it is installed or removed (§27.1)
season	the time between two updates, 90 days unless you change it (§36.1)
Secure Boot	a firmware setting that starts only boot programs signed with a trusted key (§3.6)
seeding · ratio	sharing a finished download · uploaded ÷ downloaded (ch. 20)
SELinux	the kernel's mandatory access control by labels: it limits even root; *enforcing* on Fedora (§34.1)
SELinux label	the security label of every file and process, `user:role:type:level`; `ls -Z` shows it (§34.2)
SELinux type	the part of a label that says what a process or file is: `user_home_t`, `bin_t` (§34.1)
service	a program that runs in the background with no window, started and watched by systemd (§2.5)
session (logind)	one login of one user, its processes kept in `session-N.scope` (§29.8)
set-user-ID	a permission bit that runs a program as its owner: how `sudo` becomes root (§24.2)
shebang	the `#!` first line of a script, naming the program that runs it (§25.5)
shell	the program that reads your commands and runs them: bash (§11.1)
shim	the small Microsoft-signed first boot loader, which checks GRUB (§29.1)
signal	a short numbered message the kernel delivers to a process: TERM, KILL, USR2… (§26.2)
signature	data made from a file with the packager's secret key; it proves the file unchanged (§27.5)
sink · source	in PipeWire, a sound output (speakers, headphones) · a sound input (a microphone) (§31.8)
snapshot	a frozen copy of a Btrfs subvolume at one moment, sharing unchanged blocks; not a backup (§4.2)
socket activation	starting a service when the first client connects to its socket: `pipewire.socket` (§28.1)
soname	a shared library's name with its major version: `libva.so.2` (§27.8)
spoke · hub	one item of the installer's summary screen · that summary screen (§4.1)
staging area	git's tray, also called the index: exactly what the next commit will hold (§41.1)
standard input · output · error	the three streams every program has: what it reads, what it prints, where its errors go (§25.2)
stash (git)	uncommitted changes put aside, the files back as at the last commit (§41.10)
sticky bit	on a folder: only a file's owner may delete or rename it, as in `/tmp` (§24.2)
stub	a stand-in for a real program or device in a test, answering as the real one would (§39.3)
subvolume	a part of a Btrfs file system with its own folder tree, snapshotted on its own (§4.2)
sudo	runs the one command after it as root, after asking for your own password (§4.3)
symbolic link	a tiny file holding the path of another; opening it opens what it points at (§23.6)
systemd	process 1: it starts and watches every service; each user has a manager of their own (§28.1)

Term	Meaning
tab (neovim)	a whole layout of windows — not one file each, as in a browser (ch. 17)
target	a unit that groups others and marks a point in the start-up (§28.1)
terminal emulator	the program that draws a terminal in a window: foot (§11.1)
tiled · floating · monocle	dwl's layouts: windows side by side · placed freely, by you · one at a time, at full size (§6.4)
tiling	the compositor arranging windows side by side for you, never overlapping (§2.3)
timer	a unit that starts a service on a schedule (§28.4)
tmpfs	a file system kept in memory: `/tmp`, `/run` (§23.2)
toolkit	the installed copy of the project, `/usr/local/lib/fedokawaii`, which root runs for every update (§5.5)
torrent	a set of files shared piece by piece among many computers (ch. 20)
transaction	one dnf run that installs, updates or removes a set of packages together (§27.2)
treesitter	a parser in neovim that reads the structure of code, for colours and text objects (ch. 17)
TTY	a terminal device in the kernel; `/dev/tty1` is the first console (§11.1)
tuned · profile	the service that applies a set of kernel and hardware settings · one such set, here `fedokawaii` (§30.4)
type	what kind of value something is: a number, text, a list (§43.2)
udev	the service that creates and names device files and announces new devices (§30.2)
udisks · udiskie	the system service that mounts drives for you · the session program that asks it to (§13.1)
UEFI	the modern kind of firmware, in every recent PC; it starts the boot loader (§3.6)
UID	the number the kernel knows a user by: yours is 1000, root's is 0 (§24.1)
umask	the permission bits taken away from every file and folder you create (§24.2)
unit	anything systemd manages, described by one small file: `.service`, `.timer`, `.mount`… (§28.1)
update · upgrade	newer packages of the release you have · the move to the next release, 44 → 45 (ch. 35)
upstream	the project that writes a program; Fedora, which packages it, is downstream (§2.2)
upstream (git)	the remote branch yours follows, such as `origin/main` (§41.1)
user manager	your own systemd, `user@1000.service`, which runs your services (§28.2)
user namespace	a private numbering of users for one container, borrowed from numbers nobody real owns (§34.4)
user-installed	a package asked for by name: by you, or by setup from the manifest (ch. 38)
userspace	everything that runs outside the kernel (§2.2)
VA-API	the interface for decoding and encoding video on the GPU (§15.2)
variable	a named value, read back as `$NAME` (§11.2)
Vulkan	the modern GPU programming interface (§31.2)
Wayland	the way programs and the compositor talk on modern Linux, the successor of X11 (§2.3)
weak dependency · Recommends	a dependency installed by default, but optional; it may be left out or removed (§27.4)
wheel	the administrators' group: its members may use `sudo` (§4.2)
window (neovim)	a view onto a buffer: two windows can show two files, or two places in one (ch. 17)
WirePlumber	PipeWire's session manager: it picks defaults and connects programs to devices (§31.8)
wlroots	the library dwl is built on (§31.3)
working copy	`~/repos/fedokawaii`: your own git copy of the toolkit, where the desktop is changed (§42.4)

Term	Meaning
working folder	the folder a terminal is "in"; a name without a full path is looked for there (§11.4)
working tree	the files as they are on disk now, with edits not yet committed (§41.1)
workspace	a separate set of windows, one set at a time on a screen; dwl calls them tags (§6.4)
XDG base directories	the rule for where programs keep settings, data, state and cache (§23.3)
`XDG_RUNTIME_DIR`	your per-session folder in memory, `/run/user/1000` (§23.2)
xe · i915	Intel's newer and older kernel graphics drivers (§31.2)
XKB	the keyboard layouts and options: `us,ara`, `grp:alt_shift_toggle` (§30.2)
XWayland	the bridge that lets an older X11 program, such as Steam, show on a Wayland desktop (§21.1)
zone (firewalld)	a named list of what may come in, for the network cards assigned to it (§33.3)
zram	swap kept compressed in memory (§30.4)

F. Where things stand, October 2026

A snapshot taken on 15 October 2026: the state of the parts of Linux this desktop is built from, and of the trade you are learning. Each fact carries the date it was true. A fact marked † rests on a secondary source only (a press report or an aggregator), because the primary one could not be read; treat it as likely, not settled. The survey figures in §43.2 are not repeated here. The last table says where to look when this section is out of date.

F.1 The Linux kernel

The kernel is the core of Linux: it drives the hardware and runs every program. `uname -r` shows the one you are running.

Version	Released	Worth knowing
6.18	30 November 2025	the newest longterm (LTS) kernel, projected to be kept until December 2028
6.19	8 February 2026	the last 6.x; the next was named 7.0
7.0	12 April 2026	Rust no longer marked experimental; lazy preemption on by default in the scheduler
7.1	14 June 2026	a record 530 first-time contributors
7.2	16 August 2026	**the current release**: 16,418 changes, the second-busiest cycle ever, from a record 2,652 developers
7.3	expected 18 October 2026 †	7.3-rc7 came out on 11 October

How a release is made (kernel.org; Torvalds's mails on LWN):

Stage	What happens	For example
merge window	two weeks in which new features go in; it closes with the first release candidate	7.2-rc1, 28 June 2026
release candidates	one a week, fixes only, usually up to rc7 or rc8	7.2-rc7, 9 August 2026
mainline release	a new kernel every 9–10 weeks	7.2, 16 August 2026
stable updates	fixes for the newest release, "usually once a week", until the next mainline one	7.2.2, 28 August 2026
longterm (LTS)	a release kept for years with "only important bugfixes"; it starts with a two-year projected end, which can be extended	6.18

Six LTS series are kept (kernel.org, February 2026): 6.18 and 6.12 until December 2028, 6.6 and 6.1 until December 2027, 5.15 and 5.10 until December 2026. kernel.org says LTS kernels "are picked based on various factors"; every one in its table came out between October and December, so the next is likely near the end of 2026 — a pattern, not a promise.

Rust is no longer an experiment. At the Maintainers Summit (December 2025) the kernel's maintainers judged the Rust experiment a success, and 7.0 (April 2026) dropped the "experimental" label. Rust code in the kernel grew five-fold in the year to December 2025. What has landed: Android's binder driver (6.18); a Rust ashmem module that ships on Android 16 devices with 6.12 kernels; parts of Nova, the new driver for NVIDIA GPUs, whose 7.3 work is on starting the GPU's firmware processor (August 2026 †); and the zerocopy crate (7.2). The code that runs this desktop — amdgpu, Btrfs — is still C. DRM's maintainer, Dave Airlie, said in December 2025 that the graphics subsystem is "about a year away" from refusing new drivers written in C: an expectation, not yet a rule.

F.2 Fedora

Release	Out	Supported until
42	15 April 2025	ended 27 May 2026 †
43	28 October 2025	four weeks after Fedora 45 comes out; the schedule's date is 9 December 2026 †
44 — this desktop	28 April 2026	2 June 2027
45	**not out yet**: beta 15 September 2026, release due 20 October 2026 (27 October and 3 November held in reserve †)	about thirteen months

A release is supported until four weeks after the one two later comes out (§36.5). `grep SUPPORT_END /etc/os-release` shows the date your own system was given.

What changed from Fedora 41 to 45, and what it means here:

Release	What changed	On this desktop
41 (October 2024)	DNF 5 became `dnf`	its settings live in `/etc/dnf/libdnf5.conf.d/` (§27.3); it does not show the old dnf's history
42 (April 2025)	`/usr/sbin` became a link to `bin`, and left `$PATH`	`ls -ld /usr/sbin` → `/usr/sbin -> bin`; call a command by its name, not its path
43 (October 2025)	GNOME's X11 session removed from Fedora; RPM 6.0, with enforced signatures put off	nothing for dwl, which is Wayland; Steam still runs through XWayland (§21.1)
44 (April 2026)	the `ntsync` module offered to Wine and Steam through a package recommendation	the driver GE-Proton uses (§21.1)
45	**kmscon** draws the text consoles instead of the kernel	**Ctrl+Alt+F2…F6** keep their own colours and font (§6.1)
45	RPM refuses an unsigned package by default	a downloaded `.rpm` without a valid signature is refused; `rpm --nosignature` overrides it for one command
45	`ptrace` restricted (`kernel.yama.ptrace_scope=1`) unless debugging tools are installed	`gdb -p PID` or `strace -p PID` on a running program needs `yama-ptrace-enable`, which Fedora's gdb and strace are meant to bring
45	oo7 replaces GNOME Keyring as the desktops' password store	the manifest lists no keyring: nothing changes
45	a kernel panic shows a QR code that opens an explanation page	scan it with a phone; the trace travels in the link, not to Fedora's server
45	Fedora's own repository files move from `/etc/yum.repos.d` to `/usr/share/dnf5/repos.d` (accepted)	a file you edited in `/etc` stays, and wins

Fedora 45 also brings Python 3.15, GCC 16.2, glibc 2.44 and Go 1.27 (beta, September 2026).

F.3 The Linux desktop

Wayland is now the only session on the two big desktops. X11 programs still run on both, through XWayland, as Steam does here.

Desktop	The X11 session	When
GNOME 49	switched off by default	17 September 2025
Fedora 43	GNOME's X11 packages removed	28 October 2025
GNOME 50	removed from the code	18 March 2026
KDE Plasma 6.8	removed; Plasma 6.7 keeps it, supported "into early 2027"	14 October 2026

Xorg. The X.Org server's releases are now security fixes in its 21.1 series: 21.1.22 (April 2026), 21.1.23 (June 2026) and 21.1.24 (July 2026). **XLibre** is a fork of it, started in June 2025 by a contributor whose freedesktop.org GitLab account was blocked at the time. A proposal to make it Fedora 43's X server was withdrawn (26 June 2025); for Fedora it exists only in COPR repositories, outside Fedora's own. Neither matters to dwl, which needs no X server.

wlroots and dwl. dwl 0.9 (20 September 2026) moved to wlroots 0.20 (March 2026 †) and added screen sharing through the ext-image-copy-capture protocols, input methods, mouse-axis bindings and snapping windows to screen edges. This desktop is built on exactly these two (§5.2). The release before, dwl 0.8 (28 December 2025), was for wlroots 0.19.

NVIDIA. This desktop supports AMD and Intel graphics only (§5.2), so this matters only if you change cards. NVIDIA's own driver uses its open-source kernel modules by default from the R560 series on Turing (RTX 20, GTX 16) and newer cards (announced 17 July 2024); the 580 series is the last for the older Maxwell, Pascal and Volta (1 July 2025). The fully open alternative in Mesa, NVK, is a conformant Vulkan 1.4 driver (December 2024); from Mesa 25.1 (March 2025) OpenGL on Turing and newer runs through Zink on top of it, and Mesa 25.2 (August 2025) added Blackwell, the RTX 50 series.

HDR and colour. Wayland's colour-management protocol landed on 13 February 2025, after twelve years of work, and shipped in wayland-protocols 1.41. KDE Plasma has had partial HDR since 6.0 (28 February 2024); GNOME 48 added a High Dynamic Range switch to the display settings (19 March 2025); wlroots 0.20 supports the protocol and Sway's 1.12 test release added HDR10 (March 2026 †). dwl 0.9's release notes mention neither, so treat this desktop as standard range.

How many people use Linux. Two measures, counting different things:

Measure	Linux	Month	What it counts
StatCounter, desktop systems, worldwide	**4.54 %** (Windows about 76 %)	September 2026	3 billion page views a month on more than 1 million websites
Steam Hardware & Software Survey	**3.05 %**, down from 3.90 % †	September 2026	Valve's monthly sample of Steam users; the highest yet was 5.33 %, March 2026 †

A single month of either moves easily: the Steam share often falls when more of the sample is set to Simplified Chinese (GamingOnLinux, September 2026). Read the trend over a year.

F.4 Gaming

Proton 11.0 moved Valve's compatibility layer onto Wine 11.0 (11.0-1, 7 July 2026 †); 11.0-2 (21 August 2026) is the current release. GE-Proton follows it (§21.1). **SteamOS 3.8** became the stable release in June 2026, with first support for the Steam Machine and wider support for other makers' handhelds †.

Valve hardware	Status, October 2026
Steam Deck	OLED model only: the LCD model was discontinued and prices rose on 27 May 2026, to $789 (512 GB) and $949 (1 TB); Valve gave component costs as the reason †
Steam Machine	announced 12 November 2025 with the Steam Frame headset and a new Steam Controller †; a small PC running SteamOS on a semi-custom AMD Zen 4 / RDNA 3 chip †; sold through a waitlist, with the first sales set for 29–30 June 2026, from $1,049 (512 GB) †

Anti-cheat decides whether many online games run at all. Easy Anti-Cheat has supported Proton since 23 September 2021, but each game's developer has to switch it on. Some systems do not run on Linux: EA's Javelin (no Linux support, March 2026 †; Battlefield and EA Sports FC do not work) and Activision's Ricochet (every Call of Duty title on GamingOnLinux's list broken †). Check a game on that list before you buy it.

F.5 The software industry and AI

How developers use coding agents now. The tools have moved from suggesting a line to carrying out a task: an *agent* reads a repository, edits files, runs the tests and opens a pull request, often several at once. GitHub's Copilot app (technical preview, 2 June 2026) runs each agent session in its own git worktree, an isolated copy of the branch, and its cloud agents ask before every change by default. The same post counted 1.4 billion commits a month on GitHub, nearly double a year before.

Open-source projects have written rules for it:

Project	Rule	Since
the Linux kernel	only a human may add `Signed-off-by`; the human takes full responsibility; credit the tool as `Assisted-by: AGENT_NAME:MODEL_VERSION`	in the 7.0 documentation (April 2026)
Fedora	the contributor is accountable for every line; significant AI help should be disclosed, for example with `Assisted-by:`; AI is never the final judge in review	22 October 2025

The effect is visible: 299 commits in 7.1 carried `Assisted-by:`, and LWN ties the record number of newcomers largely to LLM-based tools (June 2026). AI also finds bugs, in numbers: by May 2026 the reports had made the kernel's security list "almost entirely unmanageable" (Torvalds), and the kernel now treats a bug found with AI as public (May 2026).

The Model Context Protocol (MCP) is an open standard for connecting an AI assistant to tools and data — files, a database, a web service — through small programs called *servers*. Anthropic gave it to the **Agentic AI Foundation**, a fund under the **Linux Foundation** co-founded by Anthropic, Block and OpenAI (9 December 2025), beside Block's goose and OpenAI's AGENTS.md. At that point it had more than 10,000 public servers and 97 million SDK downloads a month, and ChatGPT, Cursor, Gemini, Microsoft Copilot and Visual Studio Code used it (Anthropic's figures). The foundation added 97 members (24 February 2026). The current specification, 2026-07-28, dropped sessions: every request now stands on its own.

Open-weight models are published as files anyone may download and run on their own computer, so the code you show them stays on it. Two with a plain Apache 2.0 licence: OpenAI's gpt-oss-20b and gpt-oss-120b (5 August 2025; the smaller needs 16 GB of memory) and Google's Gemma 4 (2 April 2026; up to 31 billion parameters).

The job market. Figures for the United States, where the official series are fullest.

Measure	Figure	Date
software developers, QA analysts and testers (BLS)	1,905,400 jobs; median pay $134,040 a year	2025
the BLS projection for them	+10 % from 2025 to 2035, "much faster than average"	page of 27 August 2026
software-development job postings on Indeed, February 2020 = 100	**78.3**: about a fifth below early 2020	2 October 2026
share of all postings that mention AI (Indeed)	5.9 %, above the 2022 peak of 3.3 %	June 2026
employment of 22–25-year-olds in AI-exposed jobs (Stanford)	about 16 % lower, on the stricter measure	to October 2025

Read together: the trade is large and projected to grow, while postings remain well below 2020 and the fall is sharpest for the youngest workers. Indeed's economists say AI roles may be much of the recent rebound in software postings (June 2026). The Stanford authors say AI is not the only cause and that the clearest change dates from 2024, not 2022 (February 2026).

What employers ask for. The World Economic Forum's *Future of Jobs Report* (7 January 2025) put AI and big data first among the fastest-growing skills, then networks and cybersecurity, then technological literacy, with creative thinking, resilience, and curiosity and lifelong learning rising too; it lists software and applications developers among the fastest-growing jobs. The Linux Foundation's *State of Tech Talent* (10 June 2026; 400

hiring and training leaders, surveyed in February 2026) found 57 % short of skills in AI security and risk and in running AI systems, security and privacy the first barrier to adopting AI (48 %, up from 17 % in 2024), and training their own staff preferred (57 %) to hiring (49 %).

F.6 Security

The xz backdoor (2024). On 29 March 2024 a report titled "backdoor in upstream xz/liblzma leading to ssh server compromise" reached the oss-security list (CVE-2024-3094): xz 5.6.0 and 5.6.1 carried hidden code. A contributor calling himself "Jia Tan" had spent a long time earning the trust of xz's one unpaid maintainer, helped by accounts, widely thought to be sock puppets, that criticised the maintainer for being slow (LWN, 2 April 2024). Fedora Rawhide and Fedora 40's pre-release received the bad versions; Fedora 40 itself never did (Fedora Magazine, 29 March 2024).

Lesson	On this desktop
a release tarball can hold code that is not in the source repository	dwl is built from one pinned git commit, and setup refuses any other (§27.7)
check every download against a known checksum	each release binary in `lib/upstream.sh` has one; a changed download is refused (§36.7)
a signature says who built a package, not only that it arrived whole	Fedora 45's RPM refuses unsigned packages; `fedokawaii verify` checks the machine against the manifest (§35.2)
one tired volunteer can be the weak point of thousands of systems	every program here comes from such projects: report bugs, test new versions

Secure Boot and the 2011 certificates. Secure Boot lets the firmware start only signed boot code. Microsoft signs Linux's first-stage loader, **shim**, so that firmware which trusts Microsoft's keys will start it; shim then checks GRUB and the kernel with the distribution's own key.

Microsoft certificate	Expires	Replaced by	Signs
Microsoft UEFI CA 2011	26–27 June 2026	Microsoft UEFI CA 2023	third-party boot loaders: shim
Microsoft Corporation KEK CA 2011	June 2026	Microsoft Corporation KEK 2K CA 2023	updates to the firmware's lists of trusted and banned keys
Microsoft Windows Production PCA 2011	October 2026	Windows UEFI CA 2023	Windows's own boot loader

What it means: a machine keeps booting with the shim it has. From October 2025 until the expiry Microsoft signed each shim with both keys; from now on a new shim carries only the 2023 signature, so a machine whose firmware lacks the 2023 certificate could fail to boot a future shim (LWN, 1 July 2026). Fedora's advice (Fedora Magazine, 11 June 2026): no need to panic, but act. Fedora 45's shim is signed with both keys; update the firmware's key list through fwupd when your maker offers it (§30.1); never remove or revoke the 2011 key, which also signs *option ROMs*, the firmware on cards such as a graphics card. To look:

```
mokutil --sb-state                    # is Secure Boot on?
mokutil --db | grep 'Subject:'        # the trusted keys: look for "Microsoft UEFI CA 2023"
sudo fwupdmgr update                  # firmware and key-list updates your maker offers
```

(`mokutil` may not be installed: `sudo dnf install mokutil` brings it — keep it in the manifest if you do, §42.12.)

Memory-safe languages (Rust, for one) build in checks — on the bounds of a buffer, on memory that has been freed — that prevent whole classes of bug by design. C, in which dwl and most of the kernel are written, has no such checks.

Who	What	When
CISA and FBI (US)	list starting a new product for critical infrastructure in a memory-unsafe language such as C or C++ among "product security bad practices"; existing products should publish a memory-safety roadmap by 1 January 2026	October 2024
NSA and CISA (US)	software makers should "plan for and begin using" memory-safe languages, step by step rather than by rewriting	24 June 2025
Google (Android)	memory-safety bugs fell below 20 % of Android's vulnerabilities for the first time; an estimated 0.2 per million lines of Rust against close to 1,000 in C and C++	13 November 2025
the Linux kernel	Rust no longer experimental	April 2026
the EU's Cyber Resilience Act	makers must send an early warning of an actively exploited vulnerability within 24 hours, from 11 September 2026; open-source stewards from 11 December 2027	2026

F.7 How to keep up

Source	Where	Good for
LWN.net	lwn.net	careful news and analysis of the kernel and free software; the weekly *Kernel release status*
kernel.org	kernel.org/releases.html	the current mainline, stable and LTS versions, with their projected ends
Torvalds's release mails	lore.kernel.org/lkml	what each release candidate and release brought, in his words
KernelNewbies	kernelnewbies.org/LinuxChanges	each kernel's changes in plain language (written by volunteers)
Fedora Magazine	fedoramagazine.org	releases, how-tos and alerts, such as the Secure Boot one
Fedora's change lists	fedoraproject.org/wiki/Releases/45/ChangeSet	every change in a coming release, with its owner and reasons
This Week in GNOME · This Week in Plasma	thisweek.gnome.org · blogs.kde.org	desktop work as it lands
dwl's releases	codeberg.org/dwl/dwl/releases	the compositor here, and the wlroots each version needs
Steam Hardware & Software Survey	store.steampowered.com/hwsurvey	Linux among Steam users, monthly; read the trend, not one month
StatCounter	gs.statcounter.com	desktop systems' share of web traffic
GitHub blog	github.blog	Copilot and agent features; the yearly Octoverse report
Stack Overflow Developer Survey	survey.stackoverflow.co	what developers use and think, once a year
BLS Occupational Outlook Handbook	bls.gov/ooh	US jobs, pay and projections
Phoronix	phoronix.com	fast news and benchmarks; confirm a claim at its source before relying on it

Sources:

- Kernel: kernel.org, Releases · LWN, Development statistics for 6.19 · 7.0 · 7.1 · 7.2 · LWN, Kernel release status, 15 April 2026 · 12 August 2026 · LWN, Linux 7.2-rc1 · LWN, Linux 7.2.2 · Phoronix, Linux 7.2 released · Phoronix, Linux 7.3-rc7 · Phoronix, Nova in 7.3
- Rust: LWN, The state of the kernel Rust experiment · rust: conclude the Rust experiment (patch)
- Fedora: Announcing Fedora Linux 44 · Announcing Fedora Linux 45 Beta · Fedora docs: Fedora 45 release milestone · Accepted changes for Fedora 45 · F43 · F44 · SwitchToDnf5 · Unify bin and sbin ·

WaylandOnlyGNOME · RPM 6.0 · UseKmsconVTConsole · Enforcing signature checking · Restrict ptrace · oo7 · DrmPanicFrontend · RelocateRpmRepoConfigsToUsr · Fedora schedule issue #218 (the EOL rule) · endoflife.date, Fedora · Warp2Search, Fedora 45 Final Freeze

- Desktop: release.gnome.org (feed) · GNOME 48 · The X11 GNOME session removal · The Register, GNOME 50 · KDE, Going all-in on a Wayland future · LWN, KDE Plasma 6.8 released · KDE's 6th Megarelease · LWN, Fedora xorg-x11-server 21.1.23 · FreeBSD VuXML, xorg-server 21.1.24 · The Register, XLibre · F43 X11Libre proposal, withdrawn · dwl releases · Phoronix, wlroots 0.20 and Sway 1.12-rc1 · NVIDIA, open-source GPU kernel modules · NVIDIA, deprecation schedule · nouveau and NVK news · Collabora, 12 years of Wayland colour management · StatCounter, desktop OS share · StatCounter, about · GamingOnLinux, Steam survey September 2026 · KitGuru, March 2026 record
- Gaming: Proton 11.0-2 (Valve, GitHub) · Phoronix, Proton 11.0-1 · Digital Citizen, SteamOS 3.8 · Game Informer, Steam Deck prices · Phoronix, Steam Machine announced · Notebookcheck, Steam Machine price and date · Epic, Easy Anti-Cheat for Linux · GamingOnLinux, EA Javelin · GamingOnLinux, anti-cheat list
- AI and industry: GitHub, the Copilot app · Linux kernel, AI coding assistants · Fedora Council, AI-assisted contributions policy · LWN, Dirk and Linus discuss AI · Anthropic, donating MCP · Agentic AI Foundation, news · MCP 2026-07-28 changelog · OpenAI, gpt-oss · Google, Gemma 4 · BLS, Software developers · FRED, Software development postings on Indeed · Indeed Hiring Lab, June 2026 · Stanford Digital Economy Lab, February 2026 · WEF, Future of Jobs Report 2025 · Linux Foundation, 2026 State of Tech Talent
- Security: oss-security, xz backdoor · LWN, Free software's not-so-eXZellent adventure · Fedora Magazine, CVE-2024-3094 · Microsoft, Secure Boot certificate expiration · LWN, Linux and Secure Boot certificate expiration · LWN, Secure Boot certificate expiration is here · Fedora Magazine, Don't Panic · Fedora test day, multi-signed shim · CISA and FBI, Product Security Bad Practices · NSA, memory-safe languages · Google, Rust in Android · European Commission, CRA reporting

G. Sources

The official documentation this book was checked against, by subject. Sources for a single section — the language surveys, *Where things stand* — are listed where they are used.

Subject	Source
Fedora's editions, releases and changes	fedoraproject.org · Release life cycle · Staying close to upstream
Fedora change pages	dbus-broker · BLS · cgroups v2 · Btrfs · zram · zram size · systemd-resolved · PipeWire · WirePlumber · DNF 5 · bin and sbin · RPM 6.0 · kmscon · signature checking · sudo-rs (proposal)
boot	Boot Loader Specification · Secure Boot on Fedora · microcode loader · dracut.conf(5)
systemd	systemd man pages · systemd-analyze(1) · loginctl(1) · desktop environments and systemd
files and processes	file-hierarchy(7) · inode(7) · signal(7) · GNU coreutils 9.0 news · Btrfs documentation
packages	rpm.org · rpmkeys(8) · RPM 6.0 release notes · dnf5.conf(5) · repoquery · history · mpv-libs in Fedora 44 · mesa-dri-drivers in Fedora 44 · amd-gpu-firmware · RPM Fusion's mesa-freeworld
graphics	Wayland architecture · wlr-layer-shell · Intel xe GPUs · AMDVLK discontinued · Mesa
sound, devices, network	PipeWire · pipewire-pulse(1) · WirePlumber · NetworkManager · firewalld
buses and specifications	D-Bus · portals.conf · Desktop Notifications · MPRIS · XDG Base Directory
security and containers	SELinux on Fedora · podman-systemd.unit(5)
changing it and learning	Pro Git · lazygit's documentation · WCAG, Contrast (Minimum) · foot, shell integration · Claude Code
secondary	the Arch Wiki — written for Arch, mostly true here
this desktop	`setup.sh`, `lib/manifest.sh`, `lib/hardware.sh`, `system/`, `configs/home/`, `bin/` and `tests/` in the toolkit, `/usr/local/lib/fedokawaii`